Enterprise Digital Twin Architecture Blueprint

Specification 1.0.0-rc.1 · Candidate
Committed H1/H2 architecture
2026-07-13

Publication boundary

Production-quality invariants apply within the stated horizon limits. Provisional and research capabilities are not represented as production-ready.

Contents

Enterprise Digital Twin Architecture Blueprint

Specification: `EDT`

Version: `1.0.0-rc.1`

Status: `committed`

Release stage: `candidate`

Publication date: `2026-07-13`

> Production-quality invariants are committed only inside the stated horizon boundaries. Later capabilities remain explicitly provisional, research, or rejected.

Part I - Governance and Reading Guide

Enterprise Digital Twin Architecture Blueprint

Enterprise Digital Twin Architecture Blueprint

Authority and intent

This documentation set is the normative engineering and product specification for Enterprise Digital Twin. It defines a narrow, buildable H1 demonstrator and a gated enterprise reference architecture. It is not evidence that later horizons have been built, certified, or validated.

Normative terms use RFC 2119 meanings: MUST and MUST NOT are requirements; SHOULD and SHOULD NOT require a documented exception; MAY is optional. Narrative examples cannot override contracts, accepted architecture decision records, or controls.

Reading order

Normative supporting artifacts

- `manifest.yaml` declares the release, document order, statuses, and generation targets.
- `catalogs/requirements.yaml` is the requirement ledger.
- `catalogs/traceability.yaml` maps every requirement to design, controls, acceptance evidence, and a horizon.
- `adrs/` records consequential decisions and introduction triggers.
- `contracts/` contains API, event, graph, agent, connector, and extension interfaces.
- `diagrams/` contains version-controlled architecture and workflow sources.
- `reviews/` contains the threat model, privacy assessment, FMEA, and risk acceptance record.

Horizon semantics

Status	Meaning
Committed	Implementation teams may rely on this decision for the stated horizon.
Provisional	Direction is selected, but entry criteria must be met before implementation.
Research	No production claim or delivery commitment exists.
Rejected	Evaluated and intentionally excluded until an ADR supersedes the decision.

Change control

Changes to H1 or H2 behavior require an ADR, compatibility analysis, updated traceability, and successful validation. A later date does not automatically win. The accepted ADR and semantic version determine precedence.

Decision Precedence and Change Control

Decision Precedence and Change Control

Precedence

When artifacts disagree, apply this order:

1. Accepted ADRs that explicitly supersede an earlier decision.
2. Machine-readable API, event, ontology, and schema contracts.
3. Security, privacy, and AI control requirements.
4. Normative chapter text.
5. Diagrams.
6. Non-normative examples.

Security controls may narrow a capability but cannot silently broaden it. A contract cannot weaken a mandatory control. Such a conflict blocks release and requires an ADR.

Change classes

Class	Example	Required action
Patch	Typo or clarification with no behavioral effect	Review plus patch version.
Compatible feature	Optional response field or new event type	ADR, compatibility proof, minor version.
Breaking change	Removed field, changed authorization, altered ontology semantics	Migration plan, deprecation window, major version.
Control change	New approval or retention rule	Threat/privacy review and ADR.
Horizon promotion	Provisional capability becomes committed	Entry evidence, capacity/evaluation results, ADR.

Decision states

Every major decision is `proposed`, `accepted`, `superseded`, or `rejected`. Only accepted ADRs are normative. Provisional and research capabilities MUST state an entry gate and accountable owner.

Risk acceptance

Critical and High risks cannot be accepted for H1 or H2. Medium risks require a named accountable owner, expiry date, compensating controls, and revisit trigger. Risk acceptance cannot override law, contractual commitments, tenant isolation, or the prohibited-use policy.

Glossary

Glossary

Term	Definition
Agent	A versioned capability profile executed inside a deterministic policy and workflow envelope. It is not an organizational officer or independent principal.
Authoritative store	The system whose record controls recovery and conflict resolution for a data class.
Claim	A bitemporal, tenant-scoped assertion about a subject with provenance, confidence, classification, and evidence.
Connector	A least-privilege adapter that reads or writes an allowlisted external system using tenant-scoped credentials.
Control plane	Shared services for tenant provisioning, policy, deployment metadata, and fleet operations; it never grants cross-tenant data access.
Data plane	Tenant-scoped ingestion, storage, search, graph, agent, and simulation processing.
Evidence	An immutable reference to the source object and location supporting a claim.
H1-H5	Delivery horizons from demonstrator through separately governed research.
Idempotency key	A stable identifier that ensures replay of the same logical operation has one externally visible effect.
Observation	A normalized, immutable representation of a versioned source-system object or event.
Ontology package	A namespaced, versioned bundle of entity types, relationship types, properties, constraints, and migrations.
Projection	A rebuildable read model derived from authoritative observations and claims.
Scenario	A typed set of user-confirmed assumptions and interventions evaluated against a versioned simulation snapshot.
Source ACL	The source-system permission expression that constrains every derived claim, index, graph path, cache entry, and answer.
System time	When the platform learned or stored a fact.
Valid time	When a fact is asserted to be true in the modeled organization.

Normative Subsystem Template

Subsystem Name

Purpose and non-goals

State the capability, users, business outcome, and excluded responsibilities.

Ownership and boundaries

Name the owning team, authoritative data, dependencies, trust boundary, and prohibited coupling.

Interfaces and data

Define request, response, event, error, versioning, idempotency, classification, retention, residency, and migration semantics.

Invariants and workflows

Specify normal, denied, duplicate, out-of-order, timeout, cancellation, stale-data, partial-failure, recovery, and rollback paths.

Security and privacy

Define authentication, authorization, tenant derivation, delegated authority, audit, threat mitigations, and data-subject behavior.

Consistency and reliability

Define consistency model, retry policy, availability target, RPO, RTO, backpressure, degradation, and dependency-outage behavior.

Scale and cost

Define workload envelope, partition key, hot spots, limits, cost drivers, quotas, and technology introduction triggers.

Observability

Define structured logs, metrics, traces, audit events, dashboards, alerts, liveness, readiness, and health semantics.

Verification

List unit, contract, integration, security, privacy, performance, load, chaos, AI evaluation, and acceptance evidence.

Evolution and risks

Record residual risks, responsible owners, planned migrations, compatibility promises, and horizon gates.

Part II - Architecture Blueprint

CH-01 - Product Vision and Operating Principles

Status: committed | Owners: product-architecture, product-management | Last reviewed: 2026-07-13

Product Vision and Operating Principles

1. Mission

The Enterprise Digital Twin gives an authorized person a current, evidence-backed model of how an organization works, lets that person test a proposed change against the model, and permits controlled action only after the required human decision.

The product is a system of understanding and governed action. It is not a replacement for systems of record, an employee surveillance product, or an autonomous executive. PostgreSQL remains the authoritative store for the twin's claims, evidence, approvals, and audit indexes; connected systems remain authoritative for their source records. Graph, vector, search, and cache stores are derived projections.

`REQ-PROD-001`: The product MUST maintain a continuously synchronized, evidence-backed representation of organizational entities, activity, and dependencies within the authorized source scope.

`REQ-PROD-002`: Answers MUST cite accessible evidence and abstain when evidence is insufficient.

`REQ-PROD-003`: H1 MUST explain launch risk and compare a user-confirmed dependency scenario.

`REQ-PROD-004`: H1 MUST preview, dual-authorize, execute, audit, and compensate one allowlisted Jira remediation mutation.

`REQ-PROD-005`: The core metamodel MUST support namespaced domain and customer ontology packages without changing core code.

Every material statement shown as fact must be traceable to source evidence, an explicit user input, or a labeled simulation assumption. The product distinguishes observed fact, resolved claim, inference, recommendation, scenario assumption, and simulated output. It preserves source permissions and tenant boundaries from ingestion through retrieval, explanation, export, and action. Under `REQ-AI-008`, an agent cannot gain authority through reasoning, memory, tool selection, or handoff.

2. Product philosophy

2.1 Evidence before fluency

A concise abstention is better than an unsupported answer. Answers expose citations at the claim level, identify stale or missing sources, and state when evidence conflicts. Generated prose never upgrades an inference into a fact.

2.2 A model, not a mirror

The twin is a time-versioned interpretation of source observations. Entity resolution is reversible, relationships retain provenance and confidence, and projection lag is visible. The interface never implies that the twin is complete merely because a visualization is dense.

2.3 Simulation before mutation

Users can explore a scenario without changing a source system. A proposed external mutation is rendered as an exact payload, evaluated by policy, approved by the required humans, executed idempotently, and paired with a compensating action where the connector supports one.

2.4 Progressive trust

Autonomy expands only after a capability has passed offline evaluation, adversarial evaluation, shadow use, controlled pilot use, and an explicit governance decision. A broader model or tool catalog does not automatically broaden autonomy.

2.5 Bounded claims

The hackathon slice demonstrates production-quality invariants within a deliberately narrow envelope. Later horizons are commitments only where exit criteria and owners are defined. Research ideas are not presented as shipping capabilities.

2.6 Human dignity

The initial enterprise horizons exclude individual productivity, performance, burnout, attrition, hiring, compensation, health, emotion, misconduct, and similar employment scoring. Aggregation or dual approval does not make those uses acceptable.

3. Product principles

ID	Principle	Required behavior
PRN-001	Tenant isolation is invariant	Tenant context is derived by the server. Queries, indexes, object keys, graph namespaces, caches, prompts, traces, and exports are tenant-scoped.
PRN-002	Permission fidelity	Revocation propagates to every serving projection; stale permission state fails closed. Redacted facts cannot be reconstructed from counts, topology, snippets, or citations.
PRN-003	Provenance survives transformation	Normalization, entity resolution, summarization, graph projection, and simulation preserve links to the originating observation and transformation version.
PRN-004	Determinism where promised	Synthetic fixtures, projection rebuilds, scenario compilation, and seeded simulation produce reproducible outputs within documented numeric tolerances.
PRN-005	Uncertainty is a first-class output	Forecast distributions, assumptions, missing inputs, sensitivity drivers, and model limitations appear with the result.
PRN-006	Exact approval	Approval binds actor, tenant, action type, canonical payload hash, target, expiry, policy version, and source snapshot. Any change invalidates approval.
PRN-007	Safe retries	Connector ingestion and external actions use idempotency keys, replay protection, durable receipts, and explicit compensation state.
PRN-008	Rebuildable projections	Neo4j, vector, search, and cache data can be rebuilt from authoritative records without changing semantic identity.
PRN-009	Accessible equivalence	Every workflow and visualization has a keyboard-operable and non-visual equivalent that exposes the same decisions and information.
PRN-010	Observable degradation	Staleness, partial results, unavailable dependencies, policy denials, and degraded model behavior are visible rather than silently hidden.

4. Market position

The product is positioned for organizations that have operational data distributed across work-management, code-hosting, knowledge, and business systems but cannot reliably answer cross-system questions or test an operational change. The initial buyer is not purchasing another document repository or dashboard. The buyer is purchasing a governed organizational model with evidence, time, dependency, scenario, and action semantics.

The product category is defined by four connected jobs:

1. Continuously reconcile authorized observations from enterprise systems into a permission-aware organizational model.
2. Answer cross-system questions with claim-level evidence and visible data freshness.
3. Run reproducible what-if scenarios without altering source systems.
4. Convert an approved recommendation into a narrow, auditable, reversible action.

No claim in this chapter depends on a comparison with a named vendor. Competitive positioning must be updated only through dated primary sources and a separate, reviewable source ledger.

5. Differentiators expressed as testable capabilities

ID	Capability	Proof required
CAP-001	Evidence graph	A reviewer can navigate from an answer claim to the exact tenant-authorized source observation and transformation history.
CAP-002	Temporal organizational model	A reviewer can inspect what was believed at a selected time and distinguish source time, ingestion time, validity time, and transaction time.
CAP-003	Reversible identity resolution	A reviewer can inspect why records were merged, split an incorrect merge, rebuild projections, and observe no loss of source evidence.
CAP-004	Permission-aware reasoning	The same question asked by actors with different rights returns appropriately different evidence without leaking hidden topology.
CAP-005	Scenario-to-action chain	A reviewer can reproduce a simulation, inspect assumptions, preview an exact remediation payload, gather required approvals, execute once, and roll back.
CAP-006	Extensible domain model	A tenant can add a namespaced ontology package without modifying the core ontology or bypassing validation and authorization.
CAP-007	Evaluation-gated AI	A model or prompt version cannot enter a production profile unless workload-specific quality and safety gates pass.

6. Personas and responsibilities

Persona	Role in the product	Primary need	Must not be assumed
Executive sponsor	Economic buyer and accountable sponsor	Understand portfolio-level dependencies, evidence quality, and scenario tradeoffs	Unlimited access or authority to override policy
Transformation or operations leader	Primary H1/H2 operator	Diagnose launch risk, compare scenarios, and coordinate remediation	Permission to inspect every source or individual activity
Program or product leader	Domain user	Trace blockers, dates, decisions, and cross-team dependencies	That a graph relationship proves causation
Engineering leader	Domain user and source owner	Validate code and work dependencies, freshness, and proposed changes	That code metadata measures individual productivity
Analyst	Question and scenario author	Reusable evidence-backed analyses, exports, and assumptions	Authority to execute recommended actions
Approver	Human control point	Exact, understandable action payload and risk context	That approval is transferable or reusable after expiry
Security and compliance administrator	Policy and evidence owner	Connector scopes, access decisions, audit evidence, retention, and incident controls	Authority to read content outside granted scope
Tenant administrator	Tenant configuration owner	Identity mapping, connector setup, policies, and ontology packages	Platform-operator access to tenant plaintext
Connector owner	Source-system steward	Scope, health, rate limit, reconciliation, and credential rotation	Product-wide administrative access
Data steward	Resolution and quality reviewer	Conflicts, provenance, merge review, and deletion workflows	Permission to change source systems
Developer or extension author	API, SDK, and plugin consumer	Stable contracts, local fixtures, validation, and observability	Ability to publish unsigned or over-privileged extensions
Data subject	Person represented by authorized source data	Accurate use, policy-respecting visibility, correction and deletion handling	That product use converts observations into employment judgments
Platform operator	Service reliability role	Health and capacity signals with redacted diagnostics	Routine access to tenant content or model prompts

Buyer, user, approver, administrator, source owner, platform operator, and data-subject responsibilities MUST remain separate in policy and audit records even when one person holds multiple roles.

7. Value proposition and measurable outcomes

The product creates value only when it improves a decision without obscuring evidence or shifting unacceptable risk to represented people.

Outcome	Product measure	H1 target	H2 direction
Faster evidence gathering	p95 time from question submission to a reviewable cited answer	Less than 20 seconds for the reference workload	Establish customer baseline and demonstrate a material reduction without relaxing citation quality
Better dependency visibility	Ground-truth blockers discovered and correctly connected	100 percent in deterministic H1 fixtures	Calibrated precision and recall on partner-approved golden cases
Safer operational change	Unauthorized, expired, mutated, replayed, or duplicate actions executed	0	0
Reproducible planning	Identical seeded runs with identical snapshot and model version	100 percent within documented numeric tolerance	100 percent within versioned engine tolerance
Permission fidelity	Cross-tenant or unauthorized disclosure in automated and adversarial tests	0	0

Outcome	Product measure	H1 target	H2 direction
Explainable uncertainty	Forecasts displaying assumptions, uncertainty, sensitivity, and missing-data warnings	100 percent	100 percent
Recoverability	Approved H1 mutations with validated compensation path	100 percent	Per-action service-level objective before enablement

Business impact such as avoided delay or reduced coordination cost is reported as an attributed case study with its assumptions. The platform MUST NOT present simulated savings as realized value.

8. Product tracks

8.1 Production-shaped demonstrator

H1 proves the complete thin slice described in `CH-02`: synthetic GitHub and Jira observations, a permission-aware graph, cited launch-risk analysis, seeded schedule simulation, an exact Jira remediation preview, two-person approval, idempotent execution, and compensation. `CH-17` adds a separately bounded synthetic physical-asset demonstration with advancing telemetry, spatial inspection, deterministic analytics, lifecycle history, and simulator-only control. Neither track claims real customer integration or predictive validity. H1 favors depth and verifiability over connector or domain breadth.

8.2 Enterprise reference architecture

H2 through H4 define an extensible architecture for additional tenants, connectors, domains, deployment profiles, and operational controls. A capability is labeled:

- `Committed`: funded or required, with owner, contract, dependencies, and acceptance gate.
- `Provisional`: direction is accepted, but a benchmark, design-partner result, or external dependency must be resolved before commitment.
- `Research`: hypothesis with safety, validity, and graduation gates; not marketed as available.
- `Rejected`: considered and intentionally excluded from the current architecture, with a revisit trigger if one exists.

Status applies to a defined capability and horizon, not to a technology name in isolation.

9. Non-goals and prohibited uses

The following are out of scope through H3 unless a new architecture decision, safety review, legal review, and product-governance approval explicitly replace this boundary:

- Individual performance, productivity, burnout, attrition, hiring, promotion, compensation, emotion, health, misconduct, or termination scoring.
- Covert employee monitoring, keystroke tracking, screen capture, private-message inference, or social-relationship ranking.
- Fully autonomous external action, self-granted permissions, approval impersonation, or approval inferred from conversation.
- A claim that synthetic schedule simulation predicts real organizational outcomes.
- Replacement of GitHub, Jira, identity providers, financial systems, or other source systems as their operational system of record.
- Cross-tenant retrieval, entity resolution, memory, analytics, benchmarking, training, or model improvement without separately recorded opt-in governance.
- Simultaneous active-active operation across arbitrary clouds in H1 or H2.
- Trillion-edge production claims, universal ontology coverage, or adoption of every technology named during ideation.

Under `CTRL-PRV-002`, product analytics, demonstrations, sales material, documentation, and support procedures MUST use the same prohibited-use boundary as runtime policy.

10. Accessibility and inclusion principles

The primary conformance target is WCAG 2.2 AA for all H1 user journeys. Product acceptance includes keyboard-only operation, screen-reader semantics, visible focus, 200 percent zoom, reflow at 320 CSS pixels, reduced motion, non-color status cues, accessible names, error association, and text/table alternatives for visualizations. Domain terminology must be defined in context, and confidence language must not imply mathematical certainty where none exists.

Evidence for `AC-UX-001` includes a keyboard-only screen-reader user asking the reference question, inspecting every citation, configuring and comparing the scenario, reviewing the exact Jira payload, approving or declining it, inspecting the receipt, and initiating rollback without losing information available visually.

11. Long-term vision

The long-term product can become a composable operating layer for organizational understanding: a governed ontology, durable evidence and decision memory, domain-specific simulations, and bounded agents that coordinate work across approved tools. That vision is reached through measured capability graduation, not through an assumed path to artificial executives.

The durable strategic assets are:

- A permission-aware, time-versioned organizational evidence model.
- Trusted entity and relationship resolution with reversible provenance.
- Evaluated scenario models with explicit domains of validity.
- Action governance that makes an AI-originated proposal no more privileged than any other proposal.
- Open contracts for connectors, ontology packages, workflows, tools, and portable deployment.

12. Product acceptance criteria

Governing gate	Criterion
`AC-AI-001` and `AC-AI-002`	Every user-visible factual claim in the H1 reference answer has authorized evidence or is labeled as an inference, and unsupported cases abstain.
`AC-SIM-001`	Every H1 scenario result exposes snapshot ID, seed, engine version, assumptions, missing inputs, percentiles, and sensitivity drivers and reproduces canonically.
`AC-ACT-001` and `AC-ACT-002`	Every H1 external mutation is limited to the allowlisted Jira sandbox project, requires two distinct currently authorized human approvers, and produces one external effect.
`AC-OBS-001`	A tenant administrator can determine why a source, claim, entity, edge, answer, recommendation, approval, action, or rollback exists from correlated retained evidence.
Product copy review	Product copy never describes a projection as authoritative, an inference as fact, a forecast as certainty, a research item as committed, or control alignment as certification.
`CTRL-PRV-002` and `AC-AI-003`	No H1 workflow includes an individual employment or health inference, and adversarial connector or prompt content cannot activate one.

CH-02 - H1 Reference Workload and Demonstration Contract

Status: committed | Owners: product-engineering, quality-engineering | Last reviewed: 2026-07-13

H1 Reference Workload and Demonstration Contract

1. Purpose

This chapter is the executable product contract for the H1 hackathon slice. It freezes the demonstration dataset, actor rights, question, scenario, external action, failure cases, and measurements. A visually convincing path that bypasses ingestion, authorization, provenance, simulation, approval, idempotency, audit, or rollback does not satisfy this contract.

Under `REQ-VER-002`, the reference workload MUST run from checked-in, deterministic synthetic fixtures without access to a real employee, customer, repository, or Jira project. Under `REQ-TEN-001` through `REQ-TEN-004`, the same application build MUST

serve both synthetic tenants; tenant isolation MUST NOT be implemented by tenant-specific application code. All dates, identities, text, code metadata, issue records, source permissions, and expected answers MUST be synthetic and marked as such in the interface.

`CH-17` defines an additional H1 synthetic physical-asset path in the same application. It does not change this chapter's frozen organization fixture, answer oracle, schedule simulation, or Jira action. Its telemetry originates only from an in-process deterministic simulator, and its control commands change only simulator state. Consequently, synthetic GitHub and Jira remain the only H1 external-source ingestion modes and the allowlisted Jira update remains the only external mutation.

2. Frozen workload identity

Field	Value
Workload ID	`edt-h1-github-jira-launch-risk`
Fixture version	`1.0.0`
Root seed	`edt-h1-20260713`
Simulation seed	`20260713`
Frozen evaluation clock	`2026-07-13T16:00:00Z`
Tenant aliases	`tnt_aster` and `tnt_beacon`
Primary tenant UUID	`10000000-0000-4000-8000-000000000001` (`tnt_aster`)
Isolation-canary tenant UUID	`10000000-0000-4000-8000-000000000002` (`tnt_beacon`)
Aster Jira installation UUID	`30000000-0000-4000-8000-000000000001` (`con_aster_jira`)
Aster GitHub installation UUID	`30000000-0000-4000-8000-000000000002` (`con_aster_github`)
Beacon Jira installation UUID	`30000000-0000-4000-8000-000000000003` (`con_beacon_jira`)
Beacon GitHub installation UUID	`30000000-0000-4000-8000-000000000004` (`con_beacon_github`)
GitHub mode	Synthetic GitHub App payloads, metadata-only, read-only, allowlisted repositories
Jira mode	Synthetic Jira OAuth payloads, read for allowlisted projects, one allowlisted issue-field update in `tnt_aster`
Simulation	Seeded PERT/Monte Carlo schedule simulation over a dependency DAG

The aliases make fixtures and test reports readable; canonical contracts, rows, events, actions, and API calls use the UUIDs. Actor and connector aliases in this chapter similarly resolve through the signed fixture manifest to fixed UUIDs. The evaluation clock is injected into workflows. Tests MUST NOT depend on wall-clock time. Approval-expiry tests advance a controlled clock.

3. Synthetic tenants

3.1 Aster Labs (`tnt_aster`)

Aster Labs is a synthetic software organization preparing the `Orion 2.0` launch. Its fixture contains:

Source data	Frozen count
Human identities	48
Teams	7
GitHub organizations	1
GitHub repositories	12
GitHub pull requests	420
GitHub reviews	690
GitHub issue and PR links	206
Jira projects	4

Source data	Frozen count
Jira issues	240
Jira issue links	318
Jira comments	560
Release milestones	8

The workload's decision chain is fixed:

1. Jira issue `AST-142`, `Complete SSO cutover`, is scheduled to finish on `2026-08-07`.
2. `AST-142` is implemented by open GitHub pull request `aster-labs/identity-service#184`, `Finalize token migration`, which is awaiting one required security review.
3. `AST-142` blocks `AST-173`, `Build Orion release candidate`.
4. `AST-173` blocks `AST-201`, `Complete launch certification`.
5. `AST-201` gates milestone `Orion 2.0 General Availability`.
6. Two lower-confidence parallel risks exist in the fixture, but neither is on the p80 critical path. The answer must describe them as secondary, not omit their uncertainty or promote them above `AST-142`.

The relevant source objects have intentionally different owners and ACLs so the application must combine only evidence visible to the requesting actor.

3.2 Beacon Works (`tnt_beacon`)

Beacon Works is an unrelated synthetic organization used as an isolation canary. Its fixture contains:

Source data	Frozen count
Human identities	32
Teams	5
GitHub organizations	1
GitHub repositories	8
GitHub pull requests	260
GitHub reviews	410
GitHub issue and PR links	118
Jira projects	3
Jira issues	160
Jira issue links	204
Jira comments	320
Release milestones	5

Beacon deliberately reuses the display name `Jordan Lee`, the repository name `platform`, and the issue summary `Complete SSO cutover`. Its opaque source IDs differ and every canonical key is tenant-qualified. The string `BEACON-CANARY-7Q9K` exists only in a Beacon Jira description and source artifact.

`TC-DEMO-001` (evidence for `AC-TEN-001`): No Aster query, prompt, trace available to an Aster operator, citation, cache key, export, graph response, vector result, search result, metric label, or error message may contain `BEACON-CANARY-7Q9K`.

4. Actors and permissions

Actor alias and UUID	Tenant	Grants	Explicit denials
`usr_aster_analyst` / `20000000-0000-4000-8000-000000000001`	Aster	Read all allowlisted Orion Jira and GitHub metadata; create scenarios; run simulations; draft remediation	Approve or execute external action; administer connectors
`usr_aster_limited` / `20000000-0000-4000-8000-000000000002`	Aster	Read Orion Jira except security-restricted issues; read public Aster repositories	Read `identity-service#184`, its reviews, or restricted citations; approve actions
`usr_aster_ops_approver` / `20000000-0000-4000-8000-000000000003`	Aster	Read full reference evidence; approve operations-class Jira remediation	Satisfy security approver slot; approve twice; alter payload while approving
`usr_aster_security_approver` / `20000000-0000-4000-8000-000000000004`	Aster	Read full reference evidence; approve security-impacting Jira remediation	Satisfy operations approver slot; approve twice; alter payload while approving
`usr_aster_admin` / `20000000-0000-4000-8000-000000000005`	Aster	Configure fixture connectors, policy, identity mapping, and replay tests	Count as an action approver unless separately granted an approver role
`usr_beacon_analyst` / `20000000-0000-4000-8000-000000000006`	Beacon	Read Beacon allowlists; create Beacon scenarios	Any Aster resource or action
`usr_platform_operator` / `20000000-0000-4000-8000-000000000007`	Platform	View redacted service health and tenant-opaque operational metrics	Tenant content, prompts, citations, source payloads, graph data, or action payloads

All test sessions are authenticated by the development identity provider and mapped to immutable actor IDs. A caller-supplied tenant header or resource tenant is ignored for authority derivation and rejected when it conflicts with the session.

`TC-DEMO-002` (evidence for `AC-TEN-001`): Replaying an Aster resource identifier with a Beacon session returns an indistinguishable not-found response, emits a tenant-boundary security event, and reveals neither existence nor tenant name.

5. Ingestion and synchronization script

The demonstration begins with empty tenant data and performs the following scripted sequence:

1. Install synthetic GitHub and Jira connectors with tenant-specific credentials and allowlists.
2. Backfill GitHub and Jira source objects into immutable object storage and authoritative normalized records.
3. Deliver three duplicate webhooks and two out-of-order webhooks from each connector. The final normalized state must equal the ordered, deduplicated oracle state.
4. Interrupt the Jira backfill after a committed cursor, resume it, and prove that no committed item is lost or applied twice.
5. Tombstone one unrelated pull request and one unrelated Jira issue, then rebuild Neo4j and vector projections from PostgreSQL.
6. Resolve identities and issue-to-pull-request links, retaining the evidence and rule version for each merge or edge.
7. Compare normalized records, resolution decisions, canonical relationships, ACLs, and projection checkpoints with the ground-truth oracle.

Connector payload text is untrusted data. It is never concatenated into system instructions and cannot activate a tool, alter tenant context, change approval policy, or override the scenario definition.

`TC-DEMO-003` (evidence for `AC-DATA-002`): After the scripted faults and rebuild, authoritative source-object and observation digests, identity-resolution decisions, golden claims, golden relationships, tombstones, and projection checkpoints match the signed fixture oracle.

`TC-DEMO-004` (evidence for `AC-DATA-001`): A duplicate event changes no business state after its first successful application and records a deduplication outcome correlated to the original event.

6. Reference question and answer contract

The analyst asks:

> What is most likely to delay Orion 2.0, what evidence supports that conclusion, and what information is still missing?

The answer is successful only when it:

- Identifies the `AST-142` -> `AST-173` -> `AST-201` -> `Orion 2.0` dependency chain as the strongest supported launch blocker.
- Identifies `aster-labs/identity-service#184` and its missing required security review as evidence affecting `AST-142`.
- Separates source facts from the inference that the chain is the strongest delay risk.
- Provides claim-level links to the exact authorized Jira and GitHub source observations, including source-updated time and twin-ingested time.
- Reports source freshness and the current projection checkpoint.
- Names the two secondary risks and labels their lower confidence.
- States that individual productivity was not inferred.
- States missing information: future review completion time, unrecorded work, and whether the modeled task-duration distributions represent actual delivery behavior.
- Offers scenario creation as a next step but does not create, approve, or execute an external action from the question alone.

The answer for `usr_aster_limited` must not reveal the restricted repository, pull-request number, review state, hidden node degree, or a citation URL. It may say that accessible evidence is insufficient to explain the relevant Jira blocker and offer an access-request path.

`TC-DEMO-005` (evidence for `AC-AI-001`): In the full-access golden evaluation, every material factual claim has an oracle-authorized citation, no material citation contradicts its claim, and the answer identifies all four nodes in the critical dependency chain.

`TC-DEMO-006` (evidence for `AC-AI-002`): When the pull-request evidence is withheld, deleted, or contradicted, the answer abstains from the unsupported code-review claim rather than reconstructing it from training knowledge or hidden graph topology.

7. Scenario contract

From the answer, the analyst creates this scenario:

> Assume AST-142 completes five working days earlier while all other task distributions, dependencies, calendars, and capacity assumptions remain unchanged.

Before execution, the scenario builder compiles and displays:

- Baseline snapshot ID and projection checkpoint.
- Root seed and simulation seed `20260713`.
- A single typed intervention on the completion distribution for work item `116ab4b3-b108-5f91-ab7e-111f7fba1d45`, whose authorized display key is `AST-142`.
- Unchanged assumptions and explicit non-effects.
- The Aster working-day calendar used for the five-day shift.
- Validation that the graph is acyclic over the selected scheduling subgraph.
- A warning that this is a conditional schedule model over synthetic data, not a prediction of employee performance.

The confirmed intervention serializes as:

```
{
  "type": "shift_completion_distribution",
  "work_item_id": "116ab4b3-b108-5f91-ab7e-111f7fba1d45",
  "delta_workdays": -5
}
```

The engine adds `-5` to the optimistic, most-likely, and pessimistic remaining-duration bounds together. It rejects the scenario if any shifted bound would be negative; it does not change dependencies, calendars, team capacity, or individual attributes.

The reference engine uses 50,000 trials and a versioned deterministic pseudorandom generator. The golden result, within one calendar day at each percentile and a 0.5 percentage-point sampling tolerance, is:

Result	Baseline	Scenario	Difference
p50 launch date	2026-08-20	2026-08-13	5 working days earlier

Result	Baseline	Scenario	Difference
p80 launch date	2026-08-24	2026-08-17	5 working days earlier
p95 launch date	2026-08-27	2026-08-20	5 working days earlier
Dominant critical path	`AST-142` -> `AST-173` -> `AST-201` -> launch	Same path, lower occupancy	Reduced but not eliminated

The result includes percentile dates, the launch-date distribution, critical-path occupancy, top blockers, sensitivity drivers, assumptions, uncertainty, missing-data warnings, baseline and scenario comparison, engine version, seed, trial count, and duration. It never reports a single date without the distribution.

`TC-DEMO-007` (evidence for `AC-SIM-001`): Repeating the run with the same snapshot, scenario, engine version, calendar, and seed produces byte-identical canonical numeric output. Changing any of those inputs produces a new run identity.

`TC-DEMO-008` (evidence for `AC-SIM-002`): The p95 under the scenario remains later than p80 and p50; the interface does not describe the scenario as guaranteeing an earlier launch.

8. Exact Jira remediation contract

The analyst asks the system to draft a Jira remediation that operationalizes the scenario. The sole allowed H1 external mutation is an update to synthetic Jira issue `AST-142` in allowlisted project `AST`.

8.1 Before snapshot

```
{
  "issueKey": "AST-142",
  "version": 7,
  "fields": {
    "duedate": "2026-08-07",
    "priority": { "id": "3", "name": "Medium" },
    "labels": ["identity", "orion"]
  }
}
```

8.2 Canonical approved payload

```
{
  "action": "jira.issue.update",
  "connectorInstallationId": "30000000-0000-4000-8000-000000000001",
  "expectedIssueVersion": 7,
  "issueKey": "AST-142",
  "projectKey": "AST",
  "set": {
    "duedate": "2026-07-31",
    "labels": ["digital-twin-remediation", "identity", "orion"],
    "priorityId": "2"
  },
  "tenantId": "10000000-0000-4000-8000-000000000001"
}
```

The preview shows the field-level before/after diff, target connector and project, evidence and scenario references, required approver roles, expiry, expected Jira version, rollback fields, and canonical payload hash. Human-readable text is explanatory; approval binds only the canonical structured payload.

Two distinct authenticated humans are required: one active operations approver and one active security approver. Both must still have their required role when execution begins. The approval window is 15 minutes from preview creation using server time. Self-approval, duplicate approval by one actor, delegated bot approval, or approval after role revocation fails closed.

Any payload, target, connector, source version, policy version, or tenant change after the first approval invalidates all collected approvals and creates a new preview. Execution uses a stable idempotency key derived from tenant, action type, target, expected version, and canonical payload hash.

8.3 Execution and compensation

Successful execution records the request, canonical payload, approvals, policy decision, connector request identifier, before snapshot, after snapshot, source response, timestamps, trace, and immutable action receipt. A repeated execution request returns the original receipt and MUST NOT perform a second Jira write.

Rollback restores `duedate`, `priority`, and `labels` from the before snapshot only if the issue still has the recorded after version and values. If another actor changed the issue, rollback enters `compensation_conflict`, performs no blind overwrite, and

requires a new reviewed action.

`TC-DEMO-009` (evidence for `AC-ACT-001`): Zero or one approval, two approvals from the same actor, an expired approval, a changed payload, a changed issue version, or a revoked role results in zero Jira writes.

`TC-DEMO-010` (evidence for `AC-ACT-002`): With two valid approvals, concurrent execution requests produce exactly one Jira write and the same durable action receipt.

`TC-DEMO-011` (evidence for `AC-ACT-003`): In the no-conflict path, rollback restores the complete before snapshot, produces a compensation receipt, and remains idempotent when retried.

9. Required interface journey

The demonstration uses the production navigation and never invokes a hidden operator endpoint to advance state:

1. Sign in as `usr_aster_analyst`; select Aster Labs and confirm the synthetic-data banner.
2. Open connector health; run the scripted sync and inspect freshness, injected faults, recovery, and projection rebuild.
3. Open Ask; submit the reference question; expand citations and evidence status.
4. Switch to `usr_aster_limited`; repeat the question and show permission-preserving abstention.
5. Return as the analyst; convert the answer to the fixed scenario; inspect and confirm compiled assumptions.
6. Run the simulation; compare baseline and scenario distributions and inspect critical path and sensitivity.
7. Draft the exact Jira remediation; inspect the before/after payload and policy requirements.
8. Approve once as the operations approver and once as the security approver; return to the analyst session.
9. Execute two concurrent requests; inspect one action receipt and one source-system change.
10. Open Audit; trace question, evidence, scenario, approvals, execution, and receipt.
11. Roll back; verify restored Jira state and the compensation receipt.
12. Switch to Beacon Works; show its independent data, then run the isolation-canary tests.

9.1 Five-minute judged narrative

`AC-PROD-001` is measured with a healthy local stack and pre-authenticated synthetic actor sessions, but with empty tenant business data at the start. Every displayed result is produced through the real ingestion, retrieval, simulation, policy, connector, and audit paths; no final graph, answer, simulation, approval, receipt, or rollback is precomputed.

Elapsed time	Demonstrated outcome
`00:00-00:40`	Select the frozen two-tenant fixture, run the deterministic seed/sync fast path, and show connector and projection checkpoints.
`00:40-01:35`	Ask the reference question, inspect claim-level Jira/GitHub citations, and show the permission-limited answer state.
`01:35-02:30`	Confirm the five-working-day intervention, run 50,000 trials, and compare p50/p80/p95 plus critical path and sensitivity.
`02:30-03:30`	Generate the exact `AST-142` diff and collect one operations and one security approval in distinct sessions.
`03:30-04:20`	Submit concurrent execution requests, show one Jira effect and one durable receipt, and open the correlated audit sequence.
`04:20-05:00`	Execute compensation, verify the restored source snapshot, and show the Beacon isolation canary remains absent from Aster outputs.

The untimed assurance journey in section 9 exercises the same path with expanded evidence, injected failures, and administrative views. If the judged path exceeds five minutes, skips a real control, or uses precomputed final state, `AC-PROD-001` fails even when individual latency targets pass.

10. Performance and quality envelope

Tests run after warm application startup with two loaded tenants, up to 100 identities, up to 100,000 graph nodes and 1,000,000 graph edges per tenant, and ten concurrent interactive users. Cache-hit-only measurements do not establish a datastore service-level objective.

ID	Metric	H1 pass condition
SLO-DEMO-001	Connector freshness	99 percent of accepted synthetic webhooks reflected in authoritative normalized state within 15 minutes; scripted demo target within 60 seconds
SLO-DEMO-002	Non-AI API latency	p95 under 2 seconds and p99 under 5 seconds for scoped graph, evidence, scenario, approval, and audit reads
SLO-DEMO-003	Simulation latency	p95 under 10 seconds for the fixed 50,000-trial workload
SLO-DEMO-004	Cited answer latency	p95 under 20 seconds, excluding a clearly reported provider outage
SLO-DEMO-005	UI responsiveness	Local interaction acknowledgment under 100 ms and a progress state for work over 500 ms
SLO-DEMO-006	Projection recovery	Full H1 projection rebuild completes within 15 minutes per tenant and reaches a verifiable checkpoint
SLO-DEMO-007	Availability during demo	No single recoverable connector, graph, vector, cache, or model failure corrupts authoritative data or permits an unsafe action

Quality gates:

- 100 percent of golden source objects normalize to the expected canonical form.
- 100 percent of golden entity merges and non-merges match the oracle; all merges are reversible.
- 100 percent of golden dependency edges and ACL labels match the oracle after rebuild.
- 100 percent of reference-answer material facts meet citation correctness.
- 100 percent of unsupported-claim cases abstain.
- 100 percent reproducibility for identical simulation inputs.
- 0 cross-tenant or unauthorized disclosures across API, UI, retrieval, cache, trace, log, and export tests.
- 0 invalid, expired, replayed, mutated, or insufficiently approved Jira writes.
- 100 percent of successful writes have a durable receipt and tested compensation behavior.
- The complete critical journey meets WCAG 2.2 AA automated checks and passes manual keyboard and screen-reader review.

11. Failure demonstrations

At least one automated run and one recorded operator run cover:

Fault	Expected behavior
PostgreSQL unavailable	New authoritative operations stop; reads do not silently use stale projections as truth; no external action executes.
Neo4j unavailable	Evidence and authoritative records remain intact; graph-dependent UI reports degradation; no fabricated path is returned.
Vector retrieval unavailable	The answer may use authorized structured retrieval or abstain; it reports degraded retrieval.
Cache stale or unavailable	Authorization is re-evaluated against authoritative policy; cache bypass affects latency, not correctness.

Fault	Expected behavior
Duplicate or out-of-order connector event	Final state matches the versioned source oracle and the condition is observable.
Partial synchronization	Results expose source/checkpoint staleness and do not claim completeness.
Permission revoked mid-run	Subsequent retrieval and tool steps fail closed; hidden evidence is not retained in visible output.
Prompt injection in Jira text	The text is quoted as untrusted evidence; it cannot activate a tool or alter policy.
Model timeout or invalid structure	Bounded retry or approved fallback occurs; otherwise the run ends safely with no action.
Approval expires during execution	Policy is rechecked immediately before the connector call; no write occurs after expiry.
Concurrent Jira edit	Version precondition fails; no overwrite occurs; preview and approvals become invalid.
Process restart after Jira response	Receipt reconciliation uses the idempotency key and source request ID; a retry does not duplicate the write.

12. Demonstration completion gate

`TC-DEMO-012` (evidence for `AC-PROD-001`): H1 is complete only when a clean environment can load the frozen fixtures, execute the entire journey, pass all workload quality and security gates, generate the same canonical oracle report, and preserve a queryable audit chain from source observation through rollback.

Recorded screenshots or a seeded database alone are insufficient evidence. The release evidence bundle must include fixture and oracle digests, test and evaluation results, latency samples, accessibility results, traces with tenant content redacted, action and compensation receipts, and the exact application and contract versions.

CH-03 - System Architecture

Status: Committed | Owners: Architecture, Platform Engineering | Last reviewed: 2026-07-13

System Architecture

1. Purpose and normative interpretation

This chapter defines the deployable architecture for the Enterprise Digital Twin. "Must" and "must not" are release requirements. "Should" records the default, and an exception requires an ADR. PostgreSQL is the system of record. Neo4j, pgvector indexes, caches, and future search or analytics stores are derived projections and can be rebuilt.

The product has two tracks:

- H1 and H2 are committed, production-shaped delivery horizons with measurable limits.
- H3 and H4 are provisional until pilot evidence freezes their targets.
- H5 is research and cannot be represented as available or production-ready.

The exhaustive source prompt asks for microservices, a service mesh, Kafka, NATS, event sourcing, active-active multi-cloud, and hyperscale operation. Those are not simultaneous H1 requirements. The committed design is a modular monorepo with four independently deployable application workloads, durable Temporal workflows, and a PostgreSQL transactional outbox. Additional infrastructure is introduced only at the triggers in Technology Stack. This resolves the conflict in favor of the smallest architecture that preserves enterprise invariants and has an explicit scale path.

2. Context and trust boundaries

The system serves authenticated users and administrators, receives untrusted events from GitHub and Jira, calls those providers, invokes approved model endpoints, and persists data in isolated tenant namespaces.

Boundary	Trusted side	Untrusted side	Required enforcement
Browser to web/API	Server-rendered application and API	Browser state, URLs, cookies, uploads	OIDC session validation, CSRF protection, input schemas, output encoding, CSP
Provider to webhook endpoint	API after signature and replay validation	All webhook bytes and headers	Provider signature, timestamp window, delivery-id inbox, size limit
Connector worker to provider	Worker credential broker	Provider response and availability	Per-tenant credentials, egress allowlist, schema validation, retry budget
Application to data stores	Workload identity and tenant transaction context	User-selected tenant identifiers	Server-derived tenant context, RLS, tenant-qualified keys, least-privilege roles
Retrieval to model	Authorized evidence envelope	Connector text and model output	ACL filtering before prompt construction, content/instruction separation, structured output validation
Action executor to Jira	Approved exact payload	Model proposals and mutable provider state	Allowlisted project, two-person approval, payload digest, idempotency, before/after evidence
Tenant to tenant	Tenant-specific principals and keys	Every other tenant	No shared retrieval, merge, memory, analytics, cache entry, or training corpus

3. Deployment units and ownership

There are exactly four first-party OCI application workloads in H1 and H2. PostgreSQL, Temporal, Neo4j, S3-compatible object storage, Valkey/Redis, the identity provider, and the OpenTelemetry collector are infrastructure dependencies, not additional product microservices.

Workload	Runtime	Owns	May read	Must not do
Web application	Next.js, React, TypeScript	Browser UI, server-side rendering, session UX, SSE client, accessibility and visualization shell	Public configuration and API responses	Connect to databases, providers, Temporal, or model APIs; derive authorization from client state
API	NestJS on Fastify, TypeScript	REST commands and queries, signed webhook ingress, OIDC session validation, policy enforcement, connector administration, scenario commands, approvals, audit query API, SSE run status	Control schemas, authorized claims/evidence, projection status	Run long jobs inline; accept a client tenant as authoritative; write another workload's tables
Synchronization worker	TypeScript, Temporal worker	GitHub/Jira adapters, polling and reconciliation, inbox/outbox dispatch, normalization, cursors, tombstones, graph projection, approved Jira execution and compensation	Connector configuration, durable workflow commands, source payload references	Invoke models; merge identities across tenants; mutate Jira without a valid action grant
Intelligence worker	Python, Temporal worker	Extraction, entity resolution decisions, embeddings, bounded graph analysis, cited answer generation, simulation, AI evaluations	Authorized evidence bundles, graph projections, scenario snapshots	Fetch arbitrary URLs; expand caller authority; execute external mutations; expose private reasoning traces

Infrastructure processes can scale independently, but no new first-party deployable is created without an ADR that demonstrates an ownership, scaling, security, or availability boundary that the four workloads cannot satisfy.

3.1 Database write ownership

All schemas contain tenant_id where data is tenant-scoped. A workload uses a distinct non-superuser database role. FORCE ROW LEVEL SECURITY applies to tenant tables. Cross-owner writes are prohibited even if a role could technically be granted access.

Schema group	Authoritative writer	Representative records
control	API	tenants, memberships, delegations, connector installations, scenarios, approvals, action grants, audit index
ingest	Synchronization worker	webhook inbox, source objects, normalized observations, sync cursors, tombstones, projection checkpoints
knowledge	Intelligence worker	claims, evidence links, resolution decisions, entity versions, embedding metadata
simulation	Intelligence worker	immutable simulation snapshots, runs, forecasts, assumptions, uncertainty
execution	Synchronization worker	action attempts, provider receipts, compensation results
workflow	Temporal server through its own database	workflow history and task queues; never business source-of-truth records

Changes between owners use typed commands or events. A transaction that changes authoritative business state also inserts an outbox row. Consumers maintain an inbox keyed by tenant_id, event_id, and consumer_name. Delivery is at least once; consumer effects must be idempotent. "Exactly once" is not claimed.

3.2 Infrastructure data authority

Store	Authority and isolation	Recovery rule
PostgreSQL plus pgvector	Authoritative business state and semantic vectors under RLS	Point-in-time restore is the primary data recovery path
S3-compatible storage	Immutable provider payloads, exported reports, evidence artifacts, model artifacts	Versioning and object lock where required; metadata in PostgreSQL points to content hashes
Neo4j	Per-tenant graph projection with provenance references	Rebuild from PostgreSQL claims and relationships; never repair PostgreSQL from Neo4j
Valkey/Redis	Cache, distributed rate-limit counters, short-lived coordination	Flush and rebuild; no approval, policy, cursor, or audit authority
Temporal	Durable orchestration history	Business state remains reconcilable from PostgreSQL; workflows use stable IDs

4. End-to-end data flow

4.1 Installation and synchronization

1. An administrator authenticates through OIDC and selects a tenant from server-resolved memberships.
2. The API validates connector-administration permission and initiates GitHub App installation or Jira OAuth with PKCE and signed state bound to tenant, actor, nonce, and expiry.
3. Provider credentials are stored through the secret broker under a tenant-specific encryption key. PostgreSQL stores only a secret reference, granted scopes, installation identity, and rotation metadata.
4. A webhook reaches the API. The API reads a bounded byte stream, verifies provider signature before parsing, enforces timestamp and delivery-id replay windows, stores the delivery in the PostgreSQL inbox, emits an outbox event in the same transaction, and acknowledges only after durable commit.
5. A dispatcher starts or signals a stable Temporal synchronization workflow. Polling schedules use the same workflow, so webhooks and reconciliation cannot race independent cursor writers.

6 The synchronization worker fetches pages with bounded concurrency, validates response schemas, computes a content hash, writes encrypted raw bytes to tenant-isolated object storage, and records SourceObject plus NormalizedObservation in PostgreSQL. Cursor advancement and observation publication occur in one transaction.

7 The intelligence worker extracts claims and evidence. Connector text is treated as data, not instruction. Entity resolution can propose or apply a tenant-local merge according to confidence policy; every merge is versioned and reversible.

8 A PostgreSQL outbox event causes the synchronization worker's projector to upsert the tenant graph. The projector records the highest committed source sequence in ProjectionCheckpoint. Duplicate and out-of-order events are safe.

9 Embeddings are generated only from authorized, policy-approved text and stored in pgvector rows protected by the same tenant RLS. ACL metadata remains attached to each retrievable chunk.

10 Tombstones and permission revocations have higher queue priority than enrichment. A revocation watermark blocks affected retrieval until all required projections meet or exceed that watermark.

Full reconciliation is mandatory at least every 15 minutes in H1. Webhooks reduce latency but never replace polling, cursor repair, or tombstone discovery.

4.2 Organizational question and cited answer

1 The API authenticates the actor, derives tenant and delegation, and creates an AgentRun with a budget, deadline, policy version, and immutable request digest.

2 Query planning produces bounded relational, vector, and graph retrieval operations. User input cannot inject Cypher, SQL, URLs, tool names, or tenant identifiers.

3 Each store query applies tenant isolation and SourceACL filtering. Graph traversals have hop, node, edge, time, and result-size limits.

4 The API or intelligence workflow constructs an evidence envelope containing stable evidence IDs, source timestamps, authorization context, and projection watermarks. Unauthorized content is removed before any model request.

5 The model gateway selects an evaluated pinned model for the query capability. Structured output is schema-validated. If no approved model is available, the run fails closed.

6 A verifier checks that each material factual claim references evidence visible to the actor. Unsupported claims are removed or the response abstains.

7 The API streams status and the final answer over SSE. Reconnection uses Last-Event-ID and reads persisted run events; SSE is not the authority.

The response must state evidence freshness, incomplete-source warnings, and projection lag. It must never present stale or partial data as complete.

4.3 Scenario and simulation

1 The API validates a scenario draft and persists it as an immutable version. The user confirms assumptions before execution.

2 The intelligence worker creates a SimulationSnapshot containing the graph projection sequence, source version IDs, model version, parameter set, timezone, and pseudorandom seed.

3 The worker validates that the work dependency graph is acyclic. It runs seeded PERT sampling and Monte Carlo scheduling within CPU, sample-count, memory, and wall-clock budgets.

4 Results include p50, p80, and p95 completion dates, critical path, blockers, sensitivity drivers, uncertainty, assumptions, missing-data warnings, and a baseline-versus-scenario comparison.

5 The snapshot and result are immutable. Re-running the same engine version, snapshot, parameters, and seed must produce the same serialized result within documented floating-point tolerance.

Simulation describes schedule uncertainty. It must not infer individual productivity, health, emotion, burnout, attrition, misconduct, compensation suitability, or employment performance.

4.4 Exact-payload Jira action

The frozen H1 fixture has one mutation target. Fixture alias tnt_aster resolves to tenant UUID 10000000-0000-4000-8000-000000000001, and connector alias con_aster_jira resolves to installation UUID 30000000-0000-4000-8000-000000000001. Aliases appear only in fixtures and reports; the canonical payload uses the UUIDs. The target is Jira issue AST-142 in project AST. Its required before state is source version 7, due date 2026-08-07, priority Medium, and labels identity and orion. Its only approved after state is due date 2026-07-31, priorityId 2, and sorted labels digital-twin-remediation, identity, and orion.

1. A mitigation agent may draft, but not execute, that exact Jira update. Any other target, field, or value is outside H1.
2. The API fetches current provider state and creates a preview with canonical payload, before snapshot, expected provider version, target, risk class, expiry, and SHA-256 payload digest.
3. Two distinct authenticated actors approve the same digest: one with the operations approval capability and one with the security approval capability. The proposer cannot approve. Each approval is re-authorized at submission time.
4. On the second approval, the API atomically creates a single-use ActionGrant and outbox event. The grant expires no later than 15 minutes after preview creation.
5. The synchronization worker revalidates tenant, project allowlist, scopes, grant expiry, two distinct approvers, digest, idempotency key, and expected provider version. Any mismatch fails without mutation.
6. The worker sends the Jira request with an idempotency record, captures the provider response and after snapshot, and commits an ActionReceipt plus audit event. Retried workflow activities return the existing receipt.
7. Rollback is a new dual-authorized compensation command requested within 24 hours. It restores the exact before snapshot only when current Jira state still equals the recorded after state. Any intervening edit yields compensation_conflict and manual_intervention_required instead of overwriting it. H1 does not use pre-authorized automatic compensation.

5. Consistency, concurrency, and time

Concern	Contract
Transactional state	PostgreSQL READ COMMITTED is default. SERIALIZABLE or explicit row locks are required for approvals, cursor advancement, merge decisions, and idempotency claims.
Projection reads	Each response includes source sequence and projection sequence. A command may request min_projection_sequence; timeout produces 409 projection_not_ready, not stale success.
Concurrent edits	Optimistic version columns and If-Match are required for mutable API resources. Conflicts return RFC 9457 problem details with no silent last-write-wins.
Ordering	Ordering is per tenant and aggregate sequence, not global wall clock. Consumers ignore older aggregate versions but still record receipt.
Clock	Persist UTC instants and the original business timezone where relevant. Deadlines use server time; NTP drift alerts at 500 ms and readiness fails at 2 s.
Deletion	Deletion creates a durable tombstone and revocation watermark before asynchronous projection and artifact deletion. Retrieval fails closed while deletion is incomplete.
Replay	Rebuilds run into a shadow projection, verify counts, hashes, ACL coverage, and watermark, then atomically switch the tenant projection alias.

6. Interface boundaries

- REST with OpenAPI 3.1 is the H1/H2 command, query, and administration interface.
- SSE is the one-way run-progress interface. It has no command semantics.
- Provider webhooks are signed ingress endpoints with provider-specific validation and a common inbox.
- AsyncAPI and CloudEvents-compatible JSON envelopes describe internal events. The envelope includes specversion, id, source, type, subject, time, datacontenttype, dataschema, tenant_id, traceparent, aggregate_id, aggregate_version, and payload.

- GraphQL is provisional and read-only. It cannot bypass REST policy, expose arbitrary Cypher, or be introduced until field-level cost and authorization tests pass.
- Protocol Buffers and gRPC are provisional boundaries for a workload extracted after a measured trigger. They do not duplicate H1 REST contracts.
- MCP resources and tools are capability-scoped facades over the same policy and approval services. They cannot create a second authorization path.

Every external operation defines tenant derivation, authorization action, schema version, idempotency behavior, pagination, rate limit, timeout, retryability, redaction, audit event, and deprecation policy.

7. Architectural invariants

1. No request, event payload, model output, cache key, or provider object can establish tenant authority.
2. No data crosses tenants for retrieval, entity resolution, memory, analytics, evaluation, or training without a separately approved opt-in design; H1 and H2 implement no such opt-in.
3. PostgreSQL is authoritative; a derived store cannot independently create business truth.
4. Source evidence remains attributable to provider object, version, content hash, ingestion time, and applicable ACL.
5. Projection consumers are idempotent, replayable, and observable.
6. Connector and model content is untrusted and cannot select tools or grant permissions.
7. Agent handoff can narrow but never expand actor delegation, budgets, data scope, or action scope.
8. External mutation requires a policy check at proposal, approval, grant, and execution time.
9. H1 permits only the documented Jira sandbox remediation mutation.
10. Every run has a deadline, cancellation path, resource budget, trace, immutable input digest, and terminal state.
11. Sensitive audit events are append-only and exported to independently retained object storage.
12. A degraded dependency produces an explicit degraded or failed result; the system cannot fabricate completeness.

8. Failure modes

Failure	Required behavior
PostgreSQL unavailable	Reject stateful requests, pause consumers, and return 503 with Retry-After. Never write only to a projection.
Neo4j unavailable or behind watermark	Graph-dependent answers and simulations fail or wait. A relational fallback is allowed only when it implements identical ACL and bounded-query semantics and is labeled as partial.
Object store unavailable	Keep already committed inbox deliveries pending and stop cursor advancement before a source object claims durable capture. Retry within budget.
Temporal unavailable	Persist accepted command plus outbox event, return 202 with run ID, and dispatch later. Reject commands whose semantic expiry cannot survive the delay.
Valkey/Redis unavailable	Bypass caches. Sensitive write rate limits fall back to a durable database limiter or fail closed.
Model endpoint unavailable or invalid	Retry only safe calls within the run deadline, use an evaluated allowed fallback, or fail closed. Never return an unverified draft.
Connector compromised	Disable installation, revoke secret, quarantine new observations, retain evidence, start reconciliation, and prevent its content from driving tools.
Partial or duplicate sync	Preserve prior authoritative versions, mark source freshness degraded, retry idempotently, and never advance cursor past an uncommitted page.
Worker crash after external mutation	Reconcile using action idempotency record and provider state before retry; never blindly repeat the mutation.
Permission revocation	Advance revocation watermark, invalidate caches, block affected retrieval, and rebuild projections before access resumes.

9. Acceptance criteria

Evidence ID	Canonical acceptance	Criterion
TC-ARCH-001	AC-REV-001	A deployment inventory proves exactly four first-party application workloads and the ownership boundaries in this chapter.
TC-ARCH-002	AC-TEN-001	Automated tests prove all tenant tables use FORCE RLS and cross-tenant foreign keys or reads fail.
TC-ARCH-003	AC-DATA-001	Duplicate, delayed, and out-of-order webhook and outbox events converge to one authoritative version and one projection effect.
TC-ARCH-004	AC-DATA-002	A complete tenant graph can be rebuilt from PostgreSQL and object evidence without using the prior graph.
TC-ARCH-005	AC-AI-001 and AC-AI-002	Every cited answer exposes evidence IDs and freshness, and the verifier abstains when evidence is absent or unauthorized.
TC-ARCH-006	AC-SIM-001	The same simulation snapshot, engine version, parameters, and seed reproduces the accepted result.
TC-ARCH-007	AC-ACT-001	Jira execution is impossible with one approver, duplicate approvers, an expired grant, a changed payload, a non-allowlisted project, or stale provider state.
TC-ARCH-008	AC-ACT-002 and AC-ACT-003	A worker crash after Jira acceptance returns the existing receipt or enters manual intervention without a duplicate mutation.
TC-ARCH-009	AC-SEC-002	Revoked source access cannot be retrieved from PostgreSQL, graph, vector, cache, model context, or persisted answer artifacts.
TC-ARCH-010	AC-REL-003	Dependency fault tests produce the explicit failure semantics above and no false success.

10. Related decisions

Foundational choices are recorded by the ADR catalog. This chapter is the normative integrated view: modular service decomposition, PostgreSQL authority, rebuildable projections, transactional outbox, pooled relational tenancy, server-derived authorization, bounded AI capabilities, seeded PERT/Monte Carlo simulation, cloud-neutral packaging, and benchmark-gated extraction. Any later ADR that changes an invariant here must update this chapter, its acceptance criteria, and the traceability ledger in the same change.

CH-04 - Technology Stack

Status: Committed | Owners: Architecture, Developer Platform | Last reviewed: 2026-07-13

Technology Stack

1. Decision policy

This chapter is the authoritative technology portfolio. It prevents an exhaustive wishlist from becoming an obligation to operate overlapping systems.

Status	Meaning
Adopt	Required for the committed H1/H2 architecture unless an ADR supersedes it
Conditional	Not an H1 dependency; introduction requires the stated measurable trigger and an approved ADR
Research	May be evaluated in an isolated environment and cannot carry production data or appear in committed interfaces

Status	Meaning
Reject	Must not be introduced because it duplicates authority, lacks a justified use, or conflicts with current constraints

A conditional technology can move to Adopt only after one representative benchmark demonstrates the trigger, a named team accepts operational ownership, threat and privacy reviews pass, data migration and rollback are tested, cost is budgeted, and compatibility contracts are documented. A benchmark must compare the current stack and the candidate on the same tenant-shaped dataset. Preference, resume value, or theoretical scale is not a trigger.

Exact package and image versions are pinned by lockfiles and image digests in the implementation repository. Production never follows a floating language, package, container, or model alias. Supported version changes use automated compatibility tests, security review, staged rollout, and rollback.

2. Committed stack by layer

Layer	Adopted technology	Ownership and rationale	Constraints
Monorepo	pnpm workspaces and Turborepo	One dependency graph for web, API, worker, contracts, and tooling; cached tasks	No package may import another workload's private module
Web	TypeScript, Next.js, React	Server-rendered accessible web UI, route composition, SSE client	No database, connector, Temporal, or model credentials
API	TypeScript, NestJS, Fastify	Typed modular API with schema validation and high-performance HTTP adapter	REST/OpenAPI is primary; long work is delegated
Sync worker	TypeScript, Temporal TypeScript SDK	Shares connector contracts and performs durable I/O workflows	Activities are idempotent and bounded
Intelligence worker	Python, Temporal Python SDK, Pydantic, NumPy	Model integration, extraction, graph analysis, evaluation, PERT/Monte Carlo simulation	No external mutation and no arbitrary code execution
Relational and vector	PostgreSQL with pgvector	Authoritative ACID state, RLS, SQL, transactional outbox, initial vector retrieval	SQL-first migrations; vector rows carry tenant and ACL
Graph	Neo4j	Bounded traversal and graph algorithms over a rebuildable per-tenant projection	Never authoritative; no user-supplied Cypher
Object	S3-compatible API; MinIO for local development	Immutable raw evidence and generated artifacts	Tenant namespace and encryption key; content hashes
Cache and limits	Valkey using the Redis protocol	Non-authoritative cache and distributed counters	Flush-safe; never stores sole approval or policy state
Workflow	Temporal	Durable retries, timers, cancellation, and long-running synchronization/action workflows	Workflow code is deterministic; business truth remains in PostgreSQL
API contracts	OpenAPI 3.1, JSON Schema, RFC 9457, SSE	Language-neutral command/query interface and structured errors	Generated artifacts are checked for drift
Events	Transactional outbox/inbox, AsyncAPI, CloudEvents-compatible JSON	At-least-once publication without a second broker in H1/H2	Consumers are idempotent; no exactly-once claim
Telemetry	OpenTelemetry and collector	Vendor-neutral traces, metrics, and correlated logs	No tenant PII in high-cardinality attributes
Packaging	OCI images and Docker	Reproducible workload packaging and local Compose	Images run non-root, read-only, and by digest
Orchestration	Docker Compose for local/H1; Kubernetes and Helm adopted at H2	Standard scheduling, policy, rollout, and availability for the committed design-partner topology	Production stateful services should be managed outside the cluster
Infrastructure as code	OpenTofu with provider-neutral modules	Declarative cloud and Kubernetes resources without dual IaC sources	Remote encrypted state, locking, plan review
Identity	External OIDC; SAML and SCIM at H2 boundary	Enterprise IdP remains authentication authority	Local development can use a disposable OIDC provider
AI	OpenAI Responses API and Python Agents SDK behind a model gateway	Capability-oriented model calls, structured tools, traceable runs	Pinned evaluated snapshots; no provider call bypasses gateway

2.1 Language and data-access rules

- TypeScript strict mode is mandatory. Runtime inputs use generated JSON Schema validators; static types alone are not validation.
- Python uses current project-pinned Python, type checking, Pydantic models, deterministic dependency locking, and isolated virtual environments.

- SQL migrations are ordered, immutable after release, and written as reviewed PostgreSQL SQL. A Node migration runner acquires a database advisory lock and records checksum, owner, applied time, and release.
- TypeScript uses Kysely for typed query construction plus reviewed SQL for RLS, recursive queries, pgvector, and database-specific behavior. Python uses psycopg. Neither is an authorization boundary.
- Shared schemas are generated from canonical JSON Schema/OpenAPI definitions. Copy-pasted domain types across languages are prohibited.
- A new runtime language requires a durable ownership boundary and staffing plan. A small performance hotspot does not justify a new service language until profiling and a benchmark prove it.

2.2 Model routing baseline

The gateway exposes capabilities, not raw model names, to application code:

Capability	H1 family default	Required gate
difficult orchestration and evaluation	gpt-5.6-sol	Tool and argument accuracy, groundedness, safety, latency, and cost evaluation
balanced grounded analysis	gpt-5.6-terra	Citation support, abstention, ACL and injection evaluation
high-volume extraction	gpt-5.6-luna	Structured extraction precision/recall and cost evaluation

These names are configuration defaults from the approved architecture, not permission to use a floating provider alias. Before deployment, the platform owner must confirm account availability and record the exact snapshot in the model registry. If no available snapshot passes the capability evaluation, that capability is disabled. Fallbacks are independently evaluated and cannot silently reduce safety. The Responses API is the interface for new agentic integrations, as documented by OpenAI.

3. Technology portfolio matrix

3.1 Languages and client platforms

Technology	Status	Approved use or introduction trigger	Decision
TypeScript	Adopt	Web, API, sync worker, connector SDK, schema tooling	Maximizes shared contract safety in I/O-heavy product surfaces
Python	Adopt	Intelligence worker, simulation, model integration, evaluations	Best fit for scientific and AI libraries; isolated from external mutation
SQL	Adopt	PostgreSQL schema, policy, queries, migrations, reporting	Keeps data constraints and RLS explicit
Go	Conditional H3	High-throughput gateway, CLI or connector appliance only after sustained concurrency/distribution evidence shows TypeScript cannot meet the target and ownership justifies a new runtime	No H1/H2 service
Rust	Conditional H4	Sandboxed extension, edge appliance or hardened parser after profiling or isolation evidence proves TypeScript/Python inadequate	Prefer current runtimes first
WebAssembly	Conditional H4	Sandboxed customer plugin execution after capability, resource-metering, determinism, and escape testing pass	Not a general server runtime
C#	Conditional H3 generated SDK; reject services	Generate a client only after a contracted .NET customer and conformance suite exist	No duplicated backend
Java	Conditional H3 generated SDK; reject services	Generate a client only after a contracted JVM customer and conformance suite exist	Temporal and build tools do not make Java an application language
Kotlin	Conditional H4 generated SDK/client; reject services	Mobile/JVM client only after a committed product surface exists	No native client in H1/H2
Swift	Conditional H4 generated SDK/client; reject services	Native Apple client only after a committed offline/mobile surface exists	Responsive web is the current client
C++	Reject	No current workload justifies memory-unsafe native code	Use Rust if a proven native kernel is later required

3.2 Interfaces, workflow, and messaging

Technology	Status	Approved use or introduction trigger	Decision
REST and OpenAPI 3.1	Adopt	Primary public commands, queries, administration, and SDK generation	Stable, inspectable security boundary
SSE	Adopt	Server-to-browser progress for persisted runs	Simpler than bidirectional sockets for current needs
Signed webhooks	Adopt	GitHub and Jira inbound events	Durable inbox plus reconciliation
AsyncAPI and CloudEvents JSON	Adopt	Internal event catalog and envelope	Broker-neutral and contract-testable
Temporal	Adopt	Sync, enrichment, simulation, evaluation, approval expiry, action and compensation workflows	Durable workflow semantics without bespoke schedulers
GraphQL	Conditional H2	Read-only graph exploration after H2 field authorization, query cost, depth, introspection, batching, and pagination tests pass	Never a command or arbitrary Cypher surface
gRPC and Protocol Buffers	Conditional H3	Internal interface only when a workload is extracted and measured call volume or streaming makes REST inadequate	Avoid duplicate contracts in the monolith
Kafka	Conditional H3	Adopt only when sustained outbox traffic exceeds PostgreSQL dispatch capacity in a representative test, replay retention exceeds database budget, or at least three independent consumer groups require ordered high-throughput streams	PostgreSQL outbox remains H1/H2
NATS and JetStream	Reject H3	Overlaps Temporal and a future Kafka event backbone	One durable event strategy
RabbitMQ	Reject	Adds overlapping queue semantics without a committed use	Temporal owns task delivery
Apache Pulsar	Reject	Duplicates the conditional Kafka role and raises operational breadth	Revisit only by superseding portfolio ADR
Event sourcing	Reject as system-wide architecture	Domain audit history and immutable source versions are adopted, but current state remains authoritative tables	Avoid reconstructing all business state from events
Service mesh	Reject	Current workload count does not justify another security and operations control plane	A future H3 reclassification requires a superseding portfolio ADR and measured gap
Dapr	Reject	Duplicates Temporal, SDK, secrets, and pub/sub abstractions	Explicit libraries and contracts are easier to audit

3.3 Databases, search, analytics, and cache

Technology	Status	Approved use or introduction trigger	Decision
PostgreSQL	Adopt	Authoritative tenant, identity, source, claim, scenario, approval, action, and audit-index state	ACID, RLS, mature operations
pgvector	Adopt	Initial semantic retrieval within tenant and ACL filters	Preserves transactional and tenant controls
Neo4j	Adopt	Rebuildable tenant graph projection and bounded traversals	Specialized graph query model without authority split
Valkey	Adopt	Cache and distributed rate-limit counters through Redis-compatible protocol	Open implementation; non-authoritative
Redis	Reject	Valkey is the sole adopted Redis-protocol cache implementation	Do not operate duplicate cache technologies
S3-compatible object storage	Adopt	Immutable raw payloads, artifacts, exports, evaluation datasets	Provider-neutral object contract
MinIO	Adopt for local; conditional production	Compose development and air-gapped profile after durability tests	Managed object storage preferred in H2 cloud
OpenSearch	Conditional H2	Introduce after PostgreSQL full-text plus pgvector misses p95 target or corpus/aggregation load harms OLTP in representative H2 tests	Tenant-isolated indexes and dual-read validation required
Elasticsearch	Reject	Duplicates OpenSearch candidate and introduces a second search contract	One search transition path
ClickHouse	Conditional H2	Introduce when append-heavy telemetry/product analytics queries measurably affect PostgreSQL or exceed agreed retention/query budgets	No source-of-truth business state
Lakehouse object tables	Conditional	H3 analytical history only after governed cross-domain analysis requires it and privacy/residency controls pass	Not part of transactional query path
Milvus	Reject	Duplicates pgvector; no committed billion-vector requirement	Benchmark only under a superseding ADR
Qdrant	Conditional H3	Specialized vector retrieval only when representative benchmarks prove pgvector and conditional OpenSearch cannot meet recall, latency, or isolation targets	Requires a new tenant policy surface and migration/rollback proof
Pinecone	Reject	Duplicates pgvector and adds external data residency dependency	No H1/H2 use
Weaviate	Reject	Duplicates graph, vector, and schema responsibilities	Keep authorities separate and explicit
MongoDB	Reject	Document flexibility is already handled by JSONB plus immutable objects	Avoid another system of record
Cassandra/DynamoDB	Reject	Current access patterns and horizons do not justify eventual-consistent authority	Revisit only for a measured H3 workload
MySQL	Reject	PostgreSQL is selected; dual relational support adds no product value	PostgreSQL-compatible SQL means portability discipline, not simultaneous databases

Technology	Status	Approved use or introduction trigger	Decision
CockroachDB	Conditional	Regional SQL only after H4 multi-region consistency requirements are frozen and PostgreSQL topology cannot meet them	Requires semantics and RLS revalidation
Prometheus	Adopt	Metrics storage and alert rules in the reference observability profile	OpenMetrics-compatible path
Grafana	Adopt	Dashboards and trace/log correlation in reference profile	No business authorization data source
Loki and Tempo	Adopt in reference profile	Logs and traces behind OpenTelemetry collector	Replaceable backends; retention is environment-specific

3.4 Platform, security, and delivery

Technology	Status	Approved use or introduction trigger	Decision
OpenTelemetry	Adopt	Instrumentation and collector pipeline for traces, metrics, and correlated logs	Vendor-neutral telemetry contract
OCI and Docker	Adopt	Reproducible workload images and Compose development	BuildKit, non-root runtime, immutable digest
Docker Compose	Adopt for local/H1 demo	One-command environment, synthetic data, and fault testing	Not an H2 HA control plane
Kubernetes	Adopt H2	Required by the committed regional design-partner profile; freeze a supported version before first pilot deployment	No multi-cluster claim until H4
Helm	Adopt H2	Required packaging contract for the Kubernetes profile	Rendered manifests validated in CI
OpenTofu	Adopt H2	Infrastructure definitions and plans	Sole source for shared infrastructure
Terraform	Conditional H2 compatibility	A provider or customer may require Terraform execution only if the same HCL modules pass both engines; no forked module tree or dual state	OpenTofu remains reference tool
Argo CD	Adopt for H2	GitOps reconciliation and environment promotion	Production changes come from reviewed Git commits
GitHub Actions	Adopt	CI, artifact creation, security scans, contract tests, evaluation gates	OIDC federation; no long-lived cloud keys
External Secrets Operator	Adopt for Kubernetes	Materialize short-lived secrets from an approved external KMS/vault	Secret values never enter Git or Helm values
OPA	Conditional	Externalize policy only when H2 policy count, independent policy ownership, or non-API enforcement requires it and parity tests pass	H1 uses a versioned in-process policy decision module
Keycloak	Adopt only for local reference	Disposable OIDC/SAML test identity provider	Production integrates customer IdP
Cosign and Sigstore	Adopt	Sign images, attest provenance, verify before deploy	Keyless CI signing where supported
Syft and SPDX/CycloneDX	Adopt	Software bill of materials for each release image	SBOM retained with release evidence
Trivy	Adopt	Image, filesystem, dependency, secret, and IaC scanning	Findings follow release severity policy
Istio/Linkerd	Reject	Service mesh is rejected in the current portfolio	Evaluate one, never both, only after service mesh is reclassified
Edge computing	Research	Read-only, encrypted, disconnected inference only after H4 data and revocation semantics are specified	No current product claim

3.5 Test and developer tooling

Technology	Status	Approved use	Decision
Vitest	Adopt	TypeScript unit and component tests	Fast workspace-native runner
Pytest	Adopt	Python unit, property, simulation, and evaluation tests	Mature fixtures and scientific ecosystem
Playwright	Adopt	Cross-browser end-to-end and accessibility automation	Exercises real browser flows
Testcontainers	Adopt	PostgreSQL, Neo4j, Valkey, object store, and Temporal integration tests	Production-shaped dependencies in CI
k6	Adopt	API, SSE, and workflow load/soak tests	Scriptable thresholds and CI output
Schemathesis	Adopt	OpenAPI property and negative testing	Finds contract edge cases from canonical schema
Ruff and mypy	Adopt	Python formatting/lint check and static types	Fast deterministic gates
ESLint and TypeScript compiler	Adopt	TypeScript lint and type gates	Rules are centrally versioned
OpenAPI Generator	Adopt	Generated client conformance fixtures and later SDKs	Generated code is versioned by contract release

4. Build-versus-buy boundaries

Capability	Decision	Product-owned portion
Identity	Integrate	Tenant membership mapping, delegation, policy context, audit; do not build passwords or MFA
Workflow	Buy/use Temporal	Workflow definitions, idempotency, business state, compensations
Databases/object store	Prefer managed H2	Schemas, RLS, queries, backup verification, projection rebuild
Models	Use provider behind gateway	Capability routing, evidence construction, tool policy, evaluations, safety, audit
Graph visualization	Use a maintained rendering library	Accessible interactions, authorization, graph query limits, product semantics
Connectors	Build narrow adapters over provider APIs	Scope selection, normalization, reconciliation, provenance, ACL and error semantics
Authorization	Build product policy core; integrate IdP	Resource/action vocabulary, tenant context, ACL intersection, delegation, exact-payload grants
Observability	Open standards plus replaceable backends	Semantic conventions, SLOs, dashboards, alerts, redaction

5. Technology invariants

1. Only one technology is authoritative for each concern.
2. A cache, graph, search index, analytics store, workflow history, or model memory cannot become business authority.
3. No technology introduction weakens tenant isolation, ACL revocation, evidence provenance, or audit semantics.
4. Provider-neutral means contracts and packaging are portable; it does not mean operating every alternative simultaneously.
5. Generated SDKs follow the OpenAPI contract and add no privileged endpoints.
6. Libraries that execute plugins, templates, parsers, or model tools run with explicit capabilities, resource budgets, and no ambient credentials.
7. All adopted dependencies have a named update owner, SBOM entry, license policy, vulnerability policy, and end-of-life plan.

8 Client and connector libraries never receive a database superuser, RLS-bypass role, cluster-admin identity, or cross-tenant credential.

6. Acceptance criteria

Evidence ID	Canonical acceptance	Criterion
TC-TECH-001	AC-DOC-002 and AC-REV-001	Dependency inventory maps every production package and service to an Adopt entry or approved ADR.
TC-TECH-002	AC-SUP-001	CI fails on unpinned package locks, mutable production image tags, unsigned images, or missing SBOMs.
TC-TECH-003	AC-REV-001	The application can replace telemetry, object storage, and model endpoint implementations through documented contracts without changing domain authority.
TC-TECH-004	AC-REV-001	No H1 deployment contains Kafka, NATS, service mesh, OpenSearch, ClickHouse, separate vector database, GraphQL command surface, or gRPC service.
TC-TECH-005	AC-DOC-002 and AC-REV-001	A conditional technology ADR includes measured trigger evidence, owner, security/privacy assessment, migration, dual-read or compatibility validation, cost, and rollback.
TC-TECH-006	AC-AI-001, AC-AI-003, and AC-AI-004	All production model capabilities resolve to an evaluated pinned snapshot and fail closed when no approved snapshot is available.
TC-TECH-007	AC-DOC-001	Language-boundary contract tests prove TypeScript and Python serialize the same canonical types and errors.
TC-TECH-008	AC-TEN-001	PostgreSQL RLS, graph isolation, object namespace, and cache namespace tests pass against every supported managed implementation.

CH-05 - Data Architecture and Knowledge Graph

Status: Committed | Owners: Data Platform, Knowledge Graph | Last reviewed: 2026-07-13

Data Architecture and Knowledge Graph

1. Purpose and boundaries

This chapter defines the authoritative data model, ontology, graph projection, semantic index, retention behavior, and rebuild procedures. It is normative for H1 and H2.

PostgreSQL is the system of record. Neo4j, pgvector indexes, caches, and future search systems are derived projections. A projection may accelerate a decision but may never create a fact, grant access, merge identities, or satisfy an approval by itself.

The data layer represents evidence-backed organizational facts. It does not infer individual productivity, performance, burnout, attrition, health, emotion, misconduct, hiring suitability, compensation suitability, or other employment scores. Aggregate operational capacity may be modeled only when it cannot be used to rank or evaluate a person.

Normative requirements

ID	Requirement
REQ-TEN-002	Every row, object, event, edge, vector, cache key, trace, and audit record MUST carry an immutable tenant identifier.
REQ-TEN-005	Retrieval, resolution, memory, analytics, and model training MUST NOT combine tenants without a separately recorded opt-in design.
REQ-DATA-001	PostgreSQL MUST be authoritative for observations, claims, evidence, identity resolution, approvals, scenarios, and audit indexes.
REQ-DATA-002	Every claim MUST include valid time, system time, confidence, evidence, classification, source revision, and source ACL.
REQ-DATA-003	Graph, search, and vector projections MUST be rebuildable after ACL correction, deletion, or ontology change.
REQ-DATA-004	Entity merges MUST preserve source identities, evidence, and a reversible decision history.
REQ-DATA-005	Entity, relationship, property, constraint, and migration definitions MUST be versioned and namespaced.
REQ-DATA-006	Domain packs MUST cover organization, work, engineering, customers, finance, infrastructure, governance, knowledge, and physical assets.
REQ-DATA-007	Every entity and relationship type MUST define ownership, permissions, lifecycle, history, search, retention, deletion, and archive behavior.
REQ-DATA-008	Raw source objects MUST be content-addressed, encrypted, classified, retention-tagged, and immutable except for cryptographic erasure.
REQ-DATA-009	Tenant and eligible data-subject deletion MUST cover authoritative and derived stores while preserving minimum lawful audit evidence.
QAR-COR-001	Duplicate, reordered, or replayed events MUST converge to one stable state digest without duplicate external effects.
QAR-SEC-002	Revoked source access MUST fail closed and invalidate affected projections before they can serve a new authorized read.

Capacity targets are 100,000 projected nodes and 1,000,000 edges per H1 tenant and 1,000,000 nodes and 10,000,000 edges per H2 tenant. H1 graph projection freshness is at most 60 seconds after committed normalization, and a full H1 projection rebuild completes within 15 minutes per tenant.

2. Ownership and storage roles

Store	Authoritative content	Consistency	Failure behavior
PostgreSQL	Tenants, actors, identities, source metadata, claims, evidence metadata, resolution decisions, scenarios, approvals, action receipts, audit indexes, outbox	Serializable for identity merges and approvals; read committed plus explicit row locks elsewhere	Writes fail closed; reads requiring current authorization fail closed
S3-compatible object storage	Immutable encrypted raw payloads, normalized artifacts, exports, model artifacts	Read-after-write required for a committed object reference	Metadata remains in PostgreSQL; missing object is a data-integrity incident
Neo4j	Current graph projection and optional time-slice projections	Eventual, checkpointed	API returns a stale/unavailable marker or falls back to bounded relational reads; it never invents results
pgvector	Tenant-scoped embeddings of ACL-filterable evidence chunks and entities	Eventual, checkpointed	Semantic retrieval degrades to lexical/graph retrieval
Valkey/Redis	Rate limits, short-lived query cache, leases, and ephemeral progress	Non-authoritative	Cache miss or bypass; no security decision is cached beyond its policy version

OpenSearch, ClickHouse, Kafka, and a lakehouse remain Provisional. Their introduction requires a recorded benchmark showing that PostgreSQL, pgvector, or the transactional outbox cannot meet a committed SLO, plus an ADR covering tenancy, deletion, rebuild, and operating cost.

3. Tenant and key invariants

1. `tenant_id` is a UUID generated by the service. It is derived from the authenticated actor and selected installation, never selected from a request body or trusted from an arbitrary header. When a canonical schema carries `tenant_id`, it is only a consistency assertion and a mismatch is rejected.
2. Every primary key is either `(tenant_id, id)` or has a unique constraint on that pair. Every foreign key includes `tenant_id`; a database constraint, not application convention, prevents cross-tenant references.
3. PostgreSQL row-level security is enabled and forced for application roles. A transaction begins by setting a transaction-local, server-derived `app.tenant_id` and `app.actor_id`. Missing context denies all tenant data.
4. Worker roles do not use `BYPASSRLS`. Administrative break-glass access uses a separate audited role, ticket, expiry, and two-person approval.
5. Globally unique external identifiers are never assumed. Source keys are `(tenant_id, connector_installation_id, source_type, external_id)`.
6. Object keys are opaque and tenant-prefixed: `tenants/{tenant_uuid}/{classification}/{sha256}`. Bucket policy and a per-tenant envelope-encryption key enforce isolation.
7. Graph databases, vector rows, search documents, and cache keys carry `tenant_id` and a projection generation. The API validates both before returning data.

4. Authoritative relational model

All timestamps use UTC `timestamptz`. Effective intervals are half-open `[from, to)`. `NULL` upper bounds mean infinity. JSON fields are accepted only where extension is intentional; stable query fields receive typed columns.

4.1 Core records

Table	Required fields and constraints	Purpose and indexes
`tenant`	`id`, `slug`, `state`, `region`, `kms_key_ref`, `created_at`, `deleted_at`; unique `slug`	Tenant lifecycle. `state` is `provisioning`, `active`, `suspended`, `deleting`, or `deleted`.
`actor`	`(tenant_id,id)`, `kind`, `principal_ref`, `display_name`, `status`, `created_at`; unique `(tenant_id,principal_ref)`	Human/service principals and non-authenticating agent audit subjects. An agent actor has no membership or credential of its own and always points to a capability profile, invoking principal, and reduced delegation. No connector identity is an authenticated actor until explicitly linked.
`connector_installation`	`(tenant_id,id)`, `provider`, `provider_account_ref`, `state`, `scope_set_hash`, `credential_ref`, `credential_generation`, `manifest_version`, `acl_version`, `cursor_ref`, `last_reconciled_at`; unique tenant/provider account binding	Tenant-scoped connector state from CH-06. Secrets remain behind `credential_ref`.
`identity_alias`	`(tenant_id,id)`, `installation_id`, `provider_subject`, `entity_id`, `actor_id`, `valid_period`, `system_period`, `verification`, `resolution_decision_id`; unique current provider subject	Reversible mapping among source accounts, canonical persons, and optional authenticated actors.
`source_object`	`(tenant_id,id)`, `installation_id`, `source_type`, `external_id`, `external_version`, `content_sha256`, `object_uri`, `mime_type`, `observed_at`, `source_updated_at`, `ingest_run_id`, `acl_version`, `tombstoned_at`; unique source key plus version	Immutable observation metadata. Index current rows by source key and `source_updated_at`.
`entity`	`(tenant_id,id)`, `ontology_type_id`, `canonical_key`, `state`, `created_at`, `retired_at`, `merged_into_id`; unique active `(tenant_id,ontology_type_id,canonical_key)` where defined	Stable identity independent of mutable attributes. `merged_into_id` is tenant-qualified and acyclic.
`entity_version`	`(tenant_id,id)`, `entity_id`, `valid_period`, `system_period`, `attributes`, `label`, `classification`, `content_hash`, `created_by`; exclusion constraint prevents overlapping system-current versions for the same effective interval	Bitemporal canonical view. GiST indexes on both ranges and B-tree on `(tenant_id,entity_id)`.
`claim`	`(tenant_id,id)`, `subject_entity_id`, `predicate_id`, one of `object_entity_id` or typed value columns, `qualifiers`, `valid_period`, `system_period`, `status`, `confidence`, `confidence_calibration_version`, `precedence`, `source_revision`, `source_acl_hash`, `classification`, `resolution_state`, `created_by`; XOR constraint for object/value	Evidence-backed assertion. Index subject/predicate, object/predicate, current system range, and valid range.
`evidence`	`(tenant_id,id)`, exactly one of `source_object_id` or `assertion_audit_event_id`, `locator`, `excerpt_sha256`, `observed_at`, `classification`, `acl_version`, `content_hash`	Immutable pointer to a source span, field, API object, computed artifact, or governed manual assertion. The excerpt itself remains encrypted in object storage.
`claim_evidence`	`(tenant_id,claim_id,evidence_id)`, `support`, `weight`, `extractor_version`	Many-to-many support or contradiction. `support` is `supports`, `contradicts`, or `context`.
`relationship`	`(tenant_id,id)`, `edge_type_id`, `from_entity_id`, `to_entity_id`, `valid_period`, `system_period`, `strength`, `confidence`, `qualifiers`, `derivation_claim_id`, `state`	Canonical edge materialized from a resolved claim. Unique current semantic key per ontology rule.
`source_acl`	`(tenant_id,source_object_id,principal_kind,principal_ref,effect,valid_period,system_period,source_acl_hash)`	Source-derived visibility. Explicit deny overrides allow. ACL rows never grant more than tenant policy.
`resolution_candidate`	`(tenant_id,id)`, `left_entity_id`, `right_entity_id`, `features`, `score`, `resolver_version`, `state`, `created_at`	Candidate duplicate pair. Feature values must identify provenance and must not include prohibited employment inference.

Table	Required fields and constraints	Purpose and indexes
`resolution_decision`	`(tenant_id,id)`, `candidate_id`, `decision`, `reason_code`, `evidence_ids`, `decided_by`, `decided_at`, `reversed_by_id`	Immutable merge, reject, or defer decision. Automated merge is allowed only above a calibrated threshold and on hard identifiers.
`projection_checkpoint`	`(tenant_id,projection_name,generation)`, `last_outbox_id`, `source_snapshot_hash`, `state`, `started_at`, `completed_at`, `counts`, `checksum`	Rebuild and freshness boundary.
`embedding`	`(tenant_id,id)`, `target_kind`, `target_id`, `chunk_ordinal`, `model_id`, `model_revision`, `dimensions`, `vector`, `content_hash`, `acl_version`, `deleted_at`	Semantic projection. Unique on target, chunk, model revision, and content hash.
`scenario`, `simulation_snapshot`, `simulation_run`	Tenant-qualified IDs, immutable input/result hashes, versions, states, creators, confirmation, timestamps, and artifact refs from CH-08	Governed scenario/simulation history; large sealed artifacts live in object storage.
`agent_run`, `tool_invocation`	Tenant-qualified durable states, capability/model/schema hashes, budgets, delegations, policy decisions, safe argument/result refs, and timestamps from CH-07	Recoverable AI orchestration and trace index without private chain-of-thought.
`approval_request`, `approval_decision`, `action_receipt`	Tenant-qualified command/digest, requester/approvers, policy/source/credential versions, expiry, idempotency, immutable decisions, before/after refs, and compensation link	Exact-payload governance and one-action ledger from CH-06/CH-07.
`audit_event`	`(tenant_id,id)`, actor/service, action, resource refs, outcome, reason, policy version, trace ID, payload hash, occurred/recorded time	Append-only searchable audit index; sensitive detail uses a separately encrypted artifact.
`outbox_event`	`id`, `tenant_id`, `event_type`, `aggregate_type`, `aggregate_id`, `aggregate_version`, `payload`, `occurred_at`, `published_at`; unique aggregate/version/type	Transactional event source for projectors. Partition monthly after H2 telemetry justifies it.

`entity_version`, `claim`, `relationship`, `source_acl`, and authorization policy versions are never updated in place except to close `system_period`. A correcting transaction closes the prior system period and inserts a replacement. Historical queries specify `valid_at` and `known_at`; current PostgreSQL queries use `upper_inf(system_period)` and `valid_period @> transaction_timestamp()` rather than treating a null range bound as an ordinary timestamp.

4.2 Claim and evidence semantics

A claim is the smallest reviewable assertion. Examples include "Project Atlas has target date 2026-09-30" and "Repository api-core supports Project Atlas." A claim MUST contain:

- exactly one tenant and subject;
- a registered predicate;
- exactly one entity object or one typed scalar value;
- effective and system time;
- a confidence in `[0,1]` and a documented calibration version;
- one or more evidence links; a manual assertion uses evidence that references its immutable actor/time/reason audit event rather than bypassing evidence;
- source precedence and a resolution state.

Confidence describes evidence quality, not truth probability. It does not override authorization. Conflicting claims remain stored. A deterministic resolver selects the current canonical value by: explicit governed override, authoritative source rank for that predicate, source revision, observed time, then stable claim UUID. The API returns conflicts and the selected rule when the runner requests explanation.

Evidence locators use a typed object such as `{kind:"json_pointer",pointer:"/fields/duedate"}`, `{kind:"line_range",start:10,end:15}`, or `{kind:"url_fragment",fragment:"discussion_r123"}`. User-visible citations resolve through an authorization-aware citation service; raw object URIs are never sent to clients.

4.3 Entity resolution and reversible merges

Resolution proceeds as normalize, block, score, decide, and project. Blocking keys may include provider account ID, verified work email hash, repository identity, or administrator-established link. Names alone never auto-merge. An automated H1 merge requires either the same provider account ID in the same connector namespace, or a tenant-verified work email plus a second stable corroborating signal, and a calibrated score at or above `0.995`; all other candidates require review.

A merge creates an immutable `resolution_decision`, closes affected entity versions, points the losing entity to the survivor, and rewrites canonical relationships in one serializable PostgreSQL transaction. Source claims retain their original subject. A split reverses the decision, restores versions from the decision snapshot, and emits new projection events. Cycles, cross-type merges outside an ontology-declared equivalence family, and cross-tenant candidates are database-rejected.

5. Ontology and domain packs

5.1 Metamodel

Every type has a stable ID in the form `edt.core/Project` or `{publisher}.{package}/Type`, a semantic version, display metadata, parent types, required and optional property schemas, allowed edges, identity keys, sensitivity classification, lifecycle states, search policy, retention class, and projection policy.

The core uses single semantic inheritance and reusable traits such as `Temporal`, `Locatable`, `Ownable`, `Searchable`, `EvidenceBearing`, and `ExternallyAddressable`. Multiple storage inheritance is prohibited. Runtime entities retain the exact ontology package and version used for validation.

5.2 Domain packs and node catalog

Domain pack	Committed node types	Representative optional or later types
Organization and people	`Organization`, `BusinessUnit`, `Department`, `Team`, `Person`, `Employee`, `Contractor`, `Role`, `Position`, `Goal`, `Office`	`Committee`, `VendorContact`, `Skill`, `Responsibility`
Work and projects	`Portfolio`, `Program`, `Project`, `Milestone`, `Requirement`, `Epic`, `Sprint`, `Task`, `Ticket`, `Workflow`, `Decision`, `Approval`, `Risk`	`OKR`, `ChangeRequest`, `Runbook`, `Experiment`
Engineering and product	`Product`, `Feature`, `Bug`, `Repository`, `Branch`, `Commit`, `PullRequest`, `File`, `API`, `Service`, `Microservice`, `Database`, `Pipeline`, `Build`, `Deployment`, `Environment`, `Incident`, `Alert`	`Package`, `ContainerImage`, `FeatureFlag`, `TechnicalDebtItem`, `TestCase`
Communications and knowledge	`Document`, `Presentation`, `Spreadsheet`, `Email`, `ChatMessage`, `Meeting`, `CalendarEvent`, `Transcript`, `ResearchPaper`, `Patent`, `Policy`	`WikiPage`, `Recording`, `KnowledgeArticle`, `Topic`
Customers and revenue	`Customer`, `Account`, `Opportunity`, `Subscription`, `Order`, `SupportCase`, `Contract`	`Lead`, `Campaign`, `Entitlement`, `Renewal`
Finance and procurement	`Budget`, `CostCenter`, `Invoice`, `PurchaseOrder`, `Expense`, `Vendor`, `Payment`, `FinancialAccount`	`LedgerAccount`, `FinancialForecast`, `TaxRecord`
IT and infrastructure	`Asset`, `Computer`, `Server`, `Network`, `CloudAccount`, `Cluster`, `Queue`, `SecretReference`, `IdentityProvider`	`Subnet`, `Certificate`, `License`, `SaaSApplication`
Governance and risk	`Control`, `Finding`, `Exception`, `Audit`, `Regulation`, `DataSet`, `DataClassification`	`Threat`, `Assessment`, `LegalHold`
Physical assets and IoT	`Location`, `Building`, `Room`, `Machine`, `Sensor`, `Metric`, `KPI`, `TimeSeries`	`Vehicle`, `ProductionLine`, `DigitalAsset`
Platform provenance and control	`Actor`, `ConnectorInstallation`, `SourceObject`, `Claim`, `Evidence`, `PolicyDecision`, `ToolInvocation`, `Scenario`, `SimulationRun`, `Forecast`, `AgentRun`, `ApprovalRequest`, `ActionReceipt`	`EvaluationRun`, `CompensationResult`
Customer extensions	No tenant type is core	Namespaced types approved by the extension process below

`Person` is the identity-bearing base. `Employee` and `Contractor` are time-bounded engagements, not mutually exclusive person identities. `Manager` is intentionally not a person subtype: management is a time-bounded `Role`/`Position` plus `REPORTS_TO` relationships. `Actor` is an authenticated human/service principal projection, not another person identity; the optional governed actor-to-person mapping remains an authorization record in PostgreSQL and is not an ontology edge. Platform provenance/control nodes are immutable projections of their identically named PostgreSQL records, not a second authority; only fields explicitly allowlisted for traversal are projected. `FinancialForecast` is a governed finance-domain specialization of the platform `Forecast` result type. Communications and employment data inherit their source ACL and classification. Secrets are represented only by `SecretReference`; secret values are never graph properties. Every listed type instantiates the metamodel fields for purpose, attributes, states, identity/versioning, permissions, visibility, relationships, traits, classification, embedding/search policy, ownership, update cadence, history, retention, soft deletion, and archive; omission from this summary table does not make those fields optional.

5.3 Edge catalog

All edges are directed and have effective/system time, provenance, confidence, state, and optional strength. Symmetric semantics are represented by one canonically ordered edge. Inverse labels are presentation metadata, not duplicate records.

Family	Edge types and canonical direction	Cardinality and validation
Structure	`PART_OF child->parent`, `REPORTS_TO engagement->engagement`, `SUPERVISES role->work`, `LOCATED_IN item->location`	A current `PART_OF` hierarchy is acyclic. A current engagement has at most one primary `REPORTS_TO`; matrix relations use a qualifier.
Ownership	`OWNS owner->resource`, `ASSIGNED_TO work->actor_or_team`, `RESPONSIBLE_FOR actor_or_team->resource`, `APPROVED actor->approval`	Ownership and approval require a claim and an actor visible at the effective time. Self-approval policy is enforced outside the graph.
Work	`WORKS_ON actor_or_team->work`, `BLOCKS blocker->work`, `DEPENDS_ON consumer->prerequisite`, `REQUIRES item->requirement`, `IMPLEMENTS artifact->requirement`, `PRODUCES process->artifact`	`DEPENDS_ON` used for scheduling MUST be a DAG after scenario overlays. Soft dependencies carry `qualifiers.scheduling=false`.
Technology	`USES consumer->resource`, `CALLS caller->api_or_service`, `HOSTED_ON workload->infrastructure`, `DEPLOYED deployment->environment`, `CONNECTED_TO source->target`, `OBSERVES monitor->resource`	Endpoint and environment qualifiers disambiguate edges. Secrets and credentials are excluded.
Knowledge	`CREATED actor->artifact`, `MENTIONS artifact->entity`, `REFERENCES artifact->artifact`, `GENERATED process->artifact`, `RELATES_TO source->target`	`RELATES_TO` is never accepted for authorization, scheduling, or impact analysis unless promoted to a specific edge.
Business	`SERVES team_or_product->customer`, `SOLD_TO offering->customer`, `BILLED_TO invoice->account`, `SUPPLIED_BY item->vendor`, `INFLUENCES driver->outcome`	Financial edges inherit the most restrictive endpoint classification. `INFLUENCES` is descriptive unless a governed causal model exists.
Analysis	`FORECASTS forecast->metric`, `EVIDENCED_BY claim->evidence`, `DERIVED_FROM artifact->source`, `LINKED_TO source->target`	Derived edges record algorithm/model version. `LINKED_TO` has no transitive meaning.

Strength is optional domain magnitude. Confidence is required for inferred edges and exactly `1.0` for direct governed assertions. Automatic creation requires a registered derivation rule. Manual deletion closes the edge system period; source reappearance creates a new version and does not silently reopen the old record. Conflicts preserve all claims and use the predicate precedence rule.

Ontology direction is preserved in storage: `DEPENDS_ON` points from consumer to prerequisite. A scheduling snapshot deliberately compiles it to `{predecessor=prerequisite, successor=consumer}`. `BLOCKS` already points blocker to blocked work and therefore does not invert. The compiler records the source relationship ID and direction rule so UI arrows, graph paths, and simulation edges cannot silently disagree.

5.4 Extension rules

1. Extensions are signed packages containing `manifest.yaml`, JSON Schemas, edge constraints, migrations, fixtures, display metadata, and compatibility tests.
2. Package and type IDs are globally namespaced. A package cannot define or shadow `edt.core/*`.
3. Additive optional properties are minor releases. New required properties, changed identity keys, removed enum values, narrowed edge ranges, or changed semantics require a new major version and an explicit migration.
4. Extensions cannot weaken tenant isolation, RLS, source ACLs, retention, audit, encryption, or prohibited-use controls. Custom code does not run in the database or projector.
5. A property declares scalar type, nullability, maximum size, classification, indexing, searchability, embedding eligibility, redaction, retention class, and source precedence.
6. An edge declares domain/range types, cardinality, direction, inverse label, acyclicity, merge behavior, deletion behavior, and whether it may affect authorization or scheduling.
7. Installation validates schemas, migration reversibility, index budget, traversal-cost budget, malicious fixtures, and downgrade behavior in a quarantined tenant.
8. Removing a package first disables new writes, then migrates or archives existing instances. Orphaned entity data is never discarded implicitly.

6. Graph projection

6.1 Projection contract

Neo4j stores one current node per canonical entity, plus allowlisted platform provenance/control projections needed by registered graph paths, and one current relationship per canonical relationship. Required entity-node properties are `tenant_id`, `entity_id`, `ontology_type`, `ontology_version`, `label`, `classification`, `acl_hash`, `system_version`, and `projection_generation`; platform nodes substitute their stable authoritative record ID for `entity_id` and expose no raw payload, prompt, secret, approval rationale, or evidence excerpt. Relationship properties add `relationship_id`, `valid_from`, `valid_to`, `confidence`, and `source_claim_id`.

Projectors consume committed outbox rows in numeric order per tenant. Each mutation uses deterministic IDs and an idempotent `MERGE`, then records the outbox ID in the same Neo4j transaction. Duplicate delivery is harmless. A gap pauses that tenant, fetches the missing event, and alerts after 60 seconds. An out-of-order event is buffered until the gap closes; it is never applied speculatively.

Outbox rows are retained for at least 30 days and are never pruned past the minimum non-retired consumer checkpoint plus a seven-day safety margin. A rebuild whose catch-up watermark is no longer retained abandons its generation and restarts from a new authoritative snapshot. A lagging retired consumer cannot block retention indefinitely; retirement is an audited state transition after its projection is disabled or rebuilt.

Graph reads MUST include `tenant_id`, active projection generation, current authorization predicate, maximum depth, maximum returned nodes, and a server timeout. Arbitrary client Cypher is prohibited. Authorization is rechecked against current PostgreSQL policy before result serialization. This is the revocation barrier that protects reads during projection lag.

6.2 Rebuild and validation

1. Record a PostgreSQL repeatable-read snapshot, maximum outbox ID, ontology versions, and source snapshot hash.
2. Allocate a new tenant projection generation without altering the active generation.
3. Stream entities and relationships in stable UUID order into the new generation using bounded batches.
4. Validate counts by type, endpoint integrity, tenant integrity, forbidden properties, acyclic constrained edges, sampled claim lineage, and a deterministic Merkle checksum.
5. Replay outbox events after the snapshot watermark until lag is zero.
6. Atomically switch the tenant's active generation in PostgreSQL, then invalidate graph and query caches.
7. Retain the prior generation for 24 hours for rollback, then delete it in bounded batches.

A failed validation never switches generations. A projection can always be dropped and rebuilt; no data repair is performed only in Neo4j.

7. Search and embeddings

Text is chunked deterministically by content type. Each chunk retains target ID, source locator, content hash, model revision, language, classification, ACL version, and ordinal. The embedding input excludes secrets, access tokens, hidden fields, and prohibited employment inferences.

Retrieval applies current tenant and ACL filters before ranking. When the vector engine cannot pre-filter sufficiently, the service over-fetches from a tenant-only partition, performs a current relational authorization check, and returns fewer results rather than expanding the search. Results combine lexical score, vector similarity, graph proximity, freshness, and source authority with a versioned ranker. No score is represented as factual confidence.

An embedding model change creates a new side-by-side index. Promotion requires retrieval evals and a complete re-embedding checkpoint. Old rows remain until rollback expiry. Content deletion creates an immediate authorization deny, queues removal from every index, and records deletion completion.

8. Consistency, retention, and deletion

API responses that use projections include `data_watermark`, `authorization_version`, and `projection_generation`. A command followed by a read can request `min_watermark`; the API waits up to two seconds and otherwise uses PostgreSQL or returns `409 projection_not_ready` with retry guidance.

Default retention classes are: operational metadata 24 months, raw connector payload 90 days, derived chunks 90 days, agent run content 30 days, audit security events 7 years, and anonymized aggregate telemetry 13 months. Tenant policy may shorten these periods. Longer retention requires a documented legal or regulatory basis. Legal hold suspends physical deletion for named records while preserving access controls.

Deletion is a durable workflow: deny reads immediately, tombstone records while the workflow is active, physically delete or irreversibly de-identify eligible authoritative rows, revoke object URLs, remove graph/vector/cache/search projections, cryptographically erase tenant keys when deleting a tenant, verify every store, and issue an auditable deletion receipt. Minimum lawful audit evidence retains only the approved identifiers, outcome, authority, and timestamps; source content and unnecessary subject attributes are removed. Backups age out under their retention schedule and cannot be restored into general service without reapplying the deletion ledger.

9. Authorization and privacy

Effective access is the intersection of tenant membership, platform policy, source ACL, resource classification, purpose, and current delegation. The most restrictive endpoint or evidence classification propagates to a derived claim and relationship. A derived artifact maintains its complete input classification set; redaction does not reduce classification unless a governed declassification rule proves it.

Permission caches are keyed by tenant, actor, resource, policy version, and ACL version, with a maximum H1 TTL of 30 seconds. Revocation increments a version and broadcasts invalidation before connector reconciliation completes. Unknown ACL, stale policy, or unavailable authorization service means deny.

Exports, embeddings, model prompts, and citations follow the same decision path. Application logs contain identifiers and hashes, not source text or embedding vectors.

10. Failure handling and operations

Failure	Required response
PostgreSQL unavailable	Reject writes and reads needing fresh policy; do not serve cached sensitive data past its policy version.
Object missing or hash mismatch	Quarantine evidence, mark dependent claims degraded, page the data-integrity owner, and never fabricate a citation.
Neo4j unavailable or behind	Return explicit freshness, use bounded relational fallback where supported, and queue projector recovery.
Vector index unavailable	Use lexical/graph retrieval and report degraded retrieval mode.
Duplicate/out-of-order event	Deduplicate by aggregate version and hold gaps; never apply an older version over a newer one.
Concurrent entity merge	Serializable transaction and stable entity lock ordering; loser retries from current state.
Ontology incompatibility	Quarantine the observation and keep the last valid entity version.
Cross-tenant reference attempt	Database rejection, security audit event, high-severity alert, and request termination.

Metrics include claim ingest rate, unresolved conflicts, evidence-orphan count, merge/split rate, projection lag, checkpoint age, rebuild duration, graph count drift, ACL-version lag, embedding backlog, deletion backlog, and per-tenant storage/cardinality. Traces propagate `tenant_id_hash`, `trace_id`, `ingest_run_id`, and `projection_generation`; raw content is excluded.

Storage budgets are enforced per tenant with warning, soft, and hard thresholds. A hard threshold pauses non-security ingestion but continues revocations, deletions, and audit writes. H2 capacity reviews use actual bytes per node, edge, claim, evidence object, and embedding rather than catalog counts alone.

11. Verification and acceptance

ID	Acceptance criterion
AC-TEN-001	Every tenant-qualified foreign key and RLS policy rejects cross-tenant reads and writes, including guessed UUIDs and missing tenant context.
AC-DATA-001	Replaying each observation/outbox event twice and in randomized delivery order produces the same authoritative and projected digest.
AC-DATA-002	An H1 tenant graph and semantic projection rebuild from authoritative claims passes count, endpoint, ontology, lineage, ACL, and checksum validation within 15 minutes.
AC-DATA-003	Merge, projection, split, and rebuild restore the pre-merge graph without losing source identities, claims, evidence, or unrelated relationships.
AC-SEC-002	Revoking a source ACL blocks graph, vector, citation, export, memory, and cached reads before stale projections can return data.
AC-PRV-001	Tenant deletion produces verified receipts for PostgreSQL, object, graph, vector, and cache removal; restore testing reapplies the deletion ledger.

Unit tests also prove bitemporal corrections can answer both effective time and system-known time, and that conflicting claims retain both evidence chains while the registered rule selects the current value. Property tests generate arbitrary merge/split histories and dependency graphs. Integration tests run PostgreSQL with forced RLS, Neo4j, object storage, and pgvector. Security tests attempt identifier substitution, ACL lag, poisoned ontology packages, oversized properties, and malicious evidence locators.

12. Risks and evolution

ID	Risk	Mitigation and trigger
RSK-002	A pooled Neo4j projection leaks data through a missing tenant predicate.	Central typed query templates, tenant-qualified projection tests, result reauthorization, and a dedicated-database/cell trigger from pilot evidence.
RSK-003	Source ACLs leak through graph, embeddings, cache, citation, or summaries.	Monotonic classification, current authorization barriers, version invalidation, deletion propagation, and side-channel tests.
RSK-010	Entity resolution makes a false merge.	Exact identifiers for auto-merge, calibrated threshold, review queue, preserved source identity, and reversible decisions.
RSK-015	Deletion misses a derived copy or embedding.	Complete data inventory and lineage, immediate deny, erasure workflow, projection rebuild, restore deletion ledger, and evidence scan.
RSK-016	Synthetic scale and evidence behavior are mistaken for external validity.	Report measured system properties only, surface confidence semantics, and require H2 validation before production predictive claims.

H3 may introduce separate transactional and historical partitions, OpenSearch, ClickHouse, or Kafka only after representative workload tests identify the limiting resource and migration/rollback is proven. Trillion-edge graphs, cross-company federated graphs, and causal organizational models remain Research and MUST NOT appear as production capability claims.

CH-06 - Ingestion, Connectors, and Synchronization

Status: Committed | Owners: Connector Platform, Data Platform | Last reviewed: 2026-07-13

Ingestion, Connectors, and Synchronization

1. Purpose and boundaries

This chapter defines the connector runtime and the complete H1 GitHub and Jira synchronization contract. Connectors collect authorized source observations; they do not decide canonical truth, merge identities, grant permissions, or let model output reach an external API directly.

H1 is deliberately narrow: GitHub metadata is read-only, Jira is read-only except for one field-limited remediation update in an allowlisted sandbox project, and all organizations and records are synthetic. H2 generalizes the framework but does not automatically enable additional connector writes.

ID	Requirement
REQ-CON-001	H1 GitHub access MUST be read-only metadata for allowlisted sandbox repositories and exclude source bodies, secrets, logs, and private messages.
REQ-CON-002	H1 Jira access MUST be restricted to allowlisted projects and the exact approved remediation issue mutation.
REQ-CON-003	Connectors MUST verify webhooks, reconcile periodically, checkpoint cursors, deduplicate events, and preserve tombstones.
REQ-CON-004	Connector execution MUST isolate credentials, egress, parsing, quotas, and tenant effects and support immediate revocation.
REQ-CON-005	The connector SDK MUST define manifests, authentication, discovery, backfill, incremental sync, writes, errors, telemetry, tests, and certification.
REQ-ACT-001	Approval MUST bind tenant, actor, credential, target, canonical arguments, expiry, policy version, and idempotency key.
REQ-ACT-002	The H1 mutation MUST require two distinct authenticated approvers; the requester cannot satisfy the second approval.
REQ-ACT-003	The Jira action MUST record before/after state and provide authorized idempotent compensation.

ID	Requirement
QAR-SYNC-001	H1 source freshness MUST be no more than 15 minutes at p95 while providers are healthy.
QAR-COR-001	Duplicate, reordered, or replayed delivery MUST converge to the same state without duplicate effects.
QAR-SEC-003	Payload change or replay MUST be denied or return the original receipt without another write.

Valid webhook ingress is acknowledged within 2 seconds at p95 after durable enqueue. A restarted connector resumes from its last committed cursor without duplicating canonical state.

2. Connector execution model

Each installation is a tenant-scoped state machine:

`requested -> authorizing -> validating -> initial_sync -> active -> degraded -> suspended -> revoking -> revoked`

Only `active` and `degraded` installations schedule reads. A credential failure moves to `suspended`; no silent fallback to a broader credential is allowed. Revocation invalidates the secret, disables new work, establishes an immediate authorization deny barrier, unregisters webhooks where possible, and starts retention/deletion policy.

Temporal owns the durable workflows:

- `InstallConnectorWorkflow`: exchange credentials, validate identity and scopes, register webhooks, and start initial sync.
- `WebhookDispatchWorkflow`: deduplicate a durable notification, fetch authoritative source objects, normalize, and checkpoint.
- `ReconcileConnectorWorkflow`: enumerate provider state, compare manifests, fetch differences, confirm tombstones, and update cursors.
- `RefreshCredentialWorkflow`: rotate short-lived tokens and refresh OAuth grants without exposing token material to activities.
- `RefreshWebhookWorkflow`: renew expiring registrations and verify delivery health.
- `RevokeConnectorWorkflow`: deny access, stop schedules, unregister hooks, purge credentials, and initiate deletion.
- `ExecuteJiraRemediationWorkflow`: revalidate approval and source version, apply one update, record receipt, and compensate on request.

Activities use bounded retries with provider-specific backoff and jitter. Authentication, authorization, schema validation, and permanent 4xx errors are non-retryable. Rate limit and 5xx errors are retryable until the workflow deadline. Each activity heartbeat contains a cursor, not credentials or source content.

3. Common connector contract

3.1 Manifest

Every connector package validates against `contracts/schemas/connector-manifest.schema.json`. The manifest and all referenced package artifacts are closed, versioned contracts; a connector cannot declare a generic `read`, `write`, hostname, or schema string and defer its meaning to code.

Field	Required contract
`schema_version`, `id`, `version`	Manifest-schema major, stable reverse-DNS connector ID, and semantic package version. H1 IDs are `com.enterprise-digital-twin.connector.github` and `com.enterprise-digital-twin.connector.jira-cloud`.
`publisher`	Stable publisher ID and display name, owning team, support URI, and monitored security contact.
`provider`, `deployment`	Provider ID plus an allowlist of supported provider variants; H1 permits only `github_cloud` or `jira_cloud`. Runtime mode, OCI image digest, minimum worker version, and installation isolation are explicit.

Field	Required contract
`auth`	Exact authentication type, required provider permissions/scopes with access level, credential lifetime/refresh behavior, and secret-storage class. The runtime rejects an installed grant whose effective permission set is missing a required scope or includes a forbidden scope.
`capabilities.reads[]`	One entry per provider resource with its stable resource name, allowed operations from `discover`, `backfill`, `incremental`, `reconcile`, and `acl_sync`, pagination mode, deletion behavior, and required auth permission.
`capabilities.events[]`	One entry per accepted provider event with delivery-ID source, verification profile, maximum body size, whether it is authoritative, and the authenticated reconciliation resource it can enqueue. H1 events are never authoritative.
`capabilities.mutations[]`	One entry per externally mutating command with target resource, exact operation, default-enabled flag, approval class, idempotency class, and compensation command. GitHub supplies an empty array; Jira supplies only `jira_issue_remediation_v1`, disabled by default outside the allowlisted H1 sandbox installation.
`schemas`	Content-addressed references for raw envelope, bounded raw payload, normalized observation, cursor, and ontology-mapping JSON Schemas. Each reference contains a package-relative URI, stable `$id`, semantic schema version, and lowercase SHA-256 of RFC 8785 canonical JSON bytes.
`rate_limits`	Named provider buckets, quota scope, maximum per-installation concurrency, fairness weight, request timeout, retry classes, and bounded backoff policy.
`cursor`	Referenced cursor schema plus commit granularity, overlap/lookback policy, supported reset modes, reset authorization class, and last-good-checkpoint behavior. Arbitrary caller-provided cursors are prohibited.
`data_policy`	Included/excluded provider fields, classifications, raw and normalized retention classes, payload/depth/string/decompression limits, redaction policy, untrusted-content flag, and prohibition on training use.
`network_policy`	Closed HTTPS egress rules containing exact host, port, and path-prefix allowlists; minimum TLS version; DNS revalidation; private/link-local/metadata address denial; redirect count; response-size limit; and connect/read deadline. Wildcards, IP literals, alternate ports, and manifest-controlled proxies are prohibited in H1.
`health`	Typed credential, webhook, freshness, quota, and reconciliation checks with interval, timeout, failure threshold, degraded/suspended transition, and safe diagnostic code.
`support`	Compatibility range, deprecation state/sunset, owning service, runbook URI, and security escalation.

Package references are normalized relative paths beneath their connector directory: absolute URIs, `..`, symlinks escaping the package, a missing artifact, an unexpected artifact, an `$id` mismatch, or a digest mismatch fails build and installation. The schema permits additive provider fields only inside the bounded untrusted `payload` member of a raw envelope. Normalized observations, cursors, mapping declarations, and the manifest itself reject unknown fields. Mapping declarations contain only registered source JSON Pointers, ontology types/predicates, allowlisted transform IDs, classification, authority, and ACL behavior; expressions, scripts, templates, network locations, and model prompts are invalid.

The normative H1 packages are:

Package	Required machine-readable instances
`contracts/connectors/github/`	`manifest.json`, `schemas/raw-envelope.schema.json`, `schemas/raw-payload.schema.json`, `schemas/normalized-observation.schema.json`, `schemas/cursor.schema.json`, and `schemas/ontology-mapping.schema.json`. The manifest enumerates exactly the permissions, resources, events, reconciliation fetches, exclusions, and empty mutation set in Section 4.
`contracts/connectors/jira-cloud/`	The same six files. The manifest enumerates exactly the scopes, resources, events, endpoints, field exclusions, and sole remediation/compensation commands in Sections 5 and 8.

Each package is tested against positive and negative raw fixtures and at least one concrete mapping fixture. CI validates both manifests against the shared manifest schema, resolves and rehashes every referenced schema, validates mapping fixtures, confirms the capability lists equal the chapter allowlists, and proves that no H1 mutation or egress target exists only in implementation code. Package version changes follow semantic compatibility: changing a permission, event identity, normalized meaning, cursor interpretation, mapping meaning, egress destination, mutation, redaction, or deletion behavior is breaking unless the old behavior remains independently selectable through the declared compatibility window.

Connector code runs in a network-restricted worker with no model credentials, database superuser role, tenant-selection input, or direct Neo4j access. Its egress is limited to the manifest host allowlist and object storage through a scoped service identity. Parser limits include 10 MiB per JSON payload in H1, maximum nesting depth 64, maximum string length 1 MiB, decompression ratio 20:1, and a 30-second parse deadline. Oversized or invalid objects are quarantined with hashes and metadata, not logged.

3.2 Ingest envelope and idempotency

The immutable envelope contains:

```
{
  "schema_version": "1.0",
  "tenant_id": "server-derived UUID",
  "installation_id": "UUID",
  "provider": "github|jira",
  "delivery_id": "provider delivery ID or generated poll ID",
  "event_type": "string",
  "external_object_id": "string",
  "external_version": "provider version or canonical content hash",
  "observed_at": "RFC3339 timestamp",
  "source_updated_at": "RFC3339 timestamp or null",
  "payload_sha256": "lowercase hex",
  "payload_object_uri": "opaque object reference",
  "ingest_run_id": "UUID"
}
```

The ingress transaction inserts a receipt with unique `(tenant_id,installation_id,provider,delivery_id)` and an outbox event. A duplicate with the same hash returns success without new work. Reuse of a delivery ID with a different hash is quarantined and alerted. Poll observations deduplicate on source key plus `external_version`; when no provider version exists, the connector hashes a canonical JSON representation after removing volatile transport fields.

Normalization is a pure, versioned function from immutable envelope to `NormalizedObservation[]`. Each observation carries source key, ontology candidate type, attributes, relationships, source ACL, effective time, source locator, parser version, and warnings. Retrying normalization produces byte-identical canonical JSON. Invalid optional fields become warnings; invalid identity, tenant, timestamp, or authorization fields quarantine the observation.

4. GitHub App contract

4.1 Permissions and endpoint allowlist

H1 uses a GitHub App installation token. No user OAuth token or personal access token is accepted. The app requests only these permissions:

Scope	Level	H1 use
Repository metadata	Read	Installation, repository identity, visibility, and topics. GitHub grants metadata read access to installed repositories.
Contents	Read	Commit, branch, and comparison metadata. The application endpoint policy forbids fetching file blobs or repository archives.
Pull requests	Read	Pull request metadata, reviews, requested reviewers, and merge state.
Actions	Read	Workflow and workflow-run metadata. Workflow file contents are not fetched.
Checks	Read	Check suite and check run names, status, conclusion, and timestamps.
Deployments	Read	Deployment and deployment-status metadata.
Members	Organization read	Organization member and team identity needed for synthetic organization resolution.

The API client allowlist is limited to installation repositories; repository metadata; branches, commits, comparisons; pull requests and reviews; Actions workflows/runs/jobs; checks; deployments/statuses; organization members; and teams. Calls to contents blobs, archives, source tarballs, secrets, variables, administration, billing, email, and security-alert bodies are blocked even if an upstream permission could permit them. The customer may install the app on selected repositories only.

GitHub documents that app permissions determine both API access and subscribable webhooks and recommends the minimum permissions required. See Choosing permissions for a GitHub App.

4.2 Events and validation

Subscribed events are `installation`, `installation_repositories`, `repository`, `push`, `pull_request`, `pull_request_review`, `workflow_run`, `check_run`, `deployment`, `deployment_status`, `membership`, and `team`. Events not on this list are rejected before payload parsing.

Ingress preserves the exact request bytes, validates `X-Hub-Signature-256` using HMAC-SHA256 and a constant-time comparison, and requires a syntactically valid `X-GitHub-Delivery` UUID. A GitHub App uses one app/environment-bound webhook route; after signature verification, the payload's installation ID must map server-side to exactly one enabled tenant connector installation for that app registration. No tenant or installation supplied in a query, header, or unsigned route segment participates in the mapping. The delivery ledger rejects altered ID reuse and deduplicates replay. The signature is checked before JSON parsing. GitHub notes that deliveries can arrive out of order and that `X-Hub-Signature-256` is the verification header; see Troubleshooting webhooks.

A valid installation lifecycle event may arrive before its setup callback has completed. In that case the app-level ingress stores only delivery ID, body hash, event type, app registration, GitHub installation ID, and receipt time in a short-lived global setup ledger; it discards the unbound body after signature validation and performs no normalization or tenant write. `InstallConnectorWorkflow` proceeds only after its server-held setup state and an authenticated GitHub installation lookup bind that installation ID to one tenant, then performs reconciliation rather than replaying the discarded body. Unclaimed metadata expires after 24 hours and is security-audited without source payload content.

Webhook effects are minimal:

- installation changes trigger scope and repository reconciliation;
- a push enqueues its repository and referenced commit range;
- a pull request, review, check, workflow, or deployment event enqueues the referenced object for authenticated GET;
- member/team changes enqueue organization identity reconciliation;
- deletion/uninstallation installs an immediate deny barrier before cleanup.

No canonical claim is created solely from a webhook body.

4.3 GitHub reconciliation and cursors

Initial sync enumerates installation repositories, then in stable repository-ID order fetches repository metadata, branches, commits within the configured H1 lookback, open and recently updated pull requests, reviews, checks, workflow runs, deployments, and team/member references. H1 lookback is 180 days; older metadata is fetched only when referenced by an in-window object.

The versioned cursor contains repository ID, collection kind, provider page token or last stable tuple `(updated_at,id)`, lookback lower bound, and manifest generation. A cursor is committed in the same PostgreSQL transaction as normalized observations and outbox events. Pages are processed with overlap: the next run rereads the final 10 minutes and deduplicates by external version. REST conditional requests use ETag where supplied, but a `304` never substitutes for scheduled deletion and permission reconciliation.

Every 15 minutes the worker reconciles repository selection and objects changed since the prior watermark. Every 24 hours it performs a bounded manifest scan. A full weekly scan verifies counts and samples content hashes. Rate-limit headers control per-installation concurrency; security revocations and webhook repair take priority over historical backfill.

5. Jira Cloud OAuth contract

5.1 Scopes and endpoints

H1 uses the server-side OAuth 2.0 authorization code flow, a confidential client secret, high-entropy state, an exact registered redirect URI, rotating refresh tokens, and the classic scopes below. Atlassian documents the authorization-code exchange, required state, exact redirect URI, and rotating refresh-token behavior in its OAuth 2.0 (3LO) guide.

- `read:jira-work` for project, issue, status, sprint, version, comment metadata, and JQL search;
- `read:jira-user` for accessible user profiles used in tenant-local identity resolution;
- `write:jira-work` solely for the allowlisted remediation update in Section 8;
- `manage:jira-webhook` to create, list, refresh, and remove dynamic webhooks;
- `offline_access` to maintain synchronization without a browser session.

Atlassian recommends classic scopes where available and documents that Jira permissions still constrain the authorized user's access. See Jira OAuth scopes. `manage:jira-project`, `manage:jira-configuration`, admin, service-management write, and granular scopes are not requested.

Read calls are limited to accessible resources, self, fields, project search, JQL issue search, issue metadata, bounded issue changelog needed for synchronization/action verification, statuses, versions, sprints, and comments. Attachments are represented by metadata only and are not downloaded in H1. Write calls are denied except `PUT /rest/api/3/issue/{issueIdOrKey}` from the approved remediation workflow and dynamic webhook lifecycle endpoints.

The installation stores Atlassian `cloudId`, authorized account ID, consented scopes, refresh-token generation, sandbox project IDs, and encrypted secret reference. The callback rejects missing/mismatched OAuth state, an unexpected callback route, or a previously consumed code. The server exchanges the one-time code only at the configured Atlassian token endpoint using its confidential client authentication and the exact registered redirect URI. After exchange, it enumerates accessible resources and rejects a `cloudId` or account binding that differs from the pending installation.

Refresh-token use is single-flight per installation under a workflow plus row lock. The activity atomically stores the newly returned access token expiry, rotated refresh-token secret reference, and incremented credential generation before releasing the lease. A stale generation cannot overwrite a newer token. Crash recovery reads the latest committed generation and never fans out concurrent refreshes; repeated invalid-grant failure suspends the installation and requires reauthorization.

5.2 Webhooks

The connector registers one dynamic issue webhook per installation with JQL `project IN (ALLOWLIST)` and events `jira:issue_created`, `jira:issue_updated`, and `jira:issue_deleted`. A second webhook for comments is enabled only if comments are included in the tenant's H1 demo data. Dynamic registrations expire after 30 days, so the refresh workflow renews them on day 20 and verifies them daily. Atlassian documents the expiration and refresh endpoint in Jira webhook REST API.

For OAuth 2.0 app webhooks, ingress validates the bearer token signature and claims against the app client secret and expected tenant binding, validates that `matchedWebhookIds` belongs to the installation, and deduplicates on `X-Atlassian-Webhook-Identifier`. The provider identifier is stable across retries. The handler acknowledges only after durable receipt and schedules an authenticated GET; Jira's duplicate and retry behavior is documented in Jira webhooks.

Webhook callback URLs contain an additional random, installation-bound path token. This token is defense in depth and does not replace bearer validation. Unknown project, issue, webhook ID, issuer, cloud ID, or event type is rejected and audited.

5.3 Jira reconciliation and cursors

Initial sync discovers allowlisted projects and fields, then uses ordered JQL:

```
project IN (ALLOWLIST) ORDER BY updated ASC, id ASC
```

Subsequent incremental scans add a lower-bound `updated >= watermark_minus_10_minutes`. The cursor stores the last `(updated,id)`, field schema hash, JQL version, page token, and lookback. The overlap prevents loss at timestamp boundaries; the source key and canonical content hash prevent duplicates. Issue fetches explicitly request only configured fields: key, project, type, summary, description when enabled, status, resolution, priority, labels, assignee, reporter, parent, subtasks, links, sprint, fix versions, due date, created, updated, and comments when enabled.

Every 15 minutes incremental reconciliation validates changes. Daily reconciliation enumerates projects, fields, statuses, and webhooks. Weekly full reconciliation enumerates all in-scope issue IDs and confirms tombstones. The connector respects

`Retry-After`, caps per-installation concurrency, and stores provider request IDs without response content.

6. Source precedence and normalization

Canonical concern	Authority and conflict rule
Jira issue/project fields	Jira is authoritative for issue status, dates, assignee reference, dependencies represented as issue links, sprint, version, and project membership.
GitHub engineering activity	GitHub is authoritative for repository, commit, pull request, review, check, workflow-run, and deployment metadata.
Cross-system project/repository link	Governed manual mapping first; exact configured key second; explicit source link third; inferred text mention remains a low-confidence candidate.
Person identity	Administrator/SCIM link first, exact provider account link second, tenant-verified work email third, review-required candidate last. Display name never auto-merges.
Dates that disagree	Preserve both claims; the domain predicate's registered authority selects the canonical view and exposes the conflict.

All provider text is untrusted. HTML and Atlassian document format are parsed with allowlisted nodes, links are normalized without fetching them, mentions become references rather than instructions, control characters are removed, and rendered output is escaped by the client. Normalization never executes macros, templates, embedded objects, URLs, or code.

7. Tombstones, permission loss, and replay

A delete webhook creates a provisional tombstone and deny barrier, then an authenticated GET confirms deletion. `404` is interpreted using installation scope: if repository/project access also disappeared, the object becomes `inaccessible`; if scope remains and the provider confirms absence, it becomes `deleted`. `403` never proves deletion.

During a full manifest scan, an object missing once is `suspect_missing`; missing in two independent successful scans at least 15 minutes apart is `tombstoned`, unless the provider offers a definitive deletion marker. A later reappearance creates a new source version and closes the tombstone; it does not erase history.

Permission loss is handled before content cleanup. The policy service receives an installation/ACL version increment, cached reads are invalidated, and inaccessible evidence is denied immediately. The retention workflow then archives or deletes payloads according to tenant policy. Replaying an ingest run is allowed only from immutable envelopes into a new parser/projection generation and must not re-enable tombstoned access.

8. H1 Jira remediation command

The only external mutation is `jira.issue.update` against an existing issue in an administrator-configured sandbox project. It may replace exactly three fields: `labels`, `duedate`, and `priority`. No transition, assignee, description, comment, attachment, issue creation, or deletion is permitted. For the frozen H1 workload, the only target is synthetic issue `AST-142` in project `AST` for tenant `10000000-0000-4000-8000-000000000001` (`tnt_aster`) through connector installation `30000000-0000-4000-8000-000000000001` (`con_aster_jira`).

8.1 Preview

The preview service performs an authenticated GET and captures this frozen H1 before snapshot:

```
{
  "issueKey": "AST-142",
  "version": 7,
  "fields": {
    "duedate": "2026-08-07",
    "priority": {"id": "3", "name": "Medium"},
    "labels": ["identity", "orion"]
  }
}
```

The exact canonical approved payload is:

```
{
  "action": "jira.issue.update",
  "connectorInstallationId": "30000000-0000-4000-8000-000000000001",
  "expectedIssueVersion": 7,
  "issueKey": "AST-142",
  "projectKey": "AST",
  "set": {
```

```
    "duedate": "2026-07-31",
    "labels": ["digital-twin-remediation", "identity", "orion"],
    "priorityId": "2"
  },
  "tenantId": "10000000-0000-4000-8000-000000000001"
}
```

`tenantId` is inserted from server context into the internal approved payload; it is never accepted from the public preview request. Labels are sorted and deduplicated, existing labels remain, and `digital-twin-remediation` is the only system-added label. Priority IDs and due dates are validated against the current Jira field metadata and tenant action policy. The preview also binds reason, evidence IDs, scenario ID, policy version, credential generation, required roles, expiry, rollback fields, stable action-level idempotency key, and the SHA-256 of RFC 8785 canonical payload JSON. It is immutable. Any field, target, connector, source version, credential, policy, tenant, idempotency key, or digest change invalidates all approvals.

8.2 Approval and execution

Two distinct active human actors approve the exact digest within 15 minutes: one with the operations approver role and one with the security approver role. Self-approval, duplicate actors, delegated bot approval, expiry, or role revocation fails closed. Execution locks the command row, verifies the idempotency key, both role-distinct approvals, current connector, tenant, and project allowlist, then refetches the issue and its current change watermark. If version is not `7` or `labels`, `duedate`, or `priority` differ from the before snapshot, it returns `409 source_changed` and requires a new preview.

The stable idempotency key is derived from tenant, action, connector, issue key, expected version, and canonical payload hash. Jira's documented edit operation accepts field updates and can return a conflict, but this design does not assume an undocumented atomic compare-and-swap contract; see the Jira edit issue operation. The connector records `send_started` durably before handing one field-limited `PUT` to the network and never sends a second `PUT` for that command. It immediately refetches the issue and relevant change history, then records provider request ID when available, exact payload, before/after snapshot, observed result, actor IDs, approval IDs, policy version, and audit event. Concurrent callers return the ledger state or original receipt. A provider conflict produces `source_changed`; an overlapping same-field edit or timeout without conclusive history enters `verification_required`/`concurrent_change_detected`, pages an operator, and never blind-retries or auto-rolls back. This provides application-level idempotency and at-most-once send, while making the residual external race explicit.

8.3 Compensation

Rollback is a new dual-approved command linked to the receipt. It is allowed only within 24 hours and only when the current issue version and `labels`, `duedate`, and `priority` exactly match the recorded after snapshot. It restores all three recorded before values and verifies them. Divergence produces `409 compensation_conflict` for manual resolution; the system never overwrites later human work. A successful retry returns the original compensation receipt. A failed or partial compensation retains both snapshots and pages an operator.

9. Consistency, failure handling, and operations

Failure	Behavior
Webhook invalid or oversized	Reject before enqueue, record hashed security metadata, and never log the body.
Provider 401	Refresh once when safe; on failure suspend installation and notify an administrator.
Provider 403	Re-evaluate scopes and source ACL; do not treat as deletion or retry indefinitely.
Provider 404	Confirm installation scope and object identity before a tombstone.
Provider 429	Honor `Retry-After`, reduce concurrency, preserve cursor, and prioritize revocations.
Provider 5xx/network failure	Exponential backoff with jitter and workflow deadline; surface stale watermark.
Cursor corrupt/incompatible	Stop that stream, retain last good checkpoint, and start a reviewed bounded rescan.
Parser regression	Quarantine the new observation; keep last valid canonical version and allow generation rollback.
Database/object storage unavailable	Do not acknowledge new webhooks until durable receipt is possible; provider retries or reconciliation recovers.
Revoked credential mid-page	Abort the page, deny reads immediately, and never commit a partial cursor.

Metrics include delivery acceptance/rejection, deduplication ratio, webhook age, provider request latency/status, rate-limit remaining, token-refresh failures, cursor age, objects scanned/changed/quarantined/tombstoned, reconciliation drift, parse duration, outbox lag, freshness by tenant, and action/compensation outcomes. Alerts fire for p95 freshness over 15 minutes for two intervals, webhook refresh inside five days of expiry, reconciliation drift, repeated signature failure, credential suspension, and any ambiguous external write.

Logs and traces include hashed tenant and installation IDs, provider request ID, ingest run, activity attempt, cursor version, and payload hash. They exclude OAuth codes, tokens, webhook secrets, raw bodies, issue descriptions, user emails, and source excerpts.

10. Testing and acceptance

ID	Acceptance criterion
AC-CON-001	Invalid, expired-token, replayed, altered, unmapped/wrong-installation, and wrong-tenant GitHub/Jira webhooks are denied or idempotently deduplicated as appropriate and audited; no GitHub write permission or non-allowlisted route is available.
AC-CON-002	Missed webhook, partial page, process termination, and stale cursor recover through reconciliation without loss or duplicate canonical state.
AC-DATA-001	Duplicate and randomly ordered webhooks plus overlapping polls produce byte-identical normalized observations and canonical checksums.
AC-ACT-001	One approver, duplicate actor, expired approval, changed payload/version, revoked role, wrong project, altered field, or reused key causes zero Jira writes.
AC-ACT-002	Two valid role-distinct approvals plus concurrent execution requests produce exactly one Jira field update and the same durable receipt.
AC-ACT-003	Ambiguous timeout triggers verification; conflict-free compensation restores due date, priority, and labels, while later human edits prevent overwrite.
AC-REL-001	H1 synthetic load meets 15-minute freshness and the committed end-to-end latency envelope while respecting provider rate limits.

Tests use provider contract fixtures plus record/replay suites with secrets removed. Fuzzing covers JSON, Unicode, Atlassian document format, headers, pagination tokens, and timestamps. Chaos tests inject rate limits, expired tokens, webhook gaps, duplicate pages, clock skew, worker termination, network partitions, provider 5xx responses, and object-store failures. A nightly

reconciliation oracle compares provider fixtures with canonical source manifests.

11. Risks and evolution

ID	Risk	Mitigation
RSK-005	Connector credential compromise or provider scopes broader than the H1 endpoint subset.	Tenant keys, exact scopes, local endpoint/egress allowlist, isolated workers, rotation, revocation, and audited calls.
RSK-006	Approval confused deputy or payload substitution.	Canonical exact payload, role-distinct approvers, 15-minute expiry, policy/source recheck, and immutable arguments.
RSK-013	Jira timeout or replay creates a duplicate external action.	Durable idempotency ledger, source refetch, provider correlation, ambiguous-state verification, receipt, and compensation.
RSK-018	Webhook/backfill backlog causes stale data.	Fair priority queues, separate historical concurrency, freshness metrics, cursor replay, backpressure, and reconciliation.
RSK-003	Compromised or stale source ACL data leaks through derived claims.	Immediate deny barrier, immutable evidence, quarantine, deterministic normalization, conflict preservation, and rapid credential revocation.

Additional connectors use this contract only after a connector-specific threat model, exact permission list, source precedence, deletion semantics, provider rate model, fixtures, and mutation review are committed. H2 connector reads are Committed by domain need; every new external mutation remains separately gated and disabled by default.

CH-07 - AI Agents, Reasoning, and Evaluations

Status: Committed | Owners: AI Platform, Applied AI, Security | Last reviewed: 2026-07-13

AI Agents, Reasoning, and Evaluations

1. Purpose and safety boundary

The AI subsystem converts an authorized question into evidence retrieval, structured analysis, simulation requests, and reviewable drafts. It is not an authority system. A model cannot authenticate a user, choose a tenant, expand a delegation, grant access, establish a canonical fact, approve its own action, or call an external mutation without deterministic policy and approval services.

Agents are bounded capability profiles, not artificial executives. Labels such as CEO, HR, legal, or finance may appear as future user-facing viewpoints, but they do not confer authority and are not H1 runtime roles.

ID	Requirement
REQ-AI-001	AI workloads MUST use the Responses API and Agents SDK behind capability, policy, budget, and evaluation interfaces.
REQ-AI-002	Agents MUST be versioned capability profiles with explicit tools, memory, context limits, handoffs, retries, evaluation, and termination.
REQ-AI-003	Deterministic authorization, retrieval, validation, execution, verification, citation, approval, and audit MUST surround model behavior.
REQ-AI-004	Connector/user content MUST NOT become privileged instructions or grant tools, and model output MUST be schema-validated.
REQ-AI-005	Memory MUST be explicit, tenant-scoped, permission-trimmed, versioned, and deletable with no invisible self-modification.
REQ-AI-006	The platform MUST retain evidence, structured rationale, action traces, and model versions rather than expose/store private chain-of-thought.
REQ-AI-007	Models, prompts, tools, and fallbacks MUST pass use-case safety, quality, latency, and cost evaluations before promotion.
REQ-AI-008	A child agent MUST inherit the intersection of user, tenant, caller, workflow, and tool-policy authority.

ID	Requirement
REQ-TEN-005	Customer data MUST NOT be used for cross-tenant retrieval, memory, evaluation, analytics, or training without separately recorded opt-in design.
REQ-ACT-001	External execution MUST bind the exact tenant, actor, credential, target, arguments, expiry, policy, and idempotency key.
REQ-ACT-004	Employment, legal, financial, production, identity, destructive, and security-control decisions MUST remain non-executable through H3.
QAR-AI-001	Missing, conflicting, or inaccessible evidence MUST produce cited uncertainty or abstention and pass the committed grounding thresholds.
QAR-AI-002	Authorization and prompt-injection suites permit zero policy bypasses.
QAR-PERF-003	H1 cited answers complete within 20 seconds at p95 with a hard per-run spend budget.
QAR-COST-001	Time, token, spend, tool, and concurrency budgets MUST terminate loops safely.

2. Runtime architecture

The Python AI worker uses the OpenAI Responses API through an internal `ModelGateway`. The OpenAI Agents SDK supplies the server-owned agent loop, tool dispatch, handoffs, guardrails, and traces; PostgreSQL and Temporal remain authoritative for durable run state and approvals. The public API never exposes provider response IDs, API keys, raw prompts, or provider-specific tool objects.

OpenAI describes the Responses API as the agentic interface for multi-turn and tool-using applications, and its current model guidance recommends Responses for reasoning and tool calling. See Responses API migration guidance and current model guidance. The Agents SDK guide explicitly supports server ownership of deployment, tools, storage, and approval decisions, which is the boundary used here.

2.1 Model gateway contract

`ModelGateway.run(request)` accepts:

- `tenant_context_ref` and `actor_context_ref`, both resolved server-side;
- `capability_profile_id` and immutable profile version;
- `prompt_template_id` and content hash;
- ordered user input and authorized context references;
- strict output schema ID;
- allowlisted tool schemas and per-tool call limits;
- model policy ID, latency/cost/token budget, deadline, and cancellation token;
- trace and audit IDs;
- a privacy-preserving `safety_identifier` derived as an HMAC of tenant and actor IDs.

It returns `AgentRunResult` with structured output, model and revision actually used, usage, finish reason, tool-invocation references, validation results, citations, safety flags, and terminal state. Provider exceptions are normalized. Raw provider request/response payloads are not retained by default; an explicitly opted-in diagnostic sample may be encrypted under a restricted key, assigned a short TTL, and accessed only through time-limited break glass.

The OpenAI request defaults to `store: false`; the application owns conversation state and supplies only the context required for that turn. Any use of provider-side storage requires an approved data-processing profile and tenant configuration. API credentials are server-side secret references, isolated per environment, never placed in a tool schema or prompt.

2.2 Model policy

Workload	H1 family default	Starting reasoning	Promotion objective
High-volume extraction and classification	`gpt-5.6-luna`	`none` or `low`	Schema validity, field accuracy, cost, and latency
Entity-resolution explanation and graph verification	`gpt-5.6-terra`	`low`	Pairwise decision accuracy and evidence use
Grounded query/research and mitigation drafting	`gpt-5.6-terra`	`medium`	Citation, abstention, usefulness, and latency
Scenario compilation and explanation	`gpt-5.6-terra`	`medium`	Operation accuracy and unsupported-change rejection
Difficult orchestration, adjudication, and eval grading	`gpt-5.6-sol`	`high`; `max` only after measurement	Quality-first benchmark with explicit cost ceiling

OpenAI currently identifies `gpt-5.6-sol` for frontier capability, `gpt-5.6-terra` for balanced capability and cost, and `gpt-5.6-luna` for efficient high-volume work in model guidance. These are workload defaults, not unconditional production aliases. Production configuration records an explicitly evaluated model revision where available, the prompt and tool-schema hashes, reasoning setting, and fallback set.

A fallback is eligible only if it has passed the same capability eval gate and data-control requirements. The gateway may fall back for transient availability or rate-limit failure, never to bypass a safety refusal, tool denial, data residency rule, or output validation error. If no approved candidate remains, the run fails closed with a retryable or terminal status.

3. Durable run model

Type	Required fields
`AgentRun`	Tenant, actor, capability/profile version, request hash, state, model policy, budgets, deadlines, cancellation state, parent run, delegation, input context refs, output ref, created/started/completed timestamps.
`ToolInvocation`	Run, tool/schema version, arguments hash and encrypted arguments ref, policy decision, authorization version, attempt, deadline, result hash/ref, side-effect class, status, error, started/completed timestamps.
`AgentHandoff`	From/to capability, purpose enum, reduced delegation, remaining budgets, summary ref, accepted/rejected status.
`RunCitation`	Output claim path, evidence ID, authorized locator, entailment score, verifier status.
`RunEvaluation`	Eval suite/version, dataset item, grader versions, scores, annotations, and release comparison.

Run states are `queued`, `retrieving`, `reasoning`, `awaiting_tool`, `awaiting_confirmation`, `awaiting_approval`, `verifying`, `completed`, `abstained`, `cancelled`, `expired`, or `failed`. State transitions use optimistic concurrency and append an audit event. A process restart resumes from durable state and never silently repeats a side-effecting tool.

The system does not request, display, or persist private chain-of-thought. It stores structured plans, alternatives, assumptions, evidence links, tool choices, concise decision rationale, verifier findings, and provider-supplied reasoning summaries when permitted. This provides auditability without making hidden reasoning a product contract.

4. Capability profiles

Every profile declares purpose, allowed tools, output schema, retrieval limits, memory scope, maximum handoffs, token/cost/time budgets, retry policy, confidence thresholds, and termination conditions.

Profile	Purpose	Allowed tools	Memory and authority	Termination
`query_research`	Answer organizational questions from accessible evidence.	Evidence search, entity lookup, bounded graph traversal, claim fetch, citation verifier.	Run context plus tenant session summary; read-only.	Complete with citations, abstain, or budget/deadline.

Profile	Purpose	Allowed tools	Memory and authority	Termination
`extraction_resolution`	Convert untrusted source observations into candidate claims and duplicate candidates.	Schema registry, ontology lookup, normalization helpers, candidate lookup.	Current source object only; writes candidates through a validated ingestion command, never canonical merges.	Valid structured candidates or quarantine.
`graph_verification`	Check whether a proposed answer or dependency follows from claims and graph paths.	Claim/evidence fetch, bounded path query, constraint checker.	Read-only; cannot add or remove graph elements.	Verified, contradicted, insufficient evidence, or limit.
`scenario_planning`	Translate a confirmed user intent into scenario operations and request simulation.	Snapshot fetch, scenario schema validator, simulation runner.	Scenario-local; cannot infer person productivity or employment outcome.	Valid preview needing confirmation, result, or rejection.
`mitigation_drafting`	Draft evidence-backed remediation options.	Answer context, simulation comparison, policy catalog, Jira preview tool.	Draft-only. It cannot approve or execute.	Ranked alternatives plus assumptions and evidence.
`action_execution`	Submit an already approved exact Jira command to the deterministic action service. This H1 profile is a policy/trace wrapper and does not invoke a model.	Approval verification and `execute_approved_action` only.	No conversational memory, model generation, or argument editing. Authority is the attached one-time execution grant.	One receipt, conflict, expiry, cancellation, or failure.

Extraction, query, and scenario profiles may run in parallel only when their inputs are independent. The orchestrator records a join policy and cancels remaining work when the deadline or budget is exhausted. Debate and voting are not default correctness mechanisms. For high-impact ambiguity, the system asks the user or routes independently produced candidates to a deterministic verifier; it does not manufacture consensus.

Self-review is implemented as a separate verification pass over structured output, evidence, policy, and tool history. The original model cannot mark its own output approved. Repeated reflection is capped at one repair attempt for H1 unless the profile specifies a measured benefit.

5. Retrieval and grounding

The retrieval service receives tenant and actor context from middleware, resolves current policy and source ACLs, and then performs lexical, vector, and bounded graph retrieval. A model never supplies SQL, Cypher, tenant ID, ACL predicate, object URI, or unbounded traversal depth.

The H1 retrieval envelope is limited to 40 evidence chunks, 20 entities, 100 claims, 500 graph nodes, depth 4, 50,000 input tokens, and 5 seconds. Each item contains a stable evidence ID, authorized display locator, classification, source authority, observed/effective time, content hash, and an explicit `UNTRUSTED_SOURCE_DATA` label. Retrieval that exceeds the limit returns a partial marker and refinement options.

An answer output uses this schema shape:

```
{
  "answer": "string",
  "claims": [
    {
      "text": "string",
      "evidence_ids": ["UUID"],
      "status": "supported|conflicted|assumption|unsupported"
    }
  ],
  "assumptions": ["string"],
  "uncertainty": {"level": "low|medium|high", "reasons": ["string"]},
  "missing_data": ["string"],
  "recommended_next_steps": ["string"]
}
```

The citation verifier checks accessibility again, locator integrity, evidence existence, textual entailment, temporal fit, and conflict disclosure. A claim without valid support is removed, rewritten as an assumption, or causes abstention. Confidence labels derive from calibrated verifier bands and data completeness; the model cannot assign its own final confidence score.

6. Tool contract and authority

6.1 Tool schemas

Tools use JSON Schema with `additionalProperties: false`, explicit enums, lengths, numeric bounds, and required fields. Structured Outputs enforce schema adherence for model responses, while function calling is used to bridge the model to application tools; this distinction follows OpenAI structured output guidance.

H1 tools are:

Tool	Side-effect class	Key limits
`evidence.search`	Read	Query <= 500 characters; filters are enums/IDs; result cap 40.
`entity.get`	Read	At most 20 authorized entity UUIDs.
`graph.traverse`	Read	Registered edge types only; depth <= 4; nodes <= 500; timeout <= 2 seconds.
`claims.verify`	Read/compute	At most 25 output claims and 100 evidence links.
`scenario.compile`	Draft	Registered operation enums only; no free-form executable expression.
`simulation.run`	Compute	Confirmed scenario ID; engine limits from CH-08.
`jira.remediation.preview`	Draft/read	Frozen H1 target `AST-142`; accepts authorized scenario/evidence references only. The deterministic service fetches the current issue and returns the exact three-field diff, expected source version, and immutable digest.
`approval.request`	Workflow	Exact preview digest; the model cannot name approvers or approve.
`approved_action.execute`	External write	One-time server-issued grant bound to command, digest, tenant, actor, and expiry. Arguments cannot be model-edited.

There is no generic shell, HTTP fetch, SQL, Cypher, filesystem, email, browser, or provider SDK tool. Tool output is schema-validated, size-bounded, classified, and marked untrusted before reentering model context.

6.2 Tool-call decision path

1. Resolve the run, actor, tenant, profile, remaining budget, and cancellation state.
2. Validate the exact arguments against the registered schema and canonicalize them.
3. Evaluate policy against the current actor, delegation, tool, resources, purpose, and side-effect class.
4. Enforce rate, call-count, traversal, content, and egress limits.
5. For sensitive tools, pause for the required approval or confirmation. A pause persists the exact argument digest.
6. Invoke through a tenant-scoped service identity and deadline.
7. Validate and classify the result, append audit/trace records, and decrement budgets.
8. On retry, use the invocation ID and tool-specific idempotency contract. Side effects never retry without verification.

An agent handoff receives the intersection of the caller delegation and callee profile. It gets a structured, provenance-bearing summary rather than the complete prompt by default. The handoff is rejected when it would increase authority, cross a classification boundary, exceed maximum depth 2, or create a cycle.

7. Human confirmation and approval

Read-only questions need no per-tool approval after the user has authenticated and the application has authorized retrieval. However, every MCP operation exposed outside this trusted internal tool layer is configured for explicit user approval. OpenAI warns that remote MCP servers can see sensitive context and supports developer-required approval; see MCP and connector guidance.

Scenario compilation requires a user confirmation of structured changes before simulation. The H1 Jira remediation command requires two distinct authenticated human approvers. Any future external or high-impact command would require at least the same control, but employment, legal, financial, production, identity, destructive, and security-control decisions remain non-executable through H3 under REQ-ACT-004.

An `ApprovalRequest` binds tenant, command, exact canonical payload digest, before snapshot hash, policy version, requester, reason, evidence IDs, impact, expiry, and required approver roles. Approval expires after 15 minutes for H1 Jira actions. Any byte-significant payload change, source version change, policy change, credential change, or scope change voids prior approvals. A one-time execution grant is minted only after two valid approvals and is consumed atomically with the command ledger.

8. Prompt-injection and model-security controls

Connector content, retrieved text, tool output, plugin metadata, URLs, and user uploads are hostile data. Controls are layered:

1. Stable developer instructions contain policy and capability boundaries only. Untrusted values are never interpolated into developer messages. OpenAI explicitly warns against putting untrusted variables in higher-priority messages in agent safety guidance.
2. External content is delimited, labeled, source-attributed, size-limited, and passed as user/tool data. Instructions found inside it have no authority.
3. An extraction stage converts source content to strict schemas. Only necessary fields flow to downstream planning. Free-form text cannot become a tool name, URL, recipient, tenant, command, or approval argument.
4. Tools are least-privilege, typed, server-authorized, and egress-restricted. Retrieval tools cannot mutate; drafting tools cannot execute; execution cannot alter its approved payload.
5. Secrets, hidden ACLs, raw credentials, internal policy text, other tenants, and irrelevant source content are absent from model context.
6. Outputs are schema-validated, citation-verified, policy-checked, escaped for rendering, and scanned for secret patterns and disallowed instructions.
7. Suspicious input reduces tool availability, disables external tools, marks the trace, and may require human review. It never relaxes controls.
8. Adversarial datasets cover direct and indirect injection, encoded instructions, tool-output poisoning, citation spoofing, data exfiltration, instruction hierarchy confusion, and multi-turn persistence.

OpenAI recommends structured data flow, tool confirmations, guardrails, trace graders, and evals as complementary controls, while noting that they do not eliminate risk; see Safety in building agents.

9. Memory and context lifecycle

Memory has four explicit scopes:

- `run`: tool results and structured state until terminal state plus 30-day debugging retention;
- `session`: user-approved conversation summary, citations, and unresolved questions, tenant and actor bound, 24-hour default TTL;
- `tenant_knowledge`: only canonical entities, claims, and evidence managed by CH-05, never free-form model memory;
- `evaluation`: de-identified or synthetic cases in a separate governed store, never silently populated from customer traffic.

Summaries are model-generated candidates and cannot create facts. Before reuse they are authorized, size-bounded, and refreshed against current claims. Revoked evidence invalidates dependent summaries. Users can start a stateless run, inspect stored summaries, and delete session memory. No memory follows a user to another tenant.

Context assembly is deterministic: policy/profile, user goal, current delegation, structured plan state, authorized evidence, and bounded prior summary. It records each included item and exclusion reason. Token pressure drops lowest-ranked evidence before policy, delegation, or user goal and reports truncation.

10. Failure handling, budgets, and observability

Failure	Required behavior
Model timeout or provider 5xx	Retry within profile budget, then use only an eval-approved fallback; otherwise fail with no fabricated answer.
Rate limit	Honor retry guidance, queue within deadline, or return retryable status. No unapproved provider switch.
Safety refusal	Return the refusal or safe alternative; do not retry a different model to evade it.
Invalid structured output	One schema-focused repair attempt with no new authority, then fail or abstain.
Tool timeout	Report partial progress, verify any ambiguous side effect, and never assume success.
Authorization/ACL change	Cancel pending tool calls, discard inaccessible context, and re-plan from current permissions.
Citation failure	Remove the claim or abstain; never expose an inaccessible locator.
Budget exhaustion	Terminate with partial, clearly labeled results and remaining questions; do not exceed hard limits.
Kill switch	Stop new runs, cancel cancellable calls, prevent approvals/execution, and retain audit state.

H1 defaults per cited-answer run are 20-second wall time, 80,000 total model input tokens, 8,000 output tokens, 12 tool calls, 2 handoffs, depth 2, and a tenant-configured currency budget. Extraction jobs have separate batch budgets. Budget changes are policy version changes, not prompt instructions.

Metrics include runs by capability/model/revision/outcome, token and cost distribution, time to first/last output, tool selection and argument errors, handoff depth, schema-repair rate, citation coverage/precision, abstention, retrieval empty rate, injection flags, policy denials, approval waits, fallback rate, and cancellation latency. Traces store hashes and IDs by default; source text and prompts use a restricted encrypted sampling policy with tenant opt-in and short retention.

11. Evaluation and release gates

OpenAI recommends task-specific evals, continuous testing, production-representative datasets, automated scoring, and calibration against human judgment in evaluation best practices. The platform implements these practices without depending on any provider-hosted eval product.

11.1 Datasets

- deterministic synthetic organizations with a complete ground-truth oracle;
- golden organizational questions with required, optional, conflicting, stale, and inaccessible evidence;
- extraction fixtures across GitHub, Jira, Unicode, malformed, and adversarial payloads;
- entity-resolution pairs including collisions, renamed users, shared names, and forbidden cross-tenant pairs;
- scenario-language examples mapped to exact structured operations;
- tool-routing and exact-argument cases;
- prompt-injection and data-exfiltration attacks in user, source, citation, and tool-result positions;
- model/provider failure, truncation, refusal, and malformed-output cases;
- human-reviewed production-like cases only under explicit governance and de-identification.

11.2 Metrics and H1 thresholds

Metric	Release threshold
Strict output schema validity after first attempt	At least 99.5%
Extraction field micro-F1 on required fields	At least 0.98
Entity-resolution precision for automated merges	At least 0.999; recall is reported but never traded for lower precision
Tool selection accuracy	At least 0.98
Exact tool-argument match	At least 0.99 on executable/draft action fields
Citation precision	At least 0.98
Citation coverage for factual answer claims	At least 0.95
Unsupported material claim rate	Less than 0.01
Unsupported-claim abstention	At least 0.95
Scenario operation exact match	At least 0.97
Cross-tenant disclosure or unauthorized action	Exactly 0 across the complete security suite
Adversarial injection prevention	100% for privileged-tool and data-exfiltration cases; all failures block release
H1 cited-answer latency	p95 under 20 seconds at target concurrency

Automated graders use deterministic checks where possible: schema, exact arguments, graph oracle, citation IDs, temporal answers, and policy decisions. Model graders are limited to rubric-scored semantic qualities, use a pinned grader revision, and are calibrated against at least two human reviewers. Disagreement samples enter adjudication; a model never grades its own release alone.

A model, prompt, retrieval, schema, or agent change runs the full affected suite. Promotion requires all safety invariants, all hard thresholds, no statistically significant critical regression, and documented cost/latency comparison. A canary starts at 5% of eligible synthetic or opted-in traffic and automatically rolls back on safety failure, schema regression over 0.5 percentage points, citation precision below threshold, or p95 latency over SLO for two windows.

11.3 Acceptance criteria

ID	Acceptance criterion
AC-AI-001	Golden cited answers meet citation precision/coverage thresholds and every material claim is reproducible from profile, prompt, model, tool, context, and evidence hashes.
AC-AI-002	Missing, conflicting, restricted, and stale evidence cases produce calibrated uncertainty or abstention; mid-run revocation prevents tool, citation, and memory reuse.
AC-AI-003	Direct, indirect, encoded, and multi-turn injection cannot select a tenant, reveal restricted evidence, alter an approved payload, or invoke an unauthorized tool.
AC-AI-004	Tool choice and exact arguments meet Section 11.2; profiles reject undeclared tools and handoffs preserve equal-or-smaller authority/budget.
AC-ACT-001	A Jira command cannot execute from model text; only a current exact-payload approval and one-time immutable grant reach the action service.
AC-REL-001	The committed cited-answer workload meets the 20-second p95 target.
AC-REL-003	Cancellation, deadline, budget, model outage, malformed output, and ambiguous tool failure reach the documented safe terminal state.

Promotion evidence also reports quality, safety, latency, and cost against the current baseline. It is rejected when any Section 11.2 gate fails.

12. Risks and evolution

ID	Risk	Mitigation
RSK-004	Indirect prompt injection causes data exfiltration or reaches a privileged tool.	Untrusted-data separation, strict schemas, least-privilege tools, egress control, approval digest, validation, and adversarial gates.
RSK-012	Provider/model outage or revision silently changes behavior.	Evaluated pinned revisions, immutable configuration hashes, canaries, regression gates, queue/fail closed, and no unapproved fallback.
RSK-003	Long-running context retains stale or revoked source data.	Server-owned scoped memory, authorization at assembly/serialization, short TTL, provenance, invalidation, and side-channel tests.
RSK-006	Model output substitutes an approved payload or becomes a confused deputy.	Exact canonical digest, role-distinct humans, deterministic executor, one-time grant, and execution-time policy check.
RSK-007	Sensitive employment inference emerges from ordinary organizational data.	Prohibited-use schemas, aggregate-only simulation, tool/prompt controls, output classifiers, policy review, and security evals.

Native OpenAI multi-agent features, remote MCP, additional model providers, fine-tuning, persistent provider state, voice agents, and autonomous background actions are Provisional. Each requires a separate data-flow review, threat model, eval suite, cost envelope, rollback, and explicit product horizon. Autonomous organizations, AI executives, and workforce prediction remain Research and have no production authority.

CH-08 - Simulation and Prediction

Status: Committed | Owners: Simulation Platform, Applied AI, Product Analytics | Last reviewed: 2026-07-13

Simulation and Prediction

1. Purpose, claim boundary, and non-goals

H1 implements one defensible simulation: a seeded PERT/Monte Carlo forecast of launch timing over a task dependency DAG. It compares an immutable baseline with a user-confirmed scenario and explains the conditional distribution of completion dates.

The result is not a promise, causal conclusion, or validated prediction of human behavior. It is conditional on task estimates, dependencies, calendars, and stated assumptions. The product MUST call it a forecast or simulation, not an objective prediction of the organization.

Individual burnout, attrition, performance, productivity, hiring, compensation, emotion, health, misconduct, or suitability scoring is prohibited through H3. H1 has no person-level rate, performance, availability, or productivity parameter. Mergers, layoffs, hiring, customer churn, pricing, security-loss magnitude, cash flow, and market simulations remain Research until separately governed models and validation exist.

ID	Requirement
REQ-SIM-001	H1 MUST use a seeded PERT/Monte Carlo dependency-DAG scheduler rather than model-authored mathematics.
REQ-SIM-002	Scenario inputs, assumptions, interventions, snapshot, engine version, and seed MUST be typed, confirmed, and persisted.
REQ-SIM-003	Simulation MUST return p50, p80, p95, critical path, blockers, sensitivity, uncertainty, assumptions, and missing-data warnings.
REQ-SIM-004	Synthetic scenarios MUST be labeled demonstrative and MUST NOT claim causal or production predictive validity.
REQ-SIM-005	Later forecasts MUST define outcome, window, lineage, calibration, backtesting, drift, abstention, fairness, explanation, and prohibited use before promotion.
QAR-SIM-001	Equal snapshot, scenario, engine version, calendar, and seed MUST produce an identical canonically rounded output digest.

ID	Requirement
QAR-PERF-002	The frozen Orion 2.0 H1 workload with 50,000 trials MUST complete in less than 10 seconds at p95.

Every run binds an immutable evidence-backed snapshot, confirmed scenario, seed, trial count, engine/container version, and calendar version. Invalid dependency graphs fail explicitly. Baseline/scenario comparisons use common random numbers for unchanged work items.

2. Ownership and execution

The Python simulation worker owns validation, sampling, scheduling, aggregation, and comparison. PostgreSQL owns `Scenario`, `SimulationSnapshot`, `SimulationRun`, and result metadata. Immutable snapshots and large result artifacts are stored in S3-compatible storage. Temporal owns long-running execution, cancellation, retries, and progress. The AI subsystem can produce a typed draft and a prose explanation; it cannot change numeric results.

The engine is a pure function:

```
result = simulate(snapshot, scenario, seed, sample_count, engine_version)
```

It performs no provider reads, graph writes, model calls, or external actions. All source resolution occurs before snapshot sealing.

3. Canonical input types

3.1 SimulationSnapshot

```
{
  "schema_version": "1.0",
  "snapshot_id": "UUID",
  "tenant_id": "server-derived UUID",
  "project_id": "UUID",
  "as_of": "RFC3339 timestamp",
  "project_start": "RFC3339 timestamp",
  "target_date": "ISO YYYY-MM-DD or null",
  "simulation_model_version": "pert-monte-carlo/1.0.0",
  "parameter_set_version": "beta-pert-lambda-4/1.0.0",
  "default_seed": "unsigned 64-bit decimal string",
  "timezone": "IANA timezone",
  "timezone_database_version": "IANA tzdb version",
  "calendar": {
    "version": "string",
    "working_weekdays": [1, 2, 3, 4, 5],
    "workday_start": "09:00",
    "hours_per_workday": 8,
    "holidays": ["YYYY-MM-DD"]
  },
  "team_capacities": [],
  "tasks": [],
  "dependencies": [],
  "assumptions": [],
  "warnings": [],
  "evidence_ids": [],
  "canonical_sha256": "hex"
}
```

`tenant_id` is stored for isolation but is omitted from model-visible content and cannot be supplied by a public caller. `canonical_sha256` is SHA-256 over RFC 8785 canonical JSON excluding the hash field. Sealing records all entity, claim, evidence, ontology, ACL, numeric-model, parameter-set, calendar, timezone-database, and default-seed versions used. Later source or numeric-input changes create a new snapshot.

3.2 Task and dependency

```
{
  "work_item_id": "stable UUID",
  "source_key": "provider display key or null",
  "label": "display-only string",
  "state": "not_started|in_progress|blocked|completed|cancelled",
  "team_id": "aggregate team UUID",
  "remaining_duration": {
    "optimistic": 2.0,
    "most_likely": 3.0,
    "pessimistic": 7.0,
    "unit": "workday",
    "source": "explicit|confirmed_imputation"
  },
  "earliest_start": "RFC3339 timestamp or null",
  "actual_finish": "RFC3339 timestamp or null",
  "external_blocker": false,
  "external_blocker_until": "RFC3339 timestamp or null",
  "evidence_ids": ["UUID"]
}
```

A dependency is `{predecessor_work_item_id, successor_work_item_id, type:"finish_to_start", lag_workdays, source_relationship_id, evidence_ids}`. H1 supports finish-to-start dependencies with non-negative lag only. CH-05 `DEPENDS_ON consumer->prerequisite` relationships are deliberately inverted during snapshot compilation to `{predecessor=prerequisite, successor=consumer}`; `BLOCKS blocker->work` retains its direction. Duplicate edges are canonicalized to the greatest lag after emitting a warning. Self-edges are invalid.

Each aggregate team has `{team_id, parallel_capacity, availability, evidence_ids}`. `parallel_capacity` is an integer from 1 to 100 representing simultaneous scheduling slots. `availability` is an aggregate scenario coefficient in `(0,1]` that scales the team's task durations; it is not derived from or exposed as any person's productivity. Team membership, person hours, and person-level rates are absent from simulation input.

Completed tasks have zero remaining duration and a known `actual_finish` at or before `as_of`. Cancelled tasks have zero remaining duration and are absent from scheduling but retained in the snapshot audit. A cancelled task cannot remain an endpoint of an active scheduling dependency; before sealing, the snapshot compiler requires an explicit source resolution or a user-confirmed assumption that excludes the edge and records it in the snapshot. In-progress tasks require an estimate of remaining work; the engine never converts percent-complete to remaining time. All durations are finite numbers in `[0, 1300]` workdays and satisfy `optimistic <= most_likely <= pessimistic`. `external_blocker_until` is an evidence-backed earliest-release constraint. When `external_blocker=true` and its release date is unknown, the compiler emits a missing-data warning and the run is explicitly conditioned on immediate release at `as_of`; it does not silently invent a date. Any task in `blocked` state without a release constraint must carry a confirmed assumption that blocker delay is already included in its three-point remaining estimate, or sealing stops for clarification.

H1 synthetic fixtures provide explicit three-point estimates. If a real source provides only one positive estimate `e`, the compiler may offer, but not silently apply, the documented imputation `(0.8e, e, 1.5e)`. The user must confirm it, `source` becomes `confirmed_imputation`, and the forecast carries a high-visibility assumption. A task with no estimate blocks sealing.

3.3 Scenario

A confirmed scenario is an immutable envelope containing `scenario_id`, baseline `snapshot_id` and hash, name, target-date assertion, simulation-model version, calendar version, compiler version, unsigned 64-bit decimal-string `seed`, `sample_count`, ordered interventions, assumptions, confirmation actor/time, and canonical digest. It references the sealed snapshot rather than duplicating its tasks, dependencies, or capacities. In H1 its model, calendar, and seed values are consistency assertions that MUST equal the snapshot values; changing one requires a new snapshot. H1 committed runs require `sample_count=50000`; a separately labeled preview may request 1,000 to 49,999 samples but cannot produce the committed demo result or an approval-bearing remediation. The engine executes exactly `sample_count` numbered iterations.

The intervention list permits only these typed operations:

- `set_duration_estimate(work_item_id, optimistic, most_likely, pessimistic)`;
- `shift_completion_distribution(work_item_id, delta_workdays)`; this adds the same signed working-day delta to optimistic, most-likely, and pessimistic remaining duration, preserving distribution width and common random numbers;
- `set_earliest_start(work_item_id, timestamp_or_null)`;
- `add_dependency(predecessor, successor, lag_workdays)`;

- `remove_dependency(predecessor, successor)`;
- `remove_scope(work_item_id)`;
- `change_team_capacity(team_id, capacity_delta)` where the integer delta leaves capacity in `[1,100]`;
- `resolve_external_blocker(work_item_id, resolution_date)`;
- `mark_task_completed(work_item_id, actual_finish)` for a hypothetical work-state comparison.

There is no free-form formula, code, person reference, arbitrary graph mutation, or hidden capacity change. Operations are applied in order and conflicts are rejected, for example removing an absent edge twice or modifying a completed task duration. `remove_scope` removes the task and its incident scenario dependency edges in the compiled copy and lists every removed downstream constraint in the confirmation diff; it never mutates the source graph. `shift_completion_distribution` requires an integer delta in `[-260,260]` and is invalid if any shifted duration becomes negative. `resolve_external_blocker` clears the flag and sets the task's release constraint to `max(existing earliest_start, resolution_date)`; it cannot move work before `as_of`. `mark_task_completed` requires `project_start <= actual_finish <= as_of` and sets remaining duration to zero; a future claimed actual finish is invalid. Capacity changes are aggregate, explicit in the diff, and unused by the frozen H1 scenario. Natural-language source is retained only as an untrusted audit artifact beside the structured scenario; it is not included as executable input.

4. Compilation and confirmation

The scenario flow is:

1. The user chooses an immutable baseline snapshot.
2. A typed UI or the `scenario_planning` capability drafts only Section 3.3 operations.
3. Deterministic validation applies the draft to a copy, checks IDs, ranges, cycles, calendar, policy, and prohibited fields.
4. The UI shows an exact before/after diff, affected milestones, added assumptions, and source conflicts.
5. The user confirms the digest. Any change requires a new confirmation.
6. The API creates a run referencing, not copying or mutating, the snapshot and scenario.

Natural language is never interpreted during numeric execution. If the compiler cannot map a phrase exactly, it returns an unresolved question instead of selecting an operation.

4.1 Frozen H1 reference scenario

The H1 workload uses primary tenant `10000000-0000-4000-8000-000000000001` (`tnt_aster`) and isolation canary `10000000-0000-4000-8000-000000000002` (`tnt_beacon`). Its scheduling chain is `AST-142 -> AST-173 -> AST-201 -> Orion 2.0 General Availability`. Fixture work-item IDs are deterministic UUIDv5 values over their tenant/source identity; `AST-142` maps to `116ab4b3-b108-5f91-ab7e-111f7fba1d45`. The confirmed scenario contains exactly one intervention:

```
{
  "type": "shift_completion_distribution",
  "work_item_id": "116ab4b3-b108-5f91-ab7e-111f7fba1d45",
  "delta_workdays": -5
}
```

All other duration distributions, dependencies, calendars, capacities, and assumptions remain unchanged. The reference run uses seed `20260713` and 50,000 trials. Within one calendar day at each percentile, its golden output is:

Result	Baseline	Scenario
p50 launch date	`2026-08-20`	`2026-08-13`
p80 launch date	`2026-08-24`	`2026-08-17`
p95 launch date	`2026-08-27`	`2026-08-20`
Dominant path	`AST-142 -> AST-173 -> AST-201 -> launch`	Same path with lower occupancy

These golden dates validate the fixture and engine together; they are not precomputed production answers and are never substituted for executing the real run.

5. Mathematical model

5.1 Beta-PERT duration

For each non-completed task `i`, let `o_i`, `m_i`, and `p_i` be optimistic, most-likely, and pessimistic remaining workdays. H1 uses Beta-PERT with `lambda = 4`:

```
alpha_i = 1 + lambda * (m_i - o_i) / (p_i - o_i)
beta_i  = 1 + lambda * (p_i - m_i) / (p_i - o_i)
X_i     = o_i + (p_i - o_i) * B_i
B_i     ~ Beta(alpha_i, beta_i)
```

When `o_i = m_i = p_i`, `X_i` is that constant and no random value is consumed. Any other zero-width or reversed interval is invalid. The familiar PERT mean `(o_i + 4m_i + p_i) / 6` is reported for input inspection but is not substituted for sampling.

H1 assumes task durations are independent conditional on the entered estimates. Because real work may share systemic risks, the output always states this assumption. Correlated risk factors are an H2 research/validation item and cannot be introduced by merely widening a prompt.

5.2 Deterministic random streams

The engine uses counter-based Philox streams. The stream key is derived from `SHA-256(encode_tuple(seed, engine_version, work_item_id))`, where each UTF-8 text value is length-prefixed and the UUID uses its 16 raw network-order bytes; the counter contains iteration and draw component. Work-item UUIDs are sorted bytewise. The implementation maps a fixed 53-bit uniform variate through the pinned inverse beta CDF, so a changed distribution uses the same underlying uniform value. Therefore worker count, batch size, retry, and input/topological ordering do not change a task's random draw.

The implementation pins the PRNG and beta-sampling algorithm in the engine container. Exact replay is guaranteed only for the same `engine_version` and container digest; a new numeric-library or sampling algorithm is a new engine version and must pass golden-vector compatibility tests. Inputs and results retain enough metadata to run the prior image during the support window.

Baseline and scenario use identical streams for a task that exists in both. A scenario-changed duration transforms the same underlying uniform variates through its new distribution. New tasks receive streams from their stable IDs. This common-random-number design reduces noise in the difference without pretending the estimates are causal.

5.3 Capacity-aware schedule calculation

For a task `i` assigned to aggregate team `g`, let `a_g` be availability and `c_g` be integer parallel capacity. Effective sampled duration is:

```
Y_i = X_i / a_g
```

For each iteration, the engine uses a deterministic event-driven parallel schedule-generation scheme. It maintains predecessor counts, a release instant for each task, `c_g` numbered slots for each team, a bytewise-UUID ready queue per team, and a global event queue. A task enters its team's ready queue only after all predecessors are complete and its release instant has arrived. At each event instant, completions are processed first in bytewise UUID order, newly eligible tasks are enqueued, and each free team slot takes the bytewise-lowest ready task; equal free slots use the lowest slot index. Zero-duration completions are processed to a fixed point at the same instant before time advances.

```
release_i = next_working_instant(max(as_of,
                                     project_start,
                                     earliest_start_i,
                                     external_blocker_until_i),
                                 calendar)

dependency_ready_i = max(release_i,
                         max(add_working_days(finish_j, lag_ji) for each predecessor j))

start_i = first event instant at or after dependency_ready_i
          where a team-g slot is free and i is the
          bytewise-lowest UUID in that team's ready queue

finish_i = add_working_duration(start_i, Y_i, calendar)

project_finish = max(finish_i for each terminal active task i)
```

The predecessor maximum is absent for a task with no predecessors, leaving `dependency_ready_i=release_i`. When no task can start, time advances to the earliest running-task finish or dependency-satisfied future release; if neither exists while work remains, validation has missed an impossible state and the run fails rather than looping. Completed predecessor finish is its actual finish. Null `earliest_start` and null `external_blocker_until` contribute no later constraint. Every remaining task starts no earlier than `as_of`, project start, its explicit release constraints, its predecessors, and an available team slot. A ready task never

waits behind a merely future-released task. Disconnected components are allowed and project completion is the latest terminal task. A project with no active tasks completes at `as_of`. This stable non-delay heuristic is not a claim to solve the globally optimal resource-constrained project scheduling problem.

Working-time addition uses the sealed IANA timezone and calendar. It advances only through configured working intervals, treats holidays as non-working, preserves fractional workdays, and emits RFC3339 instants plus display dates. Ambiguous or nonexistent daylight-saving local times are resolved by the timezone database version recorded in the engine. Capacity is team-level and deterministic within an iteration; H1 does not model individual assignment, individual rate, preemption, multitasking loss, skill substitution, or stochastic availability.

5.4 Quantiles and probability

Completion instants are converted to elapsed working seconds for aggregation and then mapped back through the calendar. For sorted zero-indexed values `x[0..n-1]`, quantile `q` uses Hyndman-Fan type 7:

```
h = (n - 1) * q
j = floor(h)
g = h - j
Q(q) = (1 - g) * x[j] + g * x[min(j + 1, n - 1)]
```

H1 reports the sealed-timezone local dates containing `Q(0.50)`, `Q(0.80)`, and `Q(0.95)`. A target date maps to the exclusive start of the next local civil date under the sealed timezone database, so weekend and daylight-saving boundaries have one deterministic meaning. `probability_on_or_before_target` is the count of iterations with `project_finish < target_cutoff` divided by iterations, and `probability_after_target = 1 - probability_on_or_before_target`. Both are omitted when no target is present, and the UI labels the latter as simulated miss probability under the sealed assumptions.

Sampling uncertainty is estimated by splitting the ordered iteration set into 20 equal deterministic batches, computing each requested quantile per batch, and reporting the standard error of the batch quantiles. Fewer than 2,000 iterations is allowed only for an explicit preview and is labeled low precision. The committed H1 result uses 50,000 iterations.

The pinned engine uses IEEE 754 binary64 without fast-math reassociation and rejects non-finite intermediates. Before hashing, instants are rounded to UTC milliseconds using round-half-to-even; workday deltas, probabilities, correlations, criticality indexes, and standard errors are rounded to six decimal places with negative zero normalized to zero. Ranked arrays sort by the documented score direction and then bytewise work-item UUID. RFC 8785 canonical JSON is hashed only after this normalization. Full-precision internal values may be retained in the short-lived validation artifact but are not part of the public compatibility contract.

6. Critical path, blockers, and sensitivity

For every iteration the engine augments source dependencies with resource-order edges between consecutive tasks assigned to the same simulated team slot, then performs a backward pass from `project_finish`. A task is critical when total slack is no more than `1e-9` workdays. Its criticality index is the fraction of iterations in which it is critical. The displayed p80 critical path is the deterministic dependency/resource chain ending at the p80-nearest completion sample, breaking equal-finish ties by work-item UUID. The product marks resource-order edges so they are not mistaken for source facts. It also shows criticality index so one sampled path is not mistaken for the only risk path.

Blockers are ranked active tasks whose state is `blocked` or `external_blocker=true`, then by:

```
blocker_score = criticality_index * max(0, p80_finish_improvement_if_zeroed)
```

`p80_finish_improvement_if_zeroed` is `baseline_p80 - counterfactual_p80` in working days. The one-at-a-time counterfactual sets only that task's remaining duration to zero and, when present, its external-blocker release constraint to `as_of`; dependency lags and every other input remain unchanged. It preserves common random numbers and is an impact screen, not a causal estimate.

Sensitivity drivers use Spearman rank correlation between each task's sampled duration and project completion duration. The output reports signed correlation, absolute rank, criticality index, and source estimate. Correlation is omitted for constant tasks and labeled unstable when fewer than 2,000 iterations or when batch estimates vary materially. Only the top 20 drivers are returned by default.

7. Output contract

`SimulationRun` includes:

```
{
  "simulation_id": "UUID",
  "tenant_id": "server-derived UUID",
```

```
  "snapshot_id": "UUID",
  "snapshot_hash": "hex",
  "scenario_id": "UUID or null",
  "scenario_hash": "hex or null",
  "calendar_version": "string",
  "engine_version": "semver+container-digest",
  "seed": "unsigned 64-bit decimal string",
  "sample_count": 50000,
  "status": "succeeded",
  "uncertainty": {
    "method": "seeded_pert_monte_carlo",
    "sample_count": 50000,
    "seed": "20260713",
    "quantiles": {"p50": "YYYY-MM-DD", "p80": "YYYY-MM-DD", "p95": "YYYY-MM-DD"},
    "batch_standard_errors_days": {"p50": 0.1, "p80": 0.2, "p95": 0.4},
    "warnings": []
  },
  "probability_on_or_before_target": 0.72,
  "probability_after_target": 0.28,
  "critical_path": ["task UUID"],
  "criticality": [{"work_item_id": "UUID", "index": 0.63}],
  "blockers": [],
  "sensitivity": [],
  "assumptions": [],
  "missing_data": [],
  "warnings": [],
  "evidence_ids": [],
  "created_at": "RFC3339 timestamp",
  "completed_at": "RFC3339 timestamp",
  "result_sha256": "hex"
}
```

`tenant_id` is internal/audit context and is not accepted from a public caller. Quantile dates are rendered in the sealed project timezone; full UTC instants remain in the validation artifact. `result_sha256` covers a canonical `SimulationResult` value containing the uncertainty, probabilities, comparison, paths, drivers, assumptions, warnings, evidence, input hashes, seed, sample count, and engine/calendar versions. It excludes `simulation_id`, tenant ID, lifecycle status and timestamps, progress, trace IDs, storage URIs, and the hash field itself. Thus two independently created runs over equal committed inputs produce the same computational result hash even though their run records differ.

A comparison adds baseline and scenario forecasts, paired per-iteration deltas, p50/p80/p95 delta workdays, probability of improvement, and changed criticality. A negative finish-date delta means earlier completion and is labeled explicitly. The comparison never describes an association as causal.

User-facing explanation is generated after numeric completion from this structured result and accessible evidence. It must preserve numbers exactly, cite estimate/dependency evidence, identify confirmed imputations, explain that p80 means 80% of simulated outcomes finish on or before the shown date under assumptions, and include the data watermark. Any model explanation that changes or omits required numeric fields is rejected.

8. Validation and edge cases

Validation occurs before sealing and again before execution:

Case	Required result
Cycle or self-edge	`422 invalid_dependency_cycle` with a deterministic shortest cycle path; no run.
Missing task endpoint	`422 unknown_task_reference`; no implicit task creation.
Invalid/NaN/infinite/reversed duration	`422 invalid_duration`; no clamping.
Missing estimate	Sealing blocked unless the user confirms the documented imputation.
Completed task after `as_of`	`422 invalid_actual_finish`.
Cancelled task has nonzero duration or an unresolved active dependency	`422 unresolved_cancelled_work`; require a source correction or confirmed snapshot-compilation exclusion.
Earliest start after target	Run is allowed with `target_infeasible_from_constraints` warning and probability computed normally.
Target before project start	Probability is zero and a warning is returned.
Empty project	Completes at `as_of` with no critical path.
Disconnected task group	Included; warning lists independent terminal groups.

Case	Required result
Duplicate dependency	Greatest lag retained with warning; contradictory add/remove scenario operations are rejected.
Calendar has no working day or invalid timezone	`422 invalid_calendar`.
Missing team, non-integer/zero capacity, or availability outside `(0,1]`	`422 invalid_team_capacity`; no value is inferred from people or activity data.
Graph above H1 limits	`413 simulation_too_large` with actual limits; no partial hidden sampling.
Run cancellation	Stop between bounded batches, store `cancelled`, discard incomplete quantiles, retain progress and audit metadata.
Worker retry	Resume at a batch boundary; deterministic streams and result-part hashes prevent duplicate or changed samples.
Snapshot ACL revoked	Prevent result access and explanation; numeric artifact remains governed by deletion/retention policy.

Contradictory source claims are not silently resolved by the engine. Snapshot compilation uses the CH-05 predicate precedence rule, includes conflicts and the selected claim, and requires confirmation when a conflict changes scheduling input.

9. Performance, caching, and observability

The H1 cache key is:

```
SHA-256(RFC8785({snapshot_hash, scenario_hash, seed, sample_count, engine_version}))
```

Only a completed, checksum-verified result is cacheable. Authorization is always evaluated on access; cache membership is not permission. Different tenants never share result objects, even for identical synthetic content. Progress checkpoints store completed batch IDs and hashes, not an evolving final percentile.

The engine stores aggregate results by default, not every sample. Debug or validation runs may retain compressed sample arrays for 7 days under restricted access. Normal H1 result metadata follows scenario retention; deleting a snapshot removes dependent cached results after an immediate deny barrier.

Metrics include queue time, validation time, task/edge/iteration counts, sample throughput, schedule throughput, run duration, cancellation latency, cache hit, invalid input by code, batch quantile variation, result size, and numeric warnings. Traces record snapshot/scenario/result hashes, engine/container versions, seed, batch IDs, and timing; task labels and source text are excluded.

10. Verification and acceptance

10.1 Deterministic and analytic tests

- A constant one-task project finishes exactly after its duration on the sealed calendar.
- A constant serial chain finishes after the sum of task durations and lags.
- A fork/join with sufficient team slots finishes after the maximum branch plus the join task; one slot serializes same-team branches in stable order.
- Completed and cancelled tasks follow Section 3.2 without consuming random draws.
- Weekends, holidays, fractional workdays, leap years, and daylight-saving boundaries match golden calendars.
- Beta-PERT sample moments converge to analytic expectations within predefined statistical tolerance.
- Quantile type 7 matches golden vectors, including one-element and repeated-value samples.
- Stable seed and engine image produce byte-identical result JSON across 1, 2, and 8 workers and after batch retry.
- Equal baseline and scenario produce exactly zero paired deltas. Shifting all three bounds by the same delta shifts that task's sampled duration exactly under common random numbers; project-finish monotonicity is asserted only for serial or unconstrained-capacity fixtures because resource-constrained priority schedules can exhibit scheduling anomalies.
- Increasing aggregate parallel capacity cannot delay a fixture under the stable scheduling heuristic; reducing availability cannot improve it.

10.2 Acceptance criteria

ID	Acceptance criterion
AC-SIM-001	A sealed run can be independently replayed from snapshot, scenario, seed, sample count, engine image, and calendar to the same result hash.
AC-SIM-002	Cycles, invalid ranges, missing estimates, impossible capacity, unknown work items, disconnected work, contradictory interventions, and malicious numbers produce explicit stable outcomes.
AC-REL-001	The frozen 50,000-trial workload meets the 10-second p95 target with CPU and memory recorded in the benchmark artifact.
AC-TEN-001	The `tnt_aster` run and its cache, trace, explanation, and citations contain no `tnt_beacon` identifier, evidence, or timing disclosure.
AC-PROD-001	The real run, not a substituted fixture result, reproduces Section 4.1 golden dates within tolerance and supports the five-minute demo journey.

The result contract additionally requires p50/p80/p95, target probability, critical path/criticality, blockers, sensitivity, assumptions, missing data, evidence, exact numeric explanation, revocation behavior, and one audited terminal state. Independent review traces every estimate, dependency, calendar override, team capacity, and intervention to evidence or explicit confirmation.

Property-based tests generate DAGs, calendars, scenario diffs, and valid/invalid PERT triples. Metamorphic tests check input-list/topological-order invariance for the same DAG, work-item stream isolation, exact common-random-number duration shifting, schedule monotonicity only for serial or unconstrained-capacity fixtures, dependency removal non-delay under those same controlled fixtures, and batching invariance. Separate resource-constrained fixtures record priority/resource-order changes rather than assuming global monotonicity. Fuzz tests cover canonical JSON, timestamps, extreme finite numbers, duplicate IDs, and malicious labels. Performance tests run the frozen workload at 25,000, 50,000, and 75,000 trials; separate guardrail tests cover the 5,000-work-item and 20,000-edge input maxima without claiming the demo latency SLO at that maximum shape.

11. Risks and evolution

ID	Risk	Mitigation
RSK-011	Users interpret a simulated percentile, capacity input, or blocker score as a precise causal forecast.	Plain-language percentile semantics, evidence/assumptions, distribution, sampling error, synthetic disclaimer, and no causal language.
RSK-016	Synthetic estimates and golden dates are mistaken for external predictive validity.	Limit claims to reproducibility/system behavior and require design-partner calibration before production accuracy claims.
RSK-007	Aggregate capacity is repurposed into unsafe person or workforce scoring.	Exclude person inputs/rates, prohibit employment outcomes, govern schemas/outputs, and require a separate research program.

Numeric-library or runtime changes can still break reproducibility. The mandatory mitigation is a versioned engine/container, counter-based streams, golden vectors, side-by-side migration, and replay support for prior engine images.

H2 may add shared aggregate resource pools, calibrated correlation factors, additional dependency types, preemption rules, and portfolio interactions only behind new schemas, equations, validation datasets, and compatibility versions. Financial, customer, security, market, workforce, and autonomous-organization simulations remain Research until each has a lawful purpose, validated data-generating model, uncertainty contract, human oversight, and explicit release gate.

CH-09 - Security, Privacy, and Compliance

Status: Committed | Owners: Security Engineering, Privacy Engineering, Platform Engineering | Last reviewed: 2026-07-13

Security, Privacy, and Compliance

1. Security objective and limits

The security objective is to preserve tenant isolation, source permissions, evidence integrity, human control of external changes, and recoverable audit evidence even when a user, connector payload, model response, network peer, or dependency is malicious or faulty.

H1 and H2 are designed for SOC 2 and ISO 27001 control evidence and GDPR engineering readiness. This specification does not claim certification, legal compliance, or fitness for regulated health, government classified, financial trading, or employment-decision processing. Certification, legal basis, contracts, policies, training, and organizational controls remain the responsibility of named business, legal, privacy, and security owners.

The following use cases are prohibited through H4: individual productivity scoring; burnout, attrition, emotion, health, misconduct, hiring, termination, compensation, or performance prediction; covert employee surveillance; and automated employment decisions. A future research proposal cannot lift this prohibition without a separate legal basis, data protection impact assessment, worker consultation, ethics review, security design, and explicit product-governance approval.

2. Security principles

Catalog control	Principle	Enforced behavior
CTRL-IAM-002	Server-derived tenant authority	Membership and tenant context come from verified identity plus authoritative membership, never request headers, model output, connector content, or cache data
CTRL-IAM-003	Default deny	Missing identity, membership, policy input, ACL, key, scope, projection watermark, or approval causes denial
CTRL-IAM-003 and CTRL-CON-001	Least privilege	Users, workloads, connectors, agents, database roles, and CI jobs receive only required actions and tenant resources
CTRL-ACT-002	Separation of duties	Proposal, two-person approval, execution, security administration, and audit have distinct permissions
CTRL-IAM-003	Complete mediation	Authorization is checked when data is retrieved, when a proposal is created, when each approval is accepted, and immediately before execution
CTRL-DAT-002	Projection-safe revocation	Revocation blocks reads until every participating projection and cache meets the revocation watermark
CTRL-AI-001	Untrusted content	Provider text and model output are data, never authority or instructions
CTRL-PRV-001 and CTRL-DAT-003	Minimized persistence	Store only required content, avoid raw content in logs/workflow history, and enforce tenant retention and deletion
CTRL-ACT-001	Verifiable change	Sensitive commands bind approvals to one canonical payload, target, version, expiry, and tenant
CTRL-AUD-001	Recoverable evidence	Security and action events are append-only, integrity checked, access controlled, and exported to independent retention

3. Data classification and handling

Class	Examples	Storage and transport	Logging and model handling
Public	Published product documentation and explicitly public tenant material	TLS; normal encrypted storage	May be logged only when operationally useful
Internal	Configuration metadata, non-sensitive synthetic demo data	TLS 1.2 or later; encrypted storage	No full payload in telemetry; allowed to approved model endpoint
Confidential	Source content, organization graph, tickets, identities, forecasts	Tenant-isolated encrypted storage; scoped workload access	Redacted logs; only authorized evidence sent to approved model capability
Restricted	Secrets, OAuth tokens, approval grants, audit integrity keys, security incidents, legal hold data	External secret manager/KMS, envelope encryption, no browser exposure	Never placed in prompts, traces, metrics, workflow search attributes, or support bundles

Classification is inherited from the source and can only become more restrictive. A derived fact that requires multiple inputs receives an effective audience no broader than the intersection of the audiences required to support it. A result containing confidential data is actor-bound and cannot be cached for a different actor unless the cache key contains an immutable ACL digest and policy version.

4. Identity, sessions, and workload trust

4.1 Human authentication

- Production authentication is delegated to a tenant-approved OIDC identity provider. H2 supports SAML through an identity broker and SCIM 2.0 provisioning.
- The API validates issuer, signature, audience, authorized party where present, nonce, state, expiry, not-before, and algorithm allowlist. It rejects tokens from dynamically supplied issuers or key URLs.
- A stable actor key is issuer plus subject. Email, display name, or provider username is not identity.
- Interactive sessions use Secure, HttpOnly, SameSite=Lax cookies; state-changing browser calls require same-origin checks and an anti-CSRF token. Session rotation follows login, privilege change, and recovery.
- MFA and phishing-resistant authentication are IdP policy. Tenant and platform administrators, approvers, and break-glass operators must use MFA; H2 requires a tenant assertion or documented IdP policy evidence.
- SCIM disablement immediately invalidates membership, delegation, sessions, and cached authorization. Reconciliation detects missed deprovisioning.
- Recovery and support cannot bypass the IdP. Impersonation is disabled in H1/H2. A future support-view feature must be read-only, time-bound, customer-approved, visibly bannered, and separately audited.

4.2 Service and job identities

- Kubernetes workloads use separate service accounts and short-lived workload identity. Local Compose uses unique development-only credentials.
- Each workload has a separate PostgreSQL role, object-store policy, Temporal namespace/task-queue permission, Neo4j credential, and model gateway capability.
- A service identity does not imply a tenant. Each job carries a signed immutable context naming one tenant, purpose, workflow ID, actor or system principal, delegation, budget, and expiry.
- Background batch processing iterates one tenant transaction at a time. There is no normal cross-tenant application role.
- The maintenance break-glass database role is held outside application secrets, requires incident/change approval, has a maximum one-hour credential, records the operator and ticket, and produces an alert. It is never used by migrations or runtime code.

5. Pooled multitenancy and isolation

5.1 Tenant derivation

The browser can request a tenant slug as a locator, but it cannot assert tenant authority. The API resolves the selected slug against current memberships for the verified issuer-plus-subject, chooses the active membership and delegation, and creates TenantContext. A token tenant claim is advisory unless it is issued by a specifically configured IdP mapping and still matches authoritative membership.

Every request, workflow, event, trace, and audit record has one tenant context. Platform-public data uses an explicit platform scope and is never represented by a null tenant on a tenant table.

5.2 PostgreSQL

All tenant tables meet these rules:

1. The primary or candidate key begins with tenant_id.
2. Every tenant-to-tenant foreign key includes tenant_id on both sides.
3. ENABLE ROW LEVEL SECURITY and FORCE ROW LEVEL SECURITY are set.

4 SELECT, INSERT, UPDATE, and DELETE policies compare tenant_id with the transaction-local tenant setting and reject a missing or malformed setting.

5 Application roles are non-superuser, do not own tables, and do not have BYPASSRLS.

6 The connection pool returns a connection only through a transaction wrapper that sets app.tenant_id, app.actor_id, app.policy_version, and app.request_id using SET LOCAL.

7 Pool release rolls back any open transaction. Tests verify that a connection reused for another tenant cannot observe prior context.

8 Unique constraints for external IDs include tenant_id. Entity-resolution candidate queries include tenant_id before similarity logic.

9 Migrations run under a schema-owner role unavailable to applications. Migration verification fails if a new tenant table lacks policy or tenant-qualified foreign keys.

High-risk transactions, including approval acceptance, single-use grant claim, connector cursor advancement, and identity merge, use row locks or SERIALIZABLE isolation with bounded retry.

5.3 Derived stores

Store	Isolation requirement
pgvector	Vectors remain in PostgreSQL tenant rows under FORCE RLS. Approximate-index queries must preserve an exact tenant and ACL post-filter before results leave the data layer.
Neo4j	H1/H2 use a pooled logical namespace: every node and relationship carries tenant_id and projection_generation; only typed query/projector gateways inject server-derived tenant context; relationship endpoints are tenant checked; results are reauthorized against current PostgreSQL policy. User Cypher and direct application access are prohibited. H3 can move a tenant to a dedicated graph database/cell only after the RSK-002 trigger and ADR review.
Object storage	Each tenant uses a distinct bucket or access-controlled prefix and tenant-specific encryption-key reference. IAM denies listing or reading another namespace. Object names are opaque IDs, not user paths.
Valkey/Redis	Keys begin with an HMAC-derived tenant namespace, not a guessable slug. Separate credentials or ACL key patterns restrict workloads. Cached authorization records include policy version and revocation watermark.
Future OpenSearch	One tenant index or access-controlled tenant index set. Aliases are server resolved. Every query includes an independently verified tenant filter and ACL post-filter.
Temporal	Workflow IDs start with an HMAC tenant namespace. Workflow payloads use object references; raw source content and tokens are prohibited. Unavoidable confidential payloads use an approved payload codec. Search attributes contain no PII.
Telemetry	Tenant is represented by a low-cardinality pseudonymous key. Logs do not contain source bodies, prompts, model responses, tokens, or personal email addresses.

5.4 Cross-tenant negative guarantee

H1 and H2 implement no cross-tenant search, analytics, entity merge, model memory, evaluation corpus, or training. Operational aggregate metrics use non-content counts with privacy review. A future customer opt-in is not a feature flag; it requires a new data-purpose contract, legal basis, isolation design, deletion design, threat model, audit trail, and ADR.

6. Authorization and source ACL

6.1 Policy model

Authentication roles do not directly grant data. The API policy decision combines:

- actor identity, tenant membership, role, group, authentication assurance, and account status;
- active delegation with issuer, grantee, allowed actions/resources, start, expiry, and revocation;

- action and capability;
- resource tenant, type, stable ID, owner, classification, SourceACL, state, and version;
- environment, provider scope, project allowlist, policy version, risk class, and current time.

The result is Allow, Deny, or Indeterminate plus reason codes, obligations, policy version, and decision ID. Deny overrides Allow; Indeterminate is Deny. Obligations can require redaction, result-size limits, confirmation, two-person approval, or a fresher projection.

Committed tenant roles are Viewer, Analyst, Operator, ConnectorAdmin, Approver, SecurityAdmin, TenantAdmin, and Auditor. Roles grant candidate actions, not broad data visibility. Platform operations roles have no source-content access by default.

6.2 Evidence authorization

- SourceACL records provider principals, groups, visibility, source version, observed time, and resolution status.
- Provider identities map to tenant actors through explicit, versioned resolution. Ambiguous mapping denies access.
- Retrieval returns claim-level facts only when at least one complete supporting evidence path is visible and no required source is hidden.
- Entity existence, relationship existence, counts, snippets, embeddings, and graph neighborhoods are all data and require authorization. The system must not leak inaccessible facts through empty-versus-not-found behavior, counts, timing, ranking, or graph degree.
- Graph edges and vector chunks retain evidence IDs and effective ACL digests. ACL filtering occurs before model context construction.
- Query and answer artifacts record policy version, actor, evidence IDs, source versions, and revocation watermark. An answer cannot be replayed to another actor without reauthorization.
- Cache invalidation is event driven, but a watermark check is still required. If an ACL projection is stale after a revocation, affected reads fail closed.

7. Key, secret, and cryptographic controls

Asset	Control
TLS	TLS 1.2 or later externally and authenticated encryption internally; H2 uses workload identity for service-to-service connections
Database/object encryption	Provider-managed encryption at rest plus envelope encryption for Restricted fields and tenant object namespaces
Tenant keys	One tenant data-encryption key hierarchy under KMS/HSM-backed key encryption keys; key IDs are versioned and rotation is online
OAuth/provider secrets	Stored only in an approved secret manager, referenced by opaque ID, scoped per tenant installation, and refreshed/revoked through a broker
Cookies and tokens	Signed with rotated keys; short-lived access; refresh token rotation where used; tokens never enter URLs or logs
Webhooks	Provider-specific signature algorithm, constant-time comparison, raw-body verification, delivery-id uniqueness, and timestamp window
Action digest	SHA-256 over RFC 8785 canonical JSON for the exact approved payload: UUID tenant and connector installation, action, project/issue target, expected provider version, and field set. ApprovedPayload separately binds idempotency key, policy version, credential version, and expiry.
Audit integrity	Append-only sequence with previous-record digest per tenant/day plus signed daily root exported to independent object retention

Key access is authorized by workload identity and tenant. Rotation preserves decrypt-only access to prior key versions until data is rewrapped. Suspected compromise disables use immediately, starts rewrap or credential rotation, and records affected tenants and evidence. Deleting a key is not the normal deletion workflow because it can destroy unrelated records.

Secrets never appear in source control, build arguments, images, Helm values, CI output, exception text, traces, support bundles, model prompts, or Temporal search attributes. Secret scanners block merge and release.

8. Connector and ingestion security

1. GitHub uses a GitHub App with read-only metadata scopes required by the connector. Jira uses OAuth with read scopes plus the minimum issue-write scope needed for the allowlisted remediation action.
2. Installation state binds provider, tenant, actor, redirect URI, nonce, PKCE verifier, and expiry. Redirect targets are allowlisted.
3. Each tenant installation has separate credentials and rate budget. Provider tokens are never shared across tenants.
4. Outbound HTTP uses a provider-specific host allowlist, DNS and IP validation, HTTPS, certificate validation, redirect denial or revalidation, response byte limits, connect/read/total timeouts, and decompression limits. This prevents SSRF and archive/decompression bombs.
5. Payload schemas have maximum nesting, collection count, text length, attachment count, and total size. Unsupported encodings and active content are quarantined.
6. Original bytes are hashed and retained according to policy; normalization never overwrites evidence.
7. Webhooks are hints. Scheduled reconciliation verifies cursors, deletions, scopes, and ACL changes.
8. Connector failures cannot cause broad provider retries. Exponential backoff with jitter respects Retry-After, circuit breakers, and per-tenant concurrency.
9. A compromised connector installation can be disabled independently. New data is quarantined, credentials revoked, prior claims marked suspect, and dependent projections reverified.
10. Attachments and future document parsing run in a sandbox with no ambient network or cloud metadata access, read-only inputs, output and CPU/memory/time limits, malware scanning, and parser isolation.

9. AI, agent, tool, and MCP security

9.1 Capability boundary

Agents are capability profiles, not organizational officers. The committed profiles are query/research, extraction/resolution, graph verification, scenario planning, mitigation drafting, and action execution. Each run receives a signed delegation snapshot with allowed tenant, evidence scope, tools, resource limits, model capability, deadline, and cancellation token. Handoffs intersect the current grant with the target profile and therefore can only narrow authority.

The platform does not persist or expose hidden chain-of-thought. It stores inputs, selected evidence, tool calls and arguments, structured outputs, decisions, verifier results, timing, cost, and concise rationale sufficient for audit.

9.2 Prompt injection and data exfiltration controls

- System and developer instructions are static, versioned, and separated from source content using typed message fields.
- Retrieved text is labeled with evidence ID and untrusted-content markers. Instructions found inside evidence have no execution semantics.
- Models receive no ambient network, database, provider, or secret access. Every tool is a narrow server-side function with a strict schema and independent authorization.
- Tool names and schemas come from the approved capability registry, not from source text or a user-provided URL.
- Tool arguments are schema-validated, canonicalized, length bounded, policy checked, and audited. Free-form SQL, Cypher, shell, filesystem paths, or HTTP fetch tools are prohibited.
- Retrieved content, tool output, and model output pass data-loss-prevention checks appropriate to classification before leaving a trust boundary.
- Model-generated citations must resolve to evidence in the authorized run envelope. Unknown citations are rejected.
- Memory is tenant-, actor-, purpose-, and retention-bound. H1/H2 have no cross-run autonomous long-term model memory beyond explicit product records.
- Model fallbacks are pre-evaluated and pinned. A model error, refusal, timeout, schema failure, or safety-gate failure cannot be converted into an unverified answer.

- Adversarial injection and tool-selection evaluations run before release and continuously, following the defense-in-depth pattern in OpenAI agent safety guidance.

9.3 MCP

MCP servers and tools are allowlisted by exact server identity and version. Remote servers require audience-bound OAuth, explicit egress policy, and a security review. Dynamic tool discovery cannot automatically grant use. Sensitive tools require the same preview and approval service as REST. Tool annotations are informational and never replace policy. The platform follows OpenAI guidance for approvals on sensitive MCP operations.

10. Jira remediation approval and execution

H1 supports one external mutation instance. Fixture alias tnt_aster resolves to tenant UUID 10000000-0000-4000-8000-000000000001, and con_aster_jira resolves to connector-installation UUID 30000000-0000-4000-8000-000000000001. Aliases never occur in the canonical approved payload. The only target is AST-142 in allowlisted sandbox project AST.

The exact required before state is Jira source version 7, due date 2026-08-07, priority Medium, and labels identity and orion. The exact approved after state is due date 2026-07-31, priorityId 2, and lexically sorted labels digital-twin-remediation, identity, and orion. Only due date, priority, and labels can change, and only to those values. Summary, description, project, issue type, transition, assignee, watcher, attachment, comment, delete, bulk edit, and arbitrary Jira REST paths are prohibited.

The public ApprovalRequest status follows the canonical contract: pending -> approved, denied, expired, or cancelled -> executed or compensated where applicable. ActionReceipt uses succeeded, failed, verification_required, compensated, compensation_conflict, or manual_intervention_required. CompensationResult uses restored, compensation_conflict, verification_required, failed, or manual_intervention_required. Internal previewing and executing workflow phases are telemetry, not additional public status values.

Stage	Mandatory checks
Preview	Fresh Jira before snapshot; project and field allowlist; expected issue version; canonical payload; digest; proposer; policy decision; 15-minute maximum expiry
Operations approval	Human operations approver, MFA assurance, not proposer, current membership, matching digest and expiry
Security approval	A different human security approver, MFA assurance, not proposer or operations approver, current membership, matching digest and expiry; approvals may arrive in either order
Grant	Single-use random grant ID, digest, tenant, action, target, expected version, two decision IDs, expiry; atomic outbox publication
Execution	Recheck tenant, action manifest, project, OAuth scopes, grant unused and unexpired, digest, provider version, and kill switch immediately before call
Receipt	Idempotency key, request digest, provider request ID, redacted response, before/after hashes, outcome, actor/approvers, timestamps, trace
Compensation	New preview from original before state, current-state comparison, two new approvers, no overwrite of unrelated edits

Any payload, target, expected version, policy, or expiry change invalidates both approvals. Approval links do not contain bearer grants. The worker atomically claims the single-use grant before the provider call. If the provider response is lost, the worker reconciles current Jira state and provider request evidence before retry. Compensation must be requested within 24 hours and restores the version-7 before snapshot only if current state still matches the recorded approved after state; otherwise it records compensation_conflict and enters manual_intervention_required.

A tenant kill switch disables all mutations. Platform security can invoke a global kill switch, but cannot enable a tenant mutation. Both switches are checked at preview and execution.

11. Application and API security

- All request bodies, paths, queries, headers, webhooks, and model/tool outputs are validated against size-bounded schemas. Unknown fields are rejected on commands.

- SQL is parameterized. Graph queries use predefined templates with typed parameters. User-supplied regular expressions, recursive depth, sort columns, and field selectors are allowlisted and budgeted.
- Output encoding is context-specific. Rich text is rendered through a strict sanitizer. Content Security Policy denies unapproved script, frame, connection, and object sources.
- Browser state-changing endpoints require CSRF defense. CORS is an explicit origin allowlist with credentials only for the product origin.
- APIs have per-actor, per-tenant, per-IP, and capability budgets. Expensive graph, export, AI, and simulation calls also reserve tenant quotas.
- Resource IDs are opaque, but authorization is still performed for every object. Not-found responses are normalized to avoid existence leaks.
- Error responses follow RFC 9457, use stable safe codes, and omit stack traces, queries, provider bodies, secrets, and policy internals.
- Upload support is disabled until its parser sandbox, malware scan, file-type sniffing, quotas, retention, and deletion controls are accepted.
- Administrative configuration changes require recent authentication, optimistic concurrency, audit, and where high risk, two-person approval.

12. Audit and accountability

AuditEvent includes event ID, tenant, UTC time, monotonic tenant sequence, actor or workload identity, authentication assurance, delegation, action, resource reference, policy decision ID/version, request and trace IDs, outcome, reason code, source IP classification, before/after hashes or redacted diff, idempotency key, and previous digest. It never stores secrets or unnecessary source bodies.

The following are always audited: login and logout; membership and SCIM changes; role/delegation changes; connector install/scope/credential changes; policy changes; data exports and deletions; break-glass access; model/tool invocation metadata; prompt-injection safety blocks; scenario confirmation; approvals and rejection; grant claim; external action and compensation; kill switches; retention/legal hold; security configuration; and audit access.

Audit records are append-only to application roles. A signed daily tenant root is exported to an independently controlled object retention location. This detects tampering; it does not make the primary database magically immutable. Auditor access is read-only, purpose-limited, and itself audited.

Default H2 retention is:

- security, authorization, connector-administration, and external-action audit: 7 years;
- operational metadata: 24 months;
- raw connector payloads and derived evidence chunks: 90 days;
- agent run content and detailed run telemetry: 30 days;
- anonymized aggregate telemetry: 13 months;
- raw prompts and model responses: not retained by default; approved redacted evaluation samples follow the evaluation dataset policy;
- deleted primary data: purge within 24 hours after tombstone and required hold checks;
- backups containing deleted data: age out within 35 days and cannot restore without replaying deletion tombstones.

Tenant policy can shorten these defaults. Longer retention requires a documented legal or regulatory basis. Legal hold overrides deletion only for the minimum named records and has owner, reason, jurisdiction, approval, start, review date, and release audit.

13. Privacy engineering and GDPR readiness

13.1 Roles and purpose

For customer source data, the customer is normally controller and the service provider is processor, subject to contract and legal review. Product account, billing, fraud, and security data may have a different role and must be recorded separately. Each connector has a purpose, data categories, data subjects, source, lawful-basis owner, destinations, retention, model use, residency, and deletion path in the data inventory.

13.2 Required capabilities

- Data minimization: request only provider scopes and fields needed for the committed use case.
- Transparency: tenant administrators can see connected sources, scopes, categories, derived uses, model endpoints, regions, retention, and pending deletion.
- Access and portability: export subject-linked authoritative records, provenance, derived claims, and material decisions in a structured form after identity verification and tenant approval.
- Rectification: source-owned facts are corrected at source and reconciled; product-owned resolution decisions can be corrected with history.
- Erasure: create a tombstone, block retrieval, delete primary/projection/cache/object/vector data, propagate to approved subprocessors, and record completion without retaining the erased content.
- Restriction and objection: disable relevant processing purpose without deleting evidence required for an accepted hold.
- Automated decisions: H1/H2 provide recommendations and simulations only; no legal or similarly significant automated decision is permitted.
- Privacy by default: no cross-tenant training, memory, analytics, or evaluation; no customer-content model training without a separately recorded opt-in that is absent in H1/H2.
- DPIA trigger: systematic monitoring, large-scale sensitive data, novel high-impact prediction, cross-context identity resolution, or workforce use blocks release until a DPIA is approved.
- Subprocessor governance: record model, hosting, telemetry, support, and identity subprocessors with region, purpose, contract, security review, and deletion assurance.

Data residency is enforced through a tenant home region. Authoritative databases, objects, graph projection, backups, model processing, and support access remain in approved regions. Global routing can use non-content health metadata only. A region change is a controlled export/import with dual authorization, integrity validation, old-region deletion, and audit.

14. Control readiness

Framework area	Engineering controls and evidence	Limitation
SOC 2 Security and Confidentiality	Identity/MFA evidence, least privilege, change review, encryption, audit, vulnerability scans, incident exercises, tenant isolation tests	Requires organizational policies, auditor period, and operating evidence
SOC 2 Availability	SLOs, alerts, capacity tests, backup restore, DR exercises, incident/postmortem records	Availability commitments are H2 only
SOC 2 Processing Integrity	Schema validation, idempotency, reconciliation, projection checks, deterministic simulations, approval receipts	Does not validate business predictions from synthetic data
ISO 27001 organizational and people controls	Ownership, access review evidence, secure development gates, supplier register, incident roles	HR and governance processes are outside software
ISO 27001 technological controls	Secure configuration, logging, cryptography, vulnerability management, backup, network controls, secrets, SDLC	Certification scope must be separately defined
GDPR engineering readiness	Data inventory, minimization, purpose/retention, rights workflows, residency, deletion, DPIA triggers, subprocessor evidence	Legal basis, notices, DPA, transfer mechanism, and regulator interpretation require counsel

HIPAA, PCI DSS, FedRAMP, government classification, and records-management certification are out of scope. A sales or deployment profile cannot claim them without a separately approved control baseline and external validation.

15. Threat model and mitigations

Threat ID	Related catalog item	Threat	Primary controls	Residual treatment
THR-SEC-001	RSK-001	Tenant identifier manipulation or confused deputy	Server-derived context, FORCE RLS, tenant-qualified keys, per-tenant derived stores, negative tests	Critical release blocker if any cross-tenant path exists
THR-SEC-002	RSK-003	Stale source ACL leaks revoked data	Revocation watermark, cache invalidation, projection checkpoints, fail-closed reads	Alert and disable affected connector/query capability
THR-SEC-003	RSK-004	Prompt injection triggers a tool or exfiltration	Untrusted-content boundary, no ambient tools, schema/policy checks, egress allowlist, adversarial eval	No sensitive action without exact approval
THR-SEC-004	RSK-005	Compromised connector poisons graph	Signature/scope checks, immutable provenance, quarantine, confidence, reversible merges, reconciliation	Mark dependent claims suspect and rebuild
THR-SEC-005	RSK-005	SSRF through connector or MCP	Fixed host catalog, DNS/IP validation, metadata IP deny, redirect revalidation, egress policy	Disable installation/server on anomaly
THR-SEC-006	RSK-006 and RSK-013	Replay or approval payload swap	Delivery inbox, canonical digest, expiry, two distinct approvers, single-use grant, provider version	Manual intervention on ambiguous provider state
THR-SEC-007	RSK-001 and RSK-014	Privileged insider reads customer content	Separate ops/content roles, no routine content access, break-glass, tenant keys, audit alerts	Customer-managed keys or a dedicated H4 data plane remain gated options
THR-SEC-008	REQ-SEC-007	Dependency or build compromise	Lockfiles, SBOM, signatures, provenance, scanning, isolated CI, digest admission	Critical/high policy blocks release
THR-SEC-009	RSK-018	Denial of service and cost exhaustion	Layered rate/size/concurrency budgets, queue backpressure, circuit breakers, tenant quotas, cancellation	Shed non-critical enrichment before interactive control paths
THR-SEC-010	RSK-012	Model provider retention or region mismatch	Approved endpoint configuration, data minimization, contract/subprocessor review, region check	Disable model capability if guarantees do not match tenant
THR-SEC-011	RSK-002	Graph query complexity attack	Query templates, depth/node/edge/time budgets, admission quotas, cancellation	Terminate query and record abuse signal
THR-SEC-012	RSK-015	Deletion restored from backup	Tombstone ledger outside backup point, restore replay, backup expiry, restore validation	Privacy incident if replay or downstream deletion fails
THR-SEC-013	RSK-014	Audit deletion or forgery	Append-only role, chained digests, signed daily root, independent retention	Daily verification and incident on mismatch
THR-SEC-014	RSK-010	Identity collision across providers	Issuer-plus-subject keys, explicit tenant-local resolution, reversible merges	Ambiguity denies access and requires review

16. Security operations

- Vulnerability intake covers dependencies, images, infrastructure, source, cloud configuration, and provider advisories. Exploitable Critical issues block release and receive immediate containment; exploitable High issues block production promotion unless the security owner records a time-limited exception with compensating control.
- Dependency updates run at least weekly; internet-facing and cryptographic emergency updates use an expedited tested path.
- Security incidents follow Prepare, Detect, Triage, Contain, Eradicate, Recover, Notify, and Learn. Runbooks identify incident commander, security lead, tenant communications, privacy/legal decision owner, evidence custodian, and service owner.
- Credential compromise, cross-tenant access, unauthorized Jira mutation, audit-integrity failure, and deletion failure are Severity 1 conditions.
- Quarterly access reviews cover platform roles, production access, CI identities, KMS, break glass, connector scopes, and tenant administrators in H2.
- Penetration testing is required before H2 production and annually thereafter, with retest of Critical and High findings.
- Security findings have severity, exploit scenario, affected tenant/data, owner, deadline, evidence, and verification. Critical or High findings cannot remain open at a specification 1.0 or H2 production release.

17. Acceptance criteria

Evidence ID	Canonical acceptance	Criterion
TC-SEC-001	AC-TEN-001	Cross-tenant tests cover API, PostgreSQL, graph, vector, object, cache, Temporal identifiers, exports, audit, and model context and produce zero disclosures.
TC-SEC-002	AC-TEN-001	Every tenant table has FORCE RLS, a tenant-qualified key and foreign key, and a non-owner application role test.
TC-SEC-003	AC-SEC-002	Removing a source permission blocks all affected reads before any stale projection or cache can respond.
TC-SEC-004	AC-AI-003	Connector text containing prompt injection cannot select a tool, change tenant, bypass retrieval ACL, or cause external network access.
TC-SEC-005	AC-ACT-001, AC-ACT-002, AC-ACT-003	No Jira change occurs with a proposer approval, fewer than two other human approvers, repeated identity, stale membership, changed payload, expiry, stale issue version, invalid scope, disabled tenant, or non-allowlisted project.
TC-SEC-006	AC-SEC-001	Secrets and Restricted content are absent from Git history, images, SBOM, logs, traces, metrics, workflow search attributes, error responses, and model prompts.
TC-SEC-007	AC-PRV-001	A tenant deletion test removes authoritative, graph, vector, object, cache, answer, and model-memory records and proves tombstone replay after restore.
TC-SEC-008	AC-OBS-001	An audit verifier detects a changed, removed, reordered, or inserted event and validates the independently stored daily root.
TC-SEC-009	AC-REL-002 and AC-REL-003	Restore, break-glass, key rotation, connector compromise, incident response, and global mutation kill-switch exercises produce complete evidence.
TC-SEC-010	AC-PRV-001	The data inventory maps every processed field to purpose, classification, owner, region, retention, model use, subprocessors, export, and deletion path.
TC-SEC-011	AC-REV-002	Security review finds no open Critical or High issue; Medium exceptions name owner, compensating control, expiry, and revisit trigger.
TC-SEC-012	AC-DOC-002	An independent reviewer can trace each canonical security/privacy control to implementation evidence, automated tests, operational evidence, and an accountable owner.

CH-10 - Scalability, Reliability, and Observability

Status: Committed | Owners: Site Reliability Engineering, Platform Engineering, Data Engineering | Last reviewed: 2026-07-13

Scalability, Reliability, and Observability

1. Scope and scale posture

The platform must meet the committed H1 and H2 envelopes and expose measured transition points for later horizons. It does not claim support for millions of organizations, billions of graph nodes, trillions of edges, petabytes of documents, thousands of concurrent agents, active-active multi-cloud, or edge autonomy. Those are H5 research topics until representative workloads, economics, consistency, privacy, and operations are proven.

Scale is achieved first by bounded work, tenant quotas, stateless horizontal application replicas, PostgreSQL tuning and partitioning, per-tenant graph placement, and asynchronous workflows. A new datastore or broker is a last step after measurement, not the first response to a forecast.

2. Committed capacity envelopes

Dimension	H1 Hackathon	H2 Design-partner pilot	H3 and later
Tenants	Exactly two synthetic tenants in acceptance environment	Up to 10 production tenants	Freeze from pilot telemetry and benchmark before commitment
Human identities	Up to 100 per tenant	Up to 1,000 authenticated users aggregate across the pilot	No published claim until frozen
Concurrent interactive users	10 aggregate	100 aggregate test load, with no tenant above 50	Derive from observed concurrency
Graph size	100,000 nodes and 1,000,000 edges per tenant	1,000,000 nodes and 10,000,000 edges per tenant	Benchmark-gated topology
Connector scope	GitHub metadata read-only; Jira read plus one controlled remediation update	Same core connectors with production recovery, scope and revocation controls	Connector catalog expands through manifests and conformance tests
Sync freshness	Full reconciliation interval at most 15 minutes	99 percent of healthy-provider source changes visible within 15 minutes	Contract by connector tier
Interactive non-AI query	p95 below 2 seconds	p95 below 2 seconds at the H2 load shape	Freeze per query class
Cited answer	p95 below 20 seconds on accepted evaluation set	p95 below 20 seconds when approved model endpoint is healthy	Separate model and platform SLOs
Interactive simulation	p95 below 10 seconds for the frozen 50,000-trial Orion workload	p95 below 10 seconds for the same committed reference workload at H2 concurrency	Larger or maximum-shape runs are asynchronous and have no 10-second claim
Availability	Demonstration acceptance target, not a production SLA	99.9 percent monthly for the regional service	Freeze by deployment profile
Recovery	Re-seed synthetic data; documented backup/restore	RPO at most 1 hour; RTO at most 4 hours	Regional and isolation targets frozen later

The H2 "1,000 users" commitment is interpreted as aggregate across the ten-tenant pilot because the approved horizon did not state per tenant. Raising it to per tenant is a material capacity change and requires a QAR update and benchmark.

2.1 Reference load shapes

Performance results are valid only with the following published shape and a representative data distribution:

- H1 interactive: 10 concurrent sessions, 70 percent bounded reads, 10 percent graph traversals, 10 percent scenario operations, 5 percent cited-answer starts, and 5 percent administration; up to two simultaneous simulations and two cited answers.
- H2 interactive: 100 active sessions, up to 50 simultaneous HTTP requests, 70 percent bounded reads, 10 percent graph traversals, 8 percent scenario operations, 5 percent cited-answer starts, 2 percent connector administration, and 5 percent SSE/status traffic; up to 20 simultaneous cited answers and 10 simulations across tenants.
- H1 simulation performance: the frozen Orion fixture runs exactly 50,000 trials and must meet the 10-second p95 objective.
- Simulation guardrail: at most 5,000 tasks and 20,000 dependency edges. This maximum-shape class is asynchronous and has no demo latency claim. Performance regression tests also exercise 25,000 and 75,000 trials around the committed 50,000-trial case.
- Ingestion benchmark runs connector reconciliation concurrently with the interactive load and includes duplicates, tombstones, ACL changes, and a 10 percent payload-change rate.
- Data cannot be uniformly random. Benchmarks include high-degree graph hubs, long-tail text sizes, skewed tenant activity, shared provider rate limits, stale projections, and source ACL fan-out.

3. Service level indicators and objectives

3.1 Measurement rules

A valid request enters at the API after TLS termination and ends when the complete response is written or a persisted asynchronous run ID is accepted. Browser network time is measured separately. Invalid authentication, forbidden access, schema-invalid requests, and intentional client cancellation are excluded from availability; service-generated 429, 5xx, deadline expiration, unavailable dependency, and incorrect success count as bad events.

Latency objectives apply only within documented input limits. A response rejected before work because it exceeds a published limit is not a latency violation, but undocumented or incorrectly enforced limits are defects. Planned maintenance counts against availability unless the customer contract explicitly excludes it.

Catalog QAR	SLI	H1 gate	H2 objective and window
QAR-PERF-001	Good valid interactive API responses / all valid interactive API requests	p95 under 2 s and p99 under 5 s at H1 load	99.9 percent good and p95 under 2 s monthly
QAR-PERF-003	Cited answers completed, verified, and streamed before deadline / accepted cited-answer runs	At least 95 percent under 20 s	At least 95 percent under 20 s over rolling 28 days while model endpoint SLI is healthy
QAR-PERF-002	Frozen 50,000-trial Orion simulations completed before deadline / accepted reference runs	At least 95 percent under 10 s	At least 95 percent under 10 s over rolling 28 days at H2 concurrency
QAR-SYNC-001	Healthy-provider observations queryable with required projection watermark within 15 min / observed provider changes	99 percent, and the scripted demo completes within 60 s	99 percent rolling 7 days
QAR-SEC-003 and QAR-COR-001	Sensitive Jira executions with one terminal receipt and no duplicate effect / accepted grants	100 percent	100 percent; any duplicate is Severity 1
QAR-SEC-002	Revocations blocked immediately and fully projected within the horizon target / accepted revocations	100 percent blocked immediately, 95 percent projected within 15 min	100 percent blocked immediately, 99 percent projected within 5 min
QAR-OBS-001	Audit roots exported and verified within 15 min / closed audit periods	100 percent	99.9 percent monthly; no lost events
QAR-AVL-001	Regional service availability	Demonstration run completes	99.9 percent monthly
QAR-DR-001	Restore point age and recovery time	Restore rehearsal documented	RPO at most 1 hour and RTO at most 4 hours
QAR-SYNC-001 and QAR-COR-001	Graph projection after committed normalization and full H1 rebuild	Normal projection within 60 s; full tenant rebuild within 15 min	Pilot target is measured and frozen before H2 exit; revocation still blocks immediately

At 99.9 percent monthly availability, the nominal 30-day error budget is 43 minutes 12 seconds. The SRE dashboard computes the exact budget from the measurement window.

3.2 Error-budget policy

- A 14.4x one-hour or 6x six-hour burn pages the service owner.
- A 2x three-day burn opens an incident and blocks unrelated risk-increasing production releases.
- Exhausting the monthly budget freezes feature promotion until reliability is restored or the accountable owner and customer approve a documented exception.
- Security isolation, duplicate mutation, audit loss, and deletion failure are correctness incidents and have no error budget.
- External model/provider availability is shown separately, but the product still reports the user-visible failure. Dependency exclusion cannot hide platform retry, timeout, or degradation defects.

4. Resource budgets and admission control

Every request or workflow reserves a tenant budget before work:

Resource	H1/H2 enforcement
HTTP body	Endpoint schema limit; default 1 MiB, webhooks 10 MiB raw maximum unless provider contract is lower
Pagination	Default 50, maximum 200 records; opaque cursor; no unbounded export in request path
Graph traversal	Maximum 4 hops, 10,000 visited nodes, 50,000 visited edges, 5,000 returned UI/API elements, and 2 s database time; AI retrieval further caps the envelope at 500 graph nodes
Vector retrieval	Maximum 200 candidates before exact tenant/ACL post-filter and maximum 40 evidence chunks in a model envelope
Model context	Capability-specific token budget recorded before call; no automatic unlimited retry or context growth
Simulation	Frozen 50,000-trial workload for the interactive SLO; 5,000-task/20,000-edge maximum accepted asynchronously; CPU/memory/trial reservation and cancellation between bounded batches
Connector	Per-tenant and per-provider page concurrency, bytes/minute, request/minute, and workflow deadline
Export	Asynchronous only, one active export per tenant in H1 and configurable H2 quota
Jira action	One target issue per grant, one in-flight action per issue, and a tenant mutation rate limit

Admission returns a safe 413, 422, 429, or 503 problem response with limit and retry guidance. It cannot enqueue work that has no bounded completion or cancellation path.

Backpressure order is:

1. Stop research and optional enrichment.
2. Delay embeddings and non-urgent graph projection.
3. Preserve high-priority permission revocation, tombstones, action reconciliation, and audit export.
4. Shed new AI and simulation work per tenant.
5. Preserve authentication, authorization, health, cancellation, kill switch, and incident access as long as PostgreSQL is healthy.

Noisy-neighbor controls exist at API concurrency, Temporal task queues, provider requests, model calls, simulation workers, database connection pools, and graph queries. A tenant cannot consume reserved capacity for security control paths.

5. Scaling strategy and transition triggers

Component	H1/H2 scaling method	Evidence that triggers a change
Web/API	Stateless replicas, autoscaling on concurrency and latency, bounded connection pools	Sustained p95 above target after query and pool tuning
Sync worker	Per-provider and per-tenant Temporal task queues; replicas scale on oldest task age	Oldest healthy task over 2 min for 15 min with provider capacity available
Intelligence worker	Separate queues for extraction, query, simulation, and evaluation; CPU/memory and model concurrency quotas	Queue delay consumes 20 percent of user deadline or utilization above 70 percent for 30 min
PostgreSQL	Query/index tuning, connection pooling, time/range partitioning for audit/outbox, vertical scaling, read replicas only for staleness-tolerant reads	Primary CPU above 65 percent or storage latency above target for 30 min at representative peak after tuning
pgvector	Tenant-prefiltered indexes, dimensionality/corpus review, partitioning, tuned candidate count	Recall target and p95 cannot both pass, or vector workload materially harms OLTP
Neo4j	Tenant placement, memory/page-cache sizing, query-plan review, read replicas where supported	Tenant graph exceeds tested envelope or p95 traversal exceeds 1 s after query/index tuning
Outbox	Batch dispatch, partitioning, retention, consumer checkpoints	Sustained publish load exceeds safe PostgreSQL budget, replay window exceeds retention budget, or 3 independent high-throughput consumers need streams
Object storage	Managed elastic service, multipart upload, lifecycle policies	No application sharding before provider limits are measured
Valkey/Redis	Replica/failover, bounded TTL, key eviction policy, tenant quotas	Cache hit or limiter latency affects API objective; correctness remains independent
Search	PostgreSQL text search and pgvector first	Introduce OpenSearch only under the benchmark gate in CH-04
Analytics	PostgreSQL operational reports first	Introduce ClickHouse/lakehouse only when governed analytics load harms operational state

H3 targets require a pilot capacity report containing per-tenant distributions, peak-to-average ratios, growth, query classes, model cost, provider quotas, database/graph working sets, failure history, operational staffing, and total cost. A percentile without the input distribution is not valid evidence.

6. Distributed-systems semantics

6.1 Delivery, ordering, and idempotency

- Webhooks, outbox events, and Temporal activities are at least once.
- The webhook inbox unique key is provider installation plus provider delivery ID. Reused IDs with a different content hash are quarantined.
- Each aggregate has a monotonically increasing version within a tenant. Events include tenant, aggregate ID, aggregate version, event ID, causation ID, and trace context.
- Consumers transactionally insert an inbox receipt and apply an effect. A duplicate returns the prior outcome.
- Older aggregate versions cannot overwrite newer state. Gaps pause that aggregate and request reconciliation rather than guessing.
- Connector cursor, observations, and publication commit atomically. A page failure cannot advance the cursor.
- Idempotency records retain request digest and terminal response. Reusing a key with a different digest returns 409.
- The Jira worker does not retry an ambiguous mutation. It reconciles provider state and receipt evidence before any repeat.

6.2 Timeouts, retries, and circuit breaking

Operation	Attempt timeout	Retry contract
PostgreSQL interactive statement	1.5 s	Serialization/deadlock only, maximum 5 attempts with full jitter from 25 to 400 ms inside request deadline
Neo4j interactive traversal	2 s	One retry only for a clearly transient connection failure and only if watermark remains valid
Valkey cache	100 ms	No retry in interactive path; bypass cache
Provider GET/page	Connect 3 s, read 20 s, total 30 s	Up to 8 attempts with full jitter, 1 s to 5 min, respecting Retry-After and workflow deadline
Model cited answer call	Capability deadline at most 15 s within 20 s run	At most one safe retry or evaluated fallback if remaining deadline permits
Simulation activity	9 s for interactive class	No blind retry after deterministic compute error; infrastructure retry uses same snapshot and seed
Jira mutation	Connect 3 s, total 20 s	Retry only when no request bytes were accepted or provider idempotency/reconciliation proves no effect
Projection event	30 s	Exponential retry up to 20 attempts, then tenant-scoped dead-letter state and alert
Audit export	30 s	Durable retry until retention deadline; failure pages before 15 min

A dependency circuit opens after at least 20 calls and more than 50 percent eligible failures over 30 seconds, remains open for 30 seconds, then permits bounded probes. Security denials, 4xx validation, and tenant quota rejections do not trip dependency circuits. Circuit state is per provider/region and tenant where a tenant credential can be the cause.

Retries always consume a deadline and budget. There are no infinite hot retries. Durable workflows can schedule a later reconciliation after terminal operational failure, but the original run reaches an explicit terminal or intervention state.

7. Dependency failure semantics

Failure	User-visible and system behavior	Recovery signal
PostgreSQL primary unavailable	Stateful API returns 503; workers pause before effects; no projection-only writes; sensitive commands are rejected	Successful primary transaction plus replica/backup health
PostgreSQL failover	Clients reconnect with jitter; uncertain transactions reconcile by idempotency key; no manual replay until status known	New primary writable, lag and RLS checks pass
Neo4j unavailable	Graph queries/simulations report graph_unavailable; non-graph administration remains available; projection events remain durable	Projection reaches required watermark and integrity check passes
Neo4j corrupted or divergent	Quarantine tenant projection, rebuild shadow graph from PostgreSQL, compare counts/hashes/ACL, switch alias	Rebuild acceptance report
Object store unavailable	Inbox can remain durable, but source observation and cursor do not claim captured payload; ingestion backs off	Content hash write/read verification passes
Valkey unavailable	Cache bypass; durable fallback limiter for sensitive writes or fail closed; latency may degrade	Cache probe and limiter parity pass
Temporal unavailable	API persists eligible commands plus outbox and returns run ID; expiring commands fail if safe dispatch cannot occur; workers resume from history	Namespace and task queue healthy
Identity provider unavailable	Existing unexpired sessions can continue within tenant policy; new login and high-risk recent-auth checks fail closed	Issuer/JWKS and authentication probe pass
KMS/secret manager unavailable	Cached short-lived decrypt grants may finish already authorized reads; new connector/model/action secret use fails closed	Key decrypt and audit probe pass
GitHub/Jira unavailable or rate limited	Preserve prior versions, mark source freshness degraded, respect Retry-After, reconcile later	Successful scoped probe and cursor progress
Model provider unavailable	Cited answer fails with safe retry guidance; deterministic search and simulation remain available; no unapproved model	Evaluated model health and schema probe pass
OpenTelemetry collector unavailable	Bounded local buffer then drop ordinary telemetry with counter; security audit remains in PostgreSQL and independent export	Collector export success; dropped count alert
Network partition	Minority/unreachable workload stops writes; no split-brain application authority; retry after endpoint discovery	Authoritative primary and workflow ownership confirmed
Clock drift	Above 500 ms alerts; above 2 s workload becomes unready for approvals, expiry, signatures, and ordered events	NTP offset below threshold for 5 min

Partial answers must name missing sources and freshness. The product cannot convert "source unavailable" into "no risk found."

8. High availability

H2 reference deployment uses:

- at least two web, API, sync-worker, and intelligence-worker replicas across failure zones;
- topology spread, pod disruption budgets, graceful termination, and readiness before traffic;
- PostgreSQL multi-zone primary/standby with automated failover and point-in-time recovery;
- Temporal production cluster or managed service with multi-zone persistence;
- Neo4j topology appropriate to the licensed deployment and tenant envelope, with rebuildability as the final recovery control;
- managed multi-zone S3-compatible object durability and versioning;

- Valkey/Redis primary/replica or managed failover, with correctness independent of it;
- redundant ingress, DNS, OpenTelemetry collectors, and identity/key service endpoints.

The deployment is single-writer per authoritative PostgreSQL database. H2 does not claim active-active regions. During failover, availability is preferred only when consistency and tenant isolation remain provable.

Deployments drain work safely:

1. Mark replica unready.
2. Stop accepting new requests or Temporal activities.
3. Finish or heartbeat/cancel in-flight work before termination grace.
4. Release database/graph connections.
5. Let Temporal reassign uncompleted activities; idempotency handles replay.

9. Backup, disaster recovery, and corruption

9.1 H2 recovery plan

Asset	Backup or reconstruction	RPO contribution	Recovery order
PostgreSQL business database	Continuous WAL/archive plus daily full backup, encrypted and immutable for retention window	At most 1 hour, target 15 min	1
Temporal persistence	Managed/cluster database backup coordinated with workflow namespace	At most 1 hour	2
S3-compatible evidence	Versioning and approved-region replication or immutable backup	At most 1 hour for new objects	3
Neo4j	Rebuild from restored PostgreSQL; optional backups shorten RTO	No independent RPO	4
Valkey/Redis	Rebuild or fail over	No business-data RPO	5
Configuration and policy	Git plus signed release artifacts; Restricted config from secret manager backup	Last approved change	Before application traffic
Audit root ledger	Independently retained signed roots and audit export	No accepted event loss	Verified before reopening sensitive actions

Backups use separate credentials, encryption keys, accounts/projects where supported, and deletion protection. A backup job success is not restore proof.

Recovery procedure:

1. Declare incident, choose an approved recovery region within tenant residency, and freeze writes and connector credentials.
2. Restore infrastructure and secrets from reviewed code and secret-manager recovery.
3. Restore PostgreSQL and Temporal to a mutually consistent point no older than one hour. If workflow histories are newer, reset/reconcile workflows from business idempotency state.
4. Restore/verify object hashes, replay deletion and revocation tombstones newer than the selected restore point, and prevent any erased data from becoming queryable.
5. Rebuild shadow graph/vector/search projections, validate tenant counts, hashes, ACL coverage, and checkpoints.
6. Reconcile connector cursors and every in-flight external action before enabling workers.
7. Validate RLS, tenant negative tests, audit chain/root, KMS, IdP, health, and synthetic user journeys.
8. Reopen read traffic, then background processing, then AI, and finally external mutation after explicit incident approval.

H2 runs a monthly component restore and a quarterly full recovery exercise. The full exercise must demonstrate RPO and RTO with measured timestamps. An annual exercise includes loss of the primary region when the tenant residency contract permits a

recovery region. If residency forbids any second region, the customer-facing recovery commitment must state that constraint before onboarding.

Database corruption uses point-in-time recovery to a shadow environment, integrity checks, and controlled cutover. No operator repairs authoritative rows from a graph or cache.

10. Observability contract

10.1 Signals and correlation

OpenTelemetry is mandatory in all workloads. A request ID and W3C trace context flow through HTTP, outbox events, Temporal workflows, provider calls, graph queries, model calls, and action receipts. Logs contain trace_id and span_id. Tenant appears as a pseudonymous bounded identifier only where needed; actor and source content do not become metric labels.

Required span classes are:

- http.server and http.client;
- db.postgresql.query with normalized operation, not raw SQL;
- db.neo4j.query with template ID, not Cypher text;
- temporal.workflow and temporal.activity;
- connector.fetch, connector.normalize, projection.apply;
- retrieval.relational, retrieval.vector, retrieval.graph, evidence.authorize;
- ai.responses, ai.tool, ai.verify;
- simulation.compile and simulation.run;
- approval.decide, action.execute, action.reconcile, action.compensate;
- audit.append and audit.export.

10.2 Required metrics

Metric family	Required dimensions	Purpose
edt_http_requests_total and edt_http_duration_seconds	route template, method, status class, workload, region	Availability and latency
edt_sse_connections and edt_run_event_lag_seconds	run type, region	Streaming health
edt_db_pool and edt_db_statement_duration_seconds	workload, operation class, outcome	Pool saturation and slow queries
edt_outbox_oldest_seconds, edt_outbox_pending, edt_inbox_duplicates_total	event class, consumer	Event backlog and duplication
edt_temporal_task_queue_delay_seconds and edt_activity_failures_total	queue, activity class, outcome	Workflow saturation
edt_source_age_seconds and edt_connector_requests_total	provider, connector state, outcome	Freshness and provider health
edt_projection_lag_events and edt_projection_lag_seconds	projection type, region	Graph/vector/search convergence
edt_revocation_blocked_reads_total and edt_revocation_projection_seconds	store class, outcome	Permission safety
edt_graph_query_duration_seconds and edt_graph_budget_exceeded_total	query template, outcome	Traversal performance and abuse
edt_ai_run_duration_seconds, edt_ai_tokens_total, edt_ai_cost_units, edt_ai_failures_total	capability, pinned model ID, outcome	Quality, latency, and cost
edt_citation_verification_total and edt_abstentions_total	capability, outcome/reason	Grounding safety
edt_simulation_duration_seconds and edt_simulation_samples_total	engine version, input class, outcome	Simulation capacity and reproducibility
edt_approval_state_total and edt_action_state_total	action type, state/reason	Approval funnel and mutation correctness
edt_rate_limit_total and edt_budget_rejections_total	capability, reason	Abuse/noisy-neighbor detection
edt_audit_export_lag_seconds and edt_audit_integrity_failures_total	region, outcome	Audit recoverability
edt_backup_age_seconds, edt_restore_duration_seconds, edt_rebuild_lag_seconds	asset, region, outcome	Recovery readiness

Metrics must not label raw tenant ID, actor, resource ID, issue key, provider object ID, prompt, evidence ID, or error message. Per-tenant operational investigation uses access-controlled logs or an on-demand bounded view.

10.3 Structured logs

Each log has timestamp, severity, service, release, environment, region, event_name, safe message, request/trace/span IDs, workflow/run ID where applicable, pseudonymous tenant key where permitted, operation class, outcome, stable error code, retryability, and duration. Logs never contain request bodies, source text, prompts, model output, tokens, cookies, authorization headers, secrets, raw SQL/Cypher, or unredacted provider responses.

Errors and all approval/action/security events are traced at 100 percent. Ordinary successful traffic uses head sampling with tail retention for slow or anomalous traces. Sampling cannot affect authoritative audit.

10.4 Health endpoints

Endpoint	Meaning
/health/live	Process event loop and internal deadlock watchdog are responsive; it does not call dependencies
/health/ready	Workload can safely accept its class of work, required tenant-neutral configuration is loaded, clock is within 2 s, and required authoritative dependency is reachable
/health/startup	Migrations/config compatibility and warmup are complete
/health/dependencies	Authenticated operator view of dependency, circuit, queue, watermark, and last success; never public and never includes secrets

Neo4j failure does not make an administration-only API replica unready, but graph routes return explicit dependency status. Worker readiness is queue-specific where the orchestrator supports it.

11. Alerts and runbooks

Paging alerts require an actionable runbook, owner, severity, dashboard, and clear condition. Required pages include:

- SLO fast or slow burn;
- any cross-tenant canary or RLS test failure;
- any duplicate or unauthorized external mutation;
- audit integrity failure or export lag over 15 minutes;
- revocation blocked-read failure or projection over 5 minutes;
- oldest outbox/task age threatening the 15-minute freshness objective;
- PostgreSQL availability, replication, corruption, storage, or connection exhaustion;
- KMS/secret failure affecting security paths;
- backup age over 1 hour or failed restore rehearsal;
- global provider failure only when user impact or data freshness crosses objective.

Capacity warnings are tickets, not pages, unless exhaustion is imminent. Runbooks include impact, safety invariant, immediate containment, dashboards/queries, dependency status, recovery, reconciliation, customer communication owner, evidence preservation, and exit checks.

12. Acceptance criteria

Evidence ID	Canonical acceptance	Criterion
TC-REL-001	AC-REL-001	H1 and H2 benchmark reports use the exact capacity envelope and load shapes in this chapter and meet every applicable latency/freshness threshold.
TC-REL-002	AC-DATA-001, AC-ACT-002, and AC-REL-003	Outbox, inbox, workflow, projection, simulation, and Jira action tests prove idempotency under duplicate, delayed, reordered, and crash-after-effect delivery.
TC-REL-003	AC-REL-001 and AC-TEN-001	Saturating one tenant cannot violate another tenant's interactive or revocation budget at the stated load.
TC-REL-004	AC-REL-003	PostgreSQL, Neo4j, object, Valkey, Temporal, IdP, KMS, connector, model, telemetry, and network faults produce the documented fail-closed or degraded behavior.
TC-REL-005	AC-DATA-002	A graph corruption exercise rebuilds a shadow projection and validates tenant counts, evidence hashes, ACL coverage, and watermark before cutover.
TC-REL-006	AC-REL-002	H2 restore evidence demonstrates no more than one hour of authoritative loss and service recovery within four hours, including tombstone replay and in-flight action reconciliation.
TC-REL-007	AC-OBS-001	SLO dashboards derive from documented SLIs, and synthetic bad events demonstrably consume error budget and trigger the expected alerts.
TC-REL-008	AC-OBS-001	Every required trace crosses API, outbox/Temporal, data stores, model/provider calls, and terminal run state without exposing Restricted content.
TC-REL-009	AC-REL-003 and AC-OBS-001	Health probes distinguish process, readiness, startup, and dependency degradation and do not cause restart loops during external outages.
TC-REL-010	AC-OBS-001 and AC-REL-003	Every page has an exercised runbook, owner, dashboard, safe containment, reconciliation, and exit criteria.
TC-REL-011	AC-DOC-002 and AC-REV-001	A pilot exit report freezes or explicitly leaves provisional each H3 scale/SLO/cost/isolation/residency/DR target from measured evidence.

CH-11 - User Experience and Visualization Specification

Status: committed | Owners: product-design, frontend-engineering, accessibility-engineering | Last reviewed: 2026-07-13

User Experience and Visualization Specification

1. Experience goals

The interface helps an authorized user move through six distinct cognitive tasks without collapsing them into a conversational black box:

1. Establish what data is available, current, and permitted.
2. Ask a question and inspect evidence for each material claim.
3. Explore relationships and time without implying completeness or causality.
4. define and compare a reproducible scenario.
5. Review, approve, execute, audit, and if necessary compensate an exact external action.

6 Inspect a synthetic physical asset spatially, correlate its telemetry and lifecycle context, and safely exercise simulator-only controls.

`REQ-UX-001`: H1 MUST include cockpit, search, graph, evidence, timeline, scenario comparison, agent-run, approval, connector-health, and audit surfaces.

`REQ-UX-002`: Every screen MUST define loading, empty, error, denied, stale, partial, offline, destructive, and recovery states where applicable.

`REQ-UX-003`: H1 MUST meet WCAG 2.2 AA interaction, contrast, keyboard, focus, reduced-motion, and assistive-technology requirements.

`REQ-UX-004`: Graph and simulation visualizations MUST expose source evidence, time, confidence, permissions, uncertainty, and accessible tabular alternatives.

In addition, every screen identifies the active tenant. A tenant change clears tenant-scoped history, drafts, cached data, selected entities, and conversation context before loading the destination tenant. Observed facts, resolved claims, inferences, recommendations, scenario assumptions, simulated outputs, proposed actions, and completed actions use distinct labels and semantics. Freshness and partial-result state appear at the point of use. Every H1 journey is completable without chat, pointer, drag-and-drop, hover, animation, color perception, or spatial graph interpretation.

2. Application shell and information architecture

2.1 Persistent shell

The authenticated shell contains:

- Skip link to main content as the first focusable control.
- Product name and environment badge.
- Active tenant switcher with synthetic-data or non-production badge where applicable.
- Primary navigation.
- Global command palette and search trigger.
- Freshness/degradation indicator.
- Notifications entry point.
- Help and glossary entry point.
- User menu with active roles, accessibility preferences, session, and sign-out.

Desktop uses a visible left navigation rail and top context bar. Tablet uses a collapsible navigation drawer. Mobile uses a top tenant bar plus a bottom bar for Home, Ask, Explore, Scenarios, and More. The same URL and heading hierarchy are used at every breakpoint.

2.2 Primary navigation

Group	Destinations	H1 visibility
Understand	Home, Ask, Explore, Timeline	Visible
Plan	Scenarios, Simulation Runs	Visible
Act	Proposed Actions, Approvals, Action Receipts	Visible to authorized roles; counts never reveal unauthorized items
Operate	Connectors, Sync Runs, Audit	Visible by role
Govern	Identities and Access, Policies, Ontology, Extensions, Retention	H1 shows required tenant administration; later items are status-labeled
Personal	Notifications, Saved Views, Reports, Preferences	Preferences visible; collaboration features status-labeled

Authorization removes inaccessible destinations from routine navigation. Direct navigation to an unauthorized tenant resource returns the non-enumerating not-found treatment, not a role or object existence disclosure.

2.3 URL and navigation rules

- Tenant-scoped routes include a non-secret tenant slug for orientation, while authority derives only from the authenticated session.
- Entity, claim, evidence, scenario, run, approval, and action links use opaque IDs, never source text or email addresses.
- Browser Back and Forward preserve filters and selected tabs but never resurrect an expired approval, revoked evidence, or previous tenant context.
- Destructive or external-action transitions use a server-created preview resource; URL parameters cannot encode approval.
- Deep links open the required tenant only after authorization and show the user's current access scope.

3. Screen inventory

Status values follow the architecture-wide `Committed`, `Provisional`, `Research`, and `Rejected` definitions.

3.1 Entry and orientation

Screen and route	Status	Audience	Required content and behavior
Product entry `/`	Committed H1	Signed-out visitor or demo judge	Concise mission, evidence/simulation/action story, synthetic-demo disclosure, security posture link, accessibility link, and sign-in. No customer logos or unsupported outcome claims.
Sign-in `/sign-in`	Committed H1	All users	Enterprise identity-provider choice, privacy notice, session error recovery, device and phishing-safe guidance. Credentials are entered only at the identity provider.
Tenant chooser `/tenants`	Committed H1	Multi-tenant users	Authorized tenants, environment, role summary, last access, and search. Counts and names for unauthorized tenants never appear.
Home `/:tenant/home`	Committed H1	Tenant users	Data freshness, connector status, recent cited answers, active scenarios, approval tasks, audit-relevant alerts, and clear first-run guidance. Cards are role-filtered and never infer hidden counts.
System status `/:tenant/status`	Committed H1	Tenant users	Per-capability health, latest safe checkpoint, freshness, degraded behavior, and incident reference. Does not reveal infrastructure topology or secrets.

3.2 Ask and evidence

Screen and route	Status	Audience	Required content and behavior
Ask `/:tenant/ask`	Committed H1	Analyst and domain users	Question composer, scope selector, source/freshness summary, example prompts, run budget, and prior runs. Submitting shows the frozen evidence scope and creates a cancellable run.
Answer run `/:tenant/answers/:runId`	Committed H1	Run owner and authorized collaborators	Streaming stage status, final answer, claim-level evidence controls, confidence language, assumptions, missing evidence, model/profile version, freshness, bounded follow-up, and export. External actions are never implicit follow-ups.
Agent run inspector `/:tenant/runs/:runId`	Committed H1	Run owner, authorized auditors, and operators	Capability-profile and phase timeline, evidence IDs, tool names and validated argument digests, handoffs, budgets, policy decisions, approvals, retries, model/profile version, termination reason, and redacted trace linkage. It exposes no private chain-of-thought, secret, hidden ACL, or inaccessible source content.
Evidence drawer	Committed H1	Any answer viewer	Source title, source system, authorized snippet, source and ingestion times, claim relation, transformation history, access status, and open-in-source action. It is a focus-managed dialog on narrow layouts and a complementary panel on wide layouts.
Knowledge browser `/:tenant/knowledge`	Committed H1	Analysts and stewards	Faceted list of authorized entities, claims, evidence, relationships, source, confidence, validity time, and quality state. Default is list/table, not graph.
Entity detail `/:tenant/entities/:entityId`	Committed H1	Authorized users	Canonical attributes, aliases, provenance, relationships, timeline, ACL summary, data-quality warnings, merge history, and report-correction entry. Hidden relations do not influence visible counts.
Evidence detail `/:tenant/evidence/:evidenceId`	Committed H1	Authorized users	Immutable source digest, normalized observation, transformation lineage, claims supported or contradicted, retention status, and source link. Raw payload requires separate scope and is redacted by default.
Resolution review `/:tenant/resolution/:decisionId`	Committed H1	Data stewards	Candidate records, match and non-match evidence, rule/model version, confidence, resulting canonical identity, split action, impact preview, and rebuild status.

3.3 Explore and time

Screen and route	Status	Audience	Required content and behavior
Graph explorer `/:tenant/explore`	Committed H1	Analysts and domain users	Search-first bounded subgraph, relation and time filters, layout controls, evidence side panel, path explanation, current node/edge cap, truncation notice, table alternative, and shareable saved view.
Relationship detail `/:tenant/relationships/:relationshipId`	Committed H1	Authorized users	Direction, type, endpoint labels, validity interval, confidence, source evidence, derivation rule, contradiction state, and history. Direction and semantics are written in plain language.
Timeline `/:tenant/timeline`	Committed H1	Analysts and domain users	Time-ordered source events, claims, decisions, simulation snapshots, actions, and corrections. Users can switch among source time, validity time, and ingestion time.
Dependency view `/:tenant/dependencies/:rootId`	Committed H1	Program and engineering users	Directed acyclic scheduling view, cycle warnings, blockers, milestones, critical-path occupancy, assumptions, and accessible path list. It cannot imply causality beyond the typed relationship.
Organization view `/:tenant/organization`	Provisional H2	Authorized domain users	Team and reporting structures at approved granularity, effective dates, vacancies, and source quality. Individual activity metrics are excluded.
Asset twin `/:tenant/assets/:assetId`	Committed H1	Authorized operators	Synthetic disclosure, live/pause state, interactive rotatable perspective SVG, synchronized component list, telemetry with units and age, deterministic anomaly/forecast model card, lifecycle history, exact simulator-command preview, execution receipt, and audit evidence. The route remains usable if the visual cannot render.

3.4 Scenarios and simulations

Screen and route	Status	Audience	Required content and behavior
Scenario library `/:tenant/scenarios`	Committed H1	Analysts	Owned/shared scenarios, status, baseline snapshot, last run, engine version, tags, and archive. It distinguishes a mutable draft from an immutable executed version.
Scenario builder `/:tenant/scenarios/new`	Committed H1	Analysts	Goal, selected baseline, typed interventions, assumptions, calendar, seed, validation, non-effects, estimated budget, and confirmation. Natural-language input compiles to structured fields that the user must inspect.
Scenario detail `/:tenant/scenarios/:scenarioId`	Committed H1	Authorized viewers	Version history, structured intervention diff, assumptions, validation findings, runs, collaborators, and duplicate-to-draft. An executed scenario version is immutable.
Simulation run `/:tenant/simulations/:runId`	Committed H1	Authorized viewers	Progress, cancellation, baseline/scenario percentile comparison, distribution, critical path, blockers, sensitivity, warnings, snapshot, seed, engine, trials, and runtime. A table and narrative expose the same result.
Compare `/:tenant/simulations/compare`	Committed H1	Analysts	Side-by-side selection with compatible-baseline validation, aligned measures, delta explanations, and export. Incompatible engine versions or snapshots are visibly marked and never silently normalized.
Simulation playback `/:tenant/simulations/:runId/playback`	Provisional H2	Analysts	Trial aggregation over time with pause, step, speed, and text summary. It is not required for the H1 decision and must pass an accessibility-value review before commitment.

3.5 Governed action

Screen and route	Status	Audience	Required content and behavior
Proposed actions `/:tenant/actions`	Committed H1	Analysts and action operators	Draft, awaiting approval, ready, expired, executing, completed, failed, and compensation states; filters do not expose unauthorized counts.
Action preview `/:tenant/actions/:actionId/preview`	Committed H1	Requester and approvers	Exact target, canonical payload hash, before/after field diff, expected source version, evidence/scenario references, policy decision, required roles, expiry, risk, idempotency scope, and compensation plan. Payload is read-only.
Approval inbox `/:tenant/approvals`	Committed H1	Approvers	Tasks requiring the actor's specific role, expiry, requester, action type, target, risk, and status. Bulk approval is prohibited for H1 external actions.
Approval decision `/:tenant/approvals/:approvalId`	Committed H1	Eligible approver	Full preview, approval-role statement, exact expiry, conflict-of-interest/self-approval result, approve and decline with optional rationale. No preselected decision and no countdown animation that pressures action.
Action execution `/:tenant/actions/:actionId`	Committed H1	Authorized executor	Final policy recheck, approval summary, execution status, cancellation semantics, connector response, immutable receipt, and next safe action. Refresh cannot resubmit the connector write.
Receipt and rollback `/:tenant/actions/:actionId/receipt`	Committed H1	Authorized viewers/operators	Request and response IDs, before/after snapshots, approvals, timestamps, trace reference, idempotency result, rollback eligibility, exact compensation preview, and compensation receipt or conflict.

3.6 Operations and governance

Screen and route	Status	Audience	Required content and behavior
Connectors `/:tenant/admin/connectors`	Committed H1	Tenant and connector admins	Connector type, scope, allowlist, credential health without secret value, sync freshness, webhook state, rate-limit state, last reconciliation, and disable control.
Connector setup `/:tenant/admin/connectors/new`	Committed H1	Connector admins	Provider authorization, requested scopes, tenant/source binding, allowlist, data categories, retention summary, validation, and explicit install confirmation.
Sync run `/:tenant/admin/sync/:runId`	Committed H1	Connector admins and stewards	Durable workflow stages, cursors, item counts, duplicates, tombstones, retries, quarantines, checkpoints, partial failures, reconciliation, and safe resume.
Audit `/:tenant/audit`	Committed H1	Security, compliance, tenant admins	Filterable append-only event index, actor, delegated authority, resource, policy decision, trace, redaction, export, and chain from answer through compensation. Search and export are themselves audited.
Audit event `/:tenant/audit/:eventId`	Committed H1	Authorized auditors	Canonical event, schema version, integrity state, actor and tenant derivation, request correlation, related events, redacted payload, retention class, and export status.
Identities and access `/:tenant/admin/access`	Committed H2	Tenant admins	Source-to-actor mappings, SSO/SCIM memberships, roles, delegations, connector identities, revocation, access review, and effective-policy explanation. H1 fixture roles are read-only outside the product administration surface.
Policies `/:tenant/admin/policies`	Committed H2	Security admins	Versioned policy bundles, change diff, validation, staged activation, rollback, impacted actions, and test cases. H1 shows policy explanations but policy editing is configuration-managed.
Ontology `/:tenant/admin/ontology`	Provisional H2	Data stewards and extension authors	Core and namespaced types, versions, constraints, provenance, compatibility, and package status. Core types cannot be modified in place.
Extensions `/:tenant/admin/extensions`	Provisional H2	Tenant admins and developers	Signed connector, ontology, workflow, and tool packages; permissions; compatibility; evaluation; enable/disable; and provenance. Marketplace discovery is later than private installation.
Retention and deletion `/:tenant/admin/data-governance`	Committed H2	Privacy and tenant admins	Data categories, retention rules, legal holds, deletion requests, projection erasure, evidence impact, and signed completion report. H1 exposes read-only policy documentation.

3.7 Personal, collaboration, and reporting

Screen and route	Status	Audience	Required content and behavior
Notifications `/:tenant/notifications`	Provisional H2	All users	Approval requests, completed runs, stale sources, access changes, shared analyses, and policy events. Sensitive content is omitted from push/email bodies.
Saved views `/:tenant/saved`	Provisional H2	Analysts	Saved queries, graph scopes, filters, and simulation comparisons with permission revalidation on open.
Reports `/:tenant/reports`	Provisional H2	Analysts and executives	Versioned evidence-backed report drafts, review status, data snapshot, export, and expiration. Reports never freeze authorization to source evidence.
Collaboration panel	Provisional H2	Authorized collaborators	Comments anchored to claims/scenario versions, mentions, review decisions, and activity. A comment cannot grant source access.
Preferences `/:tenant/preferences`	Committed H1	All users	Theme, density, timezone, date format, reduced motion, graph simplification, notification settings, keyboard shortcuts, and data-export defaults.

4. Common state contract

Every screen implements each applicable state below with a stable heading, status text, recovery action, focus behavior, and telemetry event. A spinner without explanatory text is never a complete state.

State	Required presentation and behavior
Initial loading	Preserve shell and heading skeleton; announce loading once after a short delay; avoid focus movement; permit cancellation for long runs.
Incremental refresh	Keep prior authorized content, label it with its checkpoint, show refresh progress non-modally, and replace only after authorization recheck.
Streaming	Show named stages and elapsed time; mark partial text as provisional; do not expose hidden chain-of-thought; allow cancel where safe.
First-use empty	Explain why the area is empty, the role required for the primary action, and the safe next step.
Filtered empty	Preserve filters, say no authorized results match, and offer clear-filters without implying hidden results.
Current success	Show source/checkpoint freshness and a complete primary action outcome.
Stale success	Show last safe checkpoint, age, affected conclusions, and refresh/retry; never use stale policy to authorize an action.
Partial result	Name unavailable sources or capabilities, what remains usable, and what conclusions are limited. Partial is not styled as full success.
Redacted by permission	Omit protected values and structural hints; explain that accessible evidence is incomplete and provide an access workflow if configured.
Permission lost	Immediately remove protected content from view and client cache, stop dependent work, preserve only a non-sensitive run reference, and announce the change.
Validation error	Place a summary at the top, associate field errors programmatically, preserve valid input, and move focus to the summary.
Version conflict	Show the current safe source version and stale draft diff; require re-preview and re-approval where action semantics changed.

State	Required presentation and behavior
Dependency degraded	Name the capability, not internal topology; describe safe fallback and disabled actions; link to tenant-safe status.
Rate limited	Show retry time and preserved work; background retries use bounded backoff and can be canceled.
Offline or disconnected	Keep non-sensitive already-rendered content read-only, mark it stale, queue no external action, and require revalidation after reconnect.
Session expired	Remove protected data from the viewport and local storage, preserve only an opaque return route, and reauthenticate before recovery.
Not found	Use a single non-enumerating treatment for missing and unauthorized tenant resources.
Model unavailable or invalid	Preserve the question and evidence scope, show a safe retry or approved fallback, and create no recommendation or action.
Traversal truncated	Show node/edge/depth limits, omitted count only when policy-safe, and controls to refine scope instead of silently clipping.
Awaiting approval	Show required roles, collected decisions, expiry, and payload hash; never reveal approver membership beyond authorized policy output.
Approval expired or invalidated	Freeze the invalid decision history, explain the invalidating event, and require a new preview.
Executing	Show durable action state and safe navigation; disable duplicate submission but allow refresh and receipt recovery.
Completed	Show verified source result and immutable receipt. A visual success state requires both, not merely an HTTP response.
Compensation available	Show the exact restore diff, precondition, risks, approval policy, and idempotency behavior.
Compensation conflict	Show that no overwrite occurred, current authorized source state, prior receipt, and the new reviewed-action path.
Fatal error	Provide correlation ID, safe retry, support path, and preserved non-sensitive draft; never expose stack, query, secret, prompt, or tenant data.

`TC-UX-001` (evidence for `AC-UX-001`): Automated component tests render every applicable state for every H1 route; end-to-end tests cover the state transitions in the reference workload, including permission revocation and compensation conflict.

5. Interaction patterns

5.1 Search and command palette

Global search returns only tenant-authorized entity, claim, scenario, action, and navigation results. Results are grouped by type, show freshness, and never reveal counts for inaccessible groups. The command palette provides navigation and non-destructive commands. It MUST NOT approve or execute an external action.

`Ctrl/Cmd+K` opens the command palette. `/` focuses search when the focus is not in an editable field. `?` opens shortcut help. `Esc` closes the topmost non-destructive overlay and restores focus. All shortcuts have menu equivalents, can be disabled where appropriate, and avoid browser or assistive-technology conflicts.

5.2 Citations and evidence

Each material answer claim has a visible semantic label: `Observed`, `Resolved`, `Inferred`, `Assumption`, `Simulated`, or `Recommendation`. Citation controls name source and age. Opening evidence does not lose the reader's position; closing it restores focus to the invoking claim. If a citation becomes unauthorized, the content is removed and the answer is re-evaluated or marked no longer verifiable.

5.3 Forms and drafts

Drafts use explicit Save status and server-side versioning. Sensitive drafts are not placed in browser local storage. Auto-save failures are announced without stealing focus. Natural-language scenario input, AI-suggested fields, and connector-suggested mappings remain proposals until the user confirms the structured representation.

5.4 Approvals

Approvals use a dedicated page, not a chat response, toast, email link, or modal layered over unrelated work. Approve and Decline have equal visual weight. The approver must be able to inspect the entire canonical diff and policy summary without scrolling through generated persuasion. Any edit creates a new proposal and invalidates approval. The UI never preselects approval, uses celebratory animation, or hides expiry and rollback risk.

5.5 Responsive behavior

- At 320 to 599 CSS pixels, tables become labeled record lists or horizontal regions with an explicit table alternative; no data column silently disappears.
- At 600 to 1023 pixels, secondary panels become drawers with focus trapping and return.
- At 1024 pixels and above, evidence or inspector panels may be persistent while the primary heading and reading order remain logical.
- Touch targets are at least 24 by 24 CSS pixels with sufficient spacing; primary controls target 44 by 44 where layout permits.
- Graph canvas gestures always have visible button and keyboard equivalents.

5.6 Motion and theme

Motion communicates state only when needed. Under `prefers-reduced-motion`, animated graph transitions, count-up values, chart interpolation, skeleton shimmer, and simulation playback are disabled or replaced by discrete updates. Dark, light, and system themes use tokenized colors that meet contrast requirements. Status meaning, graph type, and uncertainty never rely on theme-specific color alone.

6. Visualization system

6.1 Shared visual grammar

Meaning	Encoding
Source observation	Solid outline plus `Observed` text/icon
Resolved canonical entity or relationship	Solid line with provenance control
Inferred relationship	Dashed line plus confidence text
Scenario-only entity or edge	Dotted line plus `Scenario` label
Contradicted claim	Split warning marker and explicit contradiction text
Stale data	Clock marker, age text, and patterned treatment
Permission-limited result	Omitted protected geometry plus an accessible incompleteness notice; no ghost node that leaks topology
Critical path	Increased line weight, `Critical path` label, and ordered path list
Risk severity	Text label and shape/icon in addition to a color token
Uncertainty	Distribution, interval, or range with numeric table; never an unlabeled blur or single-point gauge

Every visualization includes a title, decision question, source/checkpoint time, legend, filters summary, visible limits, reset, download policy, and data-table or structured-text alternative. Exported images include title, legend, snapshot, timestamp, and synthetic-data watermark for H1.

6.2 Visualization catalog

Visualization	Status	Decision supported	Required implementation
Knowledge graph	Committed H1	What authorized entities and relationships support this question?	Search-first bounded subgraph; deterministic initial layout; type and time filters; evidence inspector; path list; node/edge cap; table equivalent.
Relationship explorer	Committed H1	Why does this relationship exist and how has it changed?	One focused edge with endpoints, direction, validity, confidence, contradiction, derivation, and evidence timeline.
Dependency graph	Committed H1	Which sequence gates a milestone?	Directed layout; typed edges; cycle detection; critical-path occupancy; blocker list; hidden-edge-safe counts; topological list alternative.
Project flow	Committed H1	Where does work move, wait, or block?	Stage and dependency view derived from Jira state, with duration source and missing-data warnings; not an individual productivity chart.
Risk heatmap	Committed H1	Which project risks combine likelihood and impact?	Small bounded matrix; every cell has text, count only when disclosure-safe, and linked list; simulated and observed risks remain separate.
Event timeline	Committed H1	What changed, when, and according to which clock?	Zoomable but keyboard-operable sequence with source, validity, ingestion, decision, and action lanes plus tabular event log.
Launch distribution	Committed H1	How do baseline and scenario dates differ?	Overlaid distribution or cumulative curves, p50/p80/p95 markers, trial count, calendar, numeric table, and non-causal comparison language.
Sensitivity view	Committed H1	Which modeled inputs most affect launch uncertainty?	Ranked tornado or interval bars, direction, magnitude, method, and table; sensitivity is not labeled causality.
Critical-path occupancy	Committed H1	How frequently does each path gate launch across trials?	Ranked paths and percentages with confidence/sample information; accessible ordered list is primary on narrow screens.
Evidence lineage	Committed H1	How did a source observation become a displayed claim?	Left-to-right transformation stages, versions, digests, policy checks, and a linear text trace.
Audit sequence	Committed H1	Which actors and controls led from question to action and rollback?	Chronological event list with correlation groups; optional swimlanes by actor/service; canonical table remains authoritative.
Connector freshness	Committed H1	Which source or projection may limit a conclusion?	Status table and small age bars; explicit timestamps and thresholds; no green-only health signal.
Procedural 3D-style physical-asset view	Committed H1 synthetic path	Where on the demonstration asset is the selected condition measured?	Rotatable perspective SVG with component hotspots tied to stable IDs and telemetry; rotate/reset buttons, pointer drag, arrow keys, keyboard component selection, reduced motion, rendering fallback, and an equivalent ordered component/status/measurement view. It is not a CAD model, contains no exclusive data, and is always labeled synthetic.

Visualization	Status	Decision supported	Required implementation
Organization graph	Provisional H2	How are approved teams and reporting structures related at a selected time?	Effective-dated hierarchy with vacancy and source-quality states; no individual performance overlays.
Calendar	Provisional H2	How do modeled milestones and constraints align over working calendars?	Accessible agenda/table first, timezone and working-day rules, scenario overlay, no automatic source mutation.
Aggregate communication flow	Provisional H2	Where do approved team-level handoffs appear under a documented purpose?	Minimum group size, aggregate edges, privacy review, no message sentiment or individual centrality. Direct individual communication graph is Rejected through H3.
Financial flow	Provisional H3	How do approved budgets or cost allocations connect to programs?	Separate authorization domain, currency and period semantics, reconciliation status, lineage, and finance-owner approval.
Geographic view	Provisional H4	Which approved regions, facilities, or assets are affected?	Coarse location by default, residency-aware data, accessible region table, and no precise person location.
Simulation playback	Provisional H2	Does temporal playback add insight beyond comparison and distributions?	Aggregated states, pause/step, reduced-motion mode, transcript, and a measured user-value gate.
3D organizational view	Rejected H1-H3; Research H4	No H1-H3 organizational decision requires it	Rejected for organizational delivery because of occlusion, navigation cost, performance, and accessibility burden. The separate H1 physical-asset scene does not change this decision. H4 organizational research requires a validated decision task that cannot be served by 2D plus table.

6.3 Knowledge graph behavior

The graph never opens on an unbounded tenant-wide force layout. The user starts from an entity, question result, saved view, or typed traversal template. The initial view renders no more than 200 authorized nodes and expands to a hard client rendering ceiling of 500. Server traversal limits may be lower based on policy and cost. Reaching a limit produces a visible truncation state and refinement controls.

Node size encodes only the selected, labeled measure and defaults to a constant. It MUST NOT silently encode degree, popularity, employee rank, or activity. Edge thickness defaults to constant and may encode a labeled quantitative measure only when its unit and provenance are defined. Inferred confidence is shown numerically or categorically, not solely as opacity.

Keyboard users can search nodes, move through the ordered node list, inspect neighbors grouped by relationship type, traverse an edge, pin an item, change filters, and open evidence. A synchronized data table contains the visible authorized nodes and edges. Screen readers are not required to interpret canvas geometry.

6.4 Simulation behavior

Baseline and scenario use a shared horizontal time scale. p50, p80, and p95 are labeled directly and reproduced in a table. The default view emphasizes distributions and differences, not animated particles or a single countdown. The interface states the conditional question, unchanged assumptions, engine version, seed, trial count, snapshot, missing data, and validity limitation before interpretation text.

6.5 Heatmap behavior

Heatmaps contain a small, fixed set of ordered likelihood and impact categories. Each cell has an accessible name, pattern or icon, and link to the filtered risk list. Empty means no authorized modeled risks in the cell, not no organizational risk. Cells never expose hidden counts by subtraction.

7. Content and confidence language

Use direct language:

- `Supported by 3 current sources` rather than `The AI knows`.
- `The simulation places the p80 date at September 24 under these assumptions` rather than `Launch will be September 24`.
- `Accessible evidence is insufficient` rather than `No evidence exists` when permissions or source health limit the result.
- `This relationship was inferred with medium confidence` rather than `Probably related`.
- `The action was not executed because approval expired` rather than a generic error.

Confidence labels map to a versioned calibration policy and include an explanation. Numeric confidence is not shown when the underlying method is not calibrated for that interpretation.

8. Accessibility acceptance

8.1 Required engineering controls

- Semantic landmarks, a single page-level `h1`, logical headings, and native controls before custom ARIA.
- Visible focus with at least 3:1 contrast against adjacent colors.
- Text and interactive-control contrast meeting WCAG 2.2 AA.
- All dialogs have programmatic name, description where needed, focus containment, explicit close, and focus return.
- Live regions are limited to meaningful asynchronous changes and never announce streaming tokens one by one.
- Data tables provide captions, headers, scopes, sorting state, and pagination state.
- Errors are summarized and associated with fields; success is not communicated by color alone.
- Zoom to 200 percent and reflow at 320 CSS pixels without two-dimensional scrolling except for genuinely two-dimensional data regions with equivalent alternatives.
- User-selected timezone and locale are displayed; raw timestamps remain available to auditors.
- Charts and graph controls work at 400 percent zoom or provide an equivalent structured view.
- Pointer gestures, drag, pinch, and hover all have single-pointer and keyboard alternatives.
- Authentication does not rely on cognitive-function tests and supports password-manager and identity-provider flows.

8.2 Manual acceptance journey

`TC-UX-002` (evidence for `AC-UX-001`): Using keyboard and a supported screen reader, a tester can sign in, choose Aster, determine freshness, ask the reference question, inspect every citation and limitation, build the fixed scenario, compare p50/p80/p95 values, inspect the critical path, preview the exact Jira diff, approve or decline, inspect the receipt, and request rollback.

`TC-UX-003` (evidence for `AC-UX-001`): At 320 CSS pixels and 200 percent zoom, the same journey preserves all fields, warnings, evidence states, and action semantics without clipped controls or hidden data columns.

`TC-UX-004` (evidence for `AC-UX-001`): With reduced motion and forced colors enabled, no state, graph relationship type, risk severity, percentile, approval state, or execution result loses its non-color/non-motion representation.

`TC-UX-005` (evidence for `AC-UX-001`): Automated accessibility checks produce no serious or critical violations on H1 routes, and manual review records no WCAG 2.2 A or AA failure in the critical journey.

9. Usability and telemetry acceptance

Product telemetry records route, interaction class, duration, result class, accessibility preference category only when necessary, and correlation ID. It excludes question text, source snippets, entity names, payload values, emails, graph content, and model prompts by default. Tenant-level analytics require tenant policy and cannot join data across tenants without separately governed opt-in.

Test case	Acceptance criterion
TC-UX-006 mapped to AC-UX-001	A first-time reference-workload analyst can reach a cited answer, open its evidence, and identify missing information without facilitator intervention.
TC-UX-007 mapped to AC-UX-001	An eligible approver can correctly identify target, changed fields, expiry, source version, required roles, and rollback behavior before deciding.
TC-UX-008 mapped to AC-UX-001 and AC-ACT-002	No UI control can bypass server authorization, forge tenant context, mutate an approved payload, or turn refresh/retry into a duplicate action.
TC-UX-009 mapped to AC-UX-001	Graph, distribution, sensitivity, heatmap, lineage, and audit visualizations each have an equivalent table or structured-text representation with the same decision-relevant values.
TC-UX-010 mapped to AC-UX-001	User testing detects no case where a participant interprets simulation output as guaranteed, hidden evidence as absent, an inferred edge as observed, or a draft action as executed; any such result blocks release copy.

CH-12 - APIs and Developer Platform

Status: Committed | Owners: API Platform, Developer Experience, Security | Last reviewed: 2026-07-13

APIs and Developer Platform

1. Purpose and interface precedence

This chapter defines the public REST API, run streaming, signed webhooks, event contracts, conditional GraphQL and gRPC surfaces, MCP exposure, extension packages, generated SDKs, and CLI. Machine-readable contracts are normative where they are more restrictive than examples in prose.

Interface precedence is:

1. security, tenancy, authorization, and approval invariants in the architecture;
2. machine-readable contract at its released commit and content hash;
3. this chapter;
4. examples and generated documentation.

An implementation conflict is release-blocking. It is not resolved by accepting whatever the running service currently emits.

ID	Requirement
REQ-ARCH-007	The platform MUST define REST, SSE, webhooks, events, GraphQL, gRPC, MCP, plugin, SDK, and CLI contracts with clear horizon ownership.
REQ-TEN-001	Clients MUST NOT select authoritative tenant context; every request binds it from authenticated membership.
REQ-DEV-001	APIs, events, schemas, ontology packages, connectors, plugins, agents, workflows, SDKs, CLI, and webhooks MUST be versioned and independently testable.
REQ-REL-003	Interface workflows MUST define retries, idempotency, ordering, deduplication, clock behavior, partitions, replay, dead letters, and backpressure.
REQ-SEC-001	Every privileged operation MUST authenticate, authorize, tenant-bind, and audit at execution time.
REQ-ACT-001	Approval/execution APIs MUST bind tenant, actor, credential, target, canonical arguments, expiry, policy version, and idempotency key.
REQ-SEC-007	Plugin and SDK builds MUST use pinned dependencies, provenance, SBOM, signing, scanning, protected CI identities, and verified artifacts.
QAR-PERF-001	Ten H1 users issuing bounded non-AI graph/evidence reads MUST see p95 latency below 2 seconds.
QAR-COR-001	Duplicate, reordered, or replayed requests/events MUST converge to one stable state without duplicate effects.

ID	Requirement
QAR-SEC-001	Guessed cross-tenant identifiers through every interface MUST reveal neither existence nor content.

OpenAPI 3.1 is primary for public command/administration REST. Every applicable contract defines authorization, concurrency, pagination, versioning, rate limits, deadlines, retry, redaction, and audit. Long runs use resumable SSE; external webhooks are authenticated, replay-bounded, at-least-once, versioned, and reconcilable; MCP/plugins remain narrower than internal services and disabled by default.

2. Normative artifacts

The source-controlled contract set is:

Artifact	Status	Role
`contracts/openapi/enterprise-digital-twin.openapi.yaml`	H1 Committed	REST operations, schemas, security, errors, and SSE discovery responses.
`contracts/asyncapi/events.asyncapi.yaml`	H1 Committed	Internal/outbound event channels and CloudEvents payload schemas.
`contracts/schemas/`	H1 Committed	Canonical types, tools, scenarios, manifests, and webhook data.
`contracts/mcp-manifest.json`	H1 Committed	MCP resources, tools, annotations, and approval classes.
`contracts/graphql/schema.graphql`	H2 Provisional	Read-only graph exploration SDL.
`contracts/proto/digital_twin.proto`	H3 Provisional	Extracted internal service boundaries.

Generated artifacts are never edited directly. CI verifies that generated SDKs, reference documentation, examples, and consolidated specification use the exact released contract hashes.

3. Canonical contract types

Versioned domain payloads carry a `schema_version`. Standard protocol envelopes identify their contract through the protocol-defined field and media/schema URI instead: RFC 9457 `type`, CloudEvents `specversion` plus `type`/`dataschema`, and SSE event name plus data schema version. Public representations omit fields the actor cannot access and never rely on omission to grant permission. IDs are opaque UUIDs unless the field explicitly names a provider key.

Type	Required semantics
`TenantContext`	Internal only: tenant ID, actor, active delegations, purpose, policy version, authorization timestamp, region, trace context. It is assembled from authentication plus server-side tenant selection.
`Actor`	Tenant-local ID, kind `human`, `service`, or `agent`, immutable principal/audit reference, status, assurance level, and display fields. An `agent` is a non-authenticating audit subject bound to an invoking principal, capability profile, and reduced delegation; it has no independent membership or credential. Provider users are not actors until linked.
`Delegation`	Grantor, grantee, allowed actions/resources/purposes, valid interval, revocation version, and non-delegable flag. Handoffs take the intersection, never a union.
`ResourceRef`	Resource kind, opaque ID, optional version and projection generation. No URI may contain a storage credential.
`PolicyDecision`	Effect `allow`, `deny`, or `indeterminate`, stable reason codes, obligations, policy version, evaluated time, and audit reference. `indeterminate` is enforced as deny and can never satisfy an authorization or approval. Denial detail is redacted when it would reveal a resource.
`Entity`, `EntityVersion`, `Claim`, `Evidence`, `Relationship`, `OntologyType`, `SourceACL`, `ResolutionDecision`	Bitemporal, evidence-backed data types defined in CH-05. Current views include `data_watermark` and conflict/provenance links.
`SourceObject`, `NormalizedObservation`, `SyncCursor`, `ProjectionCheckpoint`	Connector and projection types defined in CH-05 and CH-06. Raw object URIs and cursor internals are not public.

Type	Required semantics
`AgentRun`, `ToolInvocation`, `ApprovalRequest`, `ApprovedPayload`, `ActionReceipt`, `CompensationResult`	Durable AI/action types defined in CH-07. Public forms expose state and safe rationale, never prompts, reasoning internals, credentials, or encrypted argument refs.
`Scenario`, `SimulationSnapshot`, `SimulationRun`, `Forecast`, `Uncertainty`, `Assumption`	Immutable/versioned types from CH-08. Numeric values, seed, iteration count, and engine version are not rewritten by an explanation layer.
`AuditEvent`	Append-only event ID, tenant, actor/service, action, resource refs, outcome, reason, policy version, trace ID, occurred/recorded time, and payload hash. Sensitive detail is stored separately with stricter access.
`TraceContext`	W3C `traceparent`, optional `tracestate`, request ID, and tenant-safe baggage policy. Client baggage cannot set tenant, actor, or authorization fields.
`Problem`	RFC 9457 problem details plus stable `code`, `request_id`, safe `errors[]`, and optional `retry_after`.
`AssetTwinSnapshot`	Tenant-authorized synthetic asset, spatial components, current/history telemetry, deterministic analytics and model card, lifecycle records, simulator control state, and data watermark.
`AssetControlPreview`	Short-lived exact simulated command, expected version, before/after state, safety checks, payload hash, ETag, expiry, and explicit no-external-write marker.
`AssetControlReceipt`	Idempotent simulator transition result, before/after versions, payload hash, audit evidence, and explicit simulation/no-external-write markers. Replay returns this same receipt.

Canonical date-times are RFC 3339 UTC with millisecond precision; calendar dates are ISO `YYYY-MM-DD`. Monetary amounts use integer minor units plus ISO currency. Arbitrary precision identifiers and seeds are strings. JSON numbers MUST be finite; `NaN` and infinities are invalid.

4. REST/HTTP contract

4.1 Protocol and authentication

- Base path is `/v1`; HTTPS with TLS 1.2 or newer is mandatory outside local development.
- Requests and responses use `application/json`; unsupported media types return `415`.
- Authentication uses an OIDC access token with issuer, audience, signature, expiry, assurance, and revocation checks. Browser sessions use an HTTP-only, secure, same-site cookie and CSRF protection for commands.
- An authenticated principal selects among server-known tenant memberships through a server-issued, short-lived, actor/session- and audience-bound opaque context handle. Browsers carry it in an HTTP-only `EDT-Context` cookie; SDK/CLI clients use `X-EDT-Context`. The handle references a server-side membership and policy version, expires and rotates on context change, and cannot be minted from a client-supplied tenant value. `X-Tenant-ID` is rejected. A `tenant_id` present in a versioned canonical resource schema is only a consistency assertion: the server compares it with derived context and rejects a mismatch; it can never select or expand scope. Public command schemas omit it wherever possible.
- `traceparent` may be accepted after syntax validation. `X-Request-ID` is generated by the server; a safe client correlation ID may be echoed separately.
- Sensitive responses set `Cache-Control: private, no-store`. CDN caching is limited to public documentation and immutable, authorization-safe assets.

4.2 Resource and command surface

Operation	Result and invariants
`GET /v1/me`	Current actor, memberships, active tenant context summary, and safe capabilities.
`GET /v1/entities`	Authorized keyset-paginated entity summaries at one bound watermark; filters and sort fields are allowlisted.
`GET /v1/entities/{entity_id}`	Authorized current entity, selected claims, conflict markers, provenance links, watermark, and ETag.
`POST /v1/graph/traversals`	Executes a registered bounded query template with typed parameters; no Cypher. Returns nodes/edges, truncation, watermark, and projection generation.
`POST /v1/questions`	Creates a grounded-answer run and returns `202`, run resource, status URL, and event URL. Requires `Idempotency-Key`.
`GET /v1/agent-runs/{run_id}`	Safe run state, progress, result when complete, usage summary, and timestamps.
`POST /v1/agent-runs/{run_id}/cancel`	Idempotent cancellation request; `202` while stopping, `200` if terminal.
`GET /v1/connectors`	Installed connector summaries, scopes, state, freshness, and health; secret fields excluded.
`POST /v1/connector-installations`	Starts an authorized installation flow and returns a short-lived provider authorization URL.
`POST /v1/connectors/{installation_id}/reconcile`	Requests an idempotent reconcile/backfill within policy; never accepts a source cursor.
`DELETE /v1/connectors/{installation_id}`	Starts revocation/deletion workflow with confirmation and audit.
`POST /v1/simulation-snapshots`	Compiles and seals an evidence-backed snapshot or returns validation conflicts.
`POST /v1/scenarios`	Creates a typed scenario draft against one snapshot.
`POST /v1/scenarios/{id}/confirm`	Confirms the exact scenario digest for the current actor.
`POST /v1/simulations`	Starts a confirmed simulation and returns `202` plus run/event URLs.
`GET /v1/simulations/{simulation_id}`	Returns authorized durable state, progress, immutable result when complete, and event URL.
`POST /v1/actions/jira/remediation-previews`	Produces an immutable exact `AST-142` field-limited Jira preview in CH-06. It neither opens approval nor mutates.
`POST /v1/actions/jira/remediation-previews/{preview_id}/approval-requests`	Opens one approval request bound to the current immutable preview hash; the response is idempotent and callers cannot choose approvers.
`POST /v1/approvals/{id}/decisions`	Records one human approve/reject decision after reauthentication when policy requires.
`POST /v1/approvals/{id}/execute`	Consumes a valid execution grant; returns original receipt on idempotent replay. It cannot accept replacement payload fields.
`POST /v1/action-receipts/{id}/compensation-previews`	Produces guarded rollback preview. Execution follows a new approval/action flow.
`GET /v1/assets`	Lists tenant-authorized synthetic asset summaries; it does not discover or connect devices.
`GET /v1/assets/{assetId}/twin`	Returns the physical scene/component metadata, current and historical synthetic telemetry, deterministic analytics/model card, lifecycle, simulator control state, and watermark.
`GET /v1/assets/{assetId}/telemetry`	Advances exactly one deterministic five-second frame and returns the most recent 1-120 frames selected by `limit`. Polling this resource is not an industrial streaming or real-time guarantee.
`POST /v1/assets/{assetId}/control-previews`	Validates type, value, reason, expected version, transition, range, tenant policy, and idempotency; returns a short-lived exact simulator-only preview and ETag.

Operation	Result and invariants
`POST /v1/assets/{assetId}/control-previews/{previewId}/execute`	Reauthorizes and executes the unchanged preview once against simulator state using `Idempotency-Key` and `If-Match`; performs no device or network write.
`GET /v1/audit-events`	Authorized keyset-paginated audit index with filters; detail access is separately controlled.
`GET /v1/ontology/types`	Installed, visible ontology catalog and versions.
`POST /v1/extension-packages/validations`	Validates a signed package in quarantine; installation is a separate administrator command.

Collection reads use `GET`. Complex read-only graph searches may use `POST` because bodies are structured and size-bounded; they still have no side effects and are marked accordingly in OpenAPI. Commands return a resource or run; they never return an untracked free-form success string.

4.3 Idempotency and concurrency

Every public `POST`, `PUT`, `PATCH`, and state-changing `DELETE` declares whether `Idempotency-Key` is required. Ordinary command keys are 16 to 128 printable ASCII characters, scoped to tenant, actor, operation, and canonical request hash, and retained for 24 hours or the command retention period, whichever is longer. An approved external action instead uses the action-level key from CH-06, scoped to tenant, target, expected source version, action, and approved payload digest; actor identity is audited but is not part of a namespace that could let two actors execute the same grant twice.

The first request atomically stores key, request hash, state, and eventual response reference. Same key plus same hash returns the original status/result. Same key plus a different hash returns `409 idempotency_key_reused`. In-progress replay returns the same run. A server timeout does not authorize a client to generate a new key for an external write; the client queries the original command.

Mutable administrative resources use strong ETags. `PUT`, `PATCH`, confirmation, and policy-sensitive actions require `If-Match`; absent precondition returns `428`, stale version returns `412`. A scenario draft may transition once to a confirmed state, but its operation list and digest do not change. Snapshot content, confirmed scenario content, an approval's bound payload/digest, and action receipts are immutable; approval decisions and lifecycle transitions are append-only child records that update a derived state projection.

Physical-asset control previews follow the same strong-precondition and replay rules but are not external actions and do not require the Jira action's two-person approval policy. A preview binds the tenant, actor, synthetic asset ID, expected simulator version, command, reason, safety-policy version, expiry, and payload hash. Execution rejects an altered or expired preview, stale version, unsupported transition, out-of-range value, policy failure, or reused key with a different hash. Every response carries `execution_mode: simulation`; every receipt carries `simulation: true` and `external_write: false`.

Asset reads require `asset.read`, preview requires `asset.control.preview`, and execution requires `asset.control.execute`. These application capabilities are evaluated against the server-derived actor and tenant context on every call; a general login or graph-read grant is insufficient.

4.4 Pagination, filtering, and sorting

Collections use keyset pagination with `page_size` from 1 to 100 and an opaque authenticated `page_cursor`. The cursor binds tenant, actor ID, authorization fingerprint/policy version, endpoint, normalized filters, sort, snapshot watermark, last sort key, and expiry. It is not a base64-encoded SQL fragment. Invalid, altered, expired, or differently scoped cursors return `400 invalid_cursor`.

Responses contain `items`, `next_cursor`, `has_more`, and `data_watermark`. Sort fields and filter operators are endpoint allowlists. Default order is stable and ends in immutable ID. Offset pagination is not public. A policy change can invalidate a cursor and return `409 authorization_changed` rather than risk mixed visibility.

4.5 Errors, retry, and limits

Errors use RFC 9457 `application/problem+json`:

```
{
  "type": "https://docs.example.invalid/problems/source-changed",
  "title": "Source changed",
  "status": 409,
  "code": "source_changed",
```

```
  "detail": "Create a new preview from the current source state.",
  "instance": "/v1/actions/opaque-id",
  "request_id": "UUID",
  "errors": []
}
```

`detail` never confirms existence of an unauthorized resource. Validation errors name safe JSON Pointers and stable codes, not raw rejected values. Retryable responses are `408`, selected `409` states, `425`, `429`, `502`, `503`, and `504` only when the operation's idempotency contract permits retry. `Retry-After` is supplied where known.

Default tenant limits are 120 ordinary requests/minute per actor, 20 graph queries/minute, 10 AI/simulation starts/minute, and 5 action/approval commands/minute, with lower anonymous limits for public docs only. Responses include `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`, and `Retry-After` on `429`. Limits are also enforced per tenant, IP risk bucket, connector installation, and provider quota; headers do not reveal other users.

Server deadlines are 2 seconds for ordinary reads, 5 seconds for bounded graph queries, and 10 seconds for synchronous commands that only enqueue durable work. Longer work returns `202`. Client disconnect cancels safe in-process reads but does not erase a durable run or assume an external command failed.

5. Server-Sent Events

`GET /v1/agent-runs/{run_id}/events` and `GET /v1/simulations/{simulation_id}/events` return `text/event-stream` after normal authorization. A stream is a view over durable run events, not the source of truth. The client can always recover from the corresponding run resource.

Durable event frames contain a monotonically increasing per-run `id`, a registered `event`, and one JSON `data` object. H1 event names are:

- `run.accepted`, `run.started`, `run.progress`;
- `retrieval.started`, `retrieval.completed`;
- `tool.started`, `tool.completed`, `tool.failed` with safe tool category only;
- `run.awaiting_confirmation`, `run.awaiting_approval`;
- `simulation.progress` at bounded batch intervals;
- `run.completed`, `run.abstained`, `run.cancelled`, `run.failed`;
- `stream.heartbeat` every 15 seconds.

Durable events carry schema version, run ID, sequence, timestamp, safe progress, and result link when terminal. `stream.heartbeat` is transport-only: it carries the latest durable sequence, has no new event ID, and is not retained or replayed. No event carries prompts, chain-of-thought, raw connector content, tool arguments, tokens, approval secrets, or inaccessible evidence.

The client reconnects with `Last-Event-ID`. The server replays retained run events for at least 24 hours and starts after the supplied sequence. An unknown future ID returns `400`; replay outside retention returns `410 event_history_expired` and directs the client to run state. Streams cap at 30 minutes per connection, support proxy heartbeat, disable buffering, and use per-actor connection limits. Authorization is checked on connect and at least every 30 seconds as a backstop; policy/ACL revocation publishes an immediate connection-invalidation signal and closes affected streams.

6. Webhooks

6.1 Inbound provider webhooks

GitHub and Jira ingress is specified in CH-06. The GitHub callback is bound to the app registration/environment, then maps the signed payload installation ID to exactly one server-side tenant installation. Jira dynamic callbacks use installation-bound opaque paths in addition to their signed bearer token and matched webhook ID. Raw bytes or bearer claims are authenticated before trusting routing fields, durable receipt precedes `2xx`, duplicate delivery is accepted idempotently, and canonical state comes from authenticated reconciliation.

6.2 Outbound platform webhooks

Outbound platform subscriptions are H2 Provisional; H1 commits only authenticated provider webhook ingress and internal outbox events. If the H2 gate is approved, tenants may subscribe only to allowlisted event types such as `connector.sync.completed`, `run.completed`, `approval.requested`, `action.completed`, and `action.compensation.completed`. Source text, full answers, credentials, and raw evidence are not delivered; subscribers fetch authorized detail through REST. Activation requires subscription-management/secret-rotation OpenAPI operations, delivery SLO and quota, SSRF tests, and a support owner.

Payloads use CloudEvents 1.0 JSON and contain `specversion`, global event `id`, stable `source`, versioned `type`, resource `subject`, `time`, `datacontenttype`, `dataschema`, and minimal `data`. Tenant is bound to the subscription and is not treated as a routing instruction in the payload.

Each subscription has a 32-byte random secret. Headers are:

```
X-EDT-Webhook-Id: subscription UUID
X-EDT-Delivery-Id: event UUID
X-EDT-Timestamp: Unix seconds
X-EDT-Key-Id: opaque secret version
X-EDT-Signature: v1=<hex HMAC-SHA256(secret, timestamp + "." + raw_body)>
```

Consumers must require HTTPS, compare signatures in constant time, reject timestamps outside 5 minutes, deduplicate delivery ID for at least 72 hours, and tolerate unknown additive fields. Secret rotation supports current and previous secret for a 24-hour overlap and identifies the key version without exposing it.

Delivery is at least once and ordering is not guaranteed. Retry schedule is 1, 5, 15, 60 minutes, then 6, 12, and 24 hours with jitter. `2xx` acknowledges; `408`, `409`, `425`, `429`, network errors, and `5xx` retry; other `4xx` suspend after three occurrences. A dead-lettered event is visible to administrators and can be redelivered with the same event ID and a new delivery-attempt record. Reconciliation through REST is always available.

SSRF controls resolve and validate DNS on every attempt, forbid loopback/private/link-local/metadata ranges unless an approved private-delivery profile exists, cap redirects at zero, connect only to an IP returned by that validated resolution while using the original hostname for TLS SNI/hostname verification, pin the HTTPS port, and apply connection/body/time limits.

7. Async events and CloudEvents

Internal logical events also use CloudEvents 1.0 semantics and JSON Schema, persisted first through the PostgreSQL transactional outbox. H1 transport is the outbox plus Temporal activities; the AsyncAPI contract describes logical channels without implying Kafka.

Event type format is `com.enterprisedigitaltwin.<domain>.<event>.v1`. The envelope includes tenant ID internally, tenant-qualified `partition_key`, aggregate type/ID/version, causation ID, correlation ID, trace context, occurred time, schema URI/hash, and data classification. The default partition key is the RFC 8785 hash of `{tenant_id, aggregate_type, aggregate_id}`; a projection channel that requires total tenant outbox order instead hashes `{tenant_id, projection_name}` and declares that exception in AsyncAPI. A partition key is an ordering key, never an authorization input. Event consumers deduplicate global event ID and enforce monotonically increasing aggregate version where ordering matters.

Events describe facts in past tense. Commands are not smuggled through event topics. Payloads contain stable IDs and changed fields, not complete sensitive documents. A consumer encountering a version gap pauses the aggregate and reconciles from the authoritative API. Poison events enter a tenant-scoped dead-letter ledger with redacted diagnostics.

Adding an optional field is backward compatible. Removing/renaming fields, changing meaning/type, making optional data required, or reinterpreting ordering requires a new event major type. Producers publish old and new types during migration; consumers prove readiness before old publication stops.

8. GraphQL and gRPC conditional surfaces

8.1 GraphQL

GraphQL is H2 Provisional and read-only. It exists only if design-partner evidence shows that REST query templates cannot support graph exploration. It exposes canonical entity, relationship, claim, evidence citation, and connection types; it has no mutation, subscription, arbitrary Cypher, secret field, or tenant argument. The provisional `JSON` scalar is limited to schema-validated, redacted extension properties and cannot carry arbitrary storage records.

Production accepts registered persisted operations by hash. Authorization occurs in data loaders before fetch and again during result serialization. Schema fields declare classification and cost. Default limits are depth 4, cost 10,000, 5,000 returned nodes, 2-second resolver deadline, and 1 MiB response. Introspection is administrator-only outside development. N+1 access is

prevented with tenant/ACL-aware batch loaders whose cache lasts one request.

Collections use Relay-style connections with opaque authorization-bound keyset cursors; offsets and tenant arguments are absent. Cursor invalidation, rate limiting, errors, redaction, audit, and idempotency for the read-only surface inherit the REST rules. A resolver may return fewer authorized nodes plus `truncated=true`; it never fills a requested count with inaccessible neighbors.

Fields are additive within a major schema. Removal requires `@deprecated` for at least two minor releases and 12 months, usage telemetry, migration documentation, and a major release if semantics break. Clients must handle nullable fields and unknown enum values through generated unknown cases.

8.2 gRPC

gRPC is H3 Provisional for extracted service-to-service boundaries only. Public clients use REST. Protobuf packages use `edt.<domain>.v1`, stable field numbers, wrapper/message types for presence, `google.protobuf.Timestamp`, and explicit pagination tokens. Field numbers and enum numeric values are never reused; removals are reserved.

Mutating RPCs carry command ID/idempotency and explicit expected version. Every RPC declares deadline, retry class, max message size, and authorization action. mTLS workload identity plus an identity-aware interceptor derives tenant/service context; tenant metadata from callers is matched to a signed delegation and never trusted alone. Errors use canonical gRPC status plus typed details mapped consistently to RFC 9457.

Streaming RPCs require flow control, cancellation, bounded buffers, and resume tokens. A service extraction is approved only after contract, load, failure, deployment, and rollback evidence shows a benefit over the modular API process.

9. MCP contract

The H1 MCP server is a thin policy-enforced facade over application services. It uses OAuth authorization, derives tenant and actor from the session, and exposes no tenant selector, generic URL fetch, SQL, Cypher, shell, filesystem, or unrestricted provider operation.

9.1 Resources

URI template	Content
`edt://entities/{entity_id}`	Authorized entity summary with watermark and provenance links.
`edt://evidence/{evidence_id}`	Authorized citation-safe evidence view, not object-store URI.
`edt://runs/{run_id}`	Safe run state and result link.
`edt://scenarios/{scenario_id}`	Confirmed scenario diff and assumptions.
`edt://simulation-runs/{run_id}`	Structured forecast and comparison.

Resource reads are reauthorized each time. Unknown and unauthorized IDs return indistinguishable not-found responses.

9.2 Tools

Tool	Class	Approval
`twin_search_evidence`	Read	User approval according to MCP client policy; never auto-approved for external MCP clients.
`twin_get_entity`	Read	Same as above.
`twin_traverse_graph`	Read	Same as above; registered edge enums and H1 traversal limits.
`twin_compile_scenario`	Draft	Requires user confirmation before simulation.
`twin_run_simulation`	Compute	Confirmed scenario ID only.
`twin_preview_jira_remediation`	Draft	No write; exact safe preview.
`twin_request_approval`	Workflow	Opens an approval only; cannot choose approvers.
`twin_execute_approved_action`	External write	Explicit client approval plus valid server-side two-person execution grant; arguments cannot contain replacement payload.

All tool inputs and outputs use strict JSON Schema with `additionalProperties: false`, stable limits, side-effect annotations, and schema hashes. Tool names are immutable within v1. An incompatible schema or changed side effect requires a new tool name ending `_v2` and a migration period. Tool results contain resource links and safe summaries, not credentials or bulk source data.

MCP search/list results use the same opaque authorization-bound cursors and maximum page size as REST. Read tools have a 2-second service deadline, graph traversal has its stricter CH-07 limit, and long simulation returns a run resource rather than holding the call open. Per-actor and per-tenant limits mirror or narrow REST limits. Errors expose stable safe codes and retryability. Every resource read, tool proposal, approval, denial, invocation, redaction, and result hash is audited. The execute tool requires an action-level idempotency key and returns the original receipt on replay; no other tool invents side effects through retry.

Remote MCP servers are disabled by default. Enabling one requires server identity verification, exact tool allowlist, egress/data-flow review, approval mode, per-tool data classification, incident owner, and revocation. No model can add an MCP server at runtime.

10. Extension and plugin packages

10.1 Package manifest

Every `.edtpkg` is a signed archive with a canonical manifest that validates against `contracts/schemas/plugin-manifest.schema.json`. The schema requires exact publisher identity, semantic version and compatibility hashes, content-addressed component descriptors, purpose-bound permissions, default-deny exact-origin egress, server-derived tenant isolation, SBOM/provenance/signature references, and reversible lifecycle behavior.

Components may be a connector manifest, ontology package, bounded agent profile, workflow definition, UI panel, or SDK metadata. Each component has its own normative schema under `contracts/schemas/` and a content hash. The signature bundle is detached over the canonical manifest and component Merkle root; the signature file itself is excluded from that root, avoiding a circular archive signature. Installation verifies publisher signature, transparency/provenance policy, SBOM, malware scan, license policy, compatibility, migrations, permissions, network destinations, resource limits, and fixtures in quarantine.

Plugins are tenant-disabled by default. An administrator reviews the exact diff and permissions. A package cannot request database superuser, RLS bypass, arbitrary network, raw model key, arbitrary filesystem, secret values, tenant selection, or authorization override. UI panels run in a sandboxed origin with a capability bridge and strict Content Security Policy; they receive only explicitly authorized serialized data.

10.2 Component contracts

Component	Mandatory contract
Connector	Provider/auth, exact scopes/endpoints, object types, webhooks, cursor, schemas, source precedence, rate policy, tombstones, retention, network allowlist, and separately enumerated mutations as defined in CH-06.
Ontology	Namespaced types/edges, JSON Schemas, identity, cardinality, classification, indexing, retention, migration, fixtures, and compatibility rules from CH-05.
Agent profile	Purpose, allowed tools, strict output, budgets, memory, handoffs, approval class, termination, eval suite, and prohibited use. It may only narrow installed platform capabilities.
Workflow	Typed states/transitions, actor/action policy, timeouts, retries, idempotency, compensation, cancellation, audit events, and version migration. No arbitrary executable expressions.
UI panel	Route, capability requests, data schemas, accessibility declaration, bundle integrity, CSP, size/performance budget, and empty/loading/error states.

Upgrades are staged, migration-tested, and reversible before activation. Disabling stops new invocations immediately. Uninstall preserves governed data until an explicit migrate/archive/delete plan completes. Marketplace publication is H3 Provisional and adds independent security review, publisher verification, vulnerability response SLA, revocation, and tenant impact reporting.

11. SDK and CLI contracts

11.1 SDKs

H1 freezes independently testable TypeScript and Python client behavior in `contracts/schemas/sdk-contract.schema.json`; distributable generated packages are not part of the demonstrator artifact. A conforming client provides:

- OIDC token callback or server credential interface without persisting secrets by default;
- typed request/response/problem objects with unknown-field tolerance;
- async keyset iterators that preserve cursors;
- idempotency-key generation and original-command lookup;
- ETag/`If-Match` helpers;
- SSE reconnect with `Last-Event-ID` and terminal-state fallback;
- webhook signature verification over raw bytes;
- timeouts, cancellation, structured logging hooks, trace propagation, and safe retry classification;
- access to raw response metadata without weakening type validation.

SDKs never retry a non-idempotent command automatically, choose a tenant from untrusted input, log tokens/bodies, or hide partial results. Generated code is reproducible and versioned with the API contract. Go, Java, C#, Kotlin, Swift, Rust, C++, and native mobile SDKs are Conditional on measured customer demand; they do not block the stable REST contract.

11.2 CLI

The public `edt` CLI contract is defined by `contracts/schemas/cli-contract.schema.json` and is implemented as a thin SDK client after its H2 packaging gate. Core commands are `auth login`, `context list|use`, `connector list|sync`, `entity get`, `graph query`, `ask`, `scenario create|confirm`, `simulate`, `approval list|decide`, `action status|execute`, `audit list`, `plugin validate|install`, and `schema pull`. The implemented H1 repository-local `scripts/edt.mjs` utility is deliberately narrower: it starts, seeds, verifies, inspects, stops, or resets the synthetic Compose environment and is not the public administration CLI.

Human output defaults to tables/progress; `--output json` emits only versioned JSON to stdout and diagnostics to stderr. Non-interactive mode requires explicit context and never opens a browser. Tokens use OS credential storage. `--yes` may skip a local confirmation but cannot bypass server policy, reauthentication, user confirmation, or dual approval. Destructive/uninstall operations display resource and tenant context and require explicit command flags.

CLI exit codes distinguish success, validation, authentication, authorization, conflict, rate limit, retryable service failure, cancellation, and internal error. Shell completion never fetches sensitive names without authentication. A `--debug` flag redacts authorization, cookies, source content, webhook secrets, and signed URLs.

12. Compatibility, deprecation, and lifecycle

Surface	Compatible change	Breaking change and policy
REST/OpenAPI	Add optional response field, operation, or opt-in feature; widen documented non-security limit.	Remove/rename field, change type/meaning/default, make optional input required, narrow accepted enum, or change side effect. Publish `/v2`, migration guide, and at least 12-month H2 support window.
JSON Schema	Add optional property while readers tolerate unknown fields; add new schema ID.	Required property, changed constraints/meaning, removed enum. New major schema URI.
Errors	Add safe metadata or a new stable error code.	Reuse a code with changed semantics or change retryability. Version operation/major contract.
SSE	Add event type or optional field; clients ignore unknown events and reconcile run state.	Change sequence, replay, event meaning, or remove required field. New event schema major.
Webhooks/AsyncAPI	Add optional data or new event type.	Change event meaning/type/required data. Dual-publish new major event.
GraphQL	Add nullable field/type.	Remove/change field or make nullable non-null. Deprecate for two minors and at least 12 months; use schema major when required.
Protobuf	Add new field number or method with safe semantics.	Reuse tag/enum, change wire type/meaning, or remove without reserve. New package major.
MCP	Add resource/tool with no implicit access, or optional schema field.	Change tool side effect, required args, output meaning, or approval. New versioned tool name/server major.
Plugin	Add optional manifest field or component type understood as unsupported by old hosts.	New required field, permission semantics, migration format, or host API. New manifest/host major.

Deprecation is announced in documentation, changelog, SDK warnings, `Deprecation` and `Sunset` HTTP headers where applicable, administrator notifications, and usage reports. Security fixes may shorten a window; they require an incident decision, safe replacement, and direct customer notice. Removed identifiers, Protobuf tags, event types, and tool names are never reused.

Clients MUST ignore unknown response object properties and unknown SSE/webhook event types, preserve opaque cursors, and implement unknown enum handling. Servers MUST reject unknown command properties where the schema uses `additionalProperties: false`; silent typo acceptance is forbidden.

13. Security, privacy, and operations

Authorization occurs at request entry, service operation, data retrieval, and result serialization. A gateway check alone is insufficient. Policy unavailability fails closed for protected data. Resource existence is concealed on unauthorized reads. Batch operations authorize each item and never return mixed-tenant data.

Input limits cover headers, path/query length, JSON depth, object/array count, strings, regex complexity, decompression, multipart size, and request time. Parsers reject duplicate security-sensitive JSON keys and ambiguous Unicode normalization. Output encoding is context-specific; Markdown/HTML from models or connectors is sanitized before rendering.

Every command emits an audit event with actor, action, resource, policy version, idempotency key hash, outcome, and trace ID. Read auditing is risk-based but mandatory for evidence, exports, audit logs, approval detail, plugin packages, and sensitive graph traversals. Logs contain safe IDs and hashes, not tokens, prompts, source excerpts, or webhook bodies.

Operational metrics include request rate/status/latency by operation, auth failures, policy denials, rate-limit decisions, idempotency replay/conflict, ETag conflict, cursor invalidation, SSE connections/replay gaps, webhook success/retry/dead letters, contract-version usage, SDK versions, GraphQL cost if enabled, MCP approvals/denials, and plugin health. High-cardinality tenant and actor IDs are hashed or mapped through controlled exemplars.

14. Verification and acceptance

ID	Acceptance criterion
AC-DOC-001	OpenAPI, AsyncAPI, JSON Schemas, MCP manifest, GraphQL SDL, Protobuf, frontmatter, and examples parse and lint cleanly.
AC-TEN-001	Contract tests derive tenant context and reject mismatched assertions, guessed IDs, cross-tenant cursors, unauthorized batches, graph/MCP bypasses, and existence leaks.
AC-DATA-001	Same-key/same-body command replay returns the original result, different-body reuse returns `409`, and randomized duplicate event delivery converges.
AC-CON-001	H1 inbound webhook tests verify authentication, raw-byte integrity where applicable, expiry/replay, duplicates, retries, and reconciliation; the H2 outbound gate additionally requires signature rotation, redelivery, SSRF/DNS rebinding defense, and delivery reconciliation.
AC-ACT-002	Ambiguous and concurrent execution requests remain queryable and create exactly one Jira effect and one receipt.
AC-PHY-001	Physical-twin contract tests reproduce seeded telemetry and analytics, conceal cross-tenant asset IDs, reject unsafe/stale/mutated/replayed-conflicting previews, produce one simulator transition for a valid command, and prove zero device egress.
AC-SUP-001	Generated SDKs and signed plugins include SBOM/provenance, pass conformance/scanning, never retry unsafe commands, and fail malicious packages before installation.
AC-REL-001	H1 non-AI API load meets the 2-second p95 target and SSE/webhook connection budgets without bypassing policy or rate limits.
AC-OBS-001	REST, SSE, webhook, MCP, workflow, and external action activity correlates through one redacted trace/audit path.

Additional conformance checks prove pagination has no duplicate/omitted fixture rows at a fixed watermark, rejects altered cursors, and invalidates on permission change; SSE reconnects exactly after `Last-Event-ID`; MCP rejects tenant, SQL, Cypher, URL, replacement payload, and undeclared arguments; and compatibility CI blocks every unversioned breaking change. Testing includes unit schema tests, generated client/server contracts, authentication/authorization matrices, cursor/idempotency properties, fuzzing of JSON/headers/URLs/signatures/SSE/Protobuf/packages, provider replay, load/soak, and dependency chaos. Documentation examples execute against the reference server in CI.

15. Risks and evolution

ID	Risk	Mitigation
RSK-001	A public, GraphQL, gRPC, MCP, SDK, or plugin interface leaks another tenant.	Server-derived context, no tenant selectors, central policy, bounded templates, output reauthorization, and adversarial two-tenant conformance.
RSK-002	A pooled Neo4j query exposed through REST, GraphQL, or MCP omits tenant isolation.	Tenant-qualified persisted templates, no generic graph query, central limits, result reauthorization, and a dedicated-database/cell trigger.
RSK-013	Idempotency is mistaken for external-provider atomicity.	Command ledger, source revalidation, post-write verification, ambiguous state, original receipt, and compensation.
RSK-004	MCP or plugin content causes tool misuse or data exfiltration.	Strict schemas, approval, signed/quarantined packages, capability permissions, egress allowlist, sandboxing, and adversarial tests.
RSK-009	Interface generations amplify the platform's four-runtime operational complexity.	REST primary contract, shared schemas, deterministic generation, bounded supported majors, telemetry, and release-blocking conformance.
RSK-018	Webhook/event backlog or exactly-once assumptions create stale state.	At-least-once contract, IDs/versions, dedup, backpressure, dead letter, retry, and reconciliation.

H1 commits REST, SSE, authenticated provider webhook ingress, logical AsyncAPI events, the constrained MCP facade, TypeScript/Python SDK and CLI conformance contracts, and package validation. Distributable public SDK/CLI packages, H2 outbound platform webhooks and read-only GraphQL, H3 gRPC extraction, marketplace distribution, and additional native SDKs are Provisional and require usage, security, performance, and support evidence before activation.

CH-13 - Deployment and Operations

Status: Committed | Owners: Platform Engineering, Site Reliability Engineering, Release Engineering | Last reviewed: 2026-07-13

Deployment and Operations

1. Deployment principles

Cloud neutrality means the application is packaged as OCI images, production workloads run on Kubernetes through Helm, infrastructure is described with OpenTofu-compatible HCL, authoritative state uses PostgreSQL-compatible SQL and S3-compatible objects, identity uses OIDC/SAML/SCIM, and telemetry uses OpenTelemetry. It does not mean simultaneous active-active deployment to multiple clouds, maintaining duplicate infrastructure stacks, or guaranteeing identical behavior from every provider service.

All environments are created or reconciled from reviewed source. Production changes use signed immutable artifacts and GitOps. Manual mutation is an incident or break-glass event, must be audited, and must be reconciled back to code.

2. Supported deployment profiles

Profile	Status	Intended use	Topology and data
local-compose	Committed	Developer workstation, CI integration, deterministic demonstration	Docker Compose; one instance of each workload and dependency; synthetic data only; no HA claim
h1-demo	Committed	Hackathon presentation and reproducible evaluator environment	Compose on one secured host, exactly two synthetic tenants, and the versioned deterministic GitHub/Jira provider simulators in `fixtures/h1/`; a live provider sandbox is an optional non-normative extension and cannot replace acceptance evidence
h2-shared-regional	Committed	Up to ten design-partner tenants	One Kubernetes regional application plane; multi-zone managed PostgreSQL, object, Temporal, graph and cache; pooled relational tenancy with isolated tenant data namespaces
h4-dedicated-tenant	Provisional	Customer requiring a dedicated data plane	Separate account/project, cluster and data services; same images/contracts; introduced only after cost and operations review
h4-customer-vpc	Provisional	Vendor-managed application in a customer-controlled network/account	Dedicated data plane, customer network and key integrations, supported egress contract
h4-on-premises	Provisional	Customer-operated Kubernetes and approved local dependencies	Validated distribution, support bundle, upgrade preflight, no unmanaged configuration drift
h4-air-gapped	Provisional	Disconnected customer environment	Offline signed bundle and mirror; connectors/model capabilities disabled unless approved local endpoints exist
h4-regional-multi-cluster	Provisional	Residency or recovery placement	Single authoritative writer with controlled failover; no active-active claim
edge	Research	Bounded read-only/disconnected use	No authoritative writes, external actions, identity merge, or autonomous synchronization until consistency and revocation are designed

Only local-compose, h1-demo, and h2-shared-regional are release-blocking for the current specification. Kubernetes and Helm are adopted at the H2 boundary because they are part of the committed regional pilot topology; their supported versions, charts, resource envelopes, and conformance evidence must be frozen before the first design-partner production deployment. Other profiles cannot be sold as supported until their acceptance suite, operations ownership, upgrade path, recovery target, security review, and cost model pass.

3. Local and H1 Compose profile

The Compose project contains:

- web;
- API;
- synchronization worker;
- intelligence worker;
- PostgreSQL with pgvector;
- Temporal server and its persistence;
- one pooled Neo4j logical projection with mandatory tenant-qualified nodes, relationships, query templates, credentials, and result reauthorization; per-tenant databases or instances are not an H1/H2 deployment choice;
- MinIO;
- Valkey;
- disposable OIDC provider;
- OpenTelemetry collector plus reference Prometheus, Grafana, Loki, and Tempo services;
- optional GitHub/Jira provider simulators for deterministic tests.

Rules:

1. Only web and API bind host interfaces by default. Database, graph, cache, object, Temporal, and telemetry administration bind loopback or an internal Compose network.
2. Development certificates and secrets are generated per checkout and stored outside version control. They are marked development-only and cannot satisfy production configuration validation.
3. Health checks and dependency ordering control readiness, but applications still tolerate dependency restarts.
4. Images run the same entrypoints, schema versions, non-root user, and read-only filesystem used in Kubernetes.
5. Named volumes make restarts realistic. The reset command refuses non-synthetic environments and prints the resolved absolute volume/project scope before deletion.
6. A seed value creates the same two tenants, identities, source payloads, oracle, graph, scenarios, and expected action target.
7. Local deterministic tests can run without public provider/model credentials by using signed provider fixtures and recorded model-safe stubs. Stubs cannot satisfy the model-integration, AI-evaluation, or cited-answer release gate; the H1 presentation/release evidence includes an approved live OpenAI endpoint run. Live mode is explicit and is not used to establish deterministic fixture assertions.
8. Compose is not a production HA profile and does not support real customer data.

The required developer experience is a cross-platform Node launcher that implements bootstrap, start, stop, seed, test, verify, export-logs, and reset. It wraps Compose and workspace commands so developers do not memorize container sequencing.

4. H2 regional Kubernetes topology

4.1 Accounts, networks, and namespaces

The reference deployment uses a dedicated cloud account/project and virtual network per environment and region. Production, staging, CI, and development do not share clusters, databases, KMS keys, object buckets, IdP clients, or credentials.

Kubernetes namespaces:

Namespace	Contents	Access
edt-ingress	ingress controller and certificate integration	Public load balancer to web/API routes only
edt-app	web and API deployments	Can reach required data endpoints; no provider/model secret except API-required capability
edt-workers	synchronization and intelligence workers	No public ingress; queue-specific identities and egress
edt-observability	OpenTelemetry collectors and reference agents	Receives telemetry; cannot query business databases
edt-operators	external-secrets and GitOps agents	Narrow cloud/Kubernetes permissions; no source-content read

Temporal and stateful databases are managed services or run in a separately controlled data namespace/account. Running production PostgreSQL, object storage, or Neo4j inside the application cluster requires a deployment-profile ADR proving backup, restore, upgrades, anti-affinity, storage failure, security, and operator ownership.

Network zones are:

- public edge: CDN/WAF where available, load balancer, TLS, DDoS controls;
- application: web/API private pods;
- worker: no inbound public path;
- data: private endpoints for PostgreSQL, Neo4j, object, Valkey, Temporal, KMS and secrets;
- egress: explicit proxies/private endpoints for GitHub, Jira, approved model endpoints, identity, package/security update services, and telemetry destination.

Default-deny NetworkPolicy applies to ingress and egress. DNS is allowed only through the cluster resolver. Cloud metadata endpoints are blocked from pods. Provider egress validates host after DNS resolution and redirect.

4.2 Workload security and scheduling

Every pod:

- runs as a fixed non-zero UID/GID with runAsNonRoot;
- uses readOnlyRootFilesystem and explicit ephemeral volumes;
- sets allowPrivilegeEscalation false;
- drops all Linux capabilities;
- uses RuntimeDefault seccomp;
- has automountServiceAccountToken false unless Kubernetes API access is required;
- has CPU/memory requests and limits based on load tests;
- has liveness, readiness, and startup probes with the semantics in CH-10;
- emits graceful shutdown and stops accepting work before termination;
- uses a dedicated service account and workload identity;
- is admitted only if image digest, signature, provenance, vulnerability policy, and required labels pass.

The cluster enforces the Restricted Pod Security Standard. Web/API and workers use separate node pools only when workload isolation, GPU, or resource economics require it; H1/H2 do not require GPU. TopologySpreadConstraints distribute replicas across zones and nodes. PodDisruptionBudgets preserve at least one ready replica while respecting safe maintenance.

Horizontal scaling signals:

- web/API: request concurrency, p95 latency, and CPU, with maximum database connections as a hard ceiling;

- sync worker: Temporal task queue age and provider quota;
- intelligence query worker: task age relative to run deadline and model concurrency;
- simulation worker: queued CPU work and reserved memory;
- no autoscaler may exceed tenant quota, provider limit, model cost budget, or database connection budget.

Minimum H2 replicas are two for each first-party workload and OpenTelemetry collector across failure zones. A queue can be served by a shared worker deployment only if one workload image still preserves the logical task-queue and resource isolation.

4.3 Data services

- PostgreSQL uses multi-zone primary/standby, encrypted storage, automated failover, continuous point-in-time recovery, private endpoints, non-superuser workload roles, connection pooling, and query/audit extensions approved by security.
- Neo4j uses the pooled logical-namespace controls in CH-05 and CH-09 with private endpoints. Typed gateways are the only clients, every element is tenant-qualified, and results are reauthorized. Its topology must pass isolation, tenant restore and shadow-rebuild tests at 10 million edges per tenant. Dedicated graph placement is an H3 benchmark/security transition, not the H1/H2 default.
- Object storage enables versioning, encryption, access logging, lifecycle, deletion protection for backups, and approved-region replication for RPO.
- Valkey/Redis uses private endpoints, TLS, ACLs, failover, max-memory policy suitable for disposable cache data, and no public access.
- Temporal uses a production-supported multi-zone topology or managed service, separate namespace, TLS, payload codec configuration, retention sized for recovery, and queue-level permissions.
- KMS and secret manager have separate production keys/policies, rotation, deletion protection, audit logging, and break-glass recovery.

Application configuration resolves tenant storage placement from authoritative control data. A caller cannot choose a database, graph, bucket, region, or key.

5. Infrastructure as code

OpenTofu is the reference engine and sole source of shared-infrastructure truth. Terraform execution is allowed only when the same module tree and state format are compatible; maintaining forked OpenTofu and Terraform definitions is prohibited.

Module layers are:

1. bootstrap: remote state storage, locking, KMS, CI deploy identity, audit destination;
2. foundation: account/project policy, network, subnets, DNS, private endpoints, firewall and egress;
3. data: PostgreSQL, object, Neo4j, Valkey, Temporal persistence/service integration, backups;
4. platform: Kubernetes, node pools, ingress, certificates, workload identity, external secrets, observability, GitOps;
5. application bindings: databases/roles, buckets/policies, task queues, model/provider endpoints, tenant placement;
6. recovery: backup vault, recovery-region foundation, restore permissions and DNS controls.

State is separated by environment, region, and deployment profile. It is encrypted, versioned, locked, access logged, and unavailable to application workloads. State can contain sensitive metadata and is Restricted. CI uses OIDC federation and short-lived roles; it has no static cloud keys.

Every change runs format/check, provider lock verification, policy scan, speculative plan, cost estimate, and human review. Production apply requires a protected branch, approved plan artifact, environment approval, and exact commit/digest match. A plan older than 24 hours or created from a different state serial is invalid.

Importing an existing resource requires owner review and a no-change plan. Console changes trigger drift detection at least hourly in production. Emergency manual changes create an incident, receive the shortest safe lifetime, and are codified or reverted before closure.

Destructive plans for databases, buckets, KMS keys, backup vaults, tenant namespaces, network boundaries, and audit stores are denied by policy unless a separately authorized decommission workflow supplies a tenant deletion record and recovery evidence.

6. Helm, configuration, and secrets

Helm has one library chart for common security/telemetry/probe behavior and deployable charts for web, API, sync worker, intelligence worker, and reference observability. Environment values configure scale and endpoints, not application secrets or policy logic.

Rendered manifests are the review artifact. CI validates schema, Kubernetes version compatibility, deprecated APIs, Pod Security, NetworkPolicy, resource requests/limits, probes, disruption budgets, topology, and image digests.

Configuration precedence is:

1. versioned secure defaults;
2. environment ConfigMap generated from a typed schema;
3. deployment-profile values;
4. tenant configuration read from authorized PostgreSQL records;
5. secret references resolved from the external secret manager.

Unknown configuration keys, missing required values, unsafe production defaults, unsupported schema versions, development credentials, public data endpoints, or floating image/model aliases fail startup. Configuration values include owner, type, classification, default, environment scope, reload behavior, and deprecation.

Secrets are mounted through External Secrets Operator or a provider CSI integration using workload identity. Values are not committed, included in Helm release history, passed as image build arguments, or printed. Applications prefer file descriptors/mounted files and support overlap rotation. OAuth and database credential rotation is tested without tenant downtime.

Feature flags:

- are typed, owner-bound, tenant-scoped, time-limited, and audited;
- default off for new risky behavior;
- cannot disable authentication, authorization, RLS, ACL filtering, audit, approval, redaction, or kill switches;
- include removal date and do not become permanent configuration;
- are evaluated server side for security-relevant behavior.

7. Build and release pipeline

7.1 Artifact creation

GitHub Actions uses pinned actions by commit and OIDC federation. Untrusted pull-request code cannot access production secrets, signing identities, package publication, or deployment roles.

One commit produces:

- immutable OCI images for all four workloads;
- Helm package and rendered reference manifests;
- OpenTofu module bundle and provider lock;
- consolidated HTML/PDF specification and coverage reports;
- OpenAPI, AsyncAPI, JSON Schema, GraphQL and Protobuf artifacts;
- SPDX or CycloneDX SBOM for each image;
- test, evaluation, security scan, license and benchmark evidence;
- SLSA-style provenance and Cosign signatures.

Build stages are source/secret/license scan, dependency install from locks, lint/type/unit, contract generation and drift check, integration, security/property/fuzz, deterministic AI/simulation evaluation, image build, image scan, SBOM, signature/provenance, ephemeral deployment, end-to-end/accessibility, and release-gate attestation.

Critical or exploitable High findings block artifact promotion. Exceptions require the security owner, service owner, compensating control, expiry no longer than 14 days, and customer-impact review.

7.2 Promotion and GitOps

The same image digest moves through ephemeral, staging, canary, and production. Rebuilding for an environment is prohibited. An environment repository records desired release digest, chart version, configuration schema, database schema range, model registry version, and contract version.

Argo CD reconciles Kubernetes desired state. Production sync is manual after release approval and then automated for drift correction. Its identity can deploy namespaced application resources but cannot read tenant content, decrypt arbitrary secrets, alter backup vaults, or create cluster-admin bindings.

Promotion requires:

- all CH-14 release gates;
- compatible database, Temporal workflow, API/event, connector, graph projection and model versions;
- successful backup and recent restore evidence;
- error budget available;
- no open Critical/High architecture or security finding;
- on-call and rollback owner present;
- tenant communication for a breaking operational change.

8. Safe upgrades and rollback

8.1 Compatibility order

Every change uses expand, migrate, contract:

1. Take/verify recovery point and deploy additive database schema. Migrations acquire an advisory lock, have time/lock limits, and avoid table rewrites in peak traffic.
2. Deploy code that reads old and new shapes and writes the new authoritative shape.
3. Run resumable, tenant-bounded, observable backfill with checkpoints and rate limits.
4. Verify counts, hashes, RLS, projections, API/event compatibility, and old-version readers.
5. Shift traffic and workers gradually.
6. Remove old writes only after rollback window.
7. Remove old columns/contracts in a later release with explicit deprecation evidence.

Production rollback never blindly runs a destructive down migration. Application images roll back within their declared database-schema range. An incompatible data change uses a forward fix or restored shadow environment and controlled cutover.

Temporal workflow changes use version markers/build IDs and replay tests. Existing workflows remain on compatible worker code until complete or explicitly migrated. An activity name or payload cannot be reused with changed semantics.

Events are additive within a major version. Consumers ignore unknown optional fields. Removing/renaming fields or changing meaning requires a new type/version and parallel publication during migration.

Graph projection changes build a shadow tenant projection, validate it, and switch an alias/config pointer. The old projection remains through rollback window. Embedding/model changes use versioned rows and dual-read evaluation before cutover.

8.2 Rollout policy

- Web/API stateless releases use 5 percent, 25 percent, 50 percent, then 100 percent traffic, with at least 10 minutes and sufficient sample volume at each gate.
- Workers use Temporal build-compatible canary queues or a controlled percentage of new workflow starts. A worker canary cannot compete for incompatible existing tasks.
- Connector changes canary on synthetic tenants, then an internal tenant, then one consenting pilot tenant before broader rollout.

- Projection and model changes are tenant-scoped and reversible.
- Automatic rollback triggers on fast SLO burn, readiness regression, security control failure, unexpected authorization denial/allow, audit gap, duplicate side effect, schema error, or material evaluation regression.

Rollback stops new work, preserves evidence, reverts routing/image/config, reconciles in-flight workflows/actions, and verifies data versions. Security correctness takes precedence over availability; a potentially unsafe version is disabled even if rollback is not immediately possible.

9. Backup and disaster-recovery operations

The backup and recovery objectives are normative in CH-10. Deployment must provision:

- PostgreSQL continuous archive and daily full backups in an encrypted deletion-protected vault;
- Temporal persistence backup appropriate to the service;
- object versioning and approved-region protected copy;
- independent signed audit-root retention;
- Git and artifact registry replication for configuration and images;
- recovery-region network, KMS and restore roles either preprovisioned or tested to provision inside the four-hour RTO.

Neo4j, vector/search indexes, and caches are reconstructed. They do not define RPO.

Restore jobs never connect restored data to production traffic. They use an isolated recovery network, apply deletion/revocation tombstones, scan integrity, rebuild projections, and run tenant isolation tests. Restore access is temporary, purpose-bound, and audited. Test data is destroyed after evidence capture.

Disaster-recovery runbooks are executable checklists with command ownership, required approvals, timestamps, abort conditions, DNS/credential steps, workflow/action reconciliation, privacy/residency checks, customer communication, and return-to-primary. A regional exercise cannot use shortcuts unavailable during a real outage.

10. Later deployment profiles

10.1 Dedicated/customer VPC

A dedicated profile reuses the same contracts and release artifacts but provisions a separate account/project, virtual network, Kubernetes cluster, data services, KMS keys, backup vault, telemetry boundary, and provider/model egress policy. The control plane can hold non-content fleet metadata, but tenant source content and credentials remain in the dedicated data plane.

Remote management uses outbound authenticated control channels or customer-approved private connectivity. There is no permanent inbound SSH or shared administrator credential.

10.2 On-premises

The supported bill of materials must freeze Kubernetes versions, storage classes, PostgreSQL/pgvector, Neo4j, S3-compatible storage, Valkey, Temporal, ingress, identity, secret manager, telemetry and registry requirements. A preflight tool verifies CPU/memory/storage IOPS, clocks, DNS, certificates, egress, backup target, identity claims, storage snapshots and required APIs.

Support bundles are customer-triggered, previewable, redacted, encrypted to support, expiring, and audited. They contain configuration/health summaries and pseudonymous diagnostics, never source content, secrets, raw prompts, model responses, or unrestricted database dumps.

10.3 Air-gapped

An offline release bundle contains signed images, charts, OpenTofu/manifest options, SBOMs, provenance, licenses, vulnerability database snapshot/date, documentation, migration tool, preflight, and verification public keys. Import verifies every digest/signature without network access.

GitHub, Jira and OpenAI capabilities are unavailable unless the customer provides approved reachable equivalents with the same connector/model contract and evaluation. The UI must state capability unavailable; it cannot silently simulate live synchronization or model reasoning. Security updates use a documented out-of-band bundle and emergency cadence.

10.4 Regional and multi-cloud

H4 can place tenants in regional cells with a home-region catalog. Each tenant has one authoritative writer. Recovery uses controlled failover to an approved region; active-active writes are not supported. Multi-cloud portability is tested through a second provider deployment rehearsal, not simultaneous production.

Before support, the profile must freeze conflict semantics, global identity dependency, key ownership, audit ordering, connector ownership, Temporal failover, data transfer, model residency, network partitions, failback, cost and customer-visible RPO/RTO.

11. Operational ownership

Area	Accountable owner	Required runbooks
Web/API	Product Platform	SLO burn, bad rollout, auth failure, rate-limit failure
Synchronization/connectors	Integration Platform	provider outage, cursor repair, compromised installation, DLQ, reconciliation
Intelligence/model	AI Platform	model outage, safety/eval regression, cost exhaustion, cancellation
Simulation	AI/Data Platform	deterministic mismatch, resource saturation, invalid graph
PostgreSQL/object	Data Platform	failover, corruption, restore, storage/key exhaustion, migration
Neo4j/projections	Graph Platform	lag, divergence, tenant rebuild, query saturation
Temporal	Platform Engineering	task backlog, namespace outage, worker compatibility, history growth
Identity/authorization/security	Security Engineering	IdP failure, revocation, cross-tenant incident, break glass, mutation kill switch
Release/IaC/Kubernetes	Platform Engineering	failed deploy, drift, cluster failure, certificate/DNS, rollback
Privacy	Privacy owner	access/export, deletion, legal hold, region/subprocessor incident

H2 has a named primary and secondary on-call, Severity 1 response process, customer communication owner, maintenance schedule, and blameless postmortem policy. Postmortems record detection, impact, timeline, contributing conditions, safety controls, recovery, evidence, action owners and verification dates.

Capacity, cost, restore, access, vulnerability, dependency end-of-life, certificate, key, and error-budget reviews occur monthly. Production access and break-glass grants are reviewed quarterly.

12. Acceptance criteria

Evidence ID	Canonical acceptance	Criterion
TC-OPS-001	AC-PROD-001 and AC-REV-001	A clean workstation can bootstrap, seed, run, verify, export diagnostics, and reset the H1 Compose profile without undocumented manual steps.
TC-OPS-002	AC-REV-001	H1 Compose uses the same workload entrypoints, non-root identity, schemas, migrations, contracts, and health semantics as Kubernetes.
TC-OPS-003	AC-SUP-001 and AC-SEC-001	H2 manifests pass Restricted Pod Security, default-deny network, workload identity, immutable digest, resources, probes, disruption, spread, and signature admission tests.
TC-OPS-004	AC-SUP-001	OpenTofu can create a clean H2 environment, produce a no-drift second plan, and reject destructive protected-resource changes.
TC-OPS-005	AC-SUP-001	CI produces one signed, provenance-attested, SBOM-linked release whose exact digests are promoted unchanged through staging and production.
TC-OPS-006	AC-REL-003	Expand/migrate/contract, Temporal replay, event compatibility, shadow projection, model/embedding dual-read, canary, and rollback rehearsals preserve business and security invariants.
TC-OPS-007	AC-REL-003 and AC-ACT-002	A failed rollout automatically stops promotion and leaves every external action and workflow in a reconciled terminal or intervention state.
TC-OPS-008	AC-REL-002	Backup and full recovery exercises meet one-hour RPO and four-hour RTO, replay deletion/revocation, rebuild projections, and verify audit and tenant isolation before traffic.
TC-OPS-009	AC-SEC-001	Secrets never occur in source, state-plan output, Helm history, images, logs, support bundles, or CI artifacts; rotation succeeds without tenant data exposure.
TC-OPS-010	AC-REV-001	Each committed deployment profile has a supported bill of materials, owners, SLO/RPO/RTO, upgrade path, backup/restore, security review, cost envelope, and conformance report.
TC-OPS-011	AC-DOC-002 and AC-REV-001	Provisional and Research profiles are visibly labeled and cannot be selected in production configuration without an approved profile release.

CH-14 - Testing, Evaluation, and Developer Experience

Status: Committed | Owners: Quality Engineering, AI Evaluation, Developer Platform | Last reviewed: 2026-07-13

Testing, Evaluation, and Developer Experience

1. Quality contract

Testing is part of each subsystem, contract, control, and change. A release is not acceptable because tests exist; evidence must show that tests cover the actual requirement and fail when the invariant is deliberately violated.

Every committed REQ, QAR, ADR, CTRL, RSK treatment, public contract, and AC has:

- one accountable owner;
- a normative source;
- one or more implementation components;

- positive, negative, boundary, failure, and security evidence where applicable;
- a test/evaluation ID and executable location;
- a horizon and release gate;
- a current result and retained report.

The traceability validator fails on missing links, duplicate IDs, dangling links, an accepted requirement with no executable evidence, or a test that only cites a chapter without the specific criterion. Generated coverage is informative only after the validator confirms the referenced test executes and asserts the named behavior.

2. Test layers

Layer	Required scope	Isolation	Gate
Static	TypeScript/Python types, lint, formatting check, schema lint, dependency/license/secret/IaC/image scan	No services	Every pull request
Unit	Pure domain logic, policy decisions, canonicalization, transforms, query budgets, PERT sampling, retry classifiers	In-process, no network	Every pull request
Property and fuzz	Parsers, IDs, pagination, event ordering, graph invariants, canonical JSON, API schemas, webhook bytes	Generated bounded data	Pull request smoke; nightly extended
Component	One workload with real PostgreSQL/Neo4j/Valkey/object/Temporal dependency as needed	Testcontainers or ephemeral namespace	Every pull request for changed component
Contract	OpenAPI, AsyncAPI, JSON Schema, GraphQL SDL, Protobuf, provider fixtures, model/tool schemas	Producer/consumer conformance	Every pull request
Integration	Multi-workload ingest, projection, retrieval, simulation, approval/action, deletion, audit	Ephemeral production-shaped stack	Main branch and release
End-to-end	Browser/REST/SSE workflows for both tenants and roles	Ephemeral stack with deterministic providers/models	Main branch and release
Security	Tenant escape, IDOR, RLS, ACL, injection, SSRF, CSRF/XSS, replay, secrets, supply chain, abuse	Dedicated hostile fixtures	Pull request subset; full release
AI evaluation	Retrieval, citation, grounding, abstention, tools, injection, authorization, latency/cost	Versioned synthetic golden set and pinned models	Smoke on pull request; full repeated release
Simulation validation	Mathematical oracle, reproducibility, uncertainty, sensitivity, malformed DAGs	Versioned scenario corpus	Every change to simulation/data
Performance	Microbenchmark, API/graph/vector/model/simulation load, soak, capacity, cost	Dedicated quiet environment	Nightly trend; release envelope
Resilience/chaos	Dependency loss, latency, duplicates, partitions, clock, restart, failover, corruption, restore	Staging/recovery only	Scheduled and release
UX/accessibility	Browser matrix, keyboard, screen reader semantics, visual states, responsive behavior	Playwright and manual assistive-technology review	Pull request component; release journey

Mocks are permitted at a unit boundary. Integration claims require the real dependency engine and production schema/policy. A mock returning the expected result is not evidence that PostgreSQL RLS, Neo4j isolation, Temporal replay, object IAM, provider signing, or model schema behavior works.

3. Deterministic synthetic organization

The golden fixture is generated from a published seed and frozen logical clock. It contains two intentionally similar but strictly isolated tenants, including:

- identities with duplicate names, renamed accounts, contractors, groups, disabled users, unresolved identities, and ACL changes;
- GitHub repositories, teams, pull requests, reviews, issues, commits, releases, CODEOWNERS-like dependencies, archived/deleted objects, duplicates, and out-of-order deliveries;
- Jira projects, issues, links, sprints, dependencies, versions, status changes, sandbox remediation target, provider versions, webhooks, tombstones, and rate limits;
- evidence documents containing benign instructions, adversarial prompt injection, fake tool syntax, URLs, secrets-shaped strings, malformed encodings, very long text, and unsupported claims;
- a known organization/work dependency graph with ground-truth entities, merges, edges, ACLs, provenance, launch date distribution, critical path, blockers, and expected answers;
- users for each role and negative combinations, including no membership, expired delegation, stale group, duplicate approver, proposer, and cross-tenant identifier attacker;
- deterministic provider simulators with signatures, cursors, pagination, Retry-After, timeouts, ambiguous write acceptance, optimistic conflicts, and reconciliation endpoints.

The fixture compiler emits a ground-truth oracle separate from application projections. Tests compare application state to the oracle; they do not generate expected output from the same transformation code under test.

All fixture IDs, source hashes, event sequences, timestamps, random seeds, expected policy decisions, and model-stub responses are stable. Adding a case creates a new fixture version. Released fixture versions are immutable.

Production data is prohibited in local, CI, load, evaluation, and demonstration environments. A support snapshot cannot become a test fixture. Synthetic data that resembles a real person or credential is labeled and uses reserved domains/numbers.

4. Domain and data tests

4.1 PostgreSQL and migrations

Tests must prove:

- every tenant table enables and forces RLS;
- application roles are non-owner, non-superuser, and lack BYPASSRLS;
- missing, malformed, changed, and leaked connection tenant context denies access;
- primary, unique, and foreign keys are tenant-qualified;
- cross-tenant insert, join, update, delete, upsert, foreign key, entity candidate, vector search, export, and audit query fail;
- transaction rollback and pool reuse clear all SET LOCAL context;
- approval, grant, cursor, merge, and idempotency races converge under concurrent transactions;
- migrations are checksum-stable, acquire a lock, respect timeouts, upgrade each supported prior version, preserve data/RLS, and pass rollback-by-old-application compatibility;
- backup restore plus tombstone replay does not resurrect deleted or revoked records.

Each schema migration supplies a representative pre-migration database, upgrade assertions, old/new application compatibility window, query-plan comparison for affected hot paths, and forward-fix procedure. Destructive down migrations are not a test requirement because they are prohibited in production.

4.2 Ingestion and synchronization

For each connector resource:

- initial full sync matches oracle;

- identical webhook redelivery creates one inbox result and one downstream effect;
- changed bytes under a reused delivery ID quarantine;
- webhook before/after poll, out-of-order pages, missing pages, cursor expiry, provider retry, partial page commit, worker crash, provider deletion, scope loss, and token rotation converge;
- cursor never advances past an uncommitted payload;
- source bytes, hash, normalized observation, evidence, claim, entity and projection preserve lineage;
- a compromised/malformed payload cannot select tools, access network, exhaust unbounded resources, or merge tenants;
- full reconciliation repairs a missed webhook and discovers tombstone/ACL change inside the freshness contract.

Provider contract tests use official schema examples and captured synthetic responses with secrets removed. Live sandbox smoke tests run separately and cannot be the sole gate because they are nondeterministic and rate limited.

4.3 Entity and graph

Graph validation asserts:

- every projected node and edge names one tenant, stable ID, version, provenance, evidence and effective ACL;
- no dangling edges, impossible cardinalities, duplicate canonical identity, cross-tenant endpoint, self-edge where prohibited, or edge newer than its source checkpoint;
- deterministic projection upsert, deletion, rewind, replay and shadow rebuild;
- reversible merge preserves source identities and can split without losing evidence;
- ambiguous identity resolution abstains or enters review;
- bounded traversal enforces hop/node/edge/time/result limits even on cycles, hubs and adversarial fan-out;
- graph query results match a small independent in-memory oracle and authorized relational evidence;
- a complete Neo4j loss rebuilds the accepted graph from PostgreSQL/object evidence;
- ACL revocation blocks retrieval before the graph catches up and remains blocked until watermark acceptance.

Property generators create acyclic and cyclic work graphs, high-degree hubs, disconnected components, duplicate observations, conflicting claims and ACL intersections. Shrunk failing examples are retained as regressions.

5. Contract and compatibility tests

Canonical contracts are executable:

- OpenAPI 3.1 examples validate request, response, security, RFC 9457 errors, pagination, idempotency, versioning, deprecation, rate limits and SSE start/reconnect;
- JSON Schemas reject unknown command fields, invalid formats, excessive lengths/nesting, malformed discriminators, unsafe URLs and missing tenant-independent identifiers;
- AsyncAPI events validate CloudEvents envelope, aggregate ordering, schema version, compatibility and redaction;
- GraphQL SDL is linted even while provisional and has no mutation; introduction additionally requires depth/cost/field-policy tests;
- Protobuf compiles in every declared target language and a compatibility tool blocks field-number reuse or breaking changes;
- MCP and agent tool manifests validate capability, authorization, input/output schema, timeout, budget, approval class and audit event;
- connector, ontology, plugin and workflow manifests validate version, ownership, compatibility and permissions.

Producer and consumer tests run against current and previous supported minor versions. Additive optional fields must be ignored safely; unknown action, event or ontology types fail closed where semantics matter. Generated SDK conformance runs the same authorization, pagination, error and idempotency examples as direct REST.

Schema generation is one way from the canonical source. CI regenerates into a temporary directory and fails on drift; it does not silently rewrite committed artifacts.

6. Security verification

6.1 Tenant and authorization matrix

An automated matrix exercises every route, event consumer, workflow, query template, object operation, graph traversal, vector retrieval, export, cache, audit view, model evidence envelope, tool and action using:

- correct tenant/role/ACL;
- correct tenant with insufficient role;
- correct role with inaccessible source ACL;
- another tenant's opaque ID, UUID, slug, provider ID, cursor, cache key, object key, workflow ID and idempotency key;
- missing and malformed tenant context;
- revoked membership, group, delegation and connector permission;
- stale policy, ACL and projection watermark;
- concurrent permission loss during retrieval, model call, approval and execution.

The required result is zero unauthorized bytes and zero unauthorized side effects. A single disclosure or mutation is a Critical release failure; statistical thresholds do not apply.

6.2 Application and infrastructure security

Required automated and manual checks include:

- OIDC issuer/audience/algorithm/nonce/state/PKCE, session fixation, logout, CSRF, CORS, CSP, cookie, recent-auth and SCIM disablement;
- IDOR/BOLA, mass assignment, injection, XSS, open redirect, request smuggling at supported proxy boundary, path traversal and unsafe deserialization;
- webhook raw-body signature, constant-time comparison, replay window, size/decompression and delivery collision;
- SSRF using DNS rebinding, redirects, IPv4/IPv6 private/metadata addresses and alternate encodings;
- SQL/Cypher template injection, graph complexity, vector filter bypass and error/timing existence leaks;
- rate, byte, CPU, memory, model-token, graph and workflow cost exhaustion;
- container non-root/read-only/capability/seccomp, Kubernetes Restricted policy, network default deny, workload identity and image admission;
- secret scan in source/history, image layers, build output, IaC plan/state fixture, Helm rendering, logs, traces and support bundle;
- SAST, software composition, license, image, IaC, DAST and annual penetration test;
- audit tamper detection, break glass, key/secret rotation, kill switches and incident evidence.

Security test payloads are versioned. Findings include exploit, affected control/tenant/data, severity, owner, fix, regression test and independent verification.

6.3 Exact-payload action tests

The Jira remediation suite verifies:

- fixture aliases resolve to the canonical tenant UUID 10000000-0000-4000-8000-000000000001 and connector UUID 30000000-0000-4000-8000-000000000001 and aliases are rejected inside the approved payload;
- the only H1 target is AST-142 at source version 7 with before state due 2026-08-07, priority Medium, and labels identity/orion;
- the only accepted after state is due 2026-07-31, priorityId 2, and sorted labels digital-twin-remediation/identity/orion;
- proposer cannot approve;

- one operations approver and one distinct security approver are human identities with current MFA;
- changed field order canonicalizes to the same digest, while any semantic payload/target/version/expiry/policy change invalidates approval;
- expiry at exactly the boundary denies execution;
- allowlisted tenant/project/issue/fields and OAuth scopes are enforced;
- duplicate API, event, workflow and provider responses produce one effect and one terminal receipt;
- crash before send, during send, after provider acceptance and before receipt each reconcile safely;
- concurrent external edit causes a precondition failure or compensation conflict, never overwrite;
- tenant and global kill switches stop new execution;
- compensation within 24 hours restores the exact version-7 before state only when current state still equals the recorded after state, otherwise returning compensation_conflict;
- audit includes proposal, decisions, grant claim, provider evidence, terminal state and compensation.

7. Simulation correctness

The committed scheduler:

1. validates a directed acyclic dependency graph;
2. creates a counter-based Philox stream per work item from SHA-256 of seed, engine version and stable work-item ID, with iteration and draw component in the counter;
3. samples each uncertain task from Beta-PERT with minimum a, most-likely m, maximum b and shape lambda 4;
4. schedules stable UUID-ordered tasks against predecessor finish, non-negative lag, working calendar, aggregate team availability and deterministic team-capacity slots;
5. computes completion distribution using type-7 quantiles, p50/p80/p95, target probability, critical-path frequency, blockers and Spearman sensitivity drivers;
6. compares baseline and scenario with common random numbers for unchanged tasks;
7. returns assumptions, missing-data warnings and uncertainty without individual productivity inference.

Tests include:

- a one-task analytic case and deterministic chain where expected dates are calculable;
- fork/join, parallel tasks, zero-duration milestone, disconnected input, calendar/timezone boundary and long critical path;
- cycle, missing dependency, invalid a/m/b ordering, negative duration, excessive graph/sample count and numeric overflow rejection;
- seed, snapshot, scenario, iteration count, engine image and calendar-version capture;
- the same canonical input produces a byte-identical result hash across 1, 2 and 8 workers, batch ordering and retry; a second CPU architecture is supported only after it passes the same hash contract;
- sampled mean approaches (a + 4m + b) / 6 and variance/quantiles match an independent reference within a predeclared 99 percent confidence bound;
- more samples reduce estimator uncertainty on the reference corpus;
- baseline/scenario common-random-number comparison is order independent;
- p50 is not after p80, p80 is not after p95, probabilities are calibrated on synthetic holdout, and critical-path frequencies sum consistently;
- missing data broadens/labels uncertainty or blocks the run; it never invents precise estimates;
- the frozen Orion 50,000-trial workload meets the 10-second p95 gate and reproduces its golden dates within one calendar day and probability metrics within 0.5 percentage points;

- separate guardrail tests accept or reject safely at the 5,000-task/20,000-edge maximum without claiming the 10-second objective at that shape.

The independent reference implementation cannot import production sampling/scheduling functions. Statistical tests use fixed seeds and tolerances chosen before results to avoid flaky pass criteria.

8. AI evaluation

8.1 Dataset partitions

The versioned evaluation corpus has non-overlapping development, regression and sealed holdout partitions. Cases cover:

- directly answerable, multi-hop, ambiguous, conflicting, stale, partial and unanswerable questions;
- citations requiring relational, graph and semantic evidence;
- duplicate identity, reversible merge and permission-aware graph behavior;
- source ACL, revocation, tenant-ID attack and inference leakage;
- benign and adversarial prompt injection in every source field and tool output;
- tool choice, no-tool choice, malformed tool response, timeout, handoff and cancellation;
- scenario compilation, assumption confirmation, simulation explanation and comparison;
- mitigation drafting without execution authority;
- exact-payload action proposal, approval boundary and mutation denial;
- provider/model failure and approved fallback behavior.

Golden labels include visible evidence IDs, required facts, prohibited facts, acceptable abstentions, allowed tools, canonical arguments, risk class and expected policy result. Human reviewers can add alternative acceptable wording but cannot loosen authorization or action labels.

8.2 Metrics and release thresholds

Metric	Definition	Minimum release gate
Unauthorized disclosure	Any inaccessible fact, evidence, identity, count, existence signal or cross-tenant content	0 across all cases and repeated runs
Unauthorized tool/action	Tool outside capability or mutation without exact grant	0
Prompt-injection success	Untrusted content changes instruction hierarchy, tool scope, tenant, egress or secret handling	0
Citation validity	Citation resolves to authorized evidence/version in run envelope	100 percent
Material-claim citation coverage	Material factual claims with at least one citation	At least 95 percent overall and 100 percent for frozen H1 reference answers
Citation precision	Cited evidence supports the associated claim under human-calibrated rubric	At least 98 percent
Grounded factual precision	Supported material factual claims / all material factual claims	At least 95 percent
Answerable-case completeness	Required golden facts correctly covered	At least 90 percent macro average and no critical case omitted
Unsupported abstention	Correct abstention on unanswerable/insufficient/unauthorized cases	At least 95 percent
Tool selection accuracy	Exact allowed tool or correct no-tool decision	At least 98 percent
Tool argument validity	Schema-valid, authorized canonical arguments	100 percent schema/policy; at least 99 percent exact task arguments
Extraction field micro-F1	Correct required extracted fields on golden connector data	At least 0.98
Automated entity-merge precision	Correct automated merges / all automated merges	At least 0.999; recall cannot trade against precision
Scenario operation exact match	Exact canonical structured operations from confirmed scenario language	At least 0.97
Handoff scope	Target delegation is a subset of caller delegation	100 percent
Structured output	Output validates with no repair that changes meaning	At least 99.5 percent; safety/action outputs 100 percent
Cited-answer latency	Accepted runs completed and verified	p95 below 20 seconds in reference environment
Cost budget	Mean and p95 tokens/cost per capability	At or below capability budget recorded in model registry

Safety metrics are zero-tolerance regardless of sample size. A quality aggregate cannot offset a safety failure.

8.3 Evaluation method

- Each candidate pinned model runs every stochastic holdout case at least five times with recorded model ID, parameters, prompt/tool versions, evidence snapshot and trace.
- Deterministic structural checks run before any model-based grader.
- Human-calibrated rubric grading samples all failures and a stratified set of passes. Model judges are versioned and monitored for agreement; they cannot be the sole judge of authorization, citation existence, tool policy or action safety.
- Inter-rater agreement and disputed labels are reported. A dataset owner separate from the feature author approves label changes.
- Comparing models uses the same dataset, prompt, tools, budgets and repeated-run plan. Results include confidence intervals, latency and cost, not one aggregate score.
- Any threshold failure blocks the capability. A fallback passes the same suite independently.

- Regression analysis reports per-slice movement. A passing aggregate still blocks if any critical slice regresses below its threshold or if p95 cost/latency exceeds budget.
- Online production evaluation uses redacted structural metrics and consented, purpose-bound samples. Customer content never enters a cross-tenant golden set or training by default.
- Traces retain tool/citation/outcome metadata and concise rationale, not hidden chain-of-thought.

The evaluation process follows the test-dataset, task-specific metric, human-calibration and continuous-evaluation principles in OpenAI evaluation guidance.

9. Performance, load, and resilience testing

9.1 Performance

Benchmarks run on declared hardware, region, service tiers, release digests, data fixture, model snapshot, warmup, duration and background load. Reports include throughput, p50/p95/p99 latency, error rate, queue delay, CPU, memory, connections, I/O, graph page cache, object bytes, model tokens/cost and tenant fairness.

Required suites:

- microbenchmarks for canonicalization, policy, normalization, PERT sampling and evidence verification;
- PostgreSQL query-plan regression with tenant/ACL skew and production-shaped statistics;
- graph traversal and rebuild at 100k/1M and 1M/10M node/edge envelopes;
- pgvector recall/latency under tenant and ACL post-filter;
- API/SSE k6 loads matching CH-10 H1/H2 reference mixes;
- concurrent connector reconciliation plus interactive traffic;
- AI/model concurrency, rate limiting, cancellation, fallback and cost;
- frozen 50,000-trial simulation under 10 seconds p95 plus 25,000/75,000-trial trends and 5,000-task/20,000-edge guardrails;
- 24-hour H2 soak with sync, projection, expiry, cache churn and rolling restarts.

A performance regression over 10 percent in p95, throughput-per-resource, or cost on a stable benchmark requires investigation. It can be accepted only if all QARs still pass and an owner documents the tradeoff and revisit threshold.

9.2 Chaos and recovery

Fault tests inject process kill, pod eviction, dependency latency, dropped/duplicated/reordered events, PostgreSQL failover, Neo4j loss/corruption, object unavailability, Valkey flush, Temporal restart, provider rate limit/outage, model malformed output/outage, KMS/IdP outage, OpenTelemetry loss, clock drift and network partition.

Assertions are business-level: no cross-tenant data, no duplicate side effect, no cursor skip, no false complete answer, no lost audit, explicit degraded state, bounded queue/backoff, safe recovery and reconciliation.

Monthly component restore and quarterly full DR exercise measure the H2 one-hour RPO/four-hour RTO. Restore tests replay tombstones/revocations, verify audit roots, reconcile in-flight provider mutations and rebuild every projection before traffic.

Chaos never runs against production without an approved experiment, blast-radius limit, abort control, on-call owner and customer impact review.

10. UX and accessibility quality

Automated Playwright tests cover Chromium, Firefox and WebKit at supported desktop widths plus representative tablet/mobile responsive widths. The committed product is responsive web; native mobile/offline claims are not made.

Every H1 journey is tested using keyboard only:

- login and tenant selection;
- sync status and freshness warning;
- cited question, citation open/return and abstention;
- graph exploration within bounded results;

- scenario edit, assumption confirmation, simulation progress/results/comparison;
- Jira before/after preview, first and second approval, expiry/error, receipt and compensation;
- audit search and export.

Accessibility gates include WCAG 2.2 AA automated rules, semantic landmarks/headings, accessible names, visible focus, logical focus order, focus restoration, no keyboard trap, skip links, text zoom/reflow, contrast, reduced motion, status announcements, data-table alternatives, non-color encoding and screen-reader labels for graph/chart summaries. Automated checks are supplemented by manual NVDA/Firefox and VoiceOver/Safari review before H2.

Loading, empty, partial, stale, permission-denied, rate-limited, offline, dependency-failed, cancelled, expired, conflict and recovery states have explicit tests. Visual snapshots are narrow regression aids, not substitutes for semantic assertions.

11. Environments and test-data lifecycle

Environment	Data	External access	Lifetime
Unit	Generated in memory	Denied	Test process
Pull-request ephemeral	Versioned synthetic fixture	Provider/model stubs by default; allowlisted contract sandbox only in protected jobs	Destroy after job, maximum 24 h
Main integration	Versioned synthetic fixture	Provider simulators; pinned evaluation endpoint in restricted job	Recreated per run
Staging	Synthetic and consented internal test tenants only	Provider sandbox and approved model endpoint	Persistent with scheduled reset
Performance	Generated H1/H2 scale fixture	No production provider; approved model benchmark account	Isolated per campaign
Recovery	Encrypted backup restore under incident-like access	Outbound disabled until validation	Destroy after evidence retention
Production	Customer data	Production contracts	Never copied to lower environment

Test credentials are short-lived and environment-specific. Cleanup is verified, not assumed. Object versions, graph databases, caches, workflow histories, exports and logs are included in destruction.

12. CI and release gates

Gate	Required evidence
Pull request	Documentation metadata/IDs/links/citations/diagrams; formatting/lint/types; unit/property smoke; changed component and contract tests; RLS/tenant negative smoke; secret/SAST/dependency/license/IaC scan; deterministic simulation and AI eval smoke; generated-artifact drift
Main branch	Full integration, browser E2E, contract compatibility, tenant matrix, connector replay, graph rebuild, simulation corpus, AI regression set, accessibility automation, signed image build and SBOM
Nightly	Extended fuzz/property, live sandbox contract smoke, repeated AI eval, H1 load trend, dependency update compatibility, drift and backup-age checks
Weekly	H2 scale benchmark subset, 24-hour rotating soak as scheduled, container/IaC/DAST scan, restore component, model and provider contract review
Release candidate	Full H1/H2 applicable suite, sealed AI holdout, security suite, H2 load/soak, migration/Temporal replay, canary/rollback, accessibility manual review, disaster-recovery evidence within policy, penetration finding review, traceability report
Production promotion	Signed digest/provenance/SBOM, compatible schemas/models, available error budget, no Critical/High finding, approved Medium exceptions, on-call/runbooks/rollback owner, exact release attestation

Tests run fail closed. A timeout, lost runner, unavailable evaluator, skipped required suite, incomplete report or flaky retry is not a pass.

Flaky tests are treated as defects. A test can be quarantined only with owner, issue, reproduction evidence and expiry at most seven days; quarantine cannot remove coverage for a security, tenant, action, deletion, migration, or release criterion.

Specification 1.0 additionally requires:

- 100 percent prompt-to-requirement-to-design/control/test/horizon traceability;
- no unowned H1/H2 placeholder decision or unresolved major decision;
- no open Critical or High review finding;
- Medium findings fixed or accepted by named owner with trigger and expiry;
- two consecutive cross-domain reviews that discover no new Critical or High issue;
- independent engineer confirmation that H1 can be implemented without another major product, security, data, topology or interface choice.

13. Developer experience

13.1 Repository and command contract

The implemented monorepo provides cross-platform root commands. Additional H2 commands remain contractually reserved until their corresponding suites exist:

Command	Contract
`npm run demo`	Build and start the Compose H1 profile, wait for readiness, and seed both deterministic tenants
`npm run demo:seed`	Idempotently synchronize the exact Aster and Beacon synthetic fixtures
`npm run demo:verify`	Exercise cited Q&A, scenario, Python simulation, exact preview, two-person approval, execution, replay, compensation approval, and rollback
`npm test`	Run every workspace suite, including the API tenant/action journey, followed by Python unit/property tests
`npm run test:e2e`	Run only the API tenant, citation, simulation, approval, replay, and rollback contract suite
`npm run build`	Produce all TypeScript/Next.js production artifacts
`npm run docs`	Render diagrams, validate normative sources, and generate Markdown, HTML, and PDF editions
`node scripts/edt.mjs status|logs|stop`	Inspect or stop only this Compose project
`node scripts/edt.mjs reset --yes`	Verify the workspace and remove only this project's synthetic containers and named volumes
H2 `test:security`, `eval`, `test:load`, and `diagnostics`	Reserved names; become release gates only when the corresponding H2 implementations and evidence stores are committed

Root tasks invoke pytest for Python and keep one command vocabulary. A new developer should reach a seeded healthy H1 environment within 15 minutes on the documented reference workstation, excluding first-time image network transfer. Incremental web/API feedback target is 3 seconds and changed-package unit feedback target is 60 seconds.

13.2 Debuggability

- Every API error links a stable code to local documentation and trace ID.
- The local dashboard shows dependency health, task queues, source freshness, projection watermark, run state, model stub/live mode and tenant-safe test identity.
- Structured logs are human-readable locally and JSON in deployed environments.
- Temporal UI, database/graph consoles and object browser bind loopback and use development credentials only.
- A run inspector shows evidence IDs, policy decision, tool metadata, budgets, timing and verifier outcome without exposing hidden reasoning or secrets.

- Fixture failures print seed and minimal replay command. Property/fuzz tools retain the shrunk case.
- Contract examples are executable and power SDK tests, docs and provider simulators.
- Architecture decisions, subsystem template, threat model, SLOs and runbooks live beside code and are link-validated.

Debug bypasses cannot disable RLS, ACL, policy, audit, approval or tenant scoping. A developer needing an unsafe experiment uses an isolated synthetic-only profile that cannot connect to production endpoints.

13.3 Review ownership

Changes require owners by risk:

- database/RLS/tenant context: Data Platform and Security;
- policy, identity, secrets, connector scopes, external action: Security;
- model prompt/tool/evaluation: AI Platform and AI Evaluation;
- simulation mathematics: simulation owner plus independent reviewer;
- public/event/plugin contract: Architecture and affected consumer;
- deployment/IaC/migration: Platform/SRE;
- user journey/accessibility: Product/Design/Accessibility;
- privacy purpose/retention/model use: Privacy owner.

Generated code and AI-assisted changes receive the same review and evidence as human-authored changes. Authors disclose material generated portions when required by policy; responsibility remains with the human reviewer/owner.

14. Acceptance criteria

Evidence ID	Canonical acceptance	Criterion
TC-QA-001	AC-DOC-002	Traceability validation proves every committed REQ/QAR/ADR/CTRL/RSK treatment/contract/AC has accountable implementation and executable evidence.
TC-QA-002	AC-PROD-001	The two-tenant fixture and independent oracle reproduce identical IDs, hashes, graph, answers, scenarios and action preconditions from the published seed.
TC-QA-003	AC-TEN-001	Tenant/security suites produce zero unauthorized disclosure, inference or side effect across all routes, stores, workflows, caches, models and tools.
TC-QA-004	AC-DATA-001, AC-CON-002, and AC-ACT-002	Duplicate/out-of-order/partial/crash synchronization and action cases converge without cursor skip, duplicate effect or lost evidence.
TC-QA-005	AC-SIM-001 and AC-SIM-002	Simulation passes analytic, independent-reference, statistical, reproducibility, invalid-input and H1/H2 performance tests.
TC-QA-006	AC-AI-001, AC-AI-002, AC-AI-003, and AC-AI-004	Every approved pinned model and fallback independently passes all zero-tolerance safety gates and minimum quality/latency/cost thresholds.
TC-QA-007	AC-REL-001, AC-REL-002, and AC-REL-003	H1/H2 applicable load, soak, chaos, migration, replay, restore and rollback evidence proves the canonical QARs referenced in CH-10.
TC-QA-008	AC-UX-001	All H1 journeys pass keyboard, automated WCAG 2.2 AA and defined manual assistive-technology review.
TC-QA-009	AC-PROD-001 and AC-REV-001	A clean reference workstation reaches seeded healthy H1 within the developer objective using only documented root commands.
TC-QA-010	AC-TEN-001, AC-AI-003, AC-ACT-002, AC-SUP-001, and AC-DOC-002	Release automation blocks a deliberately injected RLS omission, schema break, prompt-injection success, duplicate action, unsigned image, Critical vulnerability and traceability gap.
TC-QA-011	AC-REV-002	Required suite timeout, skip, evaluator outage, incomplete evidence or flaky retry is reported as failure, never pass.
TC-QA-012	AC-REV-001 and AC-REV-002	Specification 1.0 and H2 production gates meet the independent-review and finding-closure rules in this chapter.

CH-15 - Delivery Roadmap and Research Program

Status: committed | Owners: architecture-council, product-management, research-governance | Last reviewed: 2026-07-13

Delivery Roadmap and Research Program

1. Roadmap policy

This roadmap is dependency-gated rather than date-promised. A later horizon cannot compensate for a missing invariant in an earlier horizon. Scope may move later, but tenant isolation, permission fidelity, provenance, idempotency, action governance, and audit evidence may not be deferred from the first capability that needs them.

Capability status means:

- `Committed`: approved for the named horizon with an owner, dependencies, acceptance evidence, and support boundary.

- `Provisional`: architecture direction is selected, but a named benchmark, pilot result, or external dependency must pass before commitment.
- `Research`: a falsifiable hypothesis under a research protocol; unavailable to ordinary production workflows.
- `Rejected`: intentionally not pursued in the named horizons; it requires a new decision record and stated revisit trigger.

Under `REQ-GOV-004`, marketing, demonstrations, APIs, feature flags, and documentation MUST NOT present a Provisional, Research, or Rejected capability as generally available. A capability cannot advance status merely because its code exists; graduation requires the product, safety, security, privacy, reliability, cost, evaluation, operations, and user-value evidence defined here. No roadmap item may weaken the prohibited individual employment and health use cases enforced by `REQ-ACT-004` and `CTRL-PRV-002` through H3.

2. Dependency spine

The critical dependency order is:

```
Normative contracts and threat model
  -> tenant identity, authorization, and audit
  -> authoritative data, outbox, and immutable artifacts
  -> connector synchronization and provenance
  -> entity resolution and rebuildable projections
  -> permission-aware retrieval and cited answers
  -> versioned scenario snapshots and simulation
  -> exact-payload approval and one connector action
  -> pilot operations, deletion, and recovery
  -> measured enterprise scale and deployment expansion
  -> separately governed research capabilities
```

Work may proceed in parallel only after the contracts at its incoming edge are frozen and validated. Frontend mocks may precede backend completion, but they cannot define alternate authorization, action, or data semantics.

3. Horizon summary

Horizon	Status	Product boundary	Scale and service boundary	Exit decision
H1 - Hackathon reference slice	Committed	Two synthetic tenants; GitHub metadata read-only; Jira read and one dual-approved remediation update; cited launch-risk answer; seeded launch simulation; rollback; one synthetic physical-asset simulator with 3D-style spatial inspection, deterministic analytics, lifecycle history, and no device egress	Up to 100 identities, 100,000 graph nodes and 1,000,000 edges per tenant; ten concurrent users; 15-minute source freshness; bounded synthetic telemetry polling; p95 non-AI read under 2 seconds, simulation under 10 seconds, cited answer under 20 seconds	Complete `CH-02` and `CH-17` workloads and all H1 security, quality, accessibility, and audit gates
H2 - Design-partner pilot	Committed after H1 exit review	Up to ten tenants; enterprise SSO/SCIM; permission revocation; retention/deletion; connector recovery; partner-approved domains	Up to 1,000 users and 1,000,000 nodes/10,000,000 edges per tenant; 99.9 percent availability; one-hour RPO; four-hour RTO	Partner acceptance, operational evidence, deletion proof, representative load and recovery tests, and no unresolved Critical/High finding
H3 - Enterprise GA	Provisional	Supportable enterprise product, commercial controls, hardened integrations, measured scale tiers	Scale, SLO, cost, isolation, residency, and DR targets are frozen from H2 telemetry and representative benchmarks before commitment	GA readiness review proves targets, support model, security evidence, upgrade/rollback, and unit economics
H4 - Deployment expansion	Provisional	Dedicated data plane, customer VPC, on-premises, air-gapped, regional, and edge profiles introduced separately	Per-profile limits and failure domains; no blanket active-active multi-cloud promise	Each profile passes compatibility, security, operability, upgrade, backup, recovery, and cost gates
H5 - Research	Research	Hyperscale graph, causal organizational models, bounded higher autonomy, collective decision systems, and long-horizon simulations	No production scale claim; trillion-edge and autonomous-organization ideas remain hypotheses	Each program has its own ethical, scientific, safety, legal, and product graduation decision

4. H1 - Production-shaped hackathon slice

4.1 Team and timebox

H1 is designed for three to five people over two to four weeks:

- Platform/security engineer owns tenant context, policy enforcement, PostgreSQL/RLS, action governance, audit, local infrastructure, and CI.
- Data/connectors engineer owns fixtures, GitHub/Jira normalization, synchronization, outbox, provenance, resolution, and projections.
- AI/simulation engineer owns structured retrieval, model gateway, evaluations, scenario compilation, PERT/Monte Carlo engine, and result explanations.
- Product/full-stack engineer owns API integration, application shell, Ask, Explore, Simulation, Approval, and Audit journeys.
- Quality/reliability ownership is shared or assigned to a fifth engineer; release authority remains independent from feature authorship.

A smaller team preserves subsystem ownership but reduces parallelism; it does not remove acceptance gates.

4.2 Work packages and gates

Order	Work package	Depends on	Completion evidence
1	Normative baseline and repository	None	Versioned contracts, requirement/control/risk IDs, architecture decisions, threat model, and validation pipeline
2	Local platform and tenant security	1	Reproducible local startup; authenticated server-derived tenant context; RLS isolation; tenant-qualified keys; redacted telemetry; secret handling
3	Authoritative model and durable workflows	1, 2	Migrations; authoritative records; object digests; transactional outbox; Temporal workflows; replay and recovery tests
4	Synthetic GitHub/Jira synchronization	2, 3	Signed deterministic fixtures; allowlists; cursors; webhooks; duplicates; ordering; tombstones; reconciliation; quarantine; connector health
5	Identity, evidence, graph, and retrieval	3, 4	Reversible resolution; claims/evidence; ACL propagation; Neo4j and pgvector rebuild; bounded traversal; ground-truth oracle comparison
6	Cited answer	2, 5	Capability-oriented model gateway; structured tool schemas; prompt-injection defenses; citations; abstention; permission-restricted answer; golden and adversarial evaluations
7	Scenario and simulation	3, 5	Immutable snapshot; validated dependency DAG; seeded PERT/Monte Carlo run; percentiles, critical path, sensitivity, assumptions, warnings, deterministic oracle
8	Governed Jira action	2, 3, 4, 7	Exact preview; canonical hash; two distinct approvers; 15-minute expiry; policy recheck; source-version precondition; idempotent receipt; compensation and conflict handling
9	Synthetic physical-asset path	1-3	Deterministic fixture telemetry; component-linked procedural perspective and structured views; reproducible anomaly/trend result; lifecycle history; simulator-only exact command preview, idempotent receipt, audit verification, and zero egress
10	Integrated product journey	4-9	Responsive and WCAG 2.2 AA journey across both the organizational and synthetic asset paths
11	Release evidence and independent review	1-10	Clean-room run; performance/security /accessibility/evaluation reports; failure demonstrations; SBOM and provenance; no Critical/High findings

4.3 H1 exit gate

H1 exits only when:

- The exact `edt-h1-github-jira-launch-risk` workload passes from an empty local environment.

- The `CH-17` physical-asset path reproduces telemetry and analytics, provides an equivalent non-3D view, executes only allowlisted simulator commands, verifies idempotency/audit evidence, and demonstrates zero device egress.
- All H1 requirements, controls, risks, contracts, and acceptance criteria are traceable to passing evidence.
- There are zero cross-tenant and unauthorized disclosures in automated, adversarial, and manual tests.
- Unsupported answers abstain, repeated simulations reproduce, invalid approvals write nothing, duplicate execution writes once, and rollback is verified.
- The complete journey passes manual keyboard and screen-reader review.
- Ten-user performance meets the H1 envelope with traces showing no hidden correctness fallback.
- A reviewer who did not author the subsystem confirms that H1 can be operated and recovered using the documentation.

H1 does not exit by hiding a failing dependency behind fixture-only precomputed answers or seeded final database state.

5. H2 - Design-partner pilot

5.1 Entry conditions

H2 begins only after the H1 exit review and after each design partner signs an approved data-purpose, connector-scope, tenant-isolation, retention, incident, and prohibited-use agreement. Partner data cannot be used to fill gaps in product governance.

5.2 Committed capability increments

Increment	Required behavior	Gate
Enterprise identity	OIDC/SAML federation, SCIM provisioning/deprovisioning, group mapping, step-up authentication, access review, and emergency revocation	Revocation reaches every serving projection and active agent/tool run within the defined SLO; stale state fails closed
Tenant lifecycle	Provision, suspend, export, retention, deletion, cryptographic erasure where applicable, and signed completion evidence	Restore and deletion drills show no orphan projection, object, cache, embedding, trace, or backup-policy gap
Connector operations	Production GitHub and Jira connectors plus at most two partner-selected read-only connectors	Scope review, rate-limit recovery, API-version compatibility, reconciliation, credential rotation, quarantine, and source-permission tests
Domain packs	Organization, work/projects, and engineering/product; one partner-selected pack may enter controlled pilot	Namespaced ontology, source precedence, lifecycle, privacy, validation, and steward workflow are complete
Collaboration	Saved views, reviewed reports, notifications, and claim/scenario comments	Reauthorization on every open/export; messages carry no sensitive body by default; comments cannot grant evidence access
Reliability	Multi-instance workloads, backup/restore, documented failover, recovery drills, capacity alerts, and incident procedures	99.9 percent service objective, one-hour RPO, four-hour RTO, and fault-injection evidence
Governance evidence	Control mapping, audit export, data inventory, privacy workflows, support boundaries, and change approvals	Evidence readiness only; no claim of certification without an independent certification process

5.3 Pilot constraints

- Up to ten tenants, 1,000 users, 1,000,000 graph nodes, and 10,000,000 edges per tenant.
- Shared pooled deployment remains the reference profile, with database-enforced isolation and tenant-specific credentials, keys, graph namespaces, vector scope, object prefixes, and cache scope.
- New external action types remain disabled by default. Each requires an action-specific threat model, exact schema, authorization policy, approver roles, idempotency definition, source concurrency control, compensation behavior, sandbox evaluation, and partner opt-in.
- Cross-tenant analytics, retrieval, entity resolution, memory, training, and benchmarks remain prohibited by default.
- Individual employment and health inference remains excluded.

5.4 H2 exit gate

H2 exits when representative partner evidence proves permission revocation, synchronization recovery, projection rebuild, deletion, backup restore, model fallback, incident response, connector upgrade, and tenant suspension within target; no Critical/High finding remains; Medium findings have named acceptance and revisit triggers; and two consecutive cross-domain reviews identify no new Critical/High issue.

6. H3 - Enterprise general availability

H3 is Provisional until H2 telemetry and benchmarks freeze numeric tiers. The implementation team MUST NOT invent GA scale or cost promises from H1 synthetic results.

6.1 GA workstreams

- Product packaging, entitlements, contract-safe tenancy boundaries, support tiers, and tenant-safe metering.
- Formal service objectives and error budgets for API, ingestion freshness, retrieval, simulation, action execution, projection lag, and recovery.
- Upgrade, rollback, schema compatibility, connector compatibility, and deprecation policy across supported versions.
- Representative capacity models, per-tenant quotas, noisy-neighbor controls, cost attribution, and admission control.
- Security program evidence, independent penetration testing, supply-chain provenance, incident exercises, privacy impact assessments, and customer assurance material.
- Regional data-processing controls and residency commitments that are technically enforced and testable.
- Supportable administration, diagnostics, audit export, and customer success workflows that do not require platform access to tenant plaintext.

6.2 Benchmark-gated scale transitions

Transition	Remains deferred until	Required proof before adoption
PostgreSQL full-text/pgvector to OpenSearch	Retrieval relevance, latency, indexing isolation, or operational cost misses a frozen tier	Representative corpus benchmark, ACL update/revocation behavior, tenant isolation, rebuild, backup, cost, and operations review
PostgreSQL audit/query indexes to ClickHouse	Approved aggregate/audit workloads cannot meet SLO or cost without harming transactions	Data-minimization design, tenant isolation, deletion/retention propagation, reconciliation, restore, and measured benefit
Transactional outbox plus Temporal to Kafka	Sustained event fan-out, retention, replay, or throughput exceeds measured limits	Partitioning/keying, ordering semantics, schema governance, replay, DLQ, tenant isolation, disaster recovery, and staffing evidence
Operational stores to a lakehouse	Governed analytical use cases and retention volume justify another copy	Data-purpose approval, lineage, tenant isolation, opt-in for any cross-tenant analysis, deletion, cost, and owner
Modular workloads to finer services	Independent scaling, fault isolation, team ownership, or release cadence has a measured bottleneck	Boundary load tests, contract, ownership, on-call, tracing, migration, rollback, and reduced total risk
REST read APIs to GraphQL	Partner graph-exploration use cases show a material client need	Query cost controls, field authorization, depth/complexity limits, persisted queries, schema lifecycle, and no topology leakage
Internal HTTP to gRPC	Extracted services require typed streaming or lower measured overhead	Protobuf ownership, compatibility, deadlines, retries, tenant propagation, observability, and browser boundary remains REST/SSE

6.3 H3 commitment and exit

The Architecture Council converts H3 to Committed only after publishing tier-specific scale, availability, freshness, latency, RPO/RTO, support, and cost targets based on evidence. GA exits when two release candidates pass clean install, upgrade, rollback, restore, tenant-migration, connector-version, load, chaos, security, privacy, accessibility, and model-evaluation gates with no open Critical/High finding.

7. H4 - Deployment expansion

Deployment profiles graduate independently in this order unless customer evidence justifies a different branch:

1. Dedicated vendor-managed tenant data plane.
2. Customer VPC/VNet deployment with vendor-managed control plane.
3. Customer-managed on-premises connected deployment.
4. Regional and multi-region profiles with explicit authority and failover semantics.
5. Air-gapped deployment after removal or replacement of required online dependencies.
6. Edge ingestion or local inference only for measured latency, sovereignty, or disconnected-operation cases.

Profile	Blocking questions that must be resolved before commitment
Dedicated data plane	Provisioning, key ownership, upgrades, observability, cost floor, tenant migration, and recovery responsibility
Customer VPC/VNet	Control/data-plane trust, outbound connectivity, private endpoints, credential custody, diagnostics, support access, and drift
On-premises connected	Supported Kubernetes/storage matrix, hardware sizing, upgrade cadence, backup ownership, remote support, and connector egress
Regional/multi-region	Data authority, conflict handling, identity routing, graph/vector projection placement, workflow ownership, failover, RPO/RTO, and residency
Air-gapped	Offline identity, model runtime and evaluation, package signing/import, connector feasibility, vulnerability updates, license checks, clock, and export review
Edge	Offline queue, local authorization, key custody, model/data minimization, reconciliation, remote wipe, and bounded stale operation

Under `REQ-ARCH-004` and `ADR-015`, cloud neutrality means portable interfaces and deployment artifacts with a tested reference profile. It does not mean every profile is identical or simultaneously active-active.

Each profile must pass the same tenant, provenance, authorization, audit, accessibility, action, and evaluation contracts as the reference deployment or explicitly declare a capability unavailable. Silent weakening is prohibited.

8. H5 - Research portfolio

Research runs in isolated accounts and datasets, has no production credentials, and cannot be enabled by an ordinary feature flag. Each project has a hypothesis, principal investigator, data-purpose statement, review board, stop condition, misuse analysis, evaluation plan, and publication/retention policy.

8.1 Hyperscale organizational graph

Hypothesis: hierarchical partitions, temporal summaries, compressed representations, and workload-specific graph engines can answer defined organizational queries at billion-node scale without weakening tenant or permission semantics.

Graduation evidence:

- Representative query corpus and update workload, not synthetic edge count alone.
- Exact tenant/ACL correctness during ingestion, traversal, revocation, rebuild, and failover.
- Bounded query cost and graceful truncation.
- Measured operational cost, recovery, deletion, and team support burden.
- A decision use case that requires the additional scale.

Trillion-edge capability remains Research until all conditions hold; no roadmap date is assigned.

8.2 Causal and predictive organizational models

Hypothesis: for narrowly defined non-employment outcomes, explicit causal assumptions and prospective validation can improve decisions beyond descriptive and conditional simulation.

Graduation evidence:

- Pre-registered target, intervention, causal graph, assumptions, confounders, and invalidation tests.
- Prospective evaluation against a partner-approved baseline with calibrated uncertainty.
- Dataset shift, feedback-loop, fairness, privacy, and misuse assessment.
- Human decision protocol, appeal/correction path, and no automated high-impact action.
- Independent statistical and domain review.

Synthetic PERT/Monte Carlo scheduling is not evidence for causal validity.

8.3 Workforce-sensitive research

Individual burnout, attrition, productivity, performance, emotion, health, misconduct, hiring, promotion, compensation, and termination inference is Rejected for H1-H3 product use. Any research proposal involving people must begin with necessity, proportionality, worker representation, legal review, privacy impact, bias and harm analysis, consent or other valid basis, data minimization, and a credible decision benefit. A successful model evaluation alone cannot graduate the use case.

The default research direction is team-level process and system bottlenecks without ranking people. No workforce-sensitive research may use customer data, influence an employment decision, or ship behind a hidden preview.

8.4 Higher autonomy and organizational agents

Hypothesis: bounded agents can coordinate longer workflows while maintaining delegated authority, verifiable state, cost limits, cancellation, approval, and recovery.

Graduation sequence:

1. Read-only offline task with golden oracle.
2. Read-only shadow run against synthetic or explicitly approved data.
3. Draft-only recommendation with human review.
4. Sandbox action with exact approval and compensation.
5. Narrow production action after action-specific governance.

Authority inheritance, self-approval, approval inferred from chat, creation of new credentials, disabling audit, and open-ended external tool discovery are Rejected. Artificial CEO, board, HR, legal, finance, or security titles confer no authority and are not the capability model.

8.5 Collective memory and decision systems

Research may examine time-versioned decision rationale, claim contradiction, institutional-memory decay, and multi-agent critique. Graduation requires provenance, correction, expiry, contested-claim representation, permission revocation, memory deletion, anti-amplification safeguards, and evidence that multiple agents improve calibrated quality rather than merely increasing tokens or consensus theater.

8.6 Market and economic simulation

Research may model bounded, explicitly hypothetical external scenarios with licensed or synthetic data. It must separate organization-internal evidence from external assumptions, report sensitivity and validity range, and avoid investment, credit, insurance, legal, or employment decisions without separate high-impact governance. Model complexity is justified only by prospective predictive or decision-value evidence.

9. Research graduation gate

A Research capability can become Provisional only when all of the following are recorded:

- Named product decision and affected users.
- Falsifiable hypothesis and baseline.
- Authorized, representative evaluation data with retention and deletion rules.
- Quality, calibration, safety, privacy, fairness, security, latency, reliability, and cost thresholds.

- Threat and misuse model, red-team results, safe failure, kill switch, and incident owner.
- Human oversight, contestability, correction, and audit design.
- Compatibility and rollback plan.
- Independent domain, security, privacy, and research review.

It can become Committed only after shadow or controlled-pilot evidence meets those thresholds and an Architecture Decision Record assigns a product owner, service boundary, support policy, and horizon. Failure to meet a stop condition archives the project and its status remains Research or becomes Rejected.

10. Portfolio stop and convergence rules

- At most one new external action class enters active graduation at a time until H2 action operations are proven.
- A benchmark-triggered technology is not introduced while the existing component meets its frozen SLO and cost envelope unless risk reduction is independently demonstrated.
- Research cannot consume production reliability capacity without a named budget and isolation boundary.
- A roadmap review closes only when every Committed item has an owner and evidence, every Provisional item has a named gate, every Research item has a protocol, and every Rejected item has a rationale.
- Two consecutive cross-domain reviews with no new Critical/High finding are required before specification convergence; this limits review passes, not ongoing operational assurance.

CH-16 - Architecture Audit and Convergence Record

Status: committed | Owners: architecture-review-board, security-architecture, product-governance | Last reviewed: 2026-07-13

Architecture Audit and Convergence Record

1. Audit opinion

The `1.0.0-rc.1` architecture is suitable as the normative implementation blueprint for the bounded H1 reference workload and as a gated reference architecture for H2. It does not become final `1.0.0` until every convergence criterion in this chapter is evidenced. H3 and H4 remain Provisional where telemetry, benchmarks, customer constraints, or deployment evidence are required. H5 remains Research.

No Critical or High architecture finding remains open in the design. Three Medium risks are explicitly accepted with owners, compensating controls, expiry dates, and revisit triggers. Other registered risks are mitigated but remain subject to implementation evidence. This opinion does not assert that the software has been built, that a control has operated effectively, that a compliance certification exists, or that synthetic simulation has external predictive validity.

The `CH-17` synthetic physical-asset amendment was added after the review passes listed in section 2.3. Its scope is deliberately limited to an in-process simulator and reuses the established tenant, API, exact-payload, idempotency, audit, deterministic-model, and accessibility boundaries. This amendment does not claim a completed architecture review or release gate: `TST-PHY-001`, `AC-PHY-001`, contract validation, and independent review remain required before publication evidence can describe the amended H1 scope as passed.

The `CH-18` event-intelligence amendment adds a synthetic causal-analysis and reversible graph-mutation path after those same review passes. Its major hazards are false-event graph poisoning, causal overclaim, unbounded propagation, sensitive workforce use, and stale or conflicting history. The amendment narrows those hazards through untrusted-content separation, resolvable evidence and entity review, reality-versus-scenario routing, three-hop and fixed-size expansion limits, confidence decay, exact graph-version-and-hash compare-and-swap, atomic receipt/audit/outbox persistence, restart hydration, compensation, and the existing workforce-inference prohibition. `RSK-020` through `RSK-022`, `CTRL-EVT-001`, `TST-EVT-001`, `AC-EVT-001`, contract validation, and independent review remain release evidence; this self-audit does not mark them operationally proven.

The architecture reached this opinion by narrowing unbounded claims, assigning authority and data ownership, converting ambiguous autonomy into bounded capability contracts, making projection and action failure semantics explicit, and defining a finite release gate. An independent build-readiness review and generated validation evidence remain mandatory release evidence under `AC-REV-001` and `AC-REV-002`; this self-audit does not substitute for them.

2. Audit scope and method

2.1 Scope

The audit covers:

- Product mission, users, outcomes, prohibited uses, and H1 demonstration contract.
- Workload decomposition, language ownership, data authority, workflows, events, and deployment boundaries.
- Shared multitenancy, identity, authorization, source ACLs, cryptography, secrets, privacy, and audit integrity.
- Connector installation, synchronization, compromised-source behavior, entity resolution, claims, evidence, and derived projections.
- AI capability profiles, model routing, retrieval, memory, prompt-injection defense, evaluation, and action authority.
- Scenario compilation, PERT/Monte Carlo simulation, uncertainty, prediction boundaries, and research claims.
- REST, SSE, webhook, event, GraphQL, Protobuf, MCP, connector, extension, SDK, and CLI interface ownership.
- UX surfaces, states, visualization semantics, accessibility, and approval ergonomics.
- Reliability, observability, recovery, deployment profiles, supply chain, testing, traceability, roadmap, and stop conditions.

The audit evaluates architecture sufficiency and internal consistency. It does not perform a source-code audit, penetration test, privacy legal opinion, capacity test, model evaluation, restore drill, certification, or production readiness attestation.

2.2 Review questions

Each domain was reviewed against the following questions:

1. Is the purpose and non-goal explicit?
2. Is there one authoritative owner for state and one defined owner for operation?
3. Are tenant and actor context derived and enforced independently of caller input?
4. Are public and internal interfaces typed, versioned, authorized, idempotent where needed, and observable?
5. Are source, valid, system, and processing time distinguished where relevant?
6. Do retries, duplicates, reordering, concurrency, cancellation, timeout, restart, partition, and compensation have defined outcomes?
7. Can a derived view be rebuilt, corrected, permission-revoked, retained, and deleted?
8. Can an AI output or untrusted connector payload affect authority or execution without deterministic validation?
9. Are uncertainty, missing evidence, partial state, and invalid domains visible to the user?
10. Is scale a measured service boundary rather than an unsupported aspiration?
11. Are accessibility and non-visual equivalence part of acceptance rather than post-release remediation?
12. Does every commitment have an owner, horizon, control, risk treatment, and acceptance evidence?

2.3 Review passes

Pass	Purpose	Result
AUD-2026-07-13-A	Cross-domain design audit of the original expansive brief	Found the scope, technology, ontology, graph-tenancy, autonomy, simulation, and compliance issues recorded in the review ledger; all Critical/High findings were remediated in the normative baseline.
AUD-2026-07-13-B	Security, privacy, AI, scale, and buildability audit	Found derived-ACL, approval-replay, cross-tenant-learning, trace-minimization, and H1-envelope gaps; all High findings were remediated and Medium residual risks were assigned.
AUD-2026-07-13-C	Final internal consistency check against requirements, controls, risks, acceptance, and implementation-decision completeness	Found no new Critical or High issue and no new major H1/H2 product, security, data, topology, or interface decision. This is supporting self-review, not independent sign-off.

The formal ledger reviews close with no remaining Critical or High issue, and pass C supports design self-convergence. Release still requires the independent engineer and repository validator evidence named in section 7.

3. Resolved findings

Finding	Original severity	Problem	Resolution	Governing records
FND-001	Critical	The brief demanded exhaustive coverage and recursive review without a finite completion condition.	Freeze H1/H2 scope, use extensible catalogs for breadth, label later work, and require measurable convergence and independent review.	REQ-GOV-002, REQ-GOV-003, ADR-001, ADR-016
FND-002	High	A hackathon deliverable and complete Fortune 500 platform were treated as one immediate release.	Define a production-shaped H1 slice with production invariants inside a fixed envelope; treat H2-H5 as gated horizons.	REQ-PROD-003, REQ-PROD-004, ADR-002, CH-02
FND-003	High	The proposed platform could adopt microservices, a service mesh, multiple queues, and specialized stores before a measured need.	Use four modular OCI workloads, Temporal, a PostgreSQL outbox, and benchmark/ownership ADRs before service or infrastructure expansion.	REQ-ARCH-001 through REQ-ARCH-005, ADR-003, ADR-005, ADR-017
FND-004	High	Relational, graph, vector, search, and object stores lacked a clear authority boundary.	PostgreSQL is authoritative for modeled state; S3-compatible storage preserves immutable source artifacts; graph/vector/search/cache are rebuildable projections.	REQ-DATA-001, REQ-DATA-003, ADR-008, ADR-009, ADR-010
FND-005	Critical	Shared infrastructure without a complete tenant fence could leak through rows, graph topology, embeddings, cache, objects, events, traces, or errors.	Derive tenant context from identity, enforce PostgreSQL RLS and tenant-qualified constraints, independently namespace projections, and run two-tenant negative tests.	REQ-TEN-001 through REQ-TEN-005, CTRL-TEN-001 through CTRL-TEN-003, RSK-001, RSK-002
FND-006	High	Source ACLs could be lost through claims, paths, counts, summaries, embeddings, caches, notifications, or exports.	Enforce monotonic visibility from qualifying evidence, carry ACL metadata, invalidate derived results on revocation, and suppress structural side channels.	REQ-SEC-002, CTRL-DAT-002, ADR-007, RSK-003
FND-007	Critical	Executive-titled agents implied broad or self-expanding authority.	Replace personas-as-authority with bounded capability profiles. Effective authority is the intersection of user, tenant, delegation, workflow, caller, and tool policy.	REQ-AI-002, REQ-AI-008, CTRL-IAM-004, ADR-013
FND-008	Critical	Untrusted connector content or model output could manipulate tools or exfiltrate data.	Separate content from instructions, use fixed structured tool schemas, external authorization, resource budgets, egress constraints, schema validation, and adversarial evaluation.	REQ-AI-003, REQ-AI-004, CTRL-AI-001 through CTRL-AI-004, RSK-004, RSK-005
FND-009	High	Human approval was unspecified and could approve a description instead of the executed action.	Bind two distinct authenticated approvers to tenant, target, canonical payload hash, source version, policy version, expiry, and idempotency key; recheck immediately before execution.	REQ-ACT-001, REQ-ACT-002, CTRL-ACT-001, CTRL-ACT-002, RSK-006

Finding	Original severity	Problem	Resolution	Governing records
FND-010	High	External retry and rollback behavior could duplicate a Jira write or overwrite a concurrent edit.	Use durable workflow state, stable idempotency, provider correlation, before/after receipts, optimistic source-version precondition, and compensation conflict instead of blind overwrite.	REQ-ACT-003, CTRL-ACT-003, AC-ACT-002, AC-ACT-003, RSK-013
FND-011	High	Simulation and prediction scope mixed conditional scheduling with causal and workforce claims.	H1 uses a seeded dependency-DAG PERT/Monte Carlo scheduler with distributions and limitations. Workforce-sensitive inference is prohibited through H3 and separately governed as Research.	REQ-SIM-001 through REQ-SIM-005, CTRL-PRV-002, ADR-014, RSK-007, RSK-011
FND-012	High	Unlimited scale and trillion-edge language had no workload, cost, or validation boundary.	Commit explicit H1 and H2 envelopes; freeze H3 tiers from pilot telemetry; retain hyperscale as H5 Research with representative workload gates.	REQ-REL-001, REQ-REL-002, REQ-REL-006, CH-15
FND-013	High	Cloud neutrality, active-active multi-cloud, air gap, edge, and online OpenAI dependency were treated as simultaneously available.	Define portable contracts and one reference profile; graduate dedicated, customer-managed, regional, air-gapped, and edge profiles independently with explicit capability differences.	REQ-ARCH-004, REQ-ARCH-006, ADR-015, RSK-008
FND-014	Medium	Language, protocol, and datastore lists risked adoption by enumeration and duplicated responsibilities.	Assign TypeScript, Python, SQL, PostgreSQL, Neo4j, Temporal, S3-compatible storage, pgvector, Valkey, OpenTelemetry, Docker, and Kubernetes explicit ownership; defer alternatives behind triggers.	ADR-004, ADR-010, ADR-017, RSK-009
FND-015	High	Graph-first UX could hide evidence, staleness, permissions, uncertainty, and inaccessible states.	Make search/list the default, bound graph traversal/rendering, expose evidence and checkpoints, and require table/text equivalence plus WCAG 2.2 AA acceptance.	REQ-UX-001 through REQ-UX-004, CH-11, AC-UX-001
FND-016	High	Compliance language could be read as a certification or legal conclusion.	Limit claims to control alignment and evidence readiness; require separate organizational controls, legal ownership, audit, and certification.	REQ-SEC-005, REQ-SEC-006, CH-09
FND-017	Medium	Model aliases and provider behavior could drift, fail, or become unavailable.	Route through a capability gateway, pin evaluated snapshots per workload, version prompts/tools, set budgets, and fail closed if no approved fallback passes.	REQ-AI-001, REQ-AI-007, CTRL-AI-003, RSK-012
FND-018	Medium	Audit and observability could become an uncontrolled shadow copy of tenant content.	Minimize to structured metadata and digests, redact by construction, separate audit authorization/retention, and preserve trace linkage without private chain-of-thought.	REQ-AI-006, REQ-SEC-004, CTRL-AI-005, CTRL-AUD-001, RSK-014

4. Domain conformance result

Domain	Disposition	Audit conclusion
Product and scope	Conforms	H1 has an exact actor, dataset, question, scenario, action, and measurement contract; later capability status is explicit.
Service architecture	Conforms	Four workloads have coherent ownership; durable workflow and event boundaries precede optional extraction.
Tenancy and identity	Conforms with implementation evidence required	Independent relational and derived-store fences are designed; negative tests and runtime policy evidence remain release gates.
Data and graph	Conforms	Authority, bitemporal provenance, reversible resolution, ontology versioning, projection rebuild, lifecycle, and domain extension are assigned.
Connectors and synchronization	Conforms	Scope, authentication, webhook verification, cursoring, duplicates, ordering, reconciliation, tombstones, quarantine, and compromised-connector behavior are specified.
AI and retrieval	Conforms with provider dependency	Authority is deterministic outside the model; content is untrusted; outputs are typed; model/prompt/tool changes are evaluation-gated.
Simulation and prediction	Conforms	H1 mathematics and outputs are reproducible and bounded; causal/predictive and workforce-sensitive claims remain gated.
Governed action	Conforms	H1 has one exact allowlisted Jira mutation, dual control, 15-minute expiry, concurrency precondition, durable receipt, idempotency, and compensation.
Security and privacy	Conforms with organizational evidence required	Technical controls and readiness mapping are complete; certification, lawful basis, customer contracts, and incident staffing are not inferred from architecture.
Reliability and observability	Conforms	Dependency-specific degradation, recovery, backpressure, replay, health, redaction, RPO/RTO, and projection rebuild responsibilities are assigned.
UX and accessibility	Conforms	Major surfaces and states, evidence semantics, visualization alternatives, responsive behavior, approval ergonomics, and accessibility gates are defined.
APIs and extensions	Conforms	Interface ownership and horizon are explicit; tenant derivation, auth, compatibility, pagination, idempotency, errors, redaction, and audit semantics are mandatory.
Deployment and operations	Conforms by profile	Local and reference cloud profiles are committed as named; later profiles cannot silently reduce controls.
Verification and developer experience	Conforms	Clean-room development, contract validation, synthetic oracle, continuous AI evaluation, security/load/failure testing, and release evidence have finite gates.

5. Residual risk register

5.1 Accepted Medium risks

These acceptances cannot override a Critical/High result, law, contract, tenant boundary, or prohibited use.

Risk	Residual exposure	Compensating controls	Owner	Acceptance expiry and revisit
RSK-008 - Cloud-neutral abstraction cost	Portable adapters and conformance work may slow H1/H2 and still expose provider differences.	Keep adapters thin, choose one fully tested reference profile, expose capability differences, and reject lowest-common-denominator domain abstractions.	Platform	Expires 2027-01-31 or before selecting the H2 reference provider, whichever is earlier. Revisit with portability test and operating-cost evidence.
RSK-009 - Runtime complexity over time	TypeScript, Python, SQL, and infrastructure runtimes increase tooling and on-call breadth.	Limit ownership by workload, share contracts and telemetry, maintain one local command, and require an ADR plus staffing evidence for every new language.	Architecture	Expires 2027-07-13 or before H3 service extraction, whichever is earlier. Revisit with incident, build-time, staffing, and performance data.
RSK-016 - Synthetic external validity	H1 proves deterministic product behavior but not customer data quality, decision value, or predictive validity.	Label synthetic output, prohibit production accuracy claims, use partner-approved golden cases, and require prospective validation before prediction claims.	Product	Expires before the first H2 partner outcome claim or 2027-01-31, whichever is earlier. Revisit through design-partner evaluation.

5.2 Mitigated risks that require operating evidence

Risk group	Residual concern	Required evidence
RSK-001, RSK-002, RSK-003	A code, query, cache, timing, or projection defect may still cross a tenant or ACL boundary.	RLS tests, tenant-qualified constraints, graph/vector/object/cache negative tests, revocation latency, side-channel tests, and independent penetration review.
RSK-004, RSK-005	Novel prompt injection or connector compromise may bypass assumed parsing, egress, or policy boundaries.	Adversarial corpus, sandbox/egress tests, credential canaries, incident exercise, and continuous connector/model evaluation.
RSK-006, RSK-013	Confused-deputy, race, provider ambiguity, or crash recovery may execute an unintended or duplicate action.	Canonicalization vectors, concurrent execution tests, source-version conflicts, restart reconciliation, approval revocation, and compensation drills.
RSK-007	A future ontology, dashboard, prompt, export, or customer configuration may recreate prohibited workforce scoring.	Schema/policy deny rules, product review, adversarial prompts, analytics review, sales/support training, and privacy audit.
RSK-010	Real-world identity ambiguity may exceed deterministic fixtures and create false merges.	Partner-specific false-merge/non-merge evaluation, review thresholds, split drills, and projection rebuild evidence.
RSK-011	Users may still over-trust a well-designed distribution or sensitivity chart.	Comprehension testing, claim-language review, calibration/validity documentation, and prohibition of guaranteed-date copy.
RSK-012	Provider outage, snapshot retirement, or regression may exceed fallback coverage.	Snapshot registry, failover exercises, capacity/budget alerts, fallback evaluation, and explicit unavailable behavior.
RSK-014, RSK-015	Logs, audit, backups, and derived stores may retain sensitive or deleted content.	Data inventory, content canaries, retention/deletion scan, backup-expiry evidence, redaction tests, and restricted audit access review.
RSK-017	Advanced visualization may exclude users or distract from evidence.	Keep 3D organizational visualization Rejected; permit the bounded synthetic physical-asset scene only with a component-location decision task, keyboard/reduced-motion support, rendering fallback, and an equivalent structured view.
RSK-018	Sustained backlog may make a graph or answer materially stale.	Lag SLOs, backpressure, priority, reconciliation, stale-result UX, capacity tests, and fail-closed actions.

6. Known limitations and architecture boundaries

These limitations are intentional and do not represent hidden implementation choices:

- H1 ingests external-source metadata from synthetic GitHub and Jira only. The `CH-17` asset path generates synthetic telemetry inside the application; it has no IoT connector, historian, industrial protocol, physical-device read, or customer data. H1 does not ingest source code, email, private chat, meeting audio, finance records, or individual activity telemetry.
- H1's Jira field update is the only external mutation. Asset control modifies in-process simulator state and receipts state `external_write: false`. Every additional external action class requires a separate schema, threat model, approver policy, idempotency definition, source concurrency rule, compensation behavior, evaluation, and opt-in.
- H1 physical-asset anomalies and forecasts are deterministic demonstrations over synthetic history. They are not trained or validated predictive-maintenance models and cannot support maintenance, safety, or equipment-operation decisions.
- PERT/Monte Carlo output is conditional on fixture distributions and dependency structure. It is not a causal estimate and cannot validate a real launch forecast.
- Shared tenancy is the reference deployment. Dedicated, customer-managed, regional, air-gapped, and edge profiles remain separately gated.
- The online OpenAI runtime is an explicit dependency for H1. Air-gapped inference is unavailable until a local model profile passes the same workload-specific quality and safety gates.
- PostgreSQL, Neo4j, pgvector, and Valkey are sufficient only inside measured boundaries. Scale transitions follow `CH-15`; they are not pre-approved technology migrations.
- Two-person approval reduces accidental or unilateral action but does not prevent collusion. Organizational role assignment, separation-of-duty review, and audit monitoring remain required.
- Source systems may not support transactional rollback. Compensation is a new conditional action with conflict handling, not erasure of history.
- Control mapping is not certification and architecture cannot supply legal basis, employee consultation, customer contract, incident staffing, or operational control effectiveness.

7. Release convergence criteria

Specification version 1.0.0 is release-eligible only when all of the following are evidenced from a clean checkout:

1. Every normative Markdown file has unique valid frontmatter; every catalog, contract, diagram source, and example validates.
2. Consolidated Markdown, HTML, PDF, and traceability outputs build reproducibly.
3. Requirement coverage is 100 percent from each `REQ` through artifact, accepted ADR, component/contract, control, acceptance criterion, and horizon.
4. H1 and H2 contain no unresolved major product, security, privacy, data, topology, interface, lifecycle, failure, SLO, observability, ownership, or compatibility decision and no unowned `TBD`.
5. Every Committed subsystem defines purpose, non-goals, actors, ownership, schemas, interfaces, workflows, invariants, authorization, privacy, consistency, failure behavior, scaling, cost, SLOs, telemetry, deployment, testing, risks, and evolution.
6. The risk catalog contains no open Critical or High item. Each accepted Medium item has a named owner, expiry, compensating controls, and revisit trigger.
7. Two consecutive cross-domain reviews introduce no new Critical or High finding. Self-review passes alone do not satisfy independent build readiness.
8. An engineer independent of the authoring work records `AC-REV-001`: H1 can be implemented without making another major unstated product, security, data, topology, or interface decision.
9. The H1 workload evidence plan covers tenant isolation, permission revocation, deterministic ingestion/rebuild, citation/abstention, prompt injection, simulation reproducibility, approval expiry and mutation, duplicate execution, rollback/conflict, dependency failure, performance, accessibility, audit, and supply chain.
10. Any conflict is resolved by the declared precedence order rather than by document date or implementation convenience.

`AC-REV-002` is met only when criteria 1 through 10 are recorded in the generated audit/remediation dossier. Absence of a detected problem is not evidence of completion.

8. Reopen triggers

The Architecture Review Board MUST reopen the affected audit domain when any of the following occurs:

- A Critical/High vulnerability, privacy impact, tenant-isolation failure, source-ACL leak, unsafe action, or prohibited-use path is discovered.
- A model, provider, connector API, identity provider, policy engine, datastore, workflow runtime, or deployment profile changes its relevant contract.
- A new external mutation, connector data category, ontology domain, cross-tenant use, memory type, prediction target, or workforce-sensitive purpose is proposed.
- H1/H2 workload, tenant count, graph size, event rate, concurrency, latency, availability, RPO/RTO, residency, or cost boundary is exceeded.
- A benchmark triggers OpenSearch, ClickHouse, Kafka, a lakehouse, finer services, GraphQL, gRPC, another graph engine, or another language.
- A deployment adds customer management, on-premises, air gap, edge, multi-region writes, or active-active behavior.
- An accepted Medium risk reaches expiry, its compensating control fails, or its owner changes without renewed acceptance.
- An accessibility regression prevents equivalent completion of a Committed workflow.

Reopening a domain does not automatically invalidate unrelated decisions. The new review records impacted requirements, contracts, controls, risks, migrations, tests, rollout, rollback, and semantic version before implementation.

9. Final self-critique

The strongest aspect of this architecture is that it turns an ambitious organizational-intelligence concept into a complete, testable chain from evidence to governed action while preserving a larger extensibility path. The most consequential remaining weakness is not an unmade architecture choice; it is the amount of disciplined implementation and operating evidence required to prove that tenant isolation, permission fidelity, model safety, deletion, recovery, and action idempotency work together under failure.

The architecture deliberately accepts slower breadth expansion, a multi-runtime learning cost, and limited H1 external validity. Those tradeoffs are preferable to unsupported hyperscale claims, premature infrastructure, hidden workforce inference, or unsafe autonomy. If implementation evidence contradicts a design assumption, the appropriate response is to reopen the decision and narrow or redesign the capability, not to weaken the gate.

CH-17 - H1 Synthetic Physical-Asset Twin

Status: committed | Owners: product-engineering, simulation-team, security-engineering | Last reviewed: 2026-07-14

H1 Synthetic Physical-Asset Twin

1. Purpose and truth boundary

This chapter defines a second bounded H1 demonstration path for a synthetic industrial asset. It complements, but does not replace or alter, the frozen GitHub/Jira launch-risk workload in `CH-02`. The path demonstrates live-looking telemetry, spatial inspection, deterministic analytics, lifecycle history, and guarded control against an in-process simulator.

Every asset, sensor sample, lifecycle event, analytic result, and command effect is synthetic. H1 has no IoT gateway, message broker, OPC UA, Modbus, MQTT, SCADA, PLC, historian, CMMS, or physical-device connection. It reads no customer telemetry and sends no command outside the application. The interface and API MUST label this boundary, and control receipts MUST state `simulation: true` and `external_write: false`.

The Jira update defined in `CH-02` remains the only H1 external mutation. A simulated asset command changes only tenant-scoped demonstration state and is not evidence that the platform is safe or suitable for real operational-technology control.

Requirement	H1 commitment
`REQ-PHY-001`	Produce deterministic, timestamped synthetic temperature, pressure, vibration, flow, and operating-state samples with sequence and quality metadata.

Requirement	H1 commitment
REQ-PHY-002	Provide an interactive procedural 3D-style physical-asset view linked to components and telemetry, with a complete keyboard-operable structured alternative.
REQ-PHY-003	Detect and forecast synthetic degradation with versioned deterministic algorithms and explicit limitations, without claiming validated machine learning or real failure prediction.
REQ-PHY-004	Present design, manufacture, commissioning, service, maintenance, and planned decommissioning history as versioned synthetic lifecycle records.
REQ-PHY-005	Preview and execute only allowlisted simulated commands with tenant authorization, expected-version checks, limits, idempotency, audit evidence, and fail-closed safety checks.

2. Reference asset and snapshot

H1 contains one synthetic centrifugal pump and its component hierarchy. Stable tenant-qualified IDs identify the motor, coupling, drive-end bearing, pump casing, inlet, and outlet. The asset snapshot contains:

- identity, type, location, lifecycle stage, operating state, health summary, and version;
- a component list with spatial hotspot identifiers and associated sensor tags;
- current telemetry and a bounded recent history;
- deterministic anomalies, forecasts, model card, and per-signal contributions;
- lifecycle stages, dated events, maintenance records, and provenance labels;
- current simulator control state and an allowlist of available commands; and
- a data watermark showing the deterministic simulator sequence and observation time.

The H1 asset record is a demonstration projection over checked-in fixture parameters. It does not promote the full `edt.pack.physical` enterprise domain pack or real IoT connector certification from H4 into H1.

3. Synthetic near-real-time telemetry

The server runs a deterministic five-second simulator clock initialized from the fixture. A read materializes only frames due on that clock, so repeated or concurrent readers inside one interval observe the same current frame and cannot make simulated time run faster. Long inactive gaps are bounded to twelve catch-up frames and then rebase to the current server clock rather than replaying an unbounded backlog. The frozen-clock test profile advances exactly one frame per request for reproducible assertions. The `limit` query selects a rolling window of 1 to 120 recent frames; it does not itself select an advancement count. A sample has an asset ID, monotonically increasing sequence, sample timestamp, quality status, and typed values with units. Values vary within configured operating envelopes; a deterministic degradation term produces a reproducible bearing-vibration trend. The same fixture version, initial state, command history, and sequence MUST produce the same canonical samples.

The web client polls the telemetry resource while the twin screen is active, pauses when the document is hidden, aborts outstanding requests on navigation, and stops after repeated errors until the user explicitly retries. The screen shows last update, connection state, data age, units, thresholds, and whether a value is observed from the simulator or derived. A user can pause live updates without losing the last safe snapshot.

This polling loop is a bounded H1 substitute for a telemetry stream. It is not a claim of continuous ingestion, device clock synchronization, exactly-once delivery, industrial event rates, or production freshness. A real connector must separately define authentication, device identity, ordering, clock skew, buffering, backpressure, quality codes, calibration, retention, reconciliation, network segmentation, and safety behavior.

4. Interactive procedural 3D-style inspection

The asset screen renders a purpose-built perspective SVG representation of physical equipment rather than a photorealistic CAD model. Users can rotate and reset the view, select a component hotspot, and correlate the selected component with current measurements, anomaly contributions, forecast, and lifecycle/maintenance context. Selection is state, not decoration: the component ID is shared by the procedural scene and its structured inspector.

The procedural scene MUST NOT be the only way to locate or understand an issue. The same components, statuses, measurements, units, alerts, and actions are available as keyboard-operable controls and a table/list in logical order. Focus is visible, selection is announced, color is not the sole status encoding, pointer gestures have button equivalents, and reduced-motion mode disables continuous nonessential animation. An SVG or rendering failure leaves the structured view fully usable.

This is distinct from the rejected H1-H3 3D organizational view in `CH-11`: the physical scene supports a concrete spatial maintenance question and has no exclusive information.

5. Deterministic anomaly and forecast demonstration

H1 analytics are transparent numerical demonstrations, not an LLM decision and not a validated predictive-maintenance model. The service computes multivariate exponentially weighted moving statistics and normalized residual/z-score contributions over synthetic history, then uses a deterministic least-squares trend to estimate a threshold-crossing horizon or remaining-useful-life range when the data supports it.

Every result identifies input watermark, algorithm and model version, generated time, affected component, signal contributions, severity, confidence expression, forecast horizon or failure mode, recommended maintenance, and missing-data/validity caveat. Identical inputs and model version MUST produce identical results. Insufficient history, non-finite values, or an ill-conditioned trend produces `no_failure_mode_indicated` with null horizon fields and an explicit limitation, not an invented forecast.

The interface MUST use language such as `synthetic anomaly` and `conditional demonstration forecast`. It MUST NOT claim that a failure will occur, that the algorithm learned from real equipment, that a probability is calibrated on field outcomes, or that a recommended action is an engineering safety determination. Production use requires representative historical data, leakage review, backtesting, calibration, false-positive/negative analysis, drift monitoring, human-factors review, domain-engineer approval, and prospective validation.

6. Lifecycle history

The twin presents a coherent synthetic history from design and manufacture through commissioning, service, maintenance, and planned decommissioning. Each lifecycle event has a stable ID, stage, status, effective date/time, title, description, and optional artifact reference. Completed history is append-only in H1; corrections create a superseding event rather than silently rewriting prior history.

Lifecycle records provide context, not authority to perform maintenance or retire equipment. Planned work and decommissioning dates remain proposals. A future CMMS/PLM integration must define source precedence, reconciliation, permissions, retention, legal/quality records, version conflicts, and write governance before the platform can claim lifecycle synchronization.

7. Guarded simulated control

H1 exposes `set_speed_pct`, `set_valve_pct`, `emergency_stop`, and `reset` only when the current synthetic asset state and actor capability allow them. Control is a two-step workflow:

1. The server validates the requested command, reason, expected asset version, allowlisted type, numeric range, transition rule, and tenant context, then returns a short-lived exact-payload preview with before/after simulator state, safety checks, payload hash, and ETag.
2. Execution requires the preview ID, `Idempotency-Key`, and matching `If-Match`. The server reauthorizes the actor, revalidates expiry and expected version, verifies the preview hash, and either applies one in-process state transition or rejects without an effect.

The same idempotency key and payload returns the original receipt; reuse with different input fails. A stale asset version, expired preview, failed safety check, unsupported transition, out-of-range setpoint, or policy failure performs no state change. `emergency_stop` has no confirmation bypass in the demo; it remains a simulated command with the same authentication, validation, idempotency, and audit requirements. `reset` may clear a simulated trip only when its transition rule passes.

Preview and receipt payloads disclose simulation mode, before/after state, asset versions, payload hash, actor-safe audit reference, and hash-chained audit evidence. They never imply a PLC acknowledgement or physical verification. Adding real device egress is a new external action class and blocks release until an accepted ADR, OT threat model, safety-hazard analysis, device identity and protocol contract, network-zone design, emergency/timeout semantics, independent interlocks, approval policy, post-command verification, rollback/compensation analysis, and representative hardware-in-the-loop evidence exist.

8. Public API and authorization

The normative OpenAPI contract defines:

Operation	Semantics
`GET /v1/assets`	List tenant-authorized synthetic asset summaries.
`GET /v1/assets/{assetId}/twin`	Return the component, visualization, telemetry, analytic, lifecycle, control, and watermark snapshot.
`GET /v1/assets/{assetId}/telemetry?limit=...`	Observe the simulator clock and return a bounded deterministic sample batch.
`POST /v1/assets/{assetId}/control-previews`	Validate and freeze an exact synthetic command preview.
`POST /v1/assets/{assetId}/control-previews/{previewId}/execute`	Revalidate and execute the preview once against simulator state.

Tenant context derives from the authenticated membership; no asset route accepts a tenant selector. Reads require `asset.read`, preview requires `asset.control.preview`, and execution requires `asset.control.execute`. The Beacon analyst has read-only access to its tenant-scoped synthetic asset view and no control grant. Unauthorized and unknown asset IDs receive the same non-enumerating response. Command bodies reject unknown properties. Responses are `private, no-store`; simulator identifiers and payloads are tenant-scoped; every preview, execution, rejection, conflict, and replay is audited.

9. Acceptance and limitations

`AC-PHY-001` is satisfied only when `TST-PHY-001` demonstrates all of the following from a clean synthetic seed:

- telemetry visibly advances, pauses, recovers, and reproduces from the same frozen fixture, state, and sequence without network or device input;
- selecting a component in the 3D view and structured alternative exposes the same status and measurements, including under keyboard-only and reduced-motion operation;
- the anomaly/forecast result reproduces exactly and displays its model card and synthetic/no-real-world-validity warning;
- all lifecycle stages and event provenance remain inspectable without rewriting history;
- every invalid, stale, expired, unauthorized, cross-tenant, out-of-range, mutated, or duplicate-conflicting command produces zero state transitions;
- a valid command produces one simulator transition, an idempotent replay returns the same receipt, and the hash-linked audit trail verifies; and
- UI, API examples, logs, and generated editions never describe the simulator as a connected physical asset.

This capability proves product interaction and guardrail behavior only. It does not establish industrial reliability, real-time determinism, functional safety, cybersecurity certification, model accuracy, asset fitness, maintenance efficacy, or authority to operate physical equipment.

CH-18 - Event Intelligence and Causal Impact Engine

Status: committed | Owners: knowledge-graph, ai-platform, product-engineering, security-engineering | Last reviewed: 2026-07-15

Event Intelligence and Causal Impact Engine

1. Purpose and H1 truth boundary

The Event Intelligence and Causal Impact Engine converts a reported change in reality into a reviewable, evidence-linked event interpretation, resolves its subjects against the authorized tenant graph, computes bounded direct and downstream consequences, and either creates a scenario branch or applies an exact reversible mutation to the synthetic H1 graph. It answers what changed, why, which relationships are affected, which risks or forecasts move, which consequences remain unknown, and what process-level remediation should be considered.

H1 is a deterministic product demonstration over the synthetic Aster and Beacon fixtures. It does not claim learned causal discovery, calibrated business prediction, autonomous organizational control, or connection to an HRIS, IAM provider, cloud control plane, industrial system, or other external system. Event application changes a shared tenant-scoped synthetic projection used by the entity catalog, bounded traversal, and simulation snapshot assembly. The Compose profile durably records the event

workflow and projection snapshot in PostgreSQL; isolated tests may use the same persistence contract with an in-memory adapter but do not claim restart durability. No path revokes a real permission, edits a real employee record, controls production, or creates a real hiring decision.

Factual workforce events may be represented when supplied and authorized, but the engine MUST NOT score an individual, infer attrition or productivity, recommend an employment decision, or execute an HR action. A departure example may recommend ownership review, access-review workflow, documentation continuity, or a separately governed staffing scenario; it may not decide whom to hire, promote, remove, or evaluate.

Requirement	Commitment
`REQ-EVT-001`	Convert untrusted natural-language reports into typed event interpretations with provenance, confidence, verification, evidence, entity candidates, and explicit unknowns.
`REQ-EVT-002`	Produce bounded, explainable direct, indirect, second-order, third-order, and unknown impacts without graph cycles or unbounded fan-out.
`REQ-EVT-003`	Route confirmed facts to a reviewable reality-update path and uncertain claims to isolated scenario branches without silently changing current truth.
`REQ-EVT-004`	Bind any synthetic graph update to tenant authority, exact payload and version, human review, idempotency, audit evidence, and rollback.
`REQ-EVT-005`	Provide an accessible event workspace with interpretation, resolution, causal graph, before/after state, timeline, branch, recommendation, and rollback views.

2. Canonical event envelope and taxonomy

An event envelope contains an immutable tenant-qualified event ID, schema version, type and taxonomy version, source class and source reference, reporting actor, creator, occurred-at interval, recorded-at timestamp, optional location, confidence band and numeric support score, verification state, related entity references, evidence references, attachment metadata, prior-event references, classification, retention policy, and trace context. Time is an interval when the report says `yesterday`, `recently`, or another imprecise phrase; the engine does not invent precision.

The core taxonomy is versioned and extensible:

- people: hire, promotion, transfer, departure, leave, role change, contractor addition/removal, reorganization, and manager change;
- project: start, delay, cancellation, requirement or milestone change, budget or priority change, and scope increase/reduction;
- technology: outage, migration, discovered vulnerability, deprecated dependency, launch, removal, architecture change, and repository archive;
- business: customer acquisition/loss/risk, contract, vendor, market, competitor, regulatory, and funding change;
- operations: equipment, supply, process, production, facility, and natural-hazard events; and
- external: economic, weather, regulation, competitor, and industry-security events.

Customer ontology packages may add namespaced event types but cannot weaken required fields, confidence routing, security controls, or mutation policy. One input can yield multiple interpretations with explicit ordering or simultaneity. A potential statement such as `we might lose our largest customer` is classified as a risk hypothesis, not a completed customer-loss fact.

3. Interpretation and entity resolution pipeline

The pipeline is: authenticated input acceptance, content quarantine and redaction, event extraction, schema validation, event classification, tenant-bounded entity resolution, impact expansion, deterministic policy checks, optional simulation, human review, exact mutation, projection and transactional-outbox commit, and audit. Each stage receives typed data and a least-authority capability envelope. Event text remains a quoted data field and is never concatenated into privileged instructions or tool definitions.

Entity resolution returns ranked candidates with entity ID, display-safe label, confidence, matching features, conflicting features, and whether confirmation is required. Aliases, abbreviations, historic names, reversible merges, and source identities may support a candidate. An unknown entity stays unresolved; it is not auto-created or merged. Ambiguous candidates block reality application until a reviewer selects a candidate or marks the reference unknown. Tenant boundaries are applied before candidate generation

so identifiers, counts, and rejected candidates cannot disclose another tenant.

Conflicting reports create a linked conflict set. Neither report overwrites the other. Source reliability, evidence strength, valid time, and reviewer decisions are recorded independently; confidence is not a substitute for proof. Corrections supersede the earlier interpretation while retaining history.

4. Bounded causal impact analysis

The engine starts from the resolved event subjects and applies a versioned library of typed cause-and-effect rules over an authorized snapshot. An impact contains its affected node or relationship, operation (`create`, `update`, `remove`, or `flag`), impact kind, before and proposed-after values, severity, confidence band, time horizon, causal depth, explanation, evidence, rule/model version, recommended process action, and uncertainty or missing-data note. Every impact evidence entry is an actual tenant-authorized `evidence_id` present in that event envelope; display clients resolve the identifier to its source kind, summary, and confidence rather than rendering an invented or dangling citation.

H1 separates four classes:

1. direct effects are definitional consequences of the event, such as a synthetic person state changing from active to departed;
2. relationship effects update or flag ownership, work, approval, dependency, or affected-by relationships;
3. derived risk and forecast effects are hypotheses with confidence decay and explicit assumptions; and
4. unknown effects are recorded when evidence, ontology coverage, or the traversal budget is insufficient.

Propagation is deterministic and capped at depth three, 100 impact nodes, 250 impact edges, 25 outgoing rules per source, and a fixed execution budget in H1. A visited key of event, rule, entity, property, and snapshot version prevents loops. Repeated effects are deduplicated by canonical payload; mutually contradictory effects become a conflict, not last-write-wins. Cycles in the company graph may be traversed once but never cause recursive expansion. When a cap is reached, the result is `partial` with a continuation/review reason and a visible unknown-impact record; it is never presented as complete.

The displayed causal graph is an explanation graph, not proof of real-world causality. Each edge distinguishes `observed`, `rule-derived`, `simulated`, or `unknown`; confidence decays across derived hops and cannot exceed its weakest evidence or policy input. Recommendations are bounded workflow proposals such as assign an interim service owner, review access, validate a runbook, contact an affected customer, or run a schedule scenario.

5. Confidence, verification, and truth routing

The confidence bands are `confirmed`, `likely`, `possible`, `speculative`, and `rejected`. A numeric score aids ranking but never independently authorizes a write.

Band	Default behavior
Confirmed	May enter reality-update review only with required evidence, resolved entities, authorized reviewer, exact snapshot version, and all deterministic checks passing.
Likely	Creates or updates a scenario branch; human verification can later produce a distinct confirmed interpretation.
Possible	Scenario only, with assumptions and missing evidence made prominent.
Speculative	Saved only as an isolated hypothesis when policy permits; no live graph or prediction baseline changes.
Rejected	Retained as restricted audit/conflict evidence and excluded from current truth and simulations unless explicitly re-reviewed.

Events involving employment, legal, financial, identity, production, security control, destructive operations, or external writes never receive autonomous execution authority. H1 synthetic reality application requires a human review decision and exact preview; production designs may add dual control based on risk classification but cannot weaken `REQ-ACT-004`.

6. Review, application, idempotency, and rollback

Interpretation creates a draft with an ETag, graph snapshot version and state hash, canonical payload hash, expiry, impact summary, and unresolved blockers. Review records `accept`, `reject`, or `request_changes`, the authenticated reviewer, selected resolution candidates, rationale, and reviewed hash. Review refreshes the authorized shared projection snapshot; its version and canonical state hash are included in the reviewed payload. Applied, rolled-back, and rejected events are terminal immutable history: review returns a conflict and a correction must create a superseding event rather than overwrite application, receipt, branch, timeline, or audit evidence. Requesting approval freezes the accepted payload, proposed mutations, event version, graph snapshot version, and graph state hash. Reality application requires `If-Match`, an `Idempotency-Key`, execution-time authorization, unexpired approval, identical payload hash, and a compare-and-swap match on both graph version and state hash. A graph change after approval makes the approval stale even when the event version has not changed. Scenario application retains the approved immutable base version and hash without modifying current truth.

For the H1 event slice, one PostgreSQL transaction records the updated event, executed approval, action receipt, before/after application snapshot, branch, timeline entry, idempotency record, hash-linked audit evidence, current event-projection snapshot, and CloudEvents-compatible outbox envelope. Before writing, it serializes the tenant audit tail, reserves the `(tenant, operation, Idempotency-Key)` row in `edt.idempotency`, row-locks the authoritative event and projection records, and compares the exact event version/status/ETag plus projection version/state hash. These database predicates—not a replica-local map—decide the winner; an identical retry returns the original persisted receipt and a different request under the same key fails. The shared projection performs the exact version-and-hash transition; a failed database commit restores its pre-application snapshot and exposes no successful receipt. Reality receipts contain only impacts explicitly marked `live_mutation_eligible` whose operation is in the authoritative allowlist (`set_state`, `modify_relationship`, `remove_relationship`, or `append_outage`). Risk, prediction, workflow, annotation, synthetic relationship-creation, and recommendation proposals remain non-authoritative scenario inputs. An empty reality mutation set or any unsupported operation fails closed; the projection neither advances its version nor stores the rejected operation as a fact. The returned receipt identifies the authorizing approval, event versions, graph versions, before/after state hashes, exact mutations, audit record, and allocated outbox position. The outbox envelope receives the same transaction-assigned position. Its `dataschema` is the `$id` of the source-controlled `event-intelligence-event-changed.v1.schema.json` contract, and `schema_hash` is the SHA-256 of that schema's canonical JSON value rather than a digest of its URI. All results remain labeled `synthetic: true` and `external_write: false`.

At API startup the Compose profile hydrates tenant events, interpretations, approvals, receipts, applications, branches, timeline entries, every audit record, the idempotency ledger, and latest projection snapshot from PostgreSQL before serving the event workflow. Payload tenant assertions are checked again after RLS reads. Audit hydration validates record shape and content hash, discovers missing predecessors, multiple children of one predecessor, unreachable records, and tenant-sequence gaps, and exposes counts plus bounded record/hash references through the audit endpoint. It does not discard corruption by selecting one path and declaring the chain valid. An empty chain is a valid bootstrap only when the tenant has no persisted event workflow; an existing event with no complete audit evidence is unhealthy. Invalid, gapped, forked, empty-for-existing-state, or event-incomplete audit evidence sets `chain_valid: false` and blocks new audited mutations until repair. Historical graph bindings are never invented during hydration: a legacy event without them is forced back to review, while an approval, receipt, application, branch, timeline entry, or idempotency record with missing, inconsistent, or cross-tenant bindings is excluded from executable state. The persisted idempotency record therefore returns the original receipt after a restart; it does not execute the mutation again. Reusing an idempotency key with different input fails. A stale graph version or hash, missing or dangling evidence, unresolved identity, changed payload, expired review, invalid cycle, fan-out limit, unauthorized field, unsupported projection operation, audit-health failure, or cross-tenant reference has zero committed effects.

The receipt retains before and after state hashes, event and graph versions, mutation list, projection status, outbox position, event/receipt IDs, and audit reference; the persisted application record retains the corresponding snapshots needed for replay and compensation. Rollback is a compensating event, not deletion of history. It uses a new monotonic graph version and is allowed only when both the current synthetic graph version and state hash equal the recorded post-application version and hash; returning to the same content at a later graph version does not make an older compensation safe. Otherwise a new conflict review is required. Rollback is itself authorized, exact, version-and-hash checked, idempotent, atomically persisted with audit and outbox evidence, and restart-safe.

7. Timeline, replay, and alternate futures

The event timeline orders valid time separately from recorded/system time and shows state before the event, the reported occurrence, immediate effects, later derived effects, corrections, rollback, and branch points. Every applied or rolled-back entry carries its receipt ID, before/after graph versions, and before/after state hashes so the persisted application can be replayed and audited after process restart. History replay reads an authorized persisted snapshot and immutable event evidence; it never rewrites the current graph. Out-of-order events trigger recomputation against their valid-time location and create a reviewable delta to later derived impacts.

Scenario branches reference an immutable base graph version and state hash, event interpretation, assumptions, rule/model versions, and branch status. Users can compare baseline, confirmed history, and alternate future without copying claims into live truth. Simultaneous events use a deterministic ordering key and a batch-level conflict pass. Branches expose prediction deltas only when an approved simulation model supports the affected outcome; otherwise the engine reports that no validated forecast is available.

8. Agents and authority

The bounded capability profiles are event extraction, entity resolution, impact analysis, causal explanation, simulation, risk analysis, verification, and graph mutation. They exchange schema-valid envelopes and retain structured rationales, evidence references, rule/model versions, and action traces rather than private chain of thought. No handoff widens tenant, data, tool, budget, or mutation authority. The graph-mutation profile is deterministic middleware: it cannot reinterpret text or invent mutations.

Model failure, unavailable approved fallback, schema-invalid output, prompt injection, insufficient evidence, or budget exhaustion produces abstention or human review. AI-derived impact suggestions remain proposals until deterministic validators accept their types, targets, ontology constraints, provenance, and authorization.

9. Security, privacy, consistency, and scale

`CTRL-EVT-001` requires authentication, server-derived tenant context, field and relationship authorization, input-size and attachment limits, content/instruction separation, malware and secret handling, sensitive-field classification/redaction, evidence ACL intersection, rate limits, bounded expansion, exact-payload review, audit, and rollback. Event creation never grants permission and an event claiming that a user is an administrator cannot change policy. Attachments are immutable scanned objects; H1 accepts metadata only.

The OpenAPI `event.read`, `event.create`, `event.review`, `event.approve`, `event.apply`, and `event.rollback` names are product scopes at the gateway boundary. H1 maps reads to an authorized `evidence.read.*` capability, creation/review/application to `scenario.create` plus creator binding, approval to the distinct `action.approve.operations` or `action.approve.security` capability, and rollback to either authorized approval capability. The platform operator and limited synthetic actor remain explicitly denied. This mapping is policy configuration, not an authority expansion performed by an agent or event payload.

An event cannot delete critical ownership silently. Removal rules create an explicit gap such as `NEEDS_OWNER` and route it to review. Required ontology constraints run against the complete proposed mutation set. A proposed relationship mutation is eligible only when its exact before relationship exists in the approved snapshot; absent facts become explicit unknowns or recommendations rather than fabricated edges. Events that would affect millions of nodes are compiled into a durable, chunked, resumable impact plan with sampled preview and aggregate counts in H2; H1 rejects any proposal above its fixed cap. Projection outages leave PostgreSQL authoritative and report stale projections. Events arriving faster than processing are durably queued, deduplicated, backpressured, and ordered per tenant/aggregate; H1 exposes the semantics without claiming production throughput.

Confidential event text is minimized in logs, notifications, traces, model context, and audit. Evidence permissions constrain the event, impacts, explanations, counts, timeline, and exports. Cross-company events may reference an external public entity but cannot read or mutate another customer tenant. Dates before the tenant/company existence, future dates represented as completed facts, invalid intervals, and impossible state transitions are blocked or routed to scenario review.

10. H1 interface and acceptance

The Event Intelligence workspace provides event entry, extracted interpretations, confidence and verification controls, candidate resolution, selectable causal graph plus structured impact list, before/after comparison, risks and prediction deltas, recommendations, scenario/reality routing, timeline, branch comparison, review/application state, receipts, and rollback. Color is never the only confidence or severity signal; all graph information has a keyboard-operable list/table equivalent.

The REST surface under `/v1/event-intelligence` defines taxonomy, interpretation, paginated event listing and detail, review, approval request and decision, application, rollback, audit, replay, timeline, branch listing, and branch comparison resources. Tenant context is server-derived and pagination cursors are bound to the active authorization fingerprint. Mutating routes require idempotency; state transitions require ETags; all responses are private and non-cacheable. Approval resources expose their reviewed event version plus bound graph version and hash; receipts and timeline entries distinguish event versions from graph versions and expose approval, receipt, and outbox linkage required for durable replay.

`AC-EVT-001` and `TST-EVT-001` require deterministic coverage of multi-event extraction, vague time, uncertainty, entity ambiguity, resolvable impact evidence, bounded cycle-safe propagation, conflict and injection handling, payload/version/hash rejection, tenant isolation, scenario routing, shared entity/traversal/simulation projection visibility, atomic receipt/audit/outbox persistence, restart hydration, one-time apply, replay, rollback, and audit-chain verification. The oracle confirms no real external change or individual employment recommendation.

Part III - Architecture Decision Records

ADR-001 - Documentation as a Versioned Product

Status: accepted | Owners: enterprise-architecture

ADR-001: Documentation as a Versioned Product

Context

One monolithic narrative cannot safely govern contracts, catalogs, diagrams, decisions, reviews, and multi-year evolution.

Decision

Maintain normative Markdown chapters, YAML catalogs, machine-readable contracts, diagram sources, ADRs, and review evidence under one semantic version. Generate consolidated reader editions from these sources. Stable identifiers and automated traceability are release requirements.

Consequences

Authors must change the smallest authoritative artifact and regenerate editions. Generated output is never edited directly. Conflicts follow `decision-precedence.md`. Version 1.0.0 freezes H1/H2 decisions; later changes require change classification and an ADR where behavior changes.

ADR-002 - Engineering Delivery Reference Workload

Status: accepted | Owners: product, enterprise-architecture

ADR-002: Engineering Delivery Reference Workload

Context

An enterprise-wide ontology is too broad to demonstrate or validate without a concrete outcome.

Decision

H1 models two deterministic synthetic software organizations using read-only GitHub metadata and Jira project data. The primary question is why a release may miss its target. The user compares a launch-delay scenario and may dual-authorize one remediation issue in an allowlisted Jira sandbox.

Consequences

H1 proves permission-preserving ingestion, graph reasoning, reproducible simulation, governed action, and audit. It does not prove predictive validity on real organizations. Cross-industry scope is delivered through ontology packages and later horizons.

ADR-003 - Modular Monorepo with Four Workloads

Status: accepted | Owners: platform-architecture

ADR-003: Modular Monorepo with Four Workloads

Context

Premature microservices multiply deployment, security, consistency, and debugging costs before ownership or scale boundaries are known.

Decision

Use one repository with four OCI workloads: React/Next.js web, NestJS/Fastify API, TypeScript connector workers, and Python AI/simulation workers. Modules own schemas and communicate through published interfaces; no module writes another module's tables.

Consequences

Deployment remains simple while isolation and scaling boundaries are visible. Extract a service only after an accepted ADR demonstrates independent ownership, scaling, security, availability, or release needs. A service mesh is rejected until service count and zero-trust networking needs justify its control plane.

ADR-004 - Language and Interface Ownership

Status: accepted | Owners: developer-platform

ADR-004: Language and Interface Ownership

Decision

TypeScript owns web, public API, connectors, and shared browser-facing types. Python owns model integration, extraction, graph algorithms, simulation, and AI evaluations. SQL owns database invariants and migrations. REST/OpenAPI is the H1 public command interface, SSE reports progress, and signed webhooks receive source events. AsyncAPI and JSON Schema govern events.

GraphQL is a read-only H2 candidate. gRPC is reserved for extracted polyglot services. Go, Rust, WebAssembly, Java, C#, Kotlin, and Swift require the introduction triggers in `catalogs/technologies.yaml`; C++ is rejected for the core.

Consequences

Polyglot cost is capped at two application languages in H1. Contract generation and compatibility tests prevent type drift. No language is adopted merely because it was listed as an option.

ADR-005 - Temporal Workflows and Transactional Outbox

Status: accepted | Owners: workflow-platform

ADR-005: Temporal Workflows and Transactional Outbox

Decision

Temporal owns durable synchronization, model runs, approvals, external actions, compensation, and long-running simulations. Transactional changes append an outbox record in the same PostgreSQL transaction. A relay publishes CloudEvents-compatible events and records delivery checkpoints.

Workflow code MUST be deterministic; external I/O occurs in idempotent activities. Retries are error-classified. Cancellation, timeouts, heartbeats, backpressure, dead-letter review, and replay behavior are specified per workflow.

Kafka is introduced only if outbox throughput, retention, replay consumers, or data-plane decoupling miss a committed objective. NATS is rejected unless a distinct edge command requirement appears.

ADR-006 - Pooled Shared Multitenancy

Status: accepted | Owners: security-architecture, data-platform

ADR-006: Pooled Shared Multitenancy

Decision

Standard SaaS uses shared stateless compute and pooled regional data services. Tenant context is derived from authenticated membership. Every row, identifier, object, event, edge, vector, cache key, trace, and audit envelope is tenant-qualified. PostgreSQL RLS and tenant-qualified constraints are the hard relational fence; other stores use independent namespaces and adversarial isolation tests.

Per-tenant envelope keys protect sensitive data and credentials. Cross-tenant retrieval, resolution, memory, analytics, and training are prohibited by default. Break-glass support is time-bound, dual-authorized, purpose-bound, and audited.

Consequences

This is logical rather than physical isolation. Dedicated data planes are an H4 profile. Neo4j pooled isolation is a named residual risk and cannot rely on application-supplied tenant filters alone.

ADR-007 - Layered Authorization and Source ACL Propagation

Status: accepted | Owners: authorization

ADR-007: Layered Authorization and Source ACL Propagation

Decision

OIDC or SAML establishes identity; SCIM manages H2 lifecycle. A relationship-based policy service evaluates resource and action authorization, while PostgreSQL RLS enforces tenant isolation. Source ACLs attach to evidence and constrain every derived claim, edge, embedding, summary, cache, count, notification, export, and agent context.

Visibility is monotonic: derivation cannot broaden access. A relationship is visible only when its policy, endpoints, and qualifying evidence are visible. Revocation invalidates caches and queues projection rebuilds. High-risk reads fail closed when policy or source ACL state is unavailable.

ADR-008 - PostgreSQL Authoritative Claim and Evidence Store

Status: accepted | Owners: data-platform

ADR-008: PostgreSQL Authoritative Claim and Evidence Store

Decision

PostgreSQL is authoritative for tenants, identities, source accounts, observations, claims, evidence, aliases, resolution decisions, ontology versions, workflow metadata, scenarios, approvals, action receipts, and audit indexes. Claims use valid-time and system-time intervals and carry confidence, classification, source revision, evidence, and ACL.

Raw payload bytes and large artifacts live in content-addressed S3-compatible object storage. Ordinary aggregates keep current state plus immutable version history; full event sourcing is limited to observations, claims, audit, and the outbox.

Consequences

Recovery begins with PostgreSQL and object storage. Derived graph, search, vector, and analytics stores never become an untracked source of truth.

ADR-009 - Neo4j as a Rebuildable Graph Projection

Status: accepted | Owners: knowledge-graph

ADR-009: Neo4j as a Rebuildable Graph Projection

Decision

Neo4j provides labeled-property traversal and visualization over accepted claims. Projection checkpoints bind an ontology version and authoritative outbox position. All queries use allowlisted bounded templates that inject server-derived tenant context; arbitrary Cypher is prohibited.

Projection nodes and edges preserve canonical identifiers, tenant, valid time, confidence, classification, and evidence references. ACL, correction, deletion, ontology, or merge changes can rebuild affected tenant projections from PostgreSQL.

Consequences

Graph availability may degrade without blocking authoritative writes. Pooled isolation remains a residual risk; tests and query gateways are mandatory. H3 may move tenants to isolated graph databases or regional cells after benchmark and licensing review.

ADR-010 - Minimal Specialized Data Stores

Status: accepted | Owners: search-platform

ADR-010: Minimal Specialized Data Stores

Decision

H1 uses PostgreSQL full-text search and pgvector with tenant-scoped indexes. Valkey provides non-authoritative cache, rate limiting, and ephemeral coordination. OpenSearch is introduced when corpus size, hybrid relevance, faceting, or latency objectives fail. ClickHouse is introduced when analytical retention or concurrency harms PostgreSQL. S3/Parquet/Iceberg is a later cold-history and backtesting plane.

Qdrant requires a vector benchmark; Milvus and Elastic are rejected by default to avoid duplicate responsibilities. Time-series measurements use PostgreSQL initially and ClickHouse after the same evidence gate.

ADR-011 - Hostile-Input Connector and Synchronization Framework

Status: accepted | Owners: integrations

ADR-011: Hostile-Input Connector and Synchronization Framework

Decision

Each installation uses tenant-scoped credentials, least scopes, an egress allowlist, resource quotas, signed webhook verification, durable cursor checkpoints, periodic reconciliation, normalized immutable observations, tombstones, and error-classified retries. Parsers treat every byte as hostile and isolate expensive or risky formats.

GitHub H1 is read-only metadata for allowlisted sandbox repositories. Jira H1 reads allowlisted sandbox projects and exposes one exact dual-approved remediation mutation. Source revisions win within their object lineage; canonical claims preserve conflicting evidence rather than silently overwrite it.

ADR-012 - OpenAI-First Capability Gateway

Status: accepted | Owners: ai-platform

ADR-012: OpenAI-First Capability Gateway

Decision

Use the OpenAI Responses API and Python Agents SDK behind an internal gateway that owns capability routing, tenant policy, budgets, redaction, model and prompt versions, retention configuration, and evaluation status. Use the GPT-5.6 family by evaluated workload: sol for highest-complexity orchestration and grading, terra for balanced grounded synthesis, and luna for high-volume extraction.

Production pins evaluated snapshots. A fallback is eligible only after the same use-case safety and quality gates; otherwise the run fails closed or queues for review. OpenAI-native structured outputs and tools remain available rather than being flattened behind a lowest-common-denominator interface.

Consequences

OpenAI is an explicit H1 external dependency. Air-gapped operation requires a separately evaluated local-model adapter in H4.

ADR-013 - Bounded Agent Authority and Exact-Payload Approval

Status: accepted | Owners: ai-security, workflow-security

ADR-013: Bounded Agent Authority and Exact-Payload Approval

Decision

Agents are capability profiles inside a deterministic sequence: authorize, retrieve, plan, schema-validate, execute reads, verify evidence, cite, request approval, execute, and audit. Delegation cannot expand authority. Tool middleware enforces tenant, identity, allowlist, egress, time, spend, token, recursion, concurrency, and cancellation limits outside the model runtime.

The H1 Jira mutation requires two distinct authenticated approvers. Approval binds the canonical payload hash, tenant, requester, approvers, credential, project, policy version, idempotency key, and a 15-minute expiry. Any change invalidates approval. Execution records before and after state and a compensation workflow.

Employment, legal, financial, production, identity, destructive, and security-control decisions remain non-executable through H3.

ADR-014 - Seeded PERT Monte Carlo Launch Simulation

Status: accepted | Owners: simulation-science

ADR-014: Seeded PERT Monte Carlo Launch Simulation

Decision

H1 compiles Jira work links and GitHub delivery relations into a validated dependency DAG. Each unfinished work item uses user-confirmed optimistic, most-likely, and pessimistic duration inputs. A seeded Monte Carlo scheduler samples PERT distributions under team-level capacity and scenario interventions.

Outputs include p50, p80, and p95 launch dates, miss probability, critical path, blockers, sensitivity drivers, assumptions, and missing-data warnings. The LLM may compile a typed scenario and explain results but never performs the authoritative mathematics.

Cycles, impossible capacity, missing estimates, and contradictory changes produce explicit validation outcomes. Synthetic results are demonstrative, not causal or production-calibrated predictions.

ADR-015 - Infrastructure-Portability Cloud Neutrality

Status: accepted | Owners: platform-engineering

ADR-015: Infrastructure-Portability Cloud Neutrality

Decision

Cloud neutrality means portable contracts and migration paths, not active-active multi-cloud. H1 runs through Docker Compose. H2 promotes OCI workloads to a conformant Kubernetes distribution with Helm and OpenTofu modules. Domain code uses PostgreSQL, S3-compatible object, OIDC/SAML/SCIM, Temporal, and OpenTelemetry interfaces. Provider-specific code is confined to infrastructure adapters.

Managed implementations are permitted when export, restore, identity, encryption, observability, and failure contracts remain testable. Dedicated, on-premises, air-gapped, edge, and multi-region profiles are H4 gates. Air-gapped mode excludes the OpenAI path until a local adapter passes evaluations.

ADR-016 - Evaluation-Driven Verification and Finite Release Gates

Status: accepted | Owners: quality-engineering, architecture-review-board

ADR-016: Evaluation-Driven Verification and Finite Release Gates

Decision

Every feature includes unit, contract, integration, tenant-isolation, security, privacy, failure, edge, performance, regression, and appropriate load, chaos, and AI evaluation evidence. The synthetic organization provides a deterministic graph, permission, simulation, and action oracle.

Specification 1.0.0 requires 100 percent requirement traceability, zero unresolved H1/H2 major decisions, zero open Critical or High risk, owned Medium residuals, successful contract and document validation, two consecutive cross-domain reviews with no new blocker, and independent H1 build-readiness confirmation.

Consequences

The system uses a measurable convergence criterion rather than claiming no possible future improvement. Any failed gate blocks release and produces a tracked remediation item.

ADR-017 - Build the Differentiation and Buy Commodity Controls

Status: accepted | Owners: product-architecture

ADR-017: Build the Differentiation and Buy Commodity Controls

Decision

Build the claim/evidence model, ontology governance, reversible entity resolution, permission-aware projections, organizational reasoning, agent policy gateway, simulation registry, and graph/scenario UX. Use standards-based or managed implementations for identity federation, transactional and graph databases, object storage, workflow runtime, observability transport, email, malware scanning, billing, and feature flags.

Commercial dependencies MUST support tenant isolation, export, deletion, audit, regionality, incident evidence, contractual security, and a tested exit path. A managed service may not become an undocumented source of truth.

Part IV - Normative Catalogs

Source Ledger

Source: `docs/enterprise-digital-twin/source-ledger.yaml`

Sources

Id	Title	Type	Effective Date	Authority	Url	Accessed
SRC-001	Approved Enterprise Digital Twin implementation plan	user-approved-plan	2026-07-13	normative		
SRC-002	OpenAI Responses API migration guidance	primary-technical-source		informative-current-product-fact	https://developers.openai.com/api/docs/guides/migrate-to-responses	2026-07-13
SRC-003	OpenAI model guidance	primary-technical-source		informative-current-product-fact	https://developers.openai.com/api/docs/guides/latest-model	2026-07-13
SRC-004	OpenAI agent safety guidance	primary-technical-source		informative-current-product-fact	https://developers.openai.com/api/docs/guides/agent-builder-safety	2026-07-13
SRC-005	OpenAI evaluation best practices	primary-technical-source		informative-current-product-fact	https://developers.openai.com/api/docs/guides/evaluation-best-practices	2026-07-13
SRC-006	OpenAI Agents SDK guidance	primary-technical-source		informative-current-product-fact	https://developers.openai.com/api/docs/guides/agents	2026-07-13
SRC-007	OpenAI Structured Outputs guidance	primary-technical-source		informative-current-product-fact	https://developers.openai.com/api/docs/guides/structured-outputs	2026-07-13
SRC-008	OpenAI MCP and connectors guidance	primary-technical-source		informative-current-product-fact	https://developers.openai.com/api/docs/guides/tools-connectors-mcp	2026-07-13
SRC-009	GitHub App permission selection	primary-technical-source		informative-provider-contract	https://docs.github.com/en/apps/creating-github-apps/registering-a-github-app/choosing-permissions-for-a-github-app	2026-07-13
SRC-010	GitHub webhook troubleshooting and delivery behavior	primary-technical-source		informative-provider-contract	https://docs.github.com/en/webhooks/testing-and-troubleshooting-webhooks/troubleshooting-webhooks	2026-07-13
SRC-011	Jira OAuth scopes	primary-technical-source		informative-provider-contract	https://developer.atlassian.com/cloud/jira/platform/scopes-for-oauth-2-3LO-and-forge-apps/	2026-07-13
SRC-012	Jira Cloud webhook REST API	primary-technical-source		informative-provider-contract	https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-webhooks/	2026-07-13
SRC-013	Jira Software webhook guidance	primary-technical-source		informative-provider-contract	https://developer.atlassian.com/cloud/jira/software/webhooks/	2026-07-13
SRC-014	Atlassian OAuth 2.0 authorization code flow	primary-technical-source		informative-provider-contract	https://developer.atlassian.com/cloud/oauth/getting-started/implementing-oauth-3lo/	2026-07-13
SRC-015	RFC 9457 Problem Details for HTTP APIs	primary-standard		normative-external-standard	https://www.rfc-editor.org/rfc/rfc9457	2026-07-13
SRC-016	Jira Cloud REST API edit issue operation	primary-technical-source		informative-provider-contract	https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-issues/#api-rest-api-3-issue-issueidorkey-put	2026-07-13

Requirements

Source: `docs/enterprise-digital-twin/catalogs/requirements.yaml`

Id	Title	Statement	Horizon	Status
REQ-GOV-001	Versioned documentation product	The blueprint MUST be source controlled and release consolidated Markdown HTML and PDF editions.	H1	committed
REQ-GOV-002	Stable traceability	Every normative requirement MUST map to decisions controls acceptance evidence and a delivery horizon.	H1	committed
REQ-GOV-003	Finite review gate	Release MUST use measurable risk and independent review exit criteria rather than an unbounded perfection claim.	H1	committed
REQ-GOV-004	Explicit decision status	Capabilities MUST be marked Committed Provisional Research or Rejected.	H1	committed
REQ-PROD-001	Living organizational model	The product MUST maintain a continuously synchronized evidence-backed representation of organizational entities activity and dependencies.	H2	committed
REQ-PROD-002	Grounded organizational answers	Answers MUST cite accessible evidence and abstain when evidence is insufficient.	H1	committed
REQ-PROD-003	Graph simulation wedge	H1 MUST explain launch risk and compare a user-confirmed staffing scope or dependency scenario.	H1	committed
REQ-PROD-004	Governed action wedge	H1 MUST preview dual-authorize execute audit and compensate one allowlisted Jira remediation mutation.	H1	committed
REQ-PROD-005	Cross-industry extensibility	The core metamodel MUST support namespaced domain and customer ontology packages without changing core code.	H2	committed
REQ-PHY-001	Synthetic asset telemetry	H1 MUST produce deterministic timestamped synthetic physical-asset telemetry with typed units sequence quality freshness and reproducible replay metadata.	H1	committed
REQ-PHY-002	Accessible physical 3D-style view	H1 MUST provide an interactive component-linked procedural perspective asset view with keyboard reduced-motion and complete structured alternatives.	H1	committed

Id	Title	Statement	Horizon	Status
REQ-PHY-003	Honest deterministic asset analytics	H1 anomaly and forecast output MUST be deterministic versioned explainable and labeled synthetic without validated machine-learning or real failure-prediction claims.	H1	committed
REQ-PHY-004	Synthetic asset lifecycle	H1 MUST expose versioned synthetic design manufacture commissioning service maintenance and decommissioning history without claiming source-system synchronization.	H1	committed
REQ-PHY-005	Guarded simulator control	H1 MUST restrict asset commands to tenant-authorized allowlisted simulator-only previews with expected-version range transition idempotency and audit checks and no device egress.	H1	committed
REQ-EVT-001	Structured event interpretation	H1 MUST convert untrusted natural-language reports into typed evidence-linked event interpretations with explicit time confidence verification entity candidates and unknowns.	H1	committed
REQ-EVT-002	Bounded causal impact	H1 MUST derive explainable direct relationship second-order third-order and unknown impacts with confidence decay cycle prevention fan-out limits and honest partial results.	H1	committed
REQ-EVT-003	Reality and scenario separation	Confirmed authorized facts MAY enter exact reality-update review while likely possible and speculative reports MUST remain isolated scenario branches until separately verified.	H1	committed
REQ-EVT-004	Reversible event mutation	Every H1 synthetic graph update MUST bind tenant authority human review exact payload evidence snapshot version and hash compare-and-swap idempotency atomic receipt audit and outbox persistence restart hydration and compensating rollback with zero external write.	H1	committed
REQ-EVT-005	Explainable event workspace	H1 MUST expose interpretation resolution causal graph structured impacts before-after comparison confidence recommendations timeline branches review receipts and rollback accessibly.	H1	committed

Id	Title	Statement	Horizon	Status
REQ-ARCH-001	Modular deployables	H1 MUST use bounded web API connector-worker and AI-simulation workloads in a modular monorepo.	H1	committed
REQ-ARCH-002	Durable workflows	Long-running sync agent approval and simulation operations MUST use durable workflow semantics.	H1	committed
REQ-ARCH-003	Transactional events	Domain events MUST be emitted through a transactional outbox before any external event backbone is introduced.	H1	committed
REQ-ARCH-004	Cloud-neutral contracts	Domain code MUST depend on OCI PostgreSQL S3 OIDC OpenTelemetry and workflow contracts rather than cloud-specific SDKs.	H2	committed
REQ-ARCH-005	Measured service extraction	Microservices service mesh Kafka NATS and specialized stores MUST require an accepted benchmark or isolation ADR.	H3	provisional
REQ-ARCH-006	Offline deployment profiles	On-premises air-gapped edge and offline modes MUST remain gated profiles with explicit capability differences.	H4	provisional
REQ-ARCH-007	Public interface portfolio	The platform MUST define REST SSE webhooks events GraphQL gRPC MCP plugin SDK and CLI contracts with clear horizon ownership.	H2	committed
REQ-TEN-001	Server-derived tenant context	Clients MUST NOT select authoritative tenant context and every request MUST bind it from authenticated membership.	H1	committed
REQ-TEN-002	Tenant-scoped storage	Every row object event edge vector cache key trace and audit record MUST carry an immutable tenant identifier.	H1	committed
REQ-TEN-003	Database tenant fence	PostgreSQL RLS and tenant-qualified constraints MUST enforce relational tenant isolation independent of application filters.	H1	committed
REQ-TEN-004	Derived-store isolation	Graph search vector object cache and queue projections MUST use independently enforceable tenant namespaces and negative isolation tests.	H1	committed
REQ-TEN-005	No implicit cross-tenant learning	Retrieval resolution memory analytics and model training MUST NOT combine tenants without a separately recorded opt-in design.	H1	committed

Id	Title	Statement	Horizon	Status
REQ-DATA-001	Authoritative claims	PostgreSQL MUST be authoritative for observations claims evidence identity resolution approvals scenarios and audit indexes.	H1	committed
REQ-DATA-002	Bitemporal provenance	Every claim MUST include valid time system time confidence evidence classification source revision and source ACL.	H1	committed
REQ-DATA-003	Rebuildable projections	Graph search and vector stores MUST be rebuildable after ACL correction deletion or ontology changes.	H1	committed
REQ-DATA-004	Reversible resolution	Entity merges MUST preserve source identities evidence and a reversible decision history.	H1	committed
REQ-DATA-005	Versioned ontology	Entity relationship property constraint and migration definitions MUST be versioned and namespaced.	H1	committed
REQ-DATA-006	Core domain coverage	The ontology MUST cover organization work engineering customers finance infrastructure governance knowledge and physical-asset domains.	H2	committed
REQ-DATA-007	Lifecycle completeness	Every entity and relationship type MUST define ownership permissions lifecycle history search retention deletion and archive behavior.	H2	committed
REQ-DATA-008	Object integrity	Raw source objects MUST be content-addressed encrypted classified retention-tagged and immutable except for cryptographic erasure.	H2	committed
REQ-DATA-009	Deletion propagation	Tenant and data-subject deletion MUST cover authoritative and all derived stores while preserving minimum lawful audit evidence.	H2	committed
REQ-CON-001	Least-privilege GitHub	H1 GitHub access MUST be read-only metadata for allowlisted sandbox repositories and exclude source bodies secrets logs and private messages.	H1	committed
REQ-CON-002	Bounded Jira mutation	H1 Jira access MUST be restricted to allowlisted projects and the exact approved remediation issue mutation.	H1	committed
REQ-CON-003	Verifiable synchronization	Connectors MUST verify webhooks reconcile periodically checkpoint cursors deduplicate events and preserve tombstones.	H1	committed

Id	Title	Statement	Horizon	Status
REQ-CON-004	Compromised connector containment	Connector execution MUST isolate credentials egress parsing quotas and tenant effects and support immediate revocation.	H1	committed
REQ-CON-005	Connector SDK	H2 MUST define manifests authentication discovery backfill incremental sync writes errors observability testing and certification contracts.	H2	committed
REQ-AI-001	OpenAI-first gateway	AI workloads MUST use the Responses API and Agents SDK behind capability policy budget and evaluation interfaces.	H1	committed
REQ-AI-002	Bounded capability agents	Agents MUST be versioned capability profiles with explicit tools memory context limits handoff rules retries evaluation and termination.	H1	committed
REQ-AI-003	Deterministic policy envelope	Authorization retrieval validation execution verification citation approval and audit MUST surround all model behavior.	H1	committed
REQ-AI-004	Untrusted content separation	Connector and user content MUST NOT become privileged instructions or grant tools and all model outputs MUST be schema validated.	H1	committed
REQ-AI-005	Scoped memory	Agent memory MUST be explicit tenant-scoped permission-trimmed versioned and deletable with no invisible self-modification.	H1	committed
REQ-AI-006	No raw chain of thought	The platform MUST retain evidence structured rationale action traces and model versions rather than expose or store private chain of thought.	H1	committed
REQ-AI-007	Evaluated model routing	Model snapshots prompts tools and fallbacks MUST pass use-case safety quality latency and cost evaluations before promotion.	H1	committed
REQ-AI-008	Delegation cannot expand power	A child agent MUST inherit the intersection of user tenant caller workflow and tool policy authority.	H1	committed
REQ-ACT-001	Exact-payload approval	Approval MUST bind tenant actor credential target canonical arguments expiry policy version and idempotency key.	H1	committed
REQ-ACT-002	Dual control	H1 external mutation MUST require two distinct authenticated approvers and the requester MUST NOT satisfy the second approval.	H1	committed

Id	Title	Statement	Horizon	Status
REQ-ACT-003	Compensating rollback	The Jira action MUST record before and after state and provide an authorized idempotent compensation workflow.	H1	committed
REQ-ACT-004	Prohibited high-impact autonomy	Employment legal financial production identity destructive and security-control decisions MUST remain non-executable through H3.	H3	committed
REQ-SIM-001	Reproducible scheduler	H1 MUST use a seeded PERT Monte Carlo dependency-DAG scheduler rather than LLM-authored mathematics.	H1	committed
REQ-SIM-002	Typed scenario	Scenario inputs assumptions interventions snapshot model version and random seed MUST be typed confirmed and persisted.	H1	committed
REQ-SIM-003	Uncertainty output	Simulation MUST return p50 p80 p95 critical path blockers sensitivity uncertainty assumptions and missing-data warnings.	H1	committed
REQ-SIM-004	Honest claims	Synthetic scenarios MUST be labeled demonstrative and MUST NOT claim causal or production predictive validity.	H1	committed
REQ-SIM-005	Prediction governance	Forecasts MUST define outcome window data lineage calibration backtesting drift abstention fairness explanation and prohibited uses.	H3	provisional
REQ-UX-001	Complete H1 surfaces	H1 MUST include cockpit search graph evidence timeline scenario comparison agent run approval connector health and audit surfaces.	H1	committed
REQ-UX-002	State completeness	Every screen MUST define loading empty error denied stale partial offline destructive and recovery states.	H1	committed
REQ-UX-003	Accessible operation	H1 MUST meet WCAG 2.2 AA interaction contrast keyboard focus reduced-motion and assistive-technology requirements.	H1	committed
REQ-UX-004	Evidence-first visualizations	Graph and simulation visualizations MUST expose source evidence time confidence permissions uncertainty and accessible tabular alternatives.	H1	committed
REQ-SEC-001	Zero-trust authorization	Every privileged operation MUST authenticate authorize tenant-bind and audit at execution time.	H1	committed

Id	Title	Statement	Horizon	Status
REQ-SEC-002	Source ACL propagation	Source permissions and revocation MUST constrain all facts paths embeddings summaries caches counts notifications exports and agent context.	H1	committed
REQ-SEC-003	Secret and key isolation	Credentials MUST be envelope encrypted tenant scoped rotatable non-loggable and unavailable to model context.	H1	committed
REQ-SEC-004	Audit integrity	Sensitive reads writes approvals policy decisions and administrative actions MUST emit immutable tamper-evident audit events without unnecessary content.	H1	committed
REQ-SEC-005	Privacy lifecycle	The product MUST define purpose minimization classification retention legal hold access export correction deletion residency and subprocessor behavior.	H2	committed
REQ-SEC-006	Control readiness	The blueprint MUST map technical and organizational evidence for SOC 2 ISO 27001 and GDPR readiness without claiming certification.	H2	committed
REQ-SEC-007	Supply-chain security	Builds MUST use pinned dependencies provenance SBOM signing scanning protected CI identities and verified deployment artifacts.	H2	committed
REQ-SEC-008	Incident response	Security incidents MUST support detection containment evidence preservation notification recovery postmortem and control updates.	H2	committed
REQ-REL-001	H1 performance envelope	H1 MUST validate the committed two-tenant volume latency freshness concurrency and model-budget limits.	H1	committed
REQ-REL-002	H2 service objectives	H2 MUST provide 99.9 percent availability one-hour RPO four-hour RTO and tested restore procedures.	H2	committed
REQ-REL-003	Distributed correctness	Workflows MUST define retries idempotency ordering deduplication clock behavior partitions replay dead letters and backpressure.	H1	committed
REQ-REL-004	Graceful degradation	Dependency failures MUST have explicit fail-closed fail-open stale-read queue or unavailable behavior.	H1	committed

Id	Title	Statement	Horizon	Status
REQ-REL-005	Full observability	Every subsystem MUST emit structured logs metrics traces health readiness liveness performance error and audit signals with tenant-safe redaction.	H1	committed
REQ-REL-006	Evidence-gated hyperscale	Million-organization billion-node and trillion-edge designs MUST remain research until representative benchmarks establish a committed need.	H5	research
REQ-DEV-001	Contract-first developer platform	APIs events schemas ontology packages connectors plugins agents workflows SDKs CLI and webhooks MUST be versioned and testable independently.	H2	committed
REQ-DEV-002	Reproducible local development	A documented one-command local profile MUST provide deterministic seed data dependencies validation and observability.	H1	committed
REQ-DEV-003	Deployment safety	CI CD MUST include contract migration security evaluation canary rollback and evidence gates.	H2	committed
REQ-VER-001	Comprehensive verification	Features MUST include unit contract integration security privacy edge failure performance regression and appropriate load chaos and AI evaluations.	H1	committed
REQ-VER-002	Synthetic golden organization	H1 MUST ship a versioned seed and oracle for expected graph facts permissions simulation and action outcomes.	H1	committed
REQ-VER-003	Continuous AI evaluation	AI changes MUST run task-specific golden adversarial tool-selection argument-accuracy citation abstention and policy evaluations.	H1	committed
REQ-VER-004	Independent build readiness	An independent engineer MUST confirm H1 can be built without a major unstated product data security topology or interface decision.	H1	committed
REQ-VER-005	Zero blocking findings	Specification release MUST have no open Critical or High risks and two consecutive clear cross-domain reviews.	H1	committed

Quality Attributes

Source: `docs/enterprise-digital-twin/catalogs/quality-attributes.yaml`

Id	Attribute	Scenario	Response	Measure	Horizon
QAR-SEC-001	tenant-isolation	A malicious authenticated user supplies another tenant resource identifier through every public and agent interface.	The request is denied before data access and emits a tenant-safe security audit event.	Zero existence or content disclosure in the isolation suite.	H1
QAR-SEC-002	permission-revocation	Source access is revoked while facts are present in graph search cache and agent memory.	New requests fail closed and affected projections are invalidated and rebuilt.	H1 within 15 minutes; H2 within 5 minutes for high-risk sources.	H1
QAR-SEC-003	action-integrity	An approved action payload changes before execution or is replayed.	Changed payloads are denied; exact replay returns the original receipt without a second effect.	100 percent denial or idempotent replay in adversarial tests.	H1
QAR-PRV-001	deletion	A tenant or data subject requests deletion of eligible data.	Authoritative and derived copies are removed or cryptographically erased and lawful audit minima remain.	Verified completion within the configured retention SLA with a deletion evidence record.	H2
QAR-AVL-001	availability	A normal H2 regional workload encounters a single instance failure.	Traffic shifts to healthy replicas without losing committed operations.	99.9 percent monthly availability excluding documented maintenance.	H2
QAR-DR-001	disaster-recovery	The primary regional data plane is unrecoverable.	Restore authoritative data then rebuild projections and reconcile connectors.	RPO no more than 1 hour and RTO no more than 4 hours for H2.	H2
QAR-PERF-001	graph-query-latency	Ten H1 users issue bounded non-AI graph queries at the committed data envelope.	Queries complete without unbounded traversals or noisy-neighbor starvation.	p95 less than 2 seconds.	H1
QAR-PERF-002	simulation-latency	A user runs the committed launch scenario over the H1 project graph.	The service returns reproducible percentiles and explanations.	p95 less than 10 seconds.	H1
QAR-PERF-003	cited-answer-latency	A user asks the committed launch-risk question.	The platform returns a permission-trimmed cited answer or abstention.	p95 less than 20 seconds and a hard per-run spend budget.	H1
QAR-SYNC-001	freshness	GitHub or Jira changes an allowlisted source object.	Webhook processing or reconciliation updates the authoritative observation and projections.	H1 p95 freshness no more than 15 minutes.	H1
QAR-COR-001	replay-correctness	Events are duplicated reordered or replayed after worker failure.	The final authoritative and projected state matches a single ordered application.	Stable state digest and no duplicate external effects.	H1
QAR-AI-001	grounding	Evidence is missing conflicting or inaccessible.	Material claims are cited and the system labels uncertainty or abstains.	At least 0.95 citation precision and less than 0.01 unsupported-material-claim rate on the golden set.	H1
QAR-AI-002	prompt-injection-resistance	A source field instructs the model to reveal data or call a write tool.	Content remains untrusted data and policy middleware denies unauthorized tools.	Zero successful policy bypasses in the committed adversarial suite.	H1
QAR-SIM-001	reproducibility	The same scenario snapshot model version and seed are executed twice.	Results and ranked sensitivity drivers are identical within defined floating-point tolerance.	Exact serialized output digest after canonical rounding.	H1
QAR-OBS-001	diagnosability	An end-to-end request fails across API workflow AI and connector boundaries.	Operators correlate redacted logs metrics traces audit and workflow history without exposing tenant content.	One trace identifier and documented diagnosis path for every acceptance flow.	H1
QAR-ACC-001	accessibility	A keyboard-only or screen-reader user completes graph investigation simulation and approval.	Equivalent controls evidence and table alternatives are available without motion dependence.	WCAG 2.2 AA automated and manual acceptance suite passes.	H1

Id	Attribute	Scenario	Response	Measure	Horizon
QAR-PORT-001	portability	The platform moves between conformant infrastructure providers.	Domain workloads use only documented portable contracts and provider adapters.	H2 contract test suite passes against two compatible development implementations.	H2
QAR-COST-001	cost-control	An agent run loops or requests excessive tools or tokens.	Per-run time token spend tool and concurrency budgets terminate work safely.	No run exceeds configured hard limits by more than one in-flight operation.	H1

Traceability

Source: `docs/enterprise-digital-twin/catalogs/traceability.yaml`

Rules

Requirement Prefix	Artifacts	Decisions	Components	Contracts	Controls	Acceptance
REQ-GOV-	README.md decision-precedence.md manifest.yaml chapters/16-architecture-audit.md	ADR-001 ADR-016	CMP-DOCS	manifest.yaml catalogs/requirements.yaml catalogs/traceability.yaml	CTRL-AUD-001	AC-DOC-001 AC-DOC-002 AC-DOC-003 AC-REV-002
REQ-PROD-	chapters/01-product-vision.md chapters/02-reference-workload.md chapters/11-ux-visualizations.md	ADR-002	CMP-WEB CMP-API	contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/schemas/scenario.schema.json	CTRL-DAT-001 CTRL-IAM-003	AC-PROD-001 AC-AI-001
REQ-PHY-	chapters/17-synthetic-physical-asset-twin.md chapters/11-ux-visualizations.md chapters/12-apis-developer-platform.md	ADR-002 ADR-004 ADR-013	CMP-WEB CMP-API CMP-POLICY CMP-POSTGRES	contracts/openapi/enterprise-digital-twin.openapi.yaml	CTRL-IAM-003 CTRL-TEN-001 CTRL-AUD-001 CTRL-PHY-001	AC-PHY-001
REQ-EVT-	chapters/18-event-intelligence-causal-impact.md chapters/05-data-knowledge-graph.md chapters/07-ai-agents-reasoning.md chapters/08-simulation-prediction.md chapters/09-security-privacy-compliance.md chapters/11-ux-visualizations.md chapters/12-apis-developer-platform.md	ADR-004 ADR-006 ADR-009 ADR-010 ADR-011 ADR-012	CMP-WEB CMP-API CMP-EVENT-INTELLIGENCE CMP-AI CMP-POLICY CMP-WORKFLOW CMP-POSTGRES CMP-GRAPH CMP-OBS	contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/asyncapi/events.asyncapi.yaml contracts/schemas/event-intelligence.schema.json	CTRL-AI-001 CTRL-AI-002 CTRL-AI-004 CTRL-IAM-003 CTRL-TEN-001 CTRL-AUD-001 CTRL-PRV-002 CTRL-EVT-001	AC-EVT-001
REQ-ARCH-	chapters/03-system-architecture.md chapters/04-technology-stack.md chapters/13-deployment-operations.md	ADR-003 ADR-004 ADR-005 ADR-015	CMP-WEB CMP-API CMP-CONNECTOR CMP-AI CMP-WORKFLOW CMP-PLATFORM	contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/asyncapi/events.asyncapi.yaml contracts/proto/digital_twin.proto	CTRL-OPS-002 CTRL-SUP-001	AC-DOC-003 AC-REL-003
REQ-TEN-	chapters/03-system-architecture.md chapters/05-data-knowledge-graph.md chapters/09-security-privacy-compliance.md	ADR-006 ADR-007	CMP-API CMP-POLICY CMP-POSTGRES CMP-PROJECTION	contracts/schemas/canonical.schema.json contracts/openapi/enterprise-digital-twin.openapi.yaml	CTRL-IAM-002 CTRL-TEN-001 CTRL-TEN-002 CTRL-TEN-003	AC-TEN-001
REQ-DATA-	chapters/05-data-knowledge-graph.md catalogs/ontology.yaml	ADR-008 ADR-009 ADR-010	CMP-POSTGRES CMP-OBJECT CMP-GRAPH CMP-PROJECTION	contracts/schemas/canonical.schema.json contracts/schemas/ontology-package.schema.json	CTRL-DAT-001 CTRL-DAT-002 CTRL-DAT-003	AC-DATA-001 AC-DATA-002 AC-DATA-003 AC-PRV-001

Requirement Prefix	Artifacts	Decisions	Components	Contracts	Controls	Acceptance
REQ-CON-	chapters/06-ingestion-connectors-sync.md catalogs/connectors.yaml	ADR-011	CMP-CONNECTOR CMP-WORKFLOW CMP-PROJECTION	contracts/schemas/connector-manifest.schema.json contracts/connectors/github/manifest.json contracts/connectors/github/schemas/raw-envelope.schema.json contracts/connectors/github/schemas/raw-payload.schema.json contracts/connectors/github/schemas/normalized-observation.schema.json contracts/connectors/github/schemas/cursor.schema.json contracts/connectors/github/schemas/ontology-mapping.schema.json contracts/connectors/jira-cloud/manifest.json contracts/connectors/jira-cloud/schemas/raw-envelope.schema.json contracts/connectors/jira-cloud/schemas/raw-payload.schema.json contracts/connectors/jira-cloud/schemas/normalized-observation.schema.json contracts/connectors/jira-cloud/schemas/cursor.schema.json contracts/connectors/jira-cloud/schemas/ontology-mapping.schema.json contracts/asyncapi/events.asyncapi.yaml	CTRL-CON-001 CTRL-CON-002 CTRL-CON-003 CTRL-CRY-002	AC-CON-001 AC-CON-002
REQ-AI-	chapters/07-ai-agents-reasoning.md catalogs/agents.yaml	ADR-012 ADR-013	CMP-AI CMP-MODEL-GATEWAY CMP-POLICY CMP-WORKFLOW	contracts/schemas/canonical.schema.json contracts/mcp-manifest.json contracts/openapi/enterprise-digital-twin.openapi.yaml	CTRL-AI-001 CTRL-AI-002 CTRL-AI-003 CTRL-AI-004 CTRL-AI-005	AC-AI-001 AC-AI-002 AC-AI-003 AC-AI-004
REQ-ACT-	chapters/07-ai-agents-reasoning.md chapters/09-security-privacy-compliance.md	ADR-013	CMP-API CMP-WORKFLOW CMP-POLICY CMP-CONNECTOR	contracts/schemas/canonical.schema.json contracts/openapi/enterprise-digital-twin.openapi.yaml	CTRL-ACT-001 CTRL-ACT-002 CTRL-ACT-003	AC-ACT-001 AC-ACT-002 AC-ACT-003
REQ-SIM-	chapters/08-simulation-prediction.md catalogs/simulations.yaml	ADR-014	CMP-AI CMP-WORKFLOW CMP-POSTGRES	contracts/schemas/scenario.schema.json contracts/schemas/simulation-snapshot.schema.json contracts/openapi/enterprise-digital-twin.openapi.yaml	CTRL-DAT-001 CTRL-PRV-002	AC-SIM-001 AC-SIM-002
REQ-UX-	chapters/11-ux-visualizations.md catalogs/screens.yaml	ADR-002	CMP-WEB CMP-API	contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/graphql/schema.graphql	CTRL-DAT-002 CTRL-PRV-001	AC-UX-001
REQ-SEC-	chapters/09-security-privacy-compliance.md reviews/threat-model.md reviews/privacy-impact-assessment.md	ADR-006 ADR-007 ADR-013	CMP-POLICY CMP-API CMP-CONNECTOR CMP-AI CMP-POSTGRES CMP-OBJECT CMP-GRAPH CMP-OBS	contracts/schemas/canonical.schema.json contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/asyncapi/events.asyncapi.yaml	CTRL-IAM-001 CTRL-IAM-003 CTRL-CRY-001 CTRL-CRY-002 CTRL-AUD-001 CTRL-PRV-001 CTRL-SUP-001 CTRL-INC-001	AC-TEN-001 AC-SEC-001 AC-SEC-002 AC-PRV-001 AC-SUP-001

Requirement Prefix	Artifacts	Decisions	Components	Contracts	Controls	Acceptance
REQ-REL-	chapters/10-scalability-reliability-observability.md chapters/13-deployment-operations.md reviews/fmea.md	ADR-003 ADR-005 ADR-015	CMP-WORKFLOW CMP-POSTGRES CMP-PROJECTION CMP-OBS CMP-PLATFORM	contracts/asyncapi/events.asyncapi.yaml contracts/proto/digital_twin.proto	CTRL-OPS-001 CTRL-OPS-002	AC-REL-001 AC-REL-002 AC-REL-003 AC-OBS-001
REQ-DEV-	chapters/12-apis-developer-platform.md chapters/13-deployment-operations.md chapters/14-testing-evaluation-dx.md	ADR-004 ADR-015	CMP-DEVELOPER CMP-DOCS CMP-API CMP-CONNECTOR CMP-AI CMP-PLATFORM	contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/asyncapi/events.asyncapi.yaml contracts/graphql/schema.graphql contracts/proto/digital_twin.proto contracts/mcp-manifest.json	CTRL-SUP-001 CTRL-OPS-002	AC-DOC-003 AC-SUP-001
REQ-VER-	chapters/14-testing-evaluation-dx.md chapters/16-architecture-audit.md	ADR-016	CMP-DOCS CMP-OBS CMP-PLATFORM	catalogs/acceptance.yaml contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/schemas/canonical.schema.json	CTRL-AI-003 CTRL-SUP-001	AC-DOC-001 AC-DOC-002 AC-REV-001 AC-REV-002
REQ-VER-002	fixtures/h1/README.md fixtures/h1/seed-manifest.yaml fixtures/h1/source-fixtures.yaml fixtures/h1/identity-mappings.yaml fixtures/h1/permission-matrix.yaml fixtures/h1/ground-truth-oracle.yaml	ADR-002 ADR-014 ADR-016	CMP-DOCS CMP-CONNECTOR CMP-POSTGRES CMP-PROJECTION CMP-AI CMP-WORKFLOW	contracts/schemas/scenario.schema.json contracts/schemas/simulation-snapshot.schema.json contracts/openapi/enterprise-digital-twin.openapi.yaml	CTRL-TEN-003 CTRL-DAT-001 CTRL-DAT-002 CTRL-ACT-001 CTRL-ACT-002 CTRL-ACT-003	AC-PROD-001 AC-TEN-001 AC-DATA-001 AC-DATA-002 AC-DATA-003 AC-AI-001 AC-AI-002 AC-ACT-001 AC-ACT-002 AC-ACT-003 AC-SIM-001 AC-SIM-002

Quality Rules

Quality Prefix	Artifacts	Decisions	Components	Contracts	Controls	Acceptance
QAR-SEC-	chapters/09-security-privacy-compliance.md reviews/threat-model.md	ADR-006 ADR-007 ADR-013	CMP-POLICY CMP-API CMP-PROJECTION CMP-WORKFLOW	contracts/schemas/canonical.schema.json contracts/openapi/enterprise-digital-twin.openapi.yaml	CTRL-IAM-002 CTRL-TEN-001 CTRL-TEN-002 CTRL-DAT-002 CTRL-ACT-001	AC-TEN-001 AC-SEC-002 AC-ACT-001 AC-ACT-002
QAR-PRV-	chapters/09-security-privacy-compliance.md reviews/privacy-impact-assessment.md	ADR-008 ADR-009 ADR-010	CMP-POSTGRES CMP-OBJECT CMP-GRAPH CMP-PROJECTION	contracts/schemas/canonical.schema.json	CTRL-DAT-003 CTRL-PRV-001	AC-PRV-001
QAR-AVL-	chapters/10-scalability-reliability-observability.md chapters/13-deployment-operations.md	ADR-003 ADR-005 ADR-015	CMP-API CMP-WORKFLOW CMP-POSTGRES CMP-PLATFORM	contracts/asyncapi/events.asyncapi.yaml	CTRL-OPS-001 CTRL-OPS-002	AC-REL-002 AC-REL-003
QAR-DR-	chapters/10-scalability-reliability-observability.md chapters/13-deployment-operations.md	ADR-009 ADR-015	CMP-POSTGRES CMP-OBJECT CMP-PROJECTION CMP-PLATFORM	contracts/asyncapi/events.asyncapi.yaml	CTRL-OPS-001	AC-REL-002
QAR-PERF-	chapters/02-reference-workload.md chapters/10-scalability-reliability-observability.md	ADR-003 ADR-009 ADR-014	CMP-API CMP-AI CMP-GRAPH CMP-MODEL-GATEWAY	contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/schemas/scenario.schema.json	CTRL-AI-004 CTRL-OPS-002	AC-REL-001
QAR-SYNC-	chapters/06-ingestion-connectors-sync.md chapters/10-scalability-reliability-observability.md	ADR-005 ADR-011	CMP-CONNECTOR CMP-WORKFLOW CMP-PROJECTION	contracts/asyncapi/events.asyncapi.yaml contracts/schemas/connector-manifest.schema.json contracts/connectors/github/manifest.json contracts/connectors/jira-cloud/manifest.json	CTRL-CON-001 CTRL-CON-002 CTRL-OPS-002	AC-CON-001 AC-CON-002
QAR-COR-	chapters/06-ingestion-connectors-sync.md chapters/10-scalability-reliability-observability.md	ADR-005 ADR-011	CMP-CONNECTOR CMP-WORKFLOW CMP-POSTGRES CMP-PROJECTION	contracts/asyncapi/events.asyncapi.yaml contracts/connectors/github/schemas/raw-envelope.schema.json contracts/connectors/github/schemas/normalized-observation.schema.json contracts/connectors/jira-cloud/schemas/raw-envelope.schema.json contracts/connectors/jira-cloud/schemas/normalized-observation.schema.json	CTRL-CON-002 CTRL-OPS-002	AC-DATA-001 AC-CON-002 AC-ACT-002
QAR-AI-	chapters/07-ai-agents-reasoning.md chapters/14-testing-evaluation-dx.md	ADR-012 ADR-013	CMP-AI CMP-MODEL-GATEWAY CMP-POLICY	contracts/mcp-manifest.json contracts/schemas/canonical.schema.json	CTRL-AI-001 CTRL-AI-002 CTRL-AI-003 CTRL-AI-004	AC-AI-001 AC-AI-002 AC-AI-003 AC-AI-004
QAR-SIM-	chapters/08-simulation-prediction.md chapters/14-testing-evaluation-dx.md	ADR-014	CMP-AI CMP-WORKFLOW CMP-POSTGRES	contracts/schemas/scenario.schema.json contracts/schemas/simulation-snapshot.schema.json	CTRL-DAT-001 CTRL-OPS-002	AC-SIM-001 AC-SIM-002
QAR-OBS-	chapters/10-scalability-reliability-observability.md chapters/13-deployment-operations.md	ADR-005 ADR-015	CMP-OBS CMP-API CMP-WORKFLOW CMP-CONNECTOR CMP-AI	contracts/asyncapi/events.asyncapi.yaml contracts/schemas/canonical.schema.json	CTRL-AUD-001 CTRL-OPS-002	AC-OBS-001
QAR-ACC-	chapters/11-ux-visualizations.md catalogs/screens.yaml	ADR-002	CMP-WEB CMP-API	contracts/openapi/enterprise-digital-twin.openapi.yaml	CTRL-DAT-002 CTRL-PRV-001	AC-UX-001

Quality Prefix	Artifacts	Decisions	Components	Contracts	Controls	Acceptance
QAR-PORT-	chapters/04-technology-stack.md chapters/13-deployment-operations.md	ADR-004 ADR-015 ADR-017	CMP-PLATFORM CMP-DEVELOPER	contracts/openapi/enterprise-digital-twin.openapi.yaml contracts/asyncapi/events.asyncapi.yaml contracts/proto/digital_twin.proto	CTRL-SUP-001 CTRL-OPS-002	AC-DOC-003 AC-SUP-001
QAR-COST-	chapters/07-ai-agents-reasoning.md chapters/10-scalability-reliability-observability.md	ADR-012 ADR-013	CMP-AI CMP-MODEL-GATEWAY CMP-WORKFLOW	contracts/schemas/canonical.schema.json contracts/mcp-manifest.json	CTRL-AI-003 CTRL-AI-004	AC-AI-004 AC-REL-003

Components

Source: `docs/enterprise-digital-twin/catalogs/components.yaml`

Id	Name	Workload	Owner	Authority	Interfaces
CMP-DOCS	Specification build system	repository-tooling	enterprise-architecture	Normative sources, validation results, and generated editions.	manifest catalogs contracts diagrams publication
CMP-WEB	Web application	web	product-engineering	Non-authoritative interaction state only.	REST SSE OIDC
CMP-API	Application API	api	platform-api	Command validation, tenant binding, authorization orchestration, and query composition.	OpenAPI SSE OIDC Temporal-client
CMP-CONNECTOR	Connector and synchronization workers	connector-worker	integrations	Provider credentials, source cursors, immutable source capture, and normalized observations.	signed-webhooks provider-APIs AsyncAPI S3 PostgreSQL Temporal
CMP-AI	AI, extraction, graph-analysis, and simulation workers	ai-simulation-worker	ai-platform	Typed proposals and derived analytical results; never policy or external authority.	Temporal-activities model-gateway PostgreSQL Neo4j S3
CMP-EVENT-INTELLIGENCE	Event intelligence and causal impact engine	api-and-ai-simulation-worker	knowledge-graph	Typed event interpretations bounded evidence-linked impact proposals scenario branches and exact version-and-hash synthetic graph-mutation plans over the shared rebuildable projection; never external or employment authority.	OpenAPI event-intelligence-schema shared-event-projection PostgreSQL transactional-outbox CloudEvents Neo4j Temporal audit-event-schema
CMP-WORKFLOW	Durable workflow control plane	Temporal	workflow-platform	Durable timers, retries, signals, cancellation, approvals, and action state machines.	Temporal-API workflow-history
CMP-POSTGRES	Authoritative relational and claim store	PostgreSQL	data-platform	Tenancy, identity, connector state, observations, claims, provenance, resolution, scenarios, approvals, receipts, outbox, and audit indexes.	PostgreSQL-compatible-SQL RLS transactional-outbox
CMP-OBJECT	Immutable object and artifact store	S3-compatible-storage	data-platform	Content-addressed source payloads, evidence artifacts, snapshots, and generated reports.	S3-compatible-API
CMP-GRAPH	Graph projection	Neo4j	knowledge-graph	Rebuildable, tenant-scoped traversal projection only.	bounded-query-templates projection-writes
CMP-CACHE	Cache and rate-limit store	Valkey	platform-api	Non-authoritative tenant-namespaced cache, rate-limit, and short-lived coordination state.	Redis-compatible-protocol

Id	Name	Workload	Owner	Authority	Interfaces
CMP-PROJECTION	Projection and outbox relay	connector-worker	data-platform	Ordered checkpoints and rebuildable graph, vector, search, cache, and event projections.	transactional-outbox AsyncAPI CloudEvents
CMP-POLICY	Identity and policy enforcement	api-and-worker-middleware	security-architecture	Server-derived tenant context and execution-time authorization decisions.	OIDC SAML SCIM policy-decision-contract
CMP-MODEL-GATEWAY	Capability-oriented model gateway	ai-simulation-worker	ai-platform	Approved model snapshots, budgets, structured schemas, fallbacks, and evaluation gates.	OpenAI-Responses-API OpenAI-Agents-SDK
CMP-OBS	Observability and audit pipeline	platform-services	reliability-engineering	Redacted telemetry, SLOs, trace correlation, and tamper-evident audit envelopes.	OpenTelemetry audit-event-schema
CMP-PLATFORM	Deployment platform	infrastructure	platform-engineering	OCI packaging, configuration, keys, networking, deployment profiles, backup, and recovery.	Docker-Compose Kubernetes Helm OpenTofu OCI
CMP-DEVELOPER	Developer and extension platform	repository-tooling-and-api	developer-platform	Versioned SDK, CLI, connector, plugin, MCP, ontology, and compatibility contracts.	OpenAPI AsyncAPI JSON-Schema GraphQL Protobuf MCP

Technologies

Source: `docs/enterprise-digital-twin/catalogs/technologies.yaml`

Technology	Disposition	Horizon	Ownership	Trigger	Rationale
TypeScript	adopt	H1	Web API connector workers shared contracts		Strong end-to-end typing and mature web ecosystem.
Python	adopt	H1	AI extraction graph analytics simulation evaluation		Best fit for model and scientific-computing ecosystems.
SQL	adopt	H1	Transactional invariants migrations analytics		Declarative integrity at the authoritative boundary.
Rust	conditional	H4	Sandboxed extensions edge appliance hardened parsers	Profiling or isolation proves a TypeScript or Python component inadequate.	
Go	conditional	H3	High-throughput gateway CLI or connector appliance	Sustained concurrency or distribution requirements justify another runtime.	
WebAssembly	conditional	H4	Portable untrusted extension sandbox	Marketplace permits customer code and a capability sandbox passes security review.	
CSharp	conditional	H3	Enterprise SDK and Microsoft ecosystem adapters	Design partners require first-class dotnet integration.	
Java	conditional	H3	Enterprise SDK and JVM connectors	Design partners require JVM integration.	
Kotlin	conditional	H4	Android client or JVM SDK ergonomics	Native Android demand is validated.	
Swift	conditional	H4	Native Apple client	Responsive web fails validated mobile workflows.	
C++	rejected	H1	None in core		Memory-safety and maintenance costs exceed any demonstrated need.

Technology	Disposition	Horizon	Ownership	Trigger	Rationale
Next.js	adopt	H1	Web shell routing rendering and BFF boundary		Coherent React application and server integration.
React	adopt	H1	Accessible component and visualization UI		Mature ecosystem and team fit.
NestJS	adopt	H1	Modular TypeScript API		Explicit modules dependency injection validation and OpenAPI integration.
Fastify	adopt	H1	HTTP runtime under NestJS		Efficient typed plugin architecture.
Temporal	adopt	H1	Durable sync agent approval action and simulation workflows		Durable timers retries signals cancellation and replay.
PostgreSQL	adopt	H1	Authoritative transactional and claim store		ACID constraints RLS mature operations and extension ecosystem.
pgvector	adopt	H1	Initial tenant-scoped semantic index		Avoids a separate vector control plane at H1 scale.
Neo4j	adopt	H1	Rebuildable graph traversal and visualization projection		Productive labeled-property graph queries with a documented pooled-tenancy residual risk.
S3-compatible object storage	adopt	H1	Raw payloads documents artifacts snapshots		Portable immutable object contract.
MinIO	adopt	H1	Local S3-compatible implementation		Deterministic local development without cloud coupling.
Valkey	adopt	H1	Cache rate limit ephemeral coordination		Open Redis-compatible non-authoritative runtime.
Redis	rejected	H1	None when Valkey is available		Do not operate duplicate cache technologies.
OpenSearch	conditional	H2	Hybrid lexical vector faceted search	PostgreSQL search misses corpus latency relevance or operational SLOs.	
Elastic	rejected	H2	None by default		Duplicate responsibility with OpenSearch; reconsider only for a contractual managed-service need.
Qdrant	conditional	H3	Specialized vector retrieval	Benchmarks prove OpenSearch and pgvector cannot meet recall latency or isolation needs.	
Milvus	rejected	H3	None by default		Operational complexity is not justified by committed scale.
ClickHouse	conditional	H2	High-volume product agent trace and simulation analytics	PostgreSQL retention or analytical concurrency misses SLOs.	
Kafka	conditional	H3	Replayable event backbone	Outbox relay cannot meet throughput retention or independent-consumer requirements.	
NATS	rejected	H3	None by default		Kafka and Temporal already cover committed messaging and workflow needs; avoid overlapping brokers.
REST	adopt	H1	Public command administration and resource APIs		Explicit cacheable and broadly supported contracts.

Technology	Disposition	Horizon	Ownership	Trigger	Rationale
OpenAPI	adopt	H1	REST contract and generated SDK surface		Machine-readable compatibility and validation.
SSE	adopt	H1	Agent sync and simulation progress		Simple unidirectional resumable browser streaming.
GraphQL	conditional	H2	Read-only curated graph exploration	Multiple clients demonstrate REST over-fetching or composition pain; arbitrary graph queries remain prohibited.	
gRPC	conditional	H3	Extracted internal service calls	A service boundary requires high-throughput streaming or multi-language generated contracts.	
AsyncAPI	adopt	H1	Event and webhook contracts		Machine-readable asynchronous interface governance.
OpenTelemetry	adopt	H1	Vendor-neutral logs metrics traces and context		Portable observability and correlation.
Docker	adopt	H1	OCI build and local packaging		Reproducible workload boundary.
Docker Compose	adopt	H1	Local and single-host demo profile		Low operational cost for the bounded demonstrator.
Kubernetes	adopt	H2	Shared regional pilot workload orchestration	Freeze supported version and conformance profile before first design-partner production deployment.	The committed multi-zone H2 application plane requires standardized scheduling rollout policy and workload identity controls.
Helm	adopt	H2	Kubernetes application packaging	Freeze charts and rendered-manifest conformance before first design-partner production deployment.	Versioned rendered manifests provide one reviewable release contract across the committed H2 environments.
OpenTofu	adopt	H2	Portable infrastructure modules		Open infrastructure-as-code contract.
Terraform	conditional	H2	Customer-required IaC compatibility	Provider or customer support requires Terraform specifically.	
Service mesh	rejected	H2	None initially		Workload count does not justify another security and operations control plane.
pnpm	adopt	H1	JavaScript workspace package and task entry point		Deterministic workspace installation with one lockfile and efficient content-addressed storage.
Turborepo	adopt	H1	Monorepo task graph and local or CI caching		Enforces workload boundaries while avoiding a custom build orchestrator.
Kysely	adopt	H1	Typed TypeScript SQL construction below repository and RLS boundaries		Keeps SQL visible and typed without claiming that an ORM supplies authorization.
psycopg	adopt	H1	Python PostgreSQL access		Maintained PostgreSQL-native driver with explicit transaction control.
Pydantic	adopt	H1	Python boundary validation and canonical models		Strict typed validation for model extraction simulation and evaluation inputs.

Technology	Disposition	Horizon	Ownership	Trigger	Rationale
NumPy	adopt	H1	Vectorized deterministic simulation mathematics		Mature numerical implementation for the bounded Monte Carlo workload.
uv	adopt	H1	Locked Python environment and task bootstrap		Fast reproducible Python dependency and virtual-environment management.
RabbitMQ	rejected	H1	None		Adds queue semantics that overlap Temporal and the transactional outbox without a committed requirement.
Dapr	rejected	H2	None		Duplicates workflow pub-sub secrets and service invocation abstractions already assigned to explicit contracts.
Prometheus	adopt	H1	Reference metrics storage alert rules and SLO recording		OpenMetrics-compatible and replaceable behind OpenTelemetry.
Grafana	adopt	H1	Reference operational dashboards and telemetry correlation		Provides an operator surface without becoming a business authorization data source.
Loki	adopt	H1	Reference redacted log backend		Replaceable OpenTelemetry-compatible local and shared reference profile.
Tempo	adopt	H1	Reference distributed trace backend		Replaceable OpenTelemetry-compatible correlation for the four workloads.
Argo CD	adopt	H2	Kubernetes GitOps reconciliation and environment promotion		Reviewed Git state and drift correction support the committed pilot deployment process.
GitHub Actions	adopt	H1	CI publication contract security and evaluation gates		Repository-native automation with workload identity federation and protected secrets.
External Secrets Operator	adopt	H2	Kubernetes secret materialization from approved vault or KMS		Keeps secret values out of Git and Helm release history.
Sigstore and Cosign	adopt	H2	Artifact signing attestation and deploy-time verification		Supports verifiable build identity provenance and admission checks.
Syft	adopt	H2	Release image SBOM generation		Produces retained SPDX or CycloneDX component evidence.
SPDX and CycloneDX	adopt	H2	SBOM exchange formats		Standard machine-readable software inventory evidence.
Trivy	adopt	H1	Dependency image secret and IaC scanning		One policy-integrated scanner covers the committed build surfaces.
Vitest	adopt	H1	TypeScript unit and component tests		Fast workspace-native feedback for web API and connector packages.
Pytest	adopt	H1	Python unit property simulation and evaluation tests		Mature fixtures and scientific testing ecosystem.

Technology	Disposition	Horizon	Ownership	Trigger	Rationale
Playwright	adopt	H1	Cross-browser journey and accessibility automation		Exercises the real browser and production-shaped HTTP boundaries.
Testcontainers	adopt	H1	Production-engine integration fixtures		Validates real PostgreSQL Neo4j Valkey object and Temporal semantics instead of mocks.
k6	adopt	H1	API SSE workflow load and soak tests		Scriptable thresholds produce CI-readable H1 and H2 performance evidence.
Schemathesis	adopt	H1	OpenAPI property and negative testing		Generates boundary cases from the normative API contract.
Ruff	adopt	H1	Python formatting and lint gate		Fast deterministic checks reduce overlapping formatter and lint tools.
mypy	adopt	H1	Python static type gate		Checks typed boundaries across AI extraction and simulation modules.
OpenAPI Generator	adopt	H1	Client conformance fixtures and later generated SDKs		Keeps consumers aligned with versioned public contracts.

Ontology

Source: `docs/enterprise-digital-twin/catalogs/ontology.yaml`

Id	Name	Domain	Status	Properties	Retention
TYPE-PERSON	Person				
TYPE-EMPLOYEE	Employee				
TYPE-MANAGER	Manager				
TYPE-CONTRACTOR	Contractor				
TYPE-TEAM	Team				
TYPE-DEPARTMENT	Department				
TYPE-BUSINESSUNIT	BusinessUnit				
TYPE-ORG	Organization				
TYPE-OFFICE	Office				
TYPE-ROLE	Role				
TYPE-POSITION	Position				
TYPE-VENDOR	Vendor				
TYPE-CUSTOMER	Customer				
TYPE-PROJECT	Project				
TYPE-PORTFOLIO	Portfolio				
TYPE-PROGRAM	Program				
TYPE-GOAL	Goal				
TYPE-REQUIREMENT	Requirement				
TYPE-EPIC	Epic				
TYPE-WORKITEM	WorkItem				
TYPE-TASK	Task				

Id	Name	Domain	Status	Properties	Retention
TYPE-TICKET	Ticket				
TYPE-BUG	Bug				
TYPE-FEATURE	Feature				
TYPE-SPRINT	Sprint				
TYPE-MILESTONE	Milestone				
TYPE-PRODUCT	Product				
TYPE-MEETING	Meeting				
TYPE-CALENDAR	CalendarEvent				
TYPE-EMAIL	Email				
TYPE-CHAT	ChatMessage				
TYPE-DOCUMENT	Document				
TYPE-FILE	File				
TYPE-PRESENTATION	Presentation				
TYPE-SPREADSHEET	Spreadsheet				
TYPE-DECISION	Decision				
TYPE-APPROVAL	Approval				
TYPE-POLICY	Policy				
TYPE-RISK	Risk				
TYPE-CONTROL	Control				
TYPE-CONTRACT	Contract				
TYPE-PATENT	Patent				
TYPE-PAPER	ResearchPaper				
TYPE-REPOSITORY	Repository				
TYPE-COMMIT	Commit				
TYPE-BRANCH	Branch				
TYPE-PULLREQUEST	PullRequest				
TYPE-API	API				
TYPE-SERVICE	Service				
TYPE-MICROSERVICE	Microservice				
TYPE-DATABASE	Database				
TYPE-PIPELINE	Pipeline				
TYPE-BUILD	Build				
TYPE-WORKFLOW	Workflow				
TYPE-DEPLOYMENT	Deployment				
TYPE-ENVIRONMENT	Environment				
TYPE-INCIDENT	Incident				
TYPE-INFRA	InfrastructureResource				
TYPE-ASSET	Asset				
TYPE-COMPUTER	Computer				
TYPE-SERVER	Server				
TYPE-BUILDING	Building				

Id	Name	Domain	Status	Properties	Retention
TYPE-MACHINE	Machine				
TYPE-SENSOR	Sensor				
TYPE-LOCATION	Location				
TYPE-KPI	KPI				
TYPE-METRIC	Metric				
TYPE-ALERT	Alert				
TYPE-NETWORK	Network				
TYPE-CLOUDACCOUNT	CloudAccount				
TYPE-CLUSTER	Cluster				
TYPE-QUEUE	Queue				
TYPE-SECRETREF	SecretReference				
TYPE-IDP	IdentityProvider				
TYPE-INVOICE	Invoice				
TYPE-COSTCENTER	CostCenter				
TYPE-PO	PurchaseOrder				
TYPE-BUDGET	Budget				
TYPE-EXPENSE	Expense				
TYPE-PAYMENT	Payment				
TYPE-ACCOUNT	FinancialAccount				
TYPE-CUSTOMERACCOUNT	Account				
TYPE-OPPORTUNITY	Opportunity				
TYPE-SUBSCRIPTION	Subscription				
TYPE-ORDER	Order				
TYPE-SUPPORTCASE	SupportCase				
TYPE-TRANSCRIPT	Transcript				
TYPE-FINDING	Finding				
TYPE-EXCEPTION	Exception				
TYPE-AUDIT	Audit				
TYPE-REGULATION	Regulation				
TYPE-DATASET	DataSet				
TYPE-DATACLASS	DataClassification				
TYPE-ROOM	Room				
TYPE-TIMESERIES	TimeSeries				
TYPE-ACTOR	Actor				
TYPE-SOURCEOBJECT	SourceObject				
TYPE-CLAIM	Claim				
TYPE-EVIDENCE	Evidence				
TYPE-POLICYDECISION	PolicyDecision				
TYPE-TOOLINVOCATION	ToolInvocation				
TYPE-SCENARIO	Scenario				

Id	Name	Domain	Status	Properties	Retention
TYPE-SIMRUN	SimulationRun				
TYPE-FORECAST	Forecast				
TYPE-AGENTRUN	AgentRun				
TYPE-APPROVALREQUEST	ApprovalRequest				
TYPE-ACTIONRECEIPT	ActionReceipt				
TYPE-CONNECTOR	ConnectorInstallation				

Namespace

```
edt.core
...
```

Version

```
1.0.0
...
```

Type Defaults

```
identity: UUIDv7 canonical identifier plus tenant-scoped provider aliases.
lifecycle: proposed active inactive archived deleted with versioned transitions.
versioning: Bitemporal immutable versions with optimistic concurrency.
permissions: Source ACL intersection plus platform policy; derived visibility never
  broadens source visibility.
metadata: tenant classification owner stewardship provenance confidence and tags.
embeddings: Optional tenant-scoped embedding of authorized text; model and input hash
  recorded.
history: Append-only version and resolution history with reversible canonical merges.
searchability: Explicit field allowlist and security-trimmed indexes.
deletion: Tombstone then projection purge object erasure and minimum lawful audit
  receipt.
update_frequency: Event-driven where supported plus periodic reconciliation.
```

Relationship Defaults

```
cardinality: Declared by ontology constraint; violations are quarantined rather than
  silently repaired.
directionality: Directed canonical edge with an optional declared inverse.
strength: Optional normalized 0-to-1 score with method and version.
confidence: Required for inferred edges and fixed at 1.0 only for authoritative deterministic
  mappings.
temporal: Valid-from valid-to observed-at and system version.
provenance: One or more supporting claims and evidence references.
permissions: Visible only when edge policy endpoints and qualifying evidence are all
  visible.
creation: Deterministic mapping model proposal or authorized manual assertion with
  method recorded.
deletion: Tombstone and projection removal; source claim history retained per policy.
merge: Duplicate edges combine evidence but never broaden ACLs; conflicting semantics
  remain separate claims.
```

Relationship Types

Id	Name	Meaning
EDGE-PART-OF	PART_OF	Subject belongs structurally to object.
EDGE-REPORTS-TO	REPORTS_TO	Person has a time-bounded reporting relationship to manager.
EDGE-SUPERVISES	SUPERVISES	Role oversees a person team process or asset.
EDGE-MEMBER-OF	MEMBER_OF	Person or team is a member of an organizational unit.
EDGE-WORKS-ON	WORKS_ON	Person or team contributes to a project or work item.
EDGE-OWNS	OWNS	Subject is accountable for object lifecycle.
EDGE-CREATED	CREATED	Subject created object.
EDGE-ASSIGNED	ASSIGNED_TO	Work is assigned to a person team or role.
EDGE-RESPONSIBLE	RESPONSIBLE_FOR	Actor team or role is accountable for a resource or outcome.
EDGE-DEPENDS	DEPENDS_ON	Subject requires object before or during successful operation.
EDGE-REQUIRES	REQUIRES	Subject has an explicit requirement for object.
EDGE-BLOCKS	BLOCKS	Subject prevents progress of object.
EDGE-USES	USES	Subject consumes or operates object.
EDGE-CALLS	CALLS	Software component invokes an API or service.
EDGE-HOSTED	HOSTED_ON	Software or data object runs on infrastructure.
EDGE-CONNECTED	CONNECTED_TO	Assets or systems have a declared connection.
EDGE-IMPLEMENTS	IMPLEMENTS	Subject realizes a requirement policy feature or interface.
EDGE-PRODUCES	PRODUCES	Process service or person emits an artifact or outcome.
EDGE-DEPLOYED	DEPLOYED	Artifact or service was released by a deployment.
EDGE-APPROVED	APPROVED	Actor granted an exact scoped approval.
EDGE-MENTIONS	MENTIONS	Content explicitly mentions an entity candidate.
EDGE-REFERENCES	REFERENCES	Artifact intentionally cites or links another object.
EDGE-LINKED	LINKED_TO	Source system declares a generic link pending stronger semantics.
EDGE-GENERATED	GENERATED	Process model or agent produced object.
EDGE-OBSERVES	OBSERVES	Sensor metric or analysis observes object.
EDGE-LOCATED	LOCATED_IN	Entity has a time-bounded physical or logical location.
EDGE-AFFECTS	AFFECTS	Event risk or change has evidence-backed impact on object.
EDGE-INFLUENCES	INFLUENCES	Subject is modeled as a non-causal influence with explicit method.
EDGE-RELATES	RELATES_TO	Weak generic association used only when no stronger type is justified.
EDGE-FORECASTS	FORECASTS	Forecast predicts a typed outcome for object.

Id	Name	Meaning
EDGE-SERVES	SERVES	Team product or service supports a customer or account.
EDGE-SOLD	SOLD_TO	Offering order or subscription is sold to a customer account.
EDGE-BILLED	BILLED_TO	Invoice or charge is billed to an authorized account.
EDGE-SUPPLIED	SUPPLIED_BY	Item service or asset is supplied by a vendor.
EDGE-EVIDENCE	EVIDENCED_BY	Claim relationship answer or decision is supported by evidence.
EDGE-DERIVED	DERIVED_FROM	Object or claim was deterministically or probabilistically derived from source.
EDGE-COMPENSATES	COMPENSATES	Action reverses or mitigates an earlier action.

Domain Packages

Namespace	Horizon	Types
edt.core.organization	H1	Person Employee Contractor Team BusinessUnit Department Organization Role Position Office
edt.core.work	H1	Portfolio Program Project Goal Requirement Epic WorkItem Task Ticket Sprint Milestone Workflow Decision Approval Risk
edt.pack.engineering	H1	Product Feature Bug Repository Branch Commit PullRequest File API Service Microservice Database Pipeline Build Deployment Environment Incident Alert
edt.pack.knowledge	H2	Meeting CalendarEvent Email ChatMessage Transcript Document File Presentation Spreadsheet ResearchPaper Patent Policy
edt.pack.commercial	H3	Vendor Customer Account Opportunity Subscription Order SupportCase Contract
edt.pack.finance	H3	Invoice PurchaseOrder Budget CostCenter Expense Payment FinancialAccount
edt.pack.infrastructure	H3	InfrastructureResource Asset Computer Server Network CloudAccount Cluster Queue SecretReference IdentityProvider
edt.pack.governance	H2	Control Finding Exception Audit Regulation DataSet DataClassification
edt.pack.physical	H4	Building Room Machine Sensor Location TimeSeries Metric KPI Asset
edt.platform.control	H1	Actor ConnectorInstallation SourceObject Claim Evidence PolicyDecision ToolInvocation Scenario SimulationRun Forecast AgentRun ApprovalRequest ActionReceipt

Agents

Source: `docs/enterprise-digital-twin/catalogs/agents.yaml`

Id	Name	Horizon	Purpose	Tools	Output	Termination
AGENT-QUERY	Query and research orchestrator	H1	Decompose organizational questions into authorized retrieval graph and evidence operations.	search_evidence traverse_graph fetch_evidence run_readonly_analysis	CitedAnswer	Answer with citations or abstain when evidence or budget is insufficient.
AGENT-EXTRACT	Knowledge extractor and resolver	H1	Convert normalized observations into typed candidate claims and resolution decisions.	read_observation propose_claims propose_resolution	ClaimCandidateBatch	Schema-valid candidates or quarantined failure.
AGENT-GRAPH-VERIFY	Graph verifier	H1	Check ontology constraints evidence lineage ACL monotonicity and conflicting claims.	read_claims validate_ontology open_review_case	VerificationReport	Verified rejected or routed to human review.
AGENT-SCENARIO	Scenario planner	H1	Compile natural-language intent into a typed user-confirmed simulation scenario.	read_project_snapshot validate_scenario request_missing_assumption	Scenario	Confirmed typed scenario or explicit cancellation.
AGENT-MITIGATION	Mitigation drafter	H1	Convert cited graph and simulation findings into a bounded Jira remediation draft.	read_simulation read_jira_schema draft_jira_action	ActionPreview	Immutable preview or abstention.
AGENT-ACTION	Approval-gated action executor	H1	Execute exactly one approved Jira mutation through deterministic middleware.	verify_approval create_or_update_remediation_issue compensate_jira_action	ActionReceipt	Executed once denied expired cancelled or compensated.
AGENT-RISK	Risk analysis specialist	H2	Synthesize operational project vendor and security risks without employment scoring.	search_evidence traverse_graph run_approved_model	RiskAssessment	Evidence-backed assessment with uncertainty or abstention.
AGENT-EVENT-EXTRACT	Event extraction specialist	H1	Convert untrusted reports into schema-valid event candidates without treating content as instructions.	classify_event_text identify_time_interval propose_entity_mentions attach_evidence_refs	EventInterpretationBatch	Typed candidates with unknowns or quarantined abstention.
AGENT-EVENT-RESOLVE	Event entity-resolution specialist	H1	Rank authorized tenant-local entities and explain matches conflicts and confirmation requirements.	search_authorized_entities read_alias_history propose_resolution	EventResolutionSet	Confirmed reviewer selection unresolved reference or explicit conflict.
AGENT-IMPACT	Bounded impact analysis specialist	H1	Apply versioned causal rules to propose direct relationship and downstream effects within fixed graph budgets.	read_graph_snapshot apply_impact_rules record_unknown_effect	CausalImpactGraph	Complete bounded graph or honest partial result at a configured limit.
AGENT-EVENT-VERIFY	Event verification specialist	H1	Check evidence authorization ontology constraints conflicts confidence routing and exact mutation eligibility.	read_event_draft validate_evidence validate_ontology open_review_case	EventVerificationReport	Verified scenario-only rejected or human-review-required.
AGENT-GRAPH-MUTATE	Event graph mutation executor	H1	Deterministically apply or compensate only an exact approved synthetic mutation plan.	verify_event_review apply_synthetic_mutation compensate_synthetic_mutation	EventMutationReceipt	Applied once replayed denied expired conflicted or compensated.

Id	Name	Horizon	Purpose	Tools	Output	Termination
AGENT-WORKFLOW-OPT	Workflow optimization specialist	H3	Recommend process changes and simulate effects before any approved execution.	read_workflow simulate_scenario draft_change	OptimizationProposal	Proposal and evaluation record; never direct high-impact execution.

Authority Invariant

```
A delegated agent receives the intersection of initiating user tenant workflow caller
  and tool policy authority and can never widen it.
...
```

Prohibited Personas

```
- Autonomous CEO CTO legal HR finance or security principal with implicit organizational
  authority.
- Agent that can mint credentials change policy expand its toolset or delegate broader
  authority.
- Agent that makes individual employment legal financial production identity or security-control
  decisions.
```

Common Runtime

```
retries: Exponential backoff only for classified transient errors; model retries never
  repeat external effects.
budgets: Per-run time tokens spend tools recursion concurrency and data-volume limits.
confidence: Calibrated task-specific score plus explicit abstention; not a self-reported
  probability alone.
verification: Schema validation deterministic authorization evidence checks and task-specific
  evaluations.
communication: Typed handoff envelopes containing purpose evidence references allowed
  tools budget and expiry.
audit: Model prompt tool schema policy and dataset versions plus redacted action trace
  and outcome.
```

Connectors

Source: `docs/enterprise-digital-twin/catalogs/connectors.yaml`

Id	Name	Status	Horizon	Auth	Reads	Writes	Capabilities
CON-GITHUB	GitHub	committed	H1	GitHub App installation scoped to allowlisted sandbox repositories.	repositories teams commits_metadata pull_requests reviews issues milestones labels dependency_links		
CON-JIRA	Jira Cloud	committed	H1	OAuth installation scoped to allowlisted sandbox projects.	projects issues sprints versions links comments status_history users	dual_approved_remediation_issue_only	
CON-GOOGLE	Google Workspace	provisional	H2				Drive Gmail Calendar Meet
CON-M365	Microsoft 365	provisional	H2				SharePoint OneDrive Outlook Calendar Teams
CON-SLACK	Slack	provisional	H2				channels threads huddles_metadata
CON-GITLAB	GitLab	provisional	H2				repositories merge_requests issues pipelines
CON-LINEAR	Linear	provisional	H2				teams projects issues cycles

Id	Name	Status	Horizon	Auth	Reads	Writes	Capabilities
CON-NOTION	Notion	provisional	H2				pages databases comments
CON-CONFLUENCE	Confluence	provisional	H2				spaces pages comments
CON-SALESFORCE	Salesforce	provisional	H3				accounts opportunities cases activities
CON-HUBSPOT	HubSpot	provisional	H3				companies contacts deals tickets
CON-SERVICENOW	ServiceNow	provisional	H3				incidents changes assets service_catalog
CON-AWS	AWS	provisional	H3				organizations resources cloudtrail_metadata cost
CON-AZURE	Azure	provisional	H3				tenants resources activity_metadata cost
CON-GCP	GCP	provisional	H3				organizations projects resources audit_metadata cost
CON-DATADOG	Datadog	provisional	H3				services metrics monitors incidents
CON-SPLUNK	Splunk	provisional	H3				saved_searches alerts incident_metadata
CON-GRAFANA	Grafana	provisional	H3				dashboards alerts data_source_metadata
CON-SNOWFLAKE	Snowflake	provisional	H3				databases schemas lineage query_metadata
CON-SAP	SAP	provisional	H4				finance procurement supply_chain assets
CON-ORACLE	Oracle	provisional	H4				erp finance procurement human_resources_metadata
CON-STRIPE	Stripe	provisional	H3				customers subscriptions invoices payments_metadata
CON-QUICKBOOKS	QuickBooks	provisional	H3				customers invoices bills accounts
CON-ZOOM	Zoom	provisional	H3				meetings participants transcripts_with_consent
CON-DROPBOX	Dropbox	provisional	H3				files folders permissions
CON-DISCORD	Discord	provisional	H4				servers channels messages_with_policy
CON-MATTERMOST	Mattermost	provisional	H4				teams channels posts

Id	Name	Status	Horizon	Auth	Reads	Writes	Capabilities
CON-FILESYSTEM	Filesystem and local documents	provisional	H3				allowlisted_files metadata text_extraction
CON-EMAIL	Standards-based email	provisional	H3				messages threads attachments permissions
CON-OCR	OCR pipeline	provisional	H3				images scanned_documents layout
CON-STT	Speech-to-text pipeline	provisional	H3				consented_audio transcripts speaker_labels
CON-IOT	IoT and telemetry	research	H4				devices sensors observations commands_with_safety_policy
CON-CUSTOM	Custom REST GraphQL gRPC and event APIs	committed	H2				connector_sdk manifest mappings webhooks

Connector Defaults

```
isolation: Tenant-scoped credential egress allowlist quotas sandboxed parsing and
  independent revocation.
synchronization: Signed webhook plus periodic reconciliation cursor checkpoints idempotency
  tombstones and dead-letter review.
security: Least scopes envelope-encrypted secrets no secret in model context and source
  ACL propagation.
certification: Contract tests synthetic fixture permission tests replay tests failure
  tests and security review.
```

Screens

Source: `docs/enterprise-digital-twin/catalogs/screens.yaml`

Id	Name	Horizon	Purpose	Roles	States	Acceptance
UX-COCKPIT	Organizational cockpit	H1	Launch posture risks changes connector health and next actions.			
UX-SEARCH	Search and Copilot	H1	Permission-aware questions answers citations abstention and run inspection.			
UX-GRAPH	Graph explorer	H1	Bounded 2D traversal filters evidence and accessible path table.			
UX-ENTITY	Entity and evidence detail	H1	Properties ownership claims conflict history ACL explanation and corrections.			
UX-TIMELINE	Timeline	H1	Valid-time and system-time organizational change inspection.			
UX-SCENARIO	Scenario builder	H1	Typed assumptions interventions validation and confirmation.			
UX-SIM-COMPARE	Simulation comparison	H1	Baseline versus scenario percentiles sensitivity critical path and caveats.			

Id	Name	Horizon	Purpose	Roles	States	Acceptance
UX-AGENT-RUN	Agent run inspector	H1	Plan evidence tools budgets handoffs approvals and outcome without private chain of thought.			
UX-APPROVAL	Action approval	H1	Exact immutable diff risk scope expiry two-person control and rollback preview.			
UX-CONNECTORS	Connector administration	H1	Scope freshness cursors errors revocation and reconciliation.			
UX-AUDIT	Audit explorer	H1	Filter export verify and correlate security and action evidence.			
UX-ASSET-TWIN	Synthetic physical-asset twin	H1	Live synthetic telemetry spatial component inspection deterministic analytics lifecycle history and simulator-only control.			
UX-EVENT-INTELLIGENCE	Event intelligence workspace	H1	Interpret untrusted event reports resolve entities inspect bounded causal impacts compare before and after route scenarios review synthetic mutations and roll them back.			
UX-ADMIN	Tenant administration	H2	SSO SCIM membership policy retention residency keys and break-glass.			
UX-ONTOLOGY	Ontology studio	H2	Versioned domain packages constraints mappings migrations and impact preview.			
UX-WORKFLOW	Workflow builder	H3	Typed steps policies approvals budgets simulation and test fixtures.			
UX-MARKETPLACE	Extension marketplace	H4	Signed connector agent ontology visualization and workflow packages.			
UX-MOBILE	Responsive mobile review	H2	Search evidence alerts and approvals; graph authoring remains desktop-first.			

Screen Defaults

```
states:
- loading
- empty
- error
- denied
- stale
- partial
- offline
- destructive_confirmation
- recovery
accessibility: WCAG 2.2 AA keyboard navigation visible focus semantic structure reduced
  motion and non-visual alternative.
evidence: Source provenance time confidence classification and permission explanation
  where applicable.
```

Visualizations

Id	Name	Purpose	Horizon
VIS-KNOWLEDGE	Knowledge graph	Typed evidence-backed relationships with bounded expansion.	
VIS-ORG	Organization graph	Time-bounded structure membership and ownership without individual scoring.	
VIS-DEPENDENCY	Dependency graph	Project service and work critical paths.	
VIS-COMMS	Communication graph	Aggregate consented collaboration patterns with minimum cohorts.	
VIS-RISK	Risk heatmap	Likelihood impact confidence controls and ownership.	
VIS-TIMELINE	Timeline	Valid-time system-time and event sequence.	
VIS-CALENDAR	Calendar	Milestones windows incidents and scheduled work.	
VIS-FLOW	Project flow	Work queues blockers throughput and dependencies.	
VIS-FINANCE	Financial flow	Authorized aggregate budget procurement invoice and dependency relationships.	
VIS-GEO	Geographic view	Residency facilities assets and service regions.	
VIS-PLAYBACK	Simulation playback	Reproducible scenario progression distributions and intervention points.	
VIS-3D	3D organizational view	Optional storytelling view with no exclusive information.	H4
VIS-PHYSICAL-3D	Procedural 3D-style physical-asset view	Spatially locate a synthetic component condition in a rotatable perspective SVG while preserving a complete keyboard-operable structured alternative.	H1
VIS-CAUSAL-IMPACT	Causal impact graph	Explain direct and downstream event consequences with depth evidence confidence time horizon and a complete keyboard-operable impact list.	H1

Simulations

Source: `docs/enterprise-digital-twin/catalogs/simulations.yaml`

Id	Name	Status	Horizon	Purpose	Inputs	Outputs	Prohibited Uses
SIM-LAUNCH-DELAY	Launch-delay risk	committed	H1		target_date work_items three_point_estimates dependencies team_capacity availability scope_changes staffing_changes	p50_date p80_date p95_date miss_probability critical_path blockers sensitivity assumptions warnings	
SIM-INCIDENT	Incident blast radius	provisional	H2				
SIM-CAPACITY	Team capacity and portfolio flow	provisional	H2				

Id	Name	Status	Horizon	Purpose	Inputs	Outputs	Prohibited Uses
SIM-BUDGET	Budget and portfolio allocation	provisional	H3				
SIM-CUSTOMER	Customer churn scenarios	research	H3				
SIM-PRICING	Pricing scenarios	research	H3				
SIM-MA	Merger and acquisition integration	research	H5				
SIM-SUPPLY	Supply-chain disruption	research	H4				
SIM-OFFICE	Office closure	research	H4				
SIM-MIGRATION	Technology migration	provisional	H3				
SIM-VENDOR	Vendor outage	provisional	H3				
SIM-SECURITY	Security incident	provisional	H3				
SIM-WORKFORCE	Workforce-sensitive scenarios	research	H5				

Model Defaults

```
reproducibility: Persist canonical inputs snapshot identifier code and model versions
  random seed and environment metadata.
uncertainty: Return distributions intervals sensitivity and missing-data warnings
  rather than a single deterministic claim.
governance: State intended use prohibited use calibration evidence human confirmation
  and decision limits.
```

Controls

Source: `docs/enterprise-digital-twin/catalogs/controls.yaml`

Id	Domain	Title	Requirement
CTRL-IAM-001	identity	Enterprise authentication	OIDC or SAML authentication with phishing-resistant MFA for privileged roles and SCIM lifecycle at H2.
CTRL-IAM-002	authorization	Server-derived tenant context	Tenant and actor context is derived from verified identity and membership and cannot be overridden by request input.
CTRL-IAM-003	authorization	Policy decision point	Every privileged read write export tool and administrative operation receives an execution-time policy decision.
CTRL-IAM-004	authorization	Delegation intersection	Delegated authority is the intersection of user tenant caller workflow and tool policies.
CTRL-TEN-001	tenancy	Relational tenant fence	PostgreSQL RLS and tenant-qualified constraints enforce tenant isolation.
CTRL-TEN-002	tenancy	Derived namespace isolation	Graph vector search object cache and queue projections use independently tested tenant namespaces.
CTRL-TEN-003	tenancy	Cross-tenant prohibition	Cross-tenant retrieval resolution memory analytics and training are disabled by default.
CTRL-DAT-001	data	Classification and provenance	Every source observation claim object and derived artifact carries classification provenance and lifecycle metadata.

Id	Domain	Title	Requirement
CTRL-DAT-002	data	ACL monotonicity	Derived data visibility cannot be broader than the qualifying source evidence visibility.
CTRL-DAT-003	data	Retention and deletion propagation	Retention legal hold deletion export and correction workflows cover authoritative and derived stores.
CTRL-CRY-001	cryptography	Encryption in transit and at rest	Modern TLS protects network paths and tenant-scoped envelope keys protect sensitive stored data.
CTRL-CRY-002	secrets	Connector credential isolation	Credentials are tenant scoped encrypted rotated redacted and never placed in model context.
CTRL-CON-001	connectors	Least-scope installation	Connector installations are restricted to allowlisted repositories projects resources and actions.
CTRL-CON-002	connectors	Webhook authenticity and replay defense	Signatures timestamps nonces and event identifiers are verified before durable acceptance.
CTRL-CON-003	connectors	Parser and egress containment	Untrusted payload processing is resource bounded and connector egress is allowlisted.
CTRL-AI-001	ai	Content-instruction separation	User and connector content is carried as untrusted data and cannot modify privileged policy or tool definitions.
CTRL-AI-002	ai	Structured model boundaries	Model plans handoffs claims scenarios and action drafts are schema validated before use.
CTRL-AI-003	ai	Model and prompt promotion gate	Snapshot prompt tool and fallback changes require task-specific quality safety latency and cost evaluation.
CTRL-AI-004	ai	Agent resource budgets	Time token spend tool recursion concurrency and data-volume budgets terminate excessive work.
CTRL-AI-005	ai	Trace minimization	Store evidence structured rationale tool and policy metadata without secrets unnecessary content or private chain of thought.
CTRL-ACT-001	actions	Exact-payload approval	Approval binds the canonical payload tenant actors credential target expiry policy and idempotency key.
CTRL-ACT-002	actions	Dual authorization	Two distinct authenticated people approve the H1 external mutation and requester cannot self-complete control.
CTRL-ACT-003	actions	Action compensation	Before and after state receipts and idempotent compensation are retained and tested.
CTRL-PHY-001	physical-assets	Synthetic device boundary	H1 asset telemetry and controls remain inside the tenant-scoped simulator and every preview receipt and user-visible result discloses that no physical read or write occurred.

Id	Domain	Title	Requirement
CTRL-EVT-001	event-intelligence	Untrusted event and graph-mutation boundary	Event text and attachments remain untrusted data and every interpretation resolution expansion review mutation and rollback is tenant-bound permission-trimmed resource-bounded evidence-resolvable version-and-hash checked idempotent atomically persisted with audit and outbox evidence restart-safe and externally inert.
CTRL-AUD-001	audit	Tamper-evident audit	Sensitive operations append hash-linked audit envelopes to restricted immutable storage.
CTRL-PRV-001	privacy	Purpose and minimization	Data collection and derived inference require documented purpose field minimization and retention.
CTRL-PRV-002	privacy	Sensitive workforce prohibition	Individual employment health emotion productivity and misconduct inference is disabled through H3.
CTRL-OPS-001	operations	Backup and restore verification	Authoritative backups are encrypted tested and followed by deterministic projection rebuild and source reconciliation.
CTRL-OPS-002	operations	Observable health	Every workload publishes liveness readiness dependency health SLO and redacted telemetry.
CTRL-SUP-001	supply-chain	Build provenance	Dependencies are pinned scanned and inventoried and release artifacts are signed with verifiable provenance.
CTRL-INC-001	incident-response	Incident lifecycle	Detection containment eradication recovery notification evidence and postmortem procedures have named owners.

Risks

Source: `docs/enterprise-digital-twin/catalogs/risks.yaml`

Id	Severity	Status	Title	Mitigation	Owner	Revisit
RSK-001	high	mitigated	Cross-tenant data disclosure	RLS independent namespaces server-derived tenant context and adversarial two-tenant tests	security-architecture	
RSK-002	high	mitigated	Neo4j pooled-tenant isolation weakness	Tenant-qualified repository templates bounded queries projection tests no arbitrary Cypher and migration trigger for isolated databases or cells	data-platform	
RSK-003	high	mitigated	Source ACL leakage through derived data	Claim-level evidence ACLs monotonic visibility revocation invalidation and side-channel tests	authorization	

Id	Severity	Status	Title	Mitigation	Owner	Revisit
RSK-004	high	mitigated	Prompt injection causes data exfiltration or action	Content separation typed boundaries external policy gateway egress controls approval and adversarial evals	ai-security	
RSK-005	high	mitigated	Connector credential compromise	Tenant keys minimum scopes isolated runtime rotation revocation and audit	integrations	
RSK-006	high	mitigated	Approval confused deputy or payload substitution	Canonical payload hash two approvers 15-minute expiry policy recheck and argument immutability	workflow-security	
RSK-007	high	mitigated	Unsafe workforce surveillance	Individual workforce inference prohibition minimization aggregate cohort rules and separate research gate	privacy	
RSK-008	medium	accepted	Cloud-neutral abstraction cost	Thin infrastructure adapters and one normative portable contract with H2 conformance tests	platform	H2 provider selection or any deployment-profile expansion
RSK-009	medium	accepted	Four-runtime operational complexity over time	H1 uses only TypeScript Python SQL and containers; other languages require ADR evidence	architecture	H3 service extraction or a fourth executable language proposal
RSK-010	medium	mitigated	Entity resolution false merge	Deterministic matching first confidence thresholds review queue preserved aliases and reversible merge	knowledge-graph	
RSK-011	medium	mitigated	Simulation mistaken for forecast	Scenario labels explicit assumptions distributions synthetic disclaimer and no causal claim	simulation-science	
RSK-012	medium	mitigated	Model provider outage or regression	Capability gateway pinned evaluated snapshots queue or fail closed and no unapproved fallback	ai-platform	
RSK-013	medium	mitigated	Duplicate external action	Durable workflow idempotency receipt provider correlation and compensation test	workflow-platform	
RSK-014	medium	mitigated	Audit log becomes sensitive shadow store	Metadata minimization redaction separate access retention and integrity controls	security-operations	
RSK-015	medium	mitigated	Deletion misses derived copy	Data inventory lineage erasure workflow projection rebuild and deletion evidence scan	data-governance	
RSK-016	medium	accepted	H1 synthetic data limits external validity	Claims limited to reproducibility and system behavior; predictive accuracy requires H2 customer validation	product	H2 design-partner evaluation or any external predictive-validity claim

Id	Severity	Status	Title	Mitigation	Owner	Revisit
RSK-017	low	mitigated	3D visualization distracts or excludes users	H4 only and always paired with accessible 2D and table alternatives	ux	
RSK-018	medium	mitigated	Event backlog causes stale graph	Backpressure lag SLO priority queues replay checkpoints stale indicators and reconciliation	reliability	
RSK-019	medium	accepted	Next.js pins a PostCSS version covered by a stringification advisory	H1 builds only source-controlled CSS and accepts no user CSS or template input; production dependency audit has zero High or Critical findings and the pin is rechecked on every Next.js release	developer-platform	Next.js publishes a compatible PostCSS pin or any untrusted style or template input is proposed
RSK-020	high	mitigated	False or malicious event poisons current truth	Untrusted-content separation evidence and resolution review confidence routing exact-payload authorization scenario isolation audit and compensating rollback	knowledge-graph	
RSK-021	medium	mitigated	Explanation graph is mistaken for proven causality	Observed rule-derived simulated and unknown edges are labeled confidence decays across hops predictions require approved models and the UI exposes assumptions and missing evidence	ai-platform	
RSK-022	medium	mitigated	Event propagation exhausts graph or workflow capacity	Fixed depth node edge fan-out and runtime budgets cycle suppression deduplication backpressure partial-result labels and durable chunking gate beyond H1	reliability	

Acceptance

Source: `docs/enterprise-digital-twin/catalogs/acceptance.yaml`

Id	Title	Evidence
AC-DOC-001	Metadata validity	Every normative Markdown artifact has valid unique frontmatter and every catalog and contract parses.
AC-DOC-002	Traceability coverage	Validator reports every REQ and QAR matched to artifacts decisions controls acceptance and horizon at 100 percent.
AC-DOC-003	Reproducible editions	Build produces stable consolidated Markdown HTML PDF and coverage report from a clean checkout.
AC-PROD-001	Five-minute demo narrative	Seed sync investigate simulate approve act and rollback flow completes deterministically from the golden fixture.
AC-TEN-001	Cross-tenant isolation	Tenant A cannot infer Tenant B resources through REST graph search vectors objects cache events agent traces errors or timing assertions.

Id	Title	Evidence
AC-DATA-001	Ingestion idempotency	Duplicate reordered and replayed observations produce the same canonical state digest.
AC-DATA-002	Projection rebuild	Neo4j and search/vector projections rebuild from authoritative claims and preserve expected oracle digest and ACLs.
AC-DATA-003	Reversible merge	A false entity merge is undone without losing identities evidence history or unrelated relationships.
AC-CON-001	Signed webhook defense	Invalid stale replayed and wrong-tenant GitHub and Jira webhooks are denied and audited.
AC-CON-002	Reconciliation recovery	Missed webhook and partial backfill recover through cursor reconciliation without duplicate state.
AC-AI-001	Citation quality	Golden set citation precision is at least 0.95 and material unsupported claim rate is below 0.01.
AC-AI-002	Abstention	Missing conflicting restricted and stale evidence cases produce calibrated uncertainty or abstention.
AC-AI-003	Prompt-injection resistance	Adversarial connector content cannot change policy reveal other data or invoke unauthorized tools.
AC-AI-004	Tool accuracy	Tool choice and exact argument accuracy meet the approved task rubric on golden and adversarial cases.
AC-ACT-001	Dual approval	Requester self-approval single approval expired approval changed payload and wrong-policy approval all fail.
AC-ACT-002	Single external effect	Concurrent duplicate and replayed execution creates or updates exactly one Jira remediation issue.
AC-ACT-003	Compensation	Authorized rollback restores the captured prior state or records a human-action-required terminal state.
AC-SIM-001	Reproducible simulation	Same snapshot input model and seed yield the identical canonical p50 p80 p95 and sensitivity digest.
AC-SIM-002	Simulation edge cases	Cycles missing estimates impossible capacity disconnected work and contradictory interventions produce explicit validation outcomes.
AC-PHY-001	Synthetic physical-asset journey	Deterministic fixture telemetry procedural perspective and structured component selection analytics lifecycle history and guarded simulator commands reproduce with zero device egress and explicit synthetic labels.
AC-EVT-001	Event intelligence causal journey	An untrusted report deterministically produces a typed interpretation tenant-bounded resolution evidence-resolvable bounded causal graph and scenario or exact snapshot-bound synthetic mutation visible through shared graph consumers with atomic receipt audit outbox persistence restart-safe replay rollback and zero external effect.
AC-SEC-001	Secrets containment	Scans of logs traces prompts errors exports and artifacts contain no injected canary secrets.
AC-SEC-002	ACL revocation	Revoked source access is unavailable to all query and agent paths within the committed freshness objective.
AC-PRV-001	Deletion completeness	Deletion inventory proves removal across authoritative graph vector search cache object trace and backup-expiry workflows.

Id	Title	Evidence
AC-REL-001	H1 latency	Representative load meets graph p95 under 2 seconds simulation p95 under 10 seconds and cited answer p95 under 20 seconds.
AC-REL-002	H2 recovery	Restore drill demonstrates RPO no more than 1 hour and RTO no more than 4 hours.
AC-REL-003	Dependency degradation	Database graph object workflow cache search model and connector outage tests match documented fail-safe behavior.
AC-OBS-001	Correlated telemetry	Each golden workflow has a redacted end-to-end trace metrics logs health and audit linkage.
AC-UX-001	Accessible H1 journey	Keyboard and screen-reader users complete investigation simulation and approval with WCAG 2.2 AA review.
AC-SUP-001	Supply-chain evidence	Release includes SBOM vulnerability results signatures provenance and dependency license report.
AC-REV-001	Independent build readiness	Independent engineer records that H1 requires no unstated major product security data topology or interface decision.
AC-REV-002	Clear audit convergence	No open Critical or High risk and two consecutive cross-domain reviews introduce no new blocker.

Tests Evaluations

Source: `docs/enterprise-digital-twin/catalogs/tests-evaluations.yaml`

Semantics

```
A test or evaluation covers only the exact REQ/QAR identifiers enumerated in covers.
  Prefix inheritance is prohibited in this catalog.
...
```

Tests Evaluations

Id	Title	Kind	Horizons	Owners	Method	Oracle	Evidence	Covers	Dataset
TST-GOV-001	Normative specification governance conformance	document_conformance	H1	architecture-governance	Run the blueprint validator, regenerate editions and reports, and independently inspect precedence, identifiers, citations, review gates, and frozen-scope trace expansion.	Zero validation findings; generated artifacts match the manifest; each exact covered identifier has all mandatory trace dimensions.	output/test-evidence/TST-GOV-001.json	REQ-GOV-001 REQ-GOV-002 REQ-GOV-003 REQ-GOV-004	
TST-PROD-001	Product boundary and demo outcome review	acceptance_review	H1 H2	product-architecture	Replay the frozen demo narrative and design-partner scorecard against personas, non-goals, accessibility principles, source-backed claims, and horizon metrics.	Every claimed outcome has a measurable metric and source or is explicitly labeled an assumption; excluded workforce scoring remains absent.	output/test-evidence/TST-PROD-001.json	REQ-PROD-001 REQ-PROD-002 REQ-PROD-003 REQ-PROD-004 REQ-PROD-005	
TST-ARCH-001	Workload topology and portability verification	architecture_test	H1 H2 H3 H4	platform-architecture	Validate container boundaries, durable workflow and outbox topology, system-of-record assignments, deployment profiles, and benchmark-gated transition records against the accepted ADRs.	The four H1 workloads deploy with only adopted dependencies; conditional technologies have measurable triggers and no undeclared authority.	output/test-evidence/TST-ARCH-001.json	REQ-ARCH-001 REQ-ARCH-002 REQ-ARCH-003 REQ-ARCH-004 REQ-ARCH-005 REQ-ARCH-006 REQ-ARCH-007	
TST-PHY-001	Synthetic physical-asset twin conformance	end_to_end_test	H1	product-engineering simulation-team security-engineering	Replay frozen-fixture telemetry and component selection through procedural perspective and structured views verify deterministic anomaly and trend outputs inspect lifecycle history and exercise valid invalid stale replayed cross-tenant and out-of-range simulator commands.	Identical fixture sequence state and model version reproduce canonical results every invalid command has zero effect a valid preview executes once replay returns its receipt audit hashes verify and no network or device write occurs.	output/test-evidence/TST-PHY-001.json	REQ-PHY-001 REQ-PHY-002 REQ-PHY-003 REQ-PHY-004 REQ-PHY-005	

Id	Title	Kind	Horizons	Owners	Method	Oracle	Evidence	Covers	Dataset
TST-EVT-001	Event intelligence and causal mutation conformance	security_test	H1	knowledge-graph ai-platform product-engineering security-engineering	Interpret the frozen reports through tenant-bound resolution and bounded causal expansion then resolve every impact evidence identifier and exercise scenario routing exact review snapshot-version-and-hash approval compare-and-swap shared entity traversal and simulation visibility atomic PostgreSQL receipt audit and outbox commit stale and mutated payload rejection process restart idempotent replay rollback audit verification and cross-tenant denial.	Interpretations preserve uncertainty and all impact evidence resolves within the authorized event entity candidates never cross tenants causal expansion terminates at configured budgets non-confirmed reports stay in branches exact reviewed payload applies once to every shared graph consumer stale snapshot or failed commit has zero committed effect restart restores projection timeline application approval idempotency and audit state replay returns the original receipt rollback compensates with a monotonic graph version and no external or employment action occurs.	output/test-evidence/TST-EVT-001.json	REQ-EVT-001 REQ-EVT-002 REQ-EVT-003 REQ-EVT-004 REQ-EVT-005	Deterministic Aster and Beacon event reports including multiple vague uncertain conflicting malicious stale cyclic excessive and reversible cases.
TST-TEN-001	Tenant isolation and authorization adversarial suite	security_test	H1	security-engineering data-platform	Exercise server-derived context, RLS, tenant-qualified relations, namespace isolation, ACL revocation, cache barriers, delegation ceilings, and cross-tenant identifier substitution.	Zero cross-tenant existence or content disclosure and no authorization decision derived from caller-supplied tenant context.	output/test-evidence/TST-TEN-001.json	REQ-TEN-001 REQ-TEN-002 REQ-TEN-003 REQ-TEN-004 REQ-TEN-005	
TST-DATA-001	Canonical data, provenance, projection, and lifecycle suite	contract_test	H1 H2	data-platform	Validate canonical schemas, tenant-qualified keys, claim/evidence chains, reversible resolution, deterministic graph rebuilds, ontology extension constraints, retention, deletion, and permission revocation.	Replays are byte-identical, projections rebuild from PostgreSQL/object storage, provenance is complete, and deletion/revocation removes every serving path within the declared SLO.	output/test-evidence/TST-DATA-001.json	REQ-DATA-001 REQ-DATA-002 REQ-DATA-003 REQ-DATA-004 REQ-DATA-005 REQ-DATA-006 REQ-DATA-007 REQ-DATA-008 REQ-DATA-009 QAR-PRV-001	

Id	Title	Kind	Horizons	Owners	Method	Oracle	Evidence	Covers	Dataset
TST-CON-001	GitHub and Jira connector certification suite	contract_test	H1 H2	connector-platform	Validate package manifests, content-addressed schemas, positive and negative fixtures, exact permissions/events/egress, duplicate and out-of-order ingress, cursor recovery, reconciliation, tombstones, and connector SDK lifecycle behavior.	Only frozen allowlists validate; retries and recovery produce deterministic observations; invalid, replayed, oversized, or misbound input has no canonical effect.	output/test-evidence/TST-CON-001.json	REQ-CON-001 REQ-CON-002 REQ-CON-003 REQ-CON-004 REQ-CON-005 QAR-SYNC-001 QAR-COR-001	
TST-AI-001	Grounded AI safety and quality evaluation	evaluation	H1	ai-platform security-engineering	Run golden and adversarial cases for citations, abstention, ACL-filtered context, tool selection and arguments, trace grading, prompt injection, workload routing, approved fallback, cost budgets, and model failure.	Thresholds in the acceptance catalog pass on pinned snapshots; inaccessible evidence never enters context; no unapproved fallback or tool authority is exercised.	output/test-evidence/TST-AI-001.json	REQ-AI-001 REQ-AI-002 REQ-AI-003 REQ-AI-004 REQ-AI-005 REQ-AI-006 REQ-AI-007 REQ-AI-008 QAR-AI-001 QAR-AI-002 QAR-COST-001	
TST-ACT-001	Exact-payload Jira action safety suite	security_test	H1 H3	workflow-platform security-engineering	Exercise preview canonicalization, two-role approval, expiry, payload/version/credential/policy mutation, replay, at-most-one send, ambiguous result verification, receipt evidence, and authorized compensation.	Every negative case performs zero writes; the approved fixture performs one field-limited PUT; replay returns the original receipt; rollback is idempotent and audited.	output/test-evidence/TST-ACT-001.json	REQ-ACT-001 REQ-ACT-002 REQ-ACT-003 REQ-ACT-004	
TST-SIM-001	Seeded launch-delay simulation verification	numerical_test	H1 H3	simulation-team	Validate scenario schema and compiler, dependency-DAG rejection, seeded PERT sampling, quantile and sensitivity calculations, snapshot immutability, baseline comparison, missing-data warnings, and prohibited inference labels.	Identical snapshots and seeds produce byte-identical forecasts; expected p50/p80/p95 and critical paths match the frozen oracle within declared tolerances.	output/test-evidence/TST-SIM-001.json	REQ-SIM-001 REQ-SIM-002 REQ-SIM-003 REQ-SIM-004 REQ-SIM-005 QAR-SIM-001	

Id	Title	Kind	Horizons	Owners	Method	Oracle	Evidence	Covers	Dataset
TST-UX-001	Permission-aware UX and accessibility journey suite	accessibility_test	H1	web-experience	Test keyboard and screen-reader journeys for organizational question, citation inspection, context confirmation, scenario comparison, exact Jira preview, dual approval, receipt, and rollback across permission variants.	WCAG-aligned interactions preserve provenance and uncertainty, inaccessible content is not disclosed, and sensitive actions require explicit confirmation states.	output/test-evidence/TST-UX-001.json	REQ-UX-001 REQ-UX-002 REQ-UX-003 REQ-UX-004 QAR-ACC-001	
TST-SEC-001	Security, privacy, and abuse-case verification	security_test	H1 H2	security-engineering privacy-office	Execute threat-model abuse cases for credential isolation, webhook authenticity, SSRF and egress, malicious connector text, prompt injection, encryption, audit integrity, deletion, incident response, and compliance evidence controls.	Critical controls fail closed, secrets and restricted content are absent from telemetry, deletion evidence is complete, and no prohibited workforce inference is emitted.	output/test-evidence/TST-SEC-001.json	REQ-SEC-001 REQ-SEC-002 REQ-SEC-003 REQ-SEC-004 REQ-SEC-005 REQ-SEC-006 REQ-SEC-007 REQ-SEC-008 QAR-SEC-001 QAR-SEC-002 QAR-SEC-003	
TST-REL-001	Performance, resilience, observability, and recovery suite	resilience_test	H1 H2 H5	reliability-engineering	Load the frozen scale profile and inject database, object, graph, cache, model, connector, network, and regional failures while measuring SLOs, backpressure, recovery, telemetry redaction, RPO, and RTO.	H1/H2 SLOs and recovery objectives pass; authoritative data is not lost; degraded projections fail safely; research-scale claims remain gated.	output/test-evidence/TST-REL-001.json	REQ-REL-001 REQ-REL-002 REQ-REL-003 REQ-REL-004 REQ-REL-005 REQ-REL-006 QAR-AVL-001 QAR-DR-001 QAR-PERF-001 QAR-PERF-002 QAR-PERF-003 QAR-OBS-001	
TST-DEV-001	Public contract, SDK, plugin, and deployment portability suite	compatibility_test	H1 H2	developer-platform	Lint and compatibility-test OpenAPI, AsyncAPI, JSON Schema, GraphQL, Protobuf,MCP, connector/plugin manifests, generated SDK behavior, local Compose, Kubernetes/Helm, and infrastructure profiles.	Normative contracts are valid and backward-compatible within their declared window; plugins cannot exceed tenant policy; adopted deployment profiles use only portable boundaries.	output/test-evidence/TST-DEV-001.json	REQ-DEV-001 REQ-DEV-002 REQ-DEV-003 QAR-PORT-001	

Id	Title	Kind	Horizons	Owners	Method	Oracle	Evidence	Covers	Dataset
TST-VER-001	Frozen H1 end-to-end and release-gate suite	end_to_end_test	H1	quality-engineering architecture-governance	Regenerate both synthetic tenants, execute the complete cited-question, seeded-simulation, exact-preview, dual-approval, one-write, replay, receipt, and rollback flow, then run two independent cross-domain reviews.	The frozen oracle and every acceptance identifier pass; coverage is 100 percent across all mandatory dimensions; no Critical/High or unowned committed decision remains.	output/test-evidence/TST-VER-001.json	REQ-VER-001 REQ-VER-002 REQ-VER-003 REQ-VER-004 REQ-VER-005	

Roadmap

Source: `docs/enterprise-digital-twin/catalogs/roadmap.yaml`

Semantics

```
Each committed requirement and quality attribute appears in exactly one horizon matching
  its source-catalog horizon. A trace may resolve only to that exact entry.
...
```

Horizons

Id	Title	Status	Owner	Exit Evidence	Requirements	Quality Attributes
H1	Production-shaped hackathon demonstrator	committed	hackathon-delivery-lead	output/test-evidence/H1-release.json	REQ-GOV-001 REQ-GOV-002 REQ-GOV-003 REQ-GOV-004 REQ-PROD-002 REQ-PROD-003 REQ-PROD-004 REQ-PHY-001 REQ-PHY-002 REQ-PHY-003 REQ-PHY-004 REQ-PHY-005 REQ-EVT-001 REQ-EVT-002 REQ-EVT-003 REQ-EVT-004 REQ-EVT-005 REQ-ARCH-001 REQ-ARCH-002 REQ-ARCH-003 REQ-TEN-001 REQ-TEN-002 REQ-TEN-003 REQ-TEN-004 REQ-TEN-005 REQ-DATA-001 REQ-DATA-002 REQ-DATA-003 REQ-DATA-004 REQ-DATA-005 REQ-CON-001 REQ-CON-002 REQ-CON-003 REQ-CON-004 REQ-AI-001 REQ-AI-002 REQ-AI-003 REQ-AI-004 REQ-AI-005 REQ-AI-006 REQ-AI-007 REQ-AI-008 REQ-ACT-001 REQ-ACT-002 REQ-ACT-003 REQ-SIM-001 REQ-SIM-002 REQ-SIM-003 REQ-SIM-004 REQ-UX-001 REQ-UX-002 REQ-UX-003 REQ-UX-004 REQ-SEC-001 REQ-SEC-002 REQ-SEC-003 REQ-SEC-004 REQ-REL-001 REQ-REL-003 REQ-REL-004 REQ-REL-005 REQ-DEV-002 REQ-VER-001 REQ-VER-002 REQ-VER-003 REQ-VER-004 REQ-VER-005	QAR-SEC-001 QAR-SEC-002 QAR-SEC-003 QAR-PERF-001 QAR-PERF-002 QAR-PERF-003 QAR-SYNC-001 QAR-COR-001 QAR-AI-001 QAR-AI-002 QAR-SIM-001 QAR-OBS-001 QAR-ACC-001 QAR-COST-001
H2	Design-partner pilot	committed	pilot-program-lead	output/test-evidence/H2-release.json	REQ-PROD-001 REQ-PROD-005 REQ-ARCH-004 REQ-ARCH-007 REQ-DATA-006 REQ-DATA-007 REQ-DATA-008 REQ-DATA-009 REQ-CON-005 REQ-SEC-005 REQ-SEC-006 REQ-SEC-007 REQ-SEC-008 REQ-REL-002 REQ-DEV-001 REQ-DEV-003	QAR-PRV-001 QAR-AVL-001 QAR-DR-001 QAR-PORT-001
H3	Enterprise GA decision boundary	provisional	enterprise-release-board	output/test-evidence/H3-freeze.json	REQ-ARCH-005 REQ-ACT-004 REQ-SIM-005	

Id	Title	Status	Owner	Exit Evidence	Requirements	Quality Attributes
H4	Deployment expansion	provisional	deployment-architecture	output/test-evidence/H4-profile-gates.json	REQ-ARCH-006	
H5	Governed research	research	research-governance-board	output/test-evidence/H5-research-gates.json	REQ-REL-006	

Part V - Machine-Readable Contract Register

The files listed here are normative and outrank prose when precedence applies. The edition records their immutable build digest without duplicating generated code.

Contract	Format	Contract Version	Bytes	Sha256 Prefix
contracts/openapi/enterprise-digital-twin.openapi.yaml	OpenAPI	3.1.0	90302	2a705f5527c39e93
contracts/asyncapi/events.asyncapi.yaml	AsyncAPI	3.0.0	11584	e2890a41a2ffc414
contracts/graphql/schema.graphql	GraphQL SDL	H2 read-only	1821	a2022c3047cc24cf
contracts/proto/digital_twin.proto	Protocol Buffers	proto3	2135	0df177382df6ce9e
contracts/schemas/canonical.schema.json	JSON Schema	schema	42318	e4b3a9f4e790ed88
contracts/schemas/scenario.schema.json	JSON Schema	schema	7711	31a2909691e0835b
contracts/schemas/simulation-snapshot.schema.json	JSON Schema	schema	6829	9327595d390c11df
contracts/schemas/event-intelligence.schema.json	JSON Schema	schema	41707	c6bdb7b696a59522
contracts/schemas/event-intelligence-event-changed.v1.schema.json	JSON Schema	schema	4840	cb994d57740ebd12
contracts/schemas/connector-manifest.schema.json	JSON Schema	schema	15787	9cdc3461a964833c
contracts/schemas/ontology-package.schema.json	JSON Schema	schema	2229	892a31ef901f8764
contracts/schemas/plugin-manifest.schema.json	JSON Schema	schema	4561	1b76a3407e508fa9
contracts/schemas/agent-profile.schema.json	JSON Schema	schema	3280	b3dceb30198be165
contracts/schemas/workflow-definition.schema.json	JSON Schema	schema	3663	50bbbb6978986986
contracts/schemas/ui-panel.schema.json	JSON Schema	schema	2217	a92ceb68e4d002bb
contracts/schemas/sdk-contract.schema.json	JSON Schema	schema	4917	30ab5acc7f151912
contracts/schemas/cli-contract.schema.json	JSON Schema	schema	4093	5ade2678c3d91856
contracts/mcp-manifest.json	JSON	1.0.0	8640	b6632ccc91dc8b90
contracts/connectors/github/manifest.json	JSON	1.0.0	13542	0162513e11c04768
contracts/connectors/github/schemas/raw-envelope.schema.json	JSON Schema	schema	1480	8ef2afcf6453c48c
contracts/connectors/github/schemas/raw-payload.schema.json	JSON Schema	schema	1696	a166ed285b88e904

Contract	Format	Contract Version	Bytes	Sha256 Prefix
contracts/connectors/github/schemas/normalized-observation.schema.json	JSON Schema	schema	4449	d7f619ff51acebc0
contracts/connectors/github/schemas/cursor.schema.json	JSON Schema	schema	1071	6e7864bbd1cc97d8
contracts/connectors/github/schemas/ontology-mapping.schema.json	JSON Schema	schema	4840	e3177d780b9381ed
contracts/connectors/jira-cloud/manifest.json	JSON	1.0.0	11721	a6de996534318292
contracts/connectors/jira-cloud/schemas/raw-envelope.schema.json	JSON Schema	schema	1613	51325ffdfe54b38a
contracts/connectors/jira-cloud/schemas/raw-payload.schema.json	JSON Schema	schema	1584	ac0925570700e607
contracts/connectors/jira-cloud/schemas/normalized-observation.schema.json	JSON Schema	schema	4407	479950153061ef60
contracts/connectors/jira-cloud/schemas/cursor.schema.json	JSON Schema	schema	937	052deca085141092
contracts/connectors/jira-cloud/schemas/ontology-mapping.schema.json	JSON Schema	schema	4736	c01918dc0ce1bc76

Part VI - Frozen H1 Fixture and Ground-Truth Oracle

These files are normative inputs to the reproducible two-tenant demonstration and its isolation, citation, simulation, action, and rollback tests.

Fixture contract

--- id: EDT-FIXTURE-H1 title: H1 Synthetic Fixture and Ground-Truth Oracle status: committed version: 1.0.0 owners:

last_reviewed: 2026-07-13 ---

- quality-engineering
- data-platform

H1 Synthetic Fixture and Ground-Truth Oracle

This directory freezes the deterministic two-tenant reference workload from CH-02. It is specification evidence, a test-data generator contract, and the source of expected outcomes. It contains no customer data.

Artifacts

File	Purpose
`seed-manifest.yaml`	Root seed, clock, canonical tenant and connector identifiers, exact source counts, allowlists, deterministic expansion rules, and canonical SHA-256 digests for every companion fixture.
`source-fixtures.yaml`	Decision-chain source objects, secondary risks, ACL classes, duplicate-name traps, and Beacon isolation canary.
`identity-mappings.yaml`	Fixed actor mappings, generated-person ranges, cross-source merge oracle, and required non-merges.
`permission-matrix.yaml`	Actor grants, denials, evidence visibility, approval separation, and expected indistinguishable-not-found cases.
`ground-truth-oracle.yaml`	Expected graph, answer, simulation, action, replay, rollback, and tenant-isolation outcomes.

Deterministic generation contract

The generator consumes UTF-8 YAML after normalizing line endings to LF. It uses the root seed and the manifest's `uuidv5` rule for stable internal identifiers. Synthetic filler records are generated in lexical source-key order from the declared counts; their content MUST NOT add a path to the Orion launch milestone or include the Beacon canary outside Beacon. Reordering YAML fields cannot change generated identifiers or oracle outcomes.

The fixture loader refuses a real provider credential, non-synthetic environment, undeclared tenant, undeclared project or repository, wall-clock time, or output path outside its isolated fixture namespace. Rerunning the seed is idempotent. A destructive reset requires the synthetic marker, resolved namespace preview, and explicit confirmation.

Oracle use

Tests execute ingestion and projection rather than copying final answers. The oracle is compared only after normalization, authorization, graph projection, cited retrieval, simulation, action execution, and compensation complete. Numeric simulation tolerances are those in CH-02; all authorization, action-count, identifier, digest, and citation membership checks are exact.

Seed Manifest

Source: `docs/enterprise-digital-twin/fixtures/h1/seed-manifest.yaml` | SHA-256: `2b58de8cb8d2816ff8ff56c67722b7da296fb0c91351ec9f8288c1a03e55af64`

```
schema_version: 1
fixture:
  workload_id: edt-h1-github-jira-launch-risk
  fixture_version: 1.0.0
  root_seed: edt-h1-20260713
```

```
  simulation_seed: "20260713"
  frozen_clock: 2026-07-13T16:00:00Z
  synthetic_only: true
  namespace_marker: EDT_SYNTHETIC_H1_V1
  identity_algorithm: uuidv5
  identity_namespace: "urn:edt:h1"
  ordering: Unicode code-point lexical order over tenant alias provider object type and source key
  rerun_semantics: Upsert immutable source revisions by tenant-qualified key and produce no duplicate
observation claim edge or external effect.
artifacts:
  source_fixtures: fixtures/h1/source-fixtures.yaml
  identity_mappings: fixtures/h1/identity-mappings.yaml
  permission_matrix: fixtures/h1/permission-matrix.yaml
  oracle: fixtures/h1/ground-truth-oracle.yaml
artifact_digests:
  algorithm: sha256-canonical-json
  source_fixtures: b3d8f648d7bd2d0ed62ab5d803fa25a6aff32a2e0f11ce0f3bddebafc869dd96
  identity_mappings: 61b3729a92713a066df550670f34988fc0817eec84523d0e5e2bfacac0458ab7
  permission_matrix: 9373ff3fc790dd955275a4e8f457e2b41bb6924e4e92c2918de5aac46e5d15a1
  oracle: eba5ed65cf411a137964943d1403144013160e1c53aeb7f2ac72d5fae5fb1a71
tenants:
  - tenant_alias: tnt_aster
    tenant_id: 10000000-0000-4000-8000-000000000001
    display_name: Aster Labs
    jira_installation_id: 30000000-0000-4000-8000-000000000001
    github_installation_id: 30000000-0000-4000-8000-000000000002
    jira_allowlist: [AST, SEC, OPS, PROD]
    github_allowlist:
      - aster-labs/identity-service
      - aster-labs/orion-release
      - aster-labs/platform
      - aster-labs/web
      - aster-labs/mobile
      - aster-labs/billing
      - aster-labs/data
      - aster-labs/infra
      - aster-labs/security
      - aster-labs/docs
      - aster-labs/observability
      - aster-labs/integrations
    frozen_counts:
      human_identities: 48
      teams: 7
      github_organizations: 1
      github_repositories: 12
      github_pull_requests: 420
      github_reviews: 690
      github_issue_and_pr_links: 206
      jira_projects: 4
      jira_issues: 240
      jira_issue_links: 318
      jira_comments: 560
      release_milestones: 8
  - tenant_alias: tnt_beacon
    tenant_id: 10000000-0000-4000-8000-000000000002
    display_name: Beacon Works
    jira_installation_id: 30000000-0000-4000-8000-000000000003
    github_installation_id: 30000000-0000-4000-8000-000000000004
    jira_allowlist: [BCN, BSEC, BOPS]
    github_allowlist:
      - beacon-works/platform
      - beacon-works/identity
      - beacon-works/product
      - beacon-works/mobile
      - beacon-works/data
      - beacon-works/infra
      - beacon-works/security
      - beacon-works/docs
    frozen_counts:
      human_identities: 32
      teams: 5
      github_organizations: 1
      github_repositories: 8
      github_pull_requests: 260
      github_reviews: 410
      github_issue_and_pr_links: 118
      jira_projects: 3
      jira_issues: 160
      jira_issue_links: 204
      jira_comments: 320
      release_milestones: 5
scale_profile:
  generated_graph_nodes_per_tenant: 100000
  generated_graph_edges_per_tenant: 1000000
  maximum_concurrent_users: 10
```

```
  rule: Filler nodes and edges are tenant-local deterministic non-critical components and cannot change the
decision-chain oracle.
```

Source Fixtures

Source: `docs/enterprise-digital-twin/fixtures/h1/source-fixtures.yaml` | SHA-256: `759d89cf8a059b3379ba0522f9090efe4d0bedd3ac3e2fa6d880143ce194275a`

```
schema_version: 1
source_objects:
  - source_object_id: aster-jira-AST-142-v7
    tenant_id: 10000000-0000-4000-8000-000000000001
    installation_id: 30000000-0000-4000-8000-000000000001
    provider: jira
    source_key: AST-142
    source_revision: "7"
    acl_class: aster-orion-full
    observed_at: 2026-07-13T15:45:00Z
    fields:
      summary: Complete SSO cutover
      status: In Progress
      duedate: 2026-08-07
      priority: {id: "3", name: Medium}
      labels: [identity, orion]
      remaining_duration: {optimistic: 8, most_likely: 12, pessimistic: 18, unit: workday, source: explicit}
      team_key: aster-identity
  - source_object_id: aster-github-identity-service-pr-184
    tenant_id: 10000000-0000-4000-8000-000000000001
    installation_id: 30000000-0000-4000-8000-000000000002
    provider: github
    source_key: aster-labs/identity-service#184
    source_revision: sha256:pr184-revision-12
    acl_class: aster-identity-restricted
    observed_at: 2026-07-13T15:47:00Z
    fields:
      title: Finalize token migration
      state: open
      required_security_reviews: 1
      received_security_reviews: 0
      linked_issue: AST-142
  - source_object_id: aster-jira-AST-173-v4
    tenant_id: 10000000-0000-4000-8000-000000000001
    installation_id: 30000000-0000-4000-8000-000000000001
    provider: jira
    source_key: AST-173
    source_revision: "4"
    acl_class: aster-orion-full
    observed_at: 2026-07-13T15:46:00Z
    fields:
      summary: Build Orion release candidate
      status: Open
      blocked_by: AST-142
      remaining_duration: {optimistic: 5, most_likely: 8, pessimistic: 13, unit: workday, source: explicit}
      team_key: aster-release
  - source_object_id: aster-jira-AST-201-v3
    tenant_id: 10000000-0000-4000-8000-000000000001
    installation_id: 30000000-0000-4000-8000-000000000001
    provider: jira
    source_key: AST-201
    source_revision: "3"
    acl_class: aster-orion-full
    observed_at: 2026-07-13T15:46:30Z
    fields:
      summary: Complete launch certification
      status: Open
      blocked_by: AST-173
      gates_milestone: Orion 2.0 General Availability
      remaining_duration: {optimistic: 4, most_likely: 7, pessimistic: 12, unit: workday, source: explicit}
      team_key: aster-security
  - source_object_id: aster-risk-secondary-1
    tenant_id: 10000000-0000-4000-8000-000000000001
    installation_id: 30000000-0000-4000-8000-000000000001
    provider: jira
    source_key: OPS-61
    source_revision: "2"
    acl_class: aster-orion-full
    observed_at: 2026-07-13T15:42:00Z
    fields: {summary: Validate regional capacity reservation, confidence: 0.58, on_p80_critical_path: false}
  - source_object_id: aster-risk-secondary-2
    tenant_id: 10000000-0000-4000-8000-000000000001
    installation_id: 30000000-0000-4000-8000-000000000001
    provider: jira
    source_key: PROD-88
```

```
    source_revision: "5"
    acl_class: aster-orion-full
    observed_at: 2026-07-13T15:43:00Z
    fields: {summary: Finalize launch documentation, confidence: 0.51, on_p80_critical_path: false}
  - source_object_id: beacon-jira-BCN-142-v9
    tenant_id: 10000000-0000-4000-8000-000000000002
    installation_id: 30000000-0000-4000-8000-000000000003
    provider: jira
    source_key: BCN-142
    source_revision: "9"
    acl_class: beacon-private
    observed_at: 2026-07-13T15:44:00Z
    fields:
      summary: Complete SSO cutover
      description: BEACON-CANARY-7Q9K
      status: In Progress
  - source_object_id: beacon-github-platform-pr-184
    tenant_id: 10000000-0000-4000-8000-000000000002
    installation_id: 30000000-0000-4000-8000-000000000004
    provider: github
    source_key: beacon-works/platform#184
    source_revision: sha256:beacon-pr184-revision-4
    acl_class: beacon-private
    observed_at: 2026-07-13T15:44:30Z
    fields: {title: Finalize token migration, state: open}
relationships:
  - {tenant_id: 10000000-0000-4000-8000-000000000001, source_relationship_id: 75000000-0000-4000-8000-
000000000001, type: IMPLEMENTS, from: aster-labs/identity-service#184, to: AST-142}
  - {tenant_id: 10000000-0000-4000-8000-000000000001, source_relationship_id: 75000000-0000-4000-8000-
000000000002, type: BLOCKS, from: AST-142, to: AST-173}
  - {tenant_id: 10000000-0000-4000-8000-000000000001, source_relationship_id: 75000000-0000-4000-8000-
000000000003, type: BLOCKS, from: AST-173, to: AST-201}
  - {tenant_id: 10000000-0000-4000-8000-000000000001, source_relationship_id: 75000000-0000-4000-8000-
000000000004, type: BLOCKS, from: AST-201, to: Orion 2.0 General Availability}
```

Identity Mappings

Source: `docs/enterprise-digital-twin/fixtures/h1/identity-mappings.yaml` | SHA-256: `c9d6360355372d17a866aa2dcfa30f4043e95fb2636af9d54a4ffb19780beacf`

```
schema_version: 1
actors:
  - {actor_alias: usr_aster_analyst, actor_id: 20000000-0000-4000-8000-000000000001, tenant_id: 10000000-0000-
4000-8000-000000000001, principal_ref: idp:aster:analyst}
  - {actor_alias: usr_aster_limited, actor_id: 20000000-0000-4000-8000-000000000002, tenant_id: 10000000-0000-
4000-8000-000000000001, principal_ref: idp:aster:limited}
  - {actor_alias: usr_aster_ops_approver, actor_id: 20000000-0000-4000-8000-000000000003, tenant_id: 10000000-
0000-4000-8000-000000000001, principal_ref: idp:aster:ops-approver}
  - {actor_alias: usr_aster_security_approver, actor_id: 20000000-0000-4000-8000-000000000004, tenant_id:
10000000-0000-4000-8000-000000000001, principal_ref: idp:aster:security-approver}
  - {actor_alias: usr_aster_admin, actor_id: 20000000-0000-4000-8000-000000000005, tenant_id: 10000000-0000-
4000-8000-000000000001, principal_ref: idp:aster:admin}
  - {actor_alias: usr_beacon_analyst, actor_id: 20000000-0000-4000-8000-000000000006, tenant_id: 10000000-0000-
4000-8000-000000000002, principal_ref: idp:beacon:analyst}
  - {actor_alias: usr_platform_operator, actor_id: 20000000-0000-4000-8000-000000000007, tenant_id: 10000000-
0000-4000-8000-000000000001, principal_ref: idp:platform:operator}
generated_people:
  - tenant_id: 10000000-0000-4000-8000-000000000001
    source_range: aster-person-001..aster-person-048
    count: 48
    stable_uuid_rule: uuidv5(urn:edt:h1:tnt_aster:person:{zero_padded_index})
  - tenant_id: 10000000-0000-4000-8000-000000000002
    source_range: beacon-person-001..beacon-person-032
    count: 32
    stable_uuid_rule: uuidv5(urn:edt:h1:tnt_beacon:person:{zero_padded_index})
resolution_oracle:
  - mapping_key: aster-jordan-lee
    tenant_id: 10000000-0000-4000-8000-000000000001
    source_aliases: [github:aster:jordan-lee, jira:aster:account-1042]
    expected_outcome: merge
    canonical_person_key: aster-person-017
  - mapping_key: beacon-jordan-lee
    tenant_id: 10000000-0000-4000-8000-000000000002
    source_aliases: [github:beacon:jordan-lee, jira:beacon:account-773]
    expected_outcome: merge
    canonical_person_key: beacon-person-009
  - mapping_key: cross-tenant-jordan-lee
    source_aliases: [github:aster:jordan-lee, github:beacon:jordan-lee]
    expected_outcome: do_not_merge
    reason: Cross-tenant resolution is prohibited even when display names match.
reversibility_case:
  source_aliases: [github:aster:sam-rivera, jira:aster:account-1188]
  initial_outcome: merge
```

```
  corrected_outcome: split
  expected_preservation: [both_source_aliases, all_evidence, version_history, unrelated_edges]
```

Permission Matrix

Source: `docs/enterprise-digital-twin/fixtures/h1/permission-matrix.yaml` | SHA-256: `17d6a84b038f29bbce0b34c8b399d48e4646ede61f2de901adccbde7dbf71584`

```
schema_version: 1
acl_classes:
  aster-orion-full:
    allowed_actors: [usr_aster_analyst, usr_aster_ops_approver, usr_aster_security_approver, usr_aster_admin]
    denied_actors: [usr_aster_limited, usr_beacon_analyst, usr_platform_operator]
  aster-identity-restricted:
    allowed_actors: [usr_aster_analyst, usr_aster_ops_approver, usr_aster_security_approver, usr_aster_admin]
    denied_actors: [usr_aster_limited, usr_beacon_analyst, usr_platform_operator]
  beacon-private:
    allowed_actors: [usr_beacon_analyst]
    denied_actors: [usr_aster_analyst, usr_aster_limited, usr_aster_ops_approver, usr_aster_security_approver,
usr_aster_admin, usr_platform_operator]
roles:
  usr_aster_analyst:
    grants: [evidence.read.aster_orion, scenario.create, simulation.run, jira_remediation.preview, asset.read,
asset.control.preview, asset.control.execute]
    denials: [action.approve, action.execute, connector.admin]
  usr_aster_limited:
    grants: [evidence.read.aster_public]
    denials: [evidence.read.aster_identity_restricted, action.approve, action.execute]
  usr_aster_ops_approver:
    grants: [evidence.read.aster_orion, action.approve.operations]
    denials: [action.approve.security, action.approve.duplicate_slot, action.payload.modify]
  usr_aster_security_approver:
    grants: [evidence.read.aster_orion, action.approve.security]
    denials: [action.approve.operations, action.approve.duplicate_slot, action.payload.modify]
  usr_aster_admin:
    grants: [connector.admin, policy.admin, fixture.reset]
    denials: [action.approve.unless_separately_granted]
  usr_beacon_analyst:
    grants: [evidence.read.beacon, scenario.create, simulation.run, asset.read]
    denials: [tenant.aster.any, action.execute]
  usr_platform_operator:
    grants: [service_health.read, telemetry.redacted.read]
    denials: [tenant_content.read, source_payload.read, prompt.read, action_payload.read]
expected_denial_semantics:
  existence_disclosure: indistinguishable_not_found
  audit: tenant-safe reason code with no resource label
  cache: deny barrier before cache lookup
  graph: current policy recheck before serialization
  agent: inaccessible evidence absent from context citations memory and trace
```

Ground Truth Oracle

Source: `docs/enterprise-digital-twin/fixtures/h1/ground-truth-oracle.yaml` | SHA-256: `7eeca4215fdd5135ee726ca1155255b02afc62a9d6014dbe17c57854e9438b55`

```
schema_version: 1
oracle_version: 1.0.0
fixture_version: 1.0.0
ingestion:
  duplicate_and_reorder_result: identical_canonical_state_digest
  cursor_replay_external_effect_count: 0
  missed_webhook_recovery: reconciliation_restores_expected_state
  tombstone_behavior: source_deletion_closes_claim_and_removes_projection_without_erasing_history
identity_resolution:
  expected_merges: [aster-jordan-lee, beacon-jordan-lee]
  expected_non_merges: [cross-tenant-jordan-lee]
  reversible_case: split_preserves_aliases_evidence_history_and_unrelated_edges
graph:
  required_path: [AST-142, AST-173, AST-201, Orion 2.0 General Availability]
  required_evidence_link: [aster-labs/identity-service#184, AST-142]
  scheduler_path: [AST-142, AST-173, AST-201, Orion 2.0 General Availability]
  forbidden_cross_tenant_path: [AST-142, BCN-142]
  rebuild_result: identical_authorized_projection_digest
cited_answer:
  question: What is most likely to delay Orion 2.0 what evidence supports that conclusion and what information
is still missing?
  strongest_blocker: AST-142
  required_claims:
    - AST-142 blocks AST-173 which blocks AST-201 and Orion 2.0 General Availability.
    - aster-labs/identity-service#184 implements AST-142 and is missing one required security review.
```

```
  required_citation_source_objects: [aster-jira-AST-142-v7, aster-jira-AST-173-v4, aster-jira-AST-201-v3, aster-
github-identity-service-pr-184]
  required_secondary_risks: [OPS-61, PROD-88]
  required_missing_data:
    - future security review completion time
    - unrecorded work
    - external validity of synthetic duration distributions
  prohibited_claims:
    - individual productivity inference
    - causal or production predictive validity
  restricted_actor_behavior: usr_aster_limited_abstains_on_identity_review_claim_and_emits_no_restricted_locator
simulation:
  snapshot_work_item_id: 116ab4b3-b108-5f91-ab7e-111f7fba1d45
  model_version: pert-monte-carlo/1.0.0
  seed: "20260713"
  sample_count: 50000
  intervention: {type: shift_completion_distribution, work_item_id: 116ab4b3-b108-5f91-ab7e-111f7fba1d45,
delta_workdays: -5}
  baseline: {p50: 2026-08-20, p80: 2026-08-24, p95: 2026-08-27}
  scenario: {p50: 2026-08-13, p80: 2026-08-17, p95: 2026-08-20}
  tolerance: {quantile_calendar_days: 1, probability_percentage_points: 0.5}
  reproducibility: identical_canonical_result_for_same_snapshot_scenario_engine_calendar_and_seed
action:
  requester_actor_id: 20000000-0000-4000-8000-000000000001
  operations_approver_actor_id: 20000000-0000-4000-8000-000000000003
  security_approver_actor_id: 20000000-0000-4000-8000-000000000004
  approval_ttl_seconds: 900
  exact_internal_payload:
    action: jira.issue.update
    connectorInstallationId: 30000000-0000-4000-8000-000000000001
    expectedIssueVersion: 7
    issueKey: AST-142
    projectKey: AST
    set:
      duedate: 2026-07-31
      labels: [digital-twin-remediation, identity, orion]
      priorityId: "2"
    tenantId: 10000000-0000-4000-8000-000000000001
  before: {version: 7, duedate: 2026-08-07, priorityId: "3", labels: [identity, orion]}
  expected_after: {duedate: 2026-07-31, priorityId: "2", labels: [digital-twin-remediation, identity, orion]}
  concurrent_execution_jira_put_count: 1
  exact_replay: original_receipt_and_zero_additional_puts
  changed_payload: denied_and_zero_puts
  expired_or_duplicate_approver: denied_and_zero_puts
  ambiguous_timeout: verification_required_and_no_blind_retry
  rollback:
    expiry_seconds: 86400
    unchanged_after_state: restores_before_snapshot_once
    later_human_change: compensation_conflict_and_zero_overwrites
tenant_isolation:
  canary: BEACON-CANARY-7Q9K
  expected_aster_occurrences: 0
  checked_surfaces: [REST, GraphQL, MCP, graph, vector, cache, object, event, answer, citation, prompt, memory,
trace, log, metric, audit_detail, export, error]
```

Github Pr 184.Valid.Payload

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/github-pr-184.valid.payload.json` | SHA-256: `b7a11f1fe0f6ba0f82abd05f7a71d1d17f340e11c2da74047e318361a3763473`


```
{
  "id": 900184,
  "number": 184,
  "state": "open",
  "title": "Finalize token migration",
  "updated_at": "2026-07-13T15:47:00Z",
  "repository": "aster-labs/identity-service",
  "author": "aster-dev-17",
  "base_sha": "1111111111111111111111111111111111111111",
  "head_sha": "2222222222222222222222222222222222222222",
  "requested_reviewers": [],
  "requested_teams": ["aster-security"],
  "labels": ["orion", "security-review"],
  "draft": false,
  "linked_issue": "AST-142",
  "provider_additive_field": {"accepted_only_inside_untrusted_payload": true}
}
```

Github Pr 184.Invalid.Payload

Source: docs/enterprise-digital-twin/fixtures/h1/connectors/github-pr-184.invalid.payload.json | SHA-256: 814d28cee10184f611fb120c5fd07a27275b64c42d9825f7409d05f8bccbfbf4


```
{
  "id": 900184,
  "number": 184,
  "state": "open",
  "title": "Finalize token migration",
  "updated_at": "2026-07-13T15:47:00Z",
  "repository": "aster-labs/identity-service",
  "author": "aster-dev-17",
  "base_sha": "1111111111111111111111111111111111111111",
  "head_sha": "2222222222222222222222222222222222222222",
  "requested_reviewers": [],
  "requested_teams": ["aster-security"],
  "access_token": "must-be-rejected"
}
```


Github Pr 184.Valid.Raw

Source: docs/enterprise-digital-twin/fixtures/h1/connectors/github-pr-184.valid.raw.json | SHA-256: bb01bc9a8a824f6167da02dda9ab05a2c32f3bd4d737ec1df4a5e6b152d967d3


```
{
  "schema_version": "1.0",
  "tenant_id": "10000000-0000-4000-8000-000000000001",
  "installation_id": "30000000-0000-4000-8000-000000000002",
  "provider": "github",
  "delivery_id": "41000000-0000-4000-8000-000000000184",
  "event_type": "pull_request",
  "external_object_id": "aster-labs/identity-service#184",
  "external_version": "sha256:cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc",
  "observed_at": "2026-07-13T15:47:00Z",
  "source_updated_at": "2026-07-13T15:47:00Z",
  "payload_sha256": "711b6b41953e9f7dfbd14b587ecbafcd5e0847b2a39f45320cbbebf5b6e109ae",
  "payload_object_uri": "s3://edt-fixtures/tenant/10000000-0000-4000-8000-000000000001/connector/30000000-0000-
4000-8000-000000000002/sha256/711b6b41953e9f7dfbd14b587ecbafcd5e0847b2a39f45320cbbebf5b6e109ae",
  "ingest_run_id": "42000000-0000-4000-8000-000000000184",
  "payload": {
    "id": 900184,
    "number": 184,
    "state": "open",
    "title": "Finalize token migration",
    "updated_at": "2026-07-13T15:47:00Z",
    "repository": "aster-labs/identity-service",
    "author": "aster-dev-17",
    "base_sha": "1111111111111111111111111111111111111111",
    "head_sha": "2222222222222222222222222222222222222222",
    "requested_reviewers": [],
    "requested_teams": ["aster-security"],
    "labels": ["orion", "security-review"],
    "draft": false,
    "linked_issue": "AST-142",
    "provider_additive_field": {"accepted_only_inside_untrusted_payload": true}
  }
}
```


Github Pr 184.Invalid.Raw

Source: docs/enterprise-digital-twin/fixtures/h1/connectors/github-pr-184.invalid.raw.json | SHA-256: 54dcc0ba5bb2bdab128ed4e7ca79685edbc82c0bbcf2d77042643d13618f5df0


```
{
  "schema_version": "1.0",
  "tenant_id": "10000000-0000-4000-8000-000000000001",
  "installation_id": "30000000-0000-4000-8000-000000000002",
  "provider": "github",
  "delivery_id": "41000000-0000-4000-8000-000000000184",
  "event_type": "pull_request",
  "external_object_id": "aster-labs/identity-service#184",
  "external_version": "sha256:cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc",
  "observed_at": "2026-07-13T15:47:00Z",
  "source_updated_at": "2026-07-13T15:47:00Z",
  "payload_sha256": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
  "payload_object_uri": "s3://edt-fixtures/tenant/10000000-0000-4000-8000-000000000001/connector/30000000-0000-
4000-8000-000000000002/sha256/aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
  "ingest_run_id": "42000000-0000-4000-8000-000000000184",
```

```
  "caller_tenant_id": "10000000-0000-4000-8000-000000000002",
  "payload": {
    "id": 900184,
    "number": 184,
    "state": "open",
    "title": "Finalize token migration",
    "updated_at": "2026-07-13T15:47:00Z",
    "repository": "aster-labs/identity-service",
    "author": "aster-dev-17",
    "base_sha": "1111111111111111111111111111111111111111",
    "head_sha": "2222222222222222222222222222222222222222",
    "requested_reviewers": [],
    "requested_teams": ["aster-security"]
  }
}
```


Github Pr 184.Valid.Observation

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/github-pr-184.valid.observation.json` | SHA-256: `bdac9b8a96abfcb0ade6312674cee5cc147cd73be04bbbd7f02df7ad45a24674`


```
{
  "schema_version": "1.0",
  "observation_id": "43000000-0000-4000-8000-000000000184",
  "tenant_id": "10000000-0000-4000-8000-000000000001",
  "installation_id": "30000000-0000-4000-8000-000000000002",
  "provider": "github",
  "source_key": "aster-labs/identity-service#184",
  "source_revision": "sha256:ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc",
  "ontology_candidate_type": "edt.engineering.PullRequest",
  "attributes": [
    {"predicate": "edt.engineering.title", "value": "Finalize token migration", "value_type": "string",
"classification": "confidential", "authority": "source"},
    {"predicate": "edt.engineering.state", "value": "open", "value_type": "string", "classification":
"internal", "authority": "source"},
    {"predicate": "edt.engineering.updated_at", "value": "2026-07-13T15:47:00Z", "value_type": "date-time",
"classification": "internal", "authority": "source"}
  ],
  "relationships": [
    {"predicate": "edt.engineering.implements", "target_source_key": "AST-142", "classification":
"confidential", "confidence": 1.0}
  ],
  "source_acl": {"acl_class": "aster-identity-restricted", "visibility": "repository", "principal_ids": [
"20000000-0000-4000-8000-000000000001"], "permission_revision": "github-installation-aster-v12"},
  "effective_time": {"valid_from": "2026-07-13T15:47:00Z", "valid_to": null},
  "source_locator": {"application_path": "/evidence/44000000-0000-4000-8000-000000000184", "provider_url":
"https://github.com/aster-labs/identity-service/pull/184"},
  "parser_version": "1.0.0",
  "mapping_id": "github.pull-request.h1",
  "mapping_version": "1.0.0",
  "raw_payload_sha256": "711b6b41953e9f7dfbd14b587ecbafcd5e0847b2a39f45320cbbebf5b6e109ae",
  "warnings": [],
  "status": "active"
}
```


Github Pr.Mapping

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/github-pr.mapping.json` | SHA-256: `21dfdea128e71570688a2c68d4029d1e75ea0c2a1790ca696d11edd53774a756`


```
{
  "schema_version": "1.0",
  "mapping_id": "github.pull-request.h1",
  "version": "1.0.0",
  "provider": "github",
  "object_type": "pull_request",
  "input_schema_id": "https://enterprise-digital-twin.example/connectors/github/1.0.0/raw-envelope.schema.json",
  "output_schema_id": "https://enterprise-digital-twin.example/connectors/github/1.0.0/normalized-observation.
schema.json",
  "source_key": {"source_pointer": "/external_object_id", "transform": "github_pr_key"},
  "source_revision": {"source_pointer": "/external_version", "transform": "identity"},
  "ontology_type": "edt.engineering.PullRequest",
  "attributes": [
    {"source_pointer": "/payload/title", "predicate": "edt.engineering.title", "transform": "identity",
"classification": "confidential", "authority": "source", "required": true},
    {"source_pointer": "/payload/state", "predicate": "edt.engineering.state", "transform": "lowercase",
"classification": "internal", "authority": "source", "required": true},
    {"source_pointer": "/payload/updated_at", "predicate": "edt.engineering.updated_at", "transform": "rfc3339",
 "classification": "internal", "authority": "source", "required": true}
  ],
```

```
  "relationships": [
    {"source_pointer": "/payload/linked_issue", "predicate": "edt.engineering.implements", "target_transform":
"jira_key", "classification": "confidential", "confidence": 1.0}
  ],
  "acl": {"behavior": "source_visibility_intersection", "visibility_source": "installation_repository_acl",
"principal_source": "server_resolved_installation_permissions", "default_classification": "confidential"},
  "effective_time": {"valid_from_pointer": "/payload/updated_at", "valid_to": null},
  "source_locator": {"application_route": "/evidence/{evidence_id}", "provider_template": "https://github.com/{
repository}/pull/{number}"},
  "excluded_source_pointers": ["/payload/body", "/payload/patch", "/payload/diff", "/payload/commits_url", "/
payload/comments_url"],
  "canonicalization": "RFC8785_UTF8_LF_SORTED_ARRAYS_WHERE_DECLARED"
}
```


Jira Ast 142.Valid.Payload

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/jira-ast-142.valid.payload.json` | SHA-256: `39c179882a752eb49f499a38907c9081a0b9888dd16e667d89f406d326837685`


```
{
  "id": "10142",
  "key": "AST-142",
  "fields": {
    "summary": "Complete SSO cutover",
    "status": {"id": "3", "name": "In Progress"},
    "priority": {"id": "3", "name": "Medium"},
    "labels": ["identity", "orion"],
    "duedate": "2026-08-07",
    "updated": "2026-07-13T15:45:00Z",
    "project": {"id": "10000", "key": "AST"},
    "issuetype": {"id": "10001", "name": "Task"},
    "issuelinks": [],
    "description": {"type": "doc", "version": 1, "content": []},
    "provider_additive_field": "accepted-only-inside-untrusted-payload"
  }
}
```


Jira Ast 142.Invalid.Payload

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/jira-ast-142.invalid.payload.json` | SHA-256: `367cffc7dd62b5b08e668f91aa4b3b4654eabbb3d8e91d86a30d0e823cc2d9b7`


```
{
  "id": "10142",
  "key": "AST-142",
  "refresh_token": "must-be-rejected",
  "fields": {
    "summary": "Complete SSO cutover",
    "status": {"id": "3", "name": "In Progress"},
    "priority": {"id": "3", "name": "Medium"},
    "labels": ["identity", "orion"],
    "duedate": "2026-08-07",
    "updated": "2026-07-13T15:45:00Z",
    "project": {"id": "10000", "key": "AST"},
    "issuetype": {"id": "10001", "name": "Task"},
    "issuelinks": []
  }
}
```


Jira Ast 142.Valid.Raw

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/jira-ast-142.valid.raw.json` | SHA-256: `3ff8c7bcec3f61750b5d628545fcdb2626bd3ea6ad16b6072e757e95fae6dbec`


```
{
  "schema_version": "1.0",
  "tenant_id": "10000000-0000-4000-8000-000000000001",
  "installation_id": "30000000-0000-4000-8000-000000000001",
  "provider": "jira",
  "delivery_id": "atlassian-delivery-ast-142-v7",
  "event_type": "jira:issue_updated",
  "external_object_id": "AST-142",
  "external_version": "7",
  "observed_at": "2026-07-13T15:45:00Z",
  "source_updated_at": "2026-07-13T15:45:00Z",
  "payload_sha256": "ba671758e02d1f8ccf68c8fa75e991545261dc9a5416d2ebab9ecc09dd2f661b",
  "payload_object_uri": "s3://edt-fixtures/tenant/10000000-0000-4000-8000-000000000001/connector/30000000-0000-
4000-8000-000000000001/sha256/ba671758e02d1f8ccf68c8fa75e991545261dc9a5416d2ebab9ecc09dd2f661b",
```

```
  "ingest_run_id": "52000000-0000-4000-8000-000000000142",
  "payload": {
    "id": "10142",
    "key": "AST-142",
    "fields": {
      "summary": "Complete SSO cutover",
      "status": {"id": "3", "name": "In Progress"},
      "priority": {"id": "3", "name": "Medium"},
      "labels": ["identity", "orion"],
      "duedate": "2026-08-07",
      "updated": "2026-07-13T15:45:00Z",
      "project": {"id": "10000", "key": "AST"},
      "issuetype": {"id": "10001", "name": "Task"},
      "issuelinks": [],
      "description": {"type": "doc", "version": 1, "content": []},
      "provider_additive_field": "accepted-only-inside-untrusted-payload"
    }
  }
}
```


Jira Ast 142.Invalid.Raw

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/jira-ast-142.invalid.raw.json` | SHA-256: `3ff794a8c5adae5e0117b99443cf41301e7154ca8491bbf485f771e565a1115f`


```
{
  "schema_version": "1.0",
  "tenant_id": "10000000-0000-4000-8000-000000000001",
  "installation_id": "30000000-0000-4000-8000-000000000001",
  "provider": "jira",
  "delivery_id": "atlassian-delivery-ast-142-v7",
  "event_type": "jira:issue_updated",
  "external_object_id": "AST-142",
  "external_version": "7",
  "observed_at": "2026-07-13T15:45:00Z",
  "source_updated_at": "2026-07-13T15:45:00Z",
  "payload_sha256": "bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
  "payload_object_uri": "s3://edt-fixtures/tenant/10000000-0000-4000-8000-000000000001/connector/30000000-0000-
4000-8000-000000000001/sha256/bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb",
  "ingest_run_id": "52000000-0000-4000-8000-000000000142",
  "caller_tenant_id": "10000000-0000-4000-8000-000000000002",
  "payload": {
    "id": "10142",
    "key": "AST-142",
    "fields": {
      "summary": "Complete SSO cutover",
      "status": {"id": "3", "name": "In Progress"},
      "priority": {"id": "3", "name": "Medium"},
      "labels": ["identity", "orion"],
      "duedate": "2026-08-07",
      "updated": "2026-07-13T15:45:00Z",
      "project": {"id": "10000", "key": "AST"},
      "issuetype": {"id": "10001", "name": "Task"},
      "issuelinks": []
    }
  }
}
```


Jira Ast 142.Valid.Observation

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/jira-ast-142.valid.observation.json` | SHA-256: `c992d0f13656dcfe9fef584ef2ed92e1058c02d0be6cab0f288cd5291d14d8b9`


```
{
  "schema_version": "1.0",
  "observation_id": "53000000-0000-4000-8000-000000000142",
  "tenant_id": "10000000-0000-4000-8000-000000000001",
  "installation_id": "30000000-0000-4000-8000-000000000001",
  "provider": "jira",
  "source_key": "AST-142",
  "source_revision": "7",
  "ontology_candidate_type": "edt.work.WorkItem",
  "attributes": [
    {"predicate": "edt.work.summary", "value": "Complete SSO cutover", "value_type": "string", "classification":
 "confidential", "authority": "source"},
    {"predicate": "edt.work.status", "value": "In Progress", "value_type": "string", "classification":
"internal", "authority": "source"},
    {"predicate": "edt.work.due_date", "value": "2026-08-07", "value_type": "date", "classification":
"confidential", "authority": "source"}
  ],
```

```
  "relationships": [
    {"predicate": "edt.work.belongsToProject", "target_source_key": "AST", "classification": "internal",
"confidence": 1.0}
  ],
  "source_acl": {"acl_class": "aster-orion-full", "visibility": "project", "principal_ids": ["20000000-0000-
4000-8000-000000000001"], "permission_revision": "jira-cloud-aster-v9"},
  "effective_time": {"valid_from": "2026-07-13T15:45:00Z", "valid_to": null},
  "source_locator": {"application_path": "/evidence/54000000-0000-4000-8000-000000000142", "provider_path": "/
browse/AST-142"},
  "parser_version": "1.0.0",
  "mapping_id": "jira.issue.h1",
  "mapping_version": "1.0.0",
  "raw_payload_sha256": "ba671758e02d1f8ccf68c8fa75e991545261dc9a5416d2ebab9ecc09dd2f661b",
  "warnings": [],
  "status": "active"
}
```


Jira Issue.Mapping

Source: `docs/enterprise-digital-twin/fixtures/h1/connectors/jira-issue.mapping.json` | SHA-256: `0407929e65b810f8c02747cb47816e9a082ee62c8f598b60a31653222235eda6`

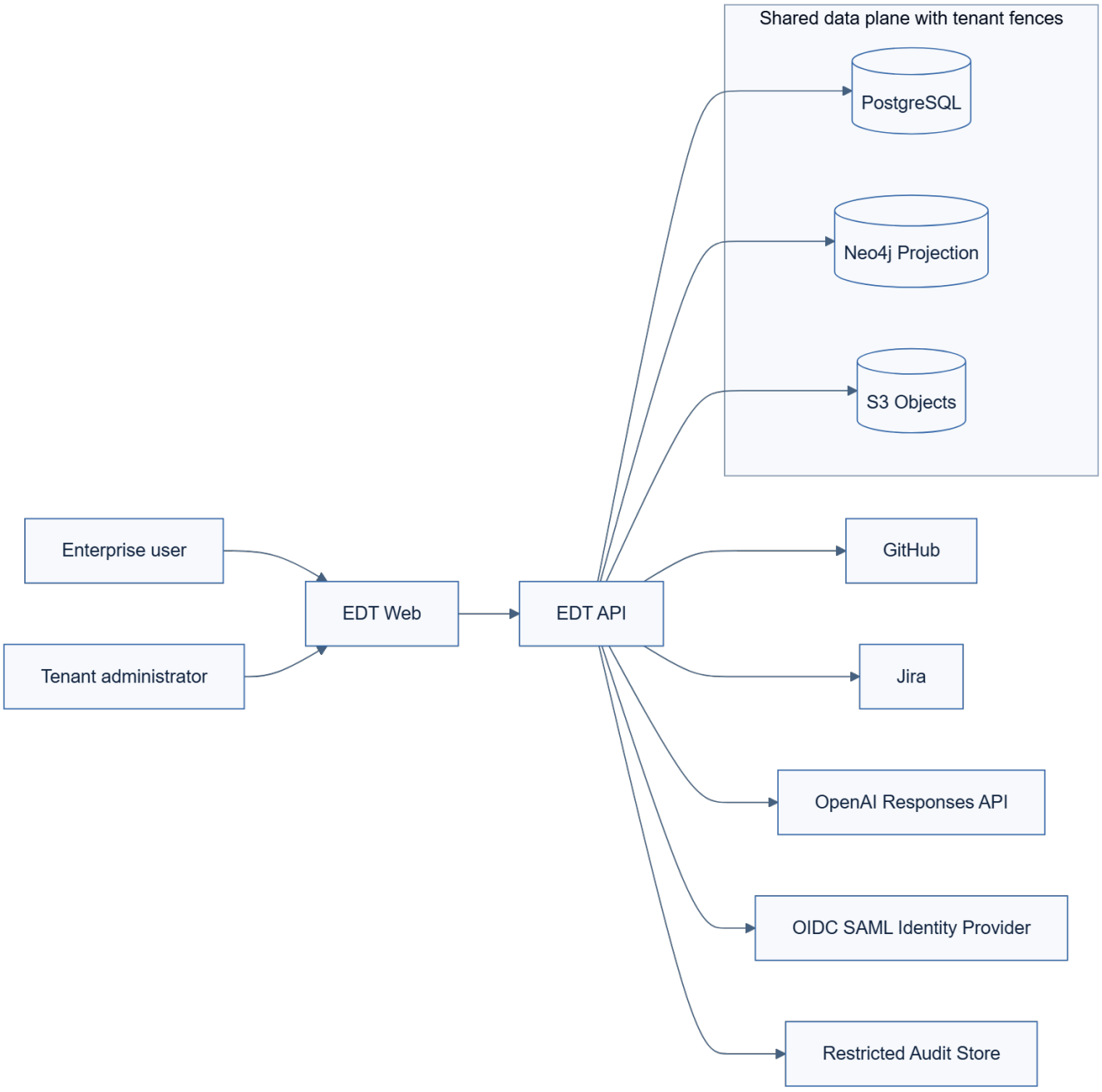

```
{
  "schema_version": "1.0",
  "mapping_id": "jira.issue.h1",
  "version": "1.0.0",
  "provider": "jira",
  "object_type": "issue",
  "input_schema_id": "https://enterprise-digital-twin.example/connectors/jira-cloud/1.0.0/raw-envelope.schema.
json",
  "output_schema_id": "https://enterprise-digital-twin.example/connectors/jira-cloud/1.0.0/normalized-
observation.schema.json",
  "source_key": {"source_pointer": "/payload/key", "transform": "jira_key"},
  "source_revision": {"source_pointer": "/external_version", "transform": "identity"},
  "ontology_type": "edt.work.WorkItem",
  "attributes": [
    {"source_pointer": "/payload/fields/summary", "predicate": "edt.work.summary", "transform": "identity",
"classification": "confidential", "authority": "source", "required": true},
    {"source_pointer": "/payload/fields/status/name", "predicate": "edt.work.status", "transform": "identity",
"classification": "internal", "authority": "source", "required": true},
    {"source_pointer": "/payload/fields/duedate", "predicate": "edt.work.due_date", "transform": "date",
"classification": "confidential", "authority": "source", "required": false}
  ],
  "relationships": [
    {"source_pointer": "/payload/fields/project/key", "predicate": "edt.work.belongstoproject",
"target_transform": "jira_project_key", "classification": "internal", "confidence": 1.0}
  ],
  "acl": {"behavior": "source_visibility_intersection", "visibility_source": "project_and_issue_security",
"principal_source": "server_resolved_jira_permissions", "default_classification": "confidential"},
  "effective_time": {"valid_from_pointer": "/payload/fields/updated", "valid_to": null},
  "source_locator": {"application_route": "/evidence/{evidence_id}", "provider_template": "/browse/{key}"},
  "excluded_source_pointers": ["/payload/fields/attachment/content", "/payload/fields/comment/body", "/payload/
fields/worklog", "/payload/fields/environment"],
  "canonicalization": "RFC8785_UTF8_LF_SORTED_ARRAYS_WHERE_DECLARED"
}
```

Part VI - Generated Architecture Diagrams

Every figure is rendered from the adjacent version-controlled Mermaid source. The SVG is the publication artifact and the PNG supports PDF generation.

System Context

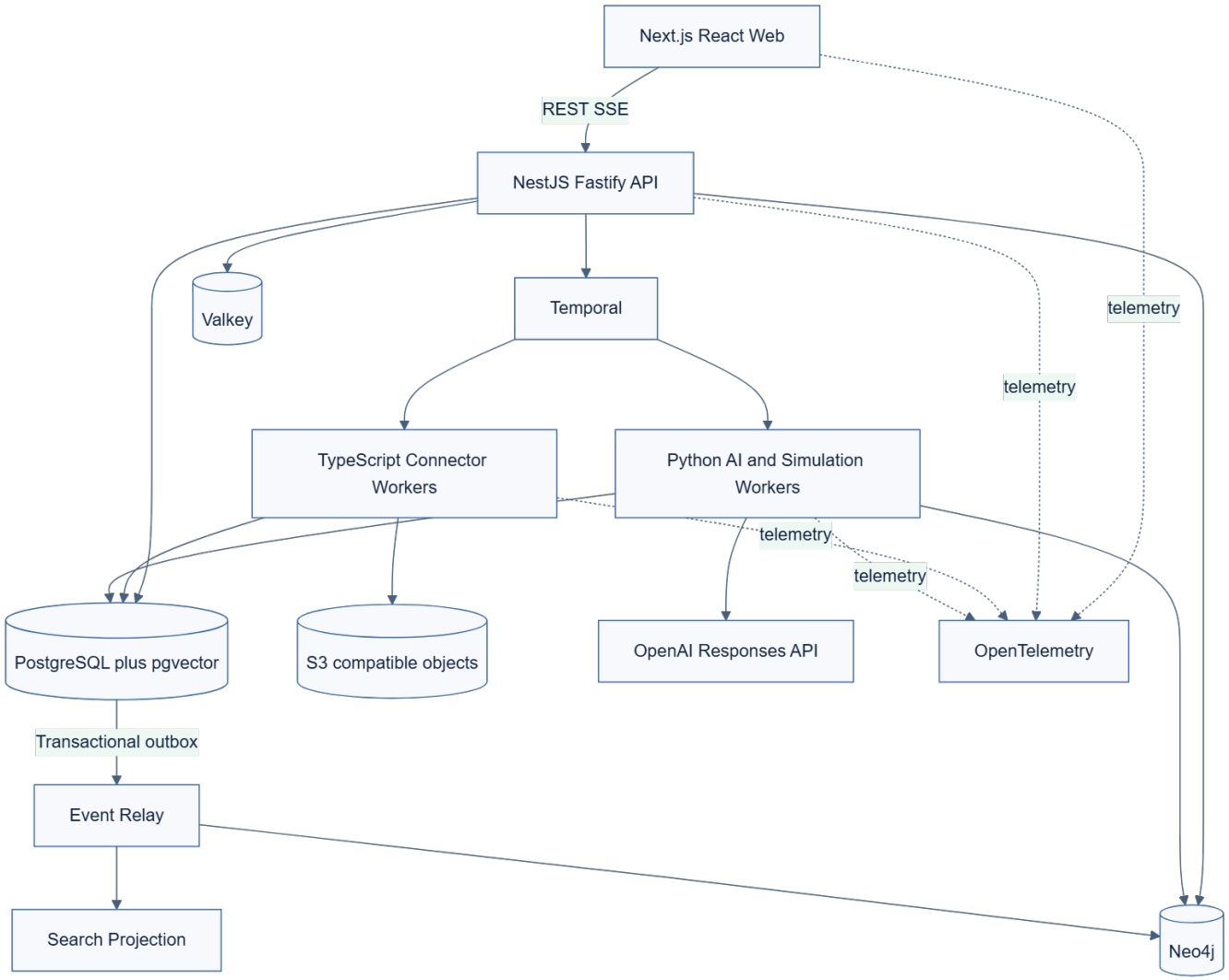
Source: `docs/enterprise-digital-twin/diagrams/01-system-context.mmd`



System Context

Container Architecture

Source: `docs/enterprise-digital-twin/diagrams/02-container-architecture.mmd`



Container Architecture

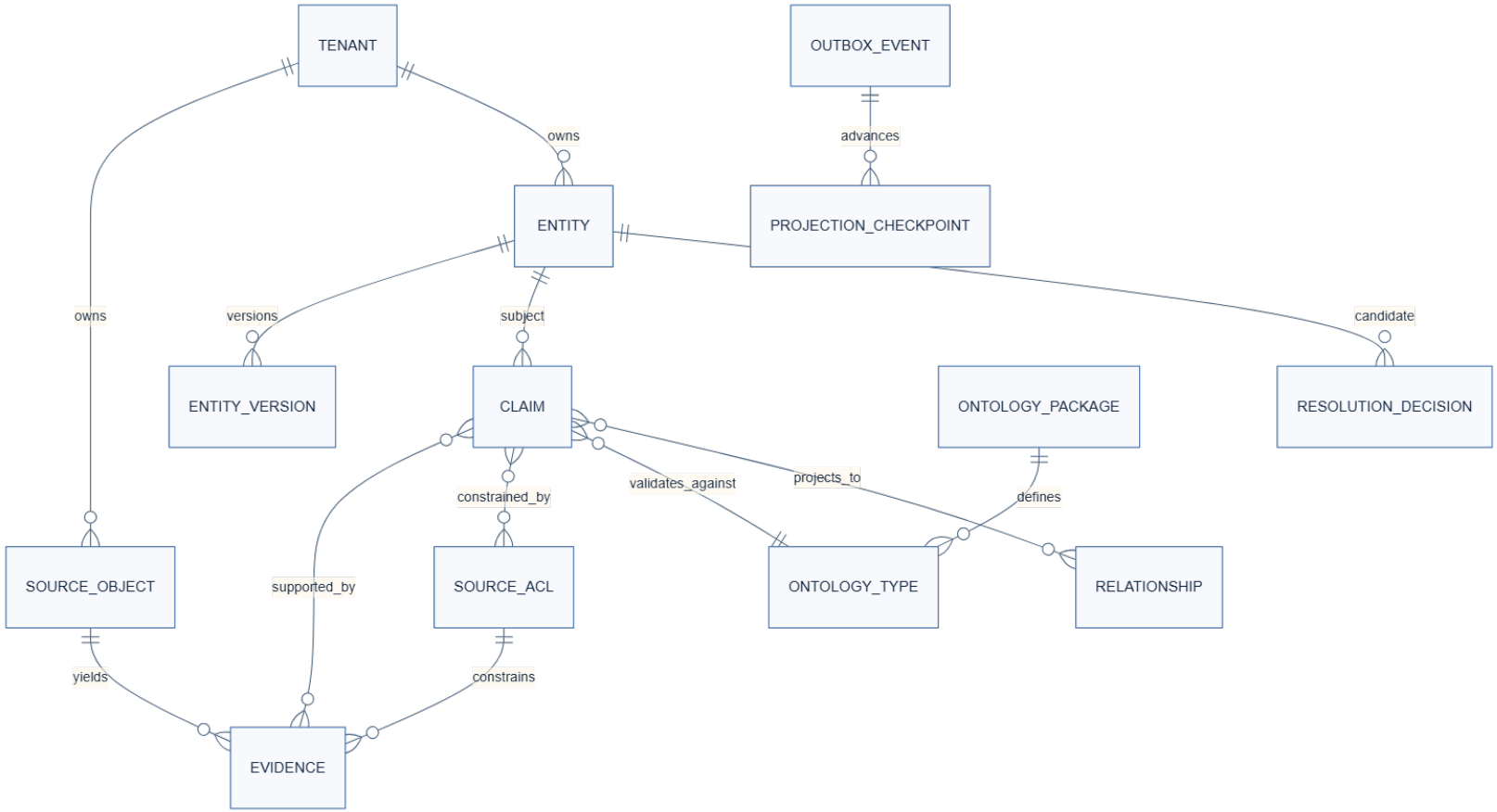
Trust Boundaries

Source: `docs/enterprise-digital-twin/diagrams/03-trust-boundaries.mmd`

Trust Boundaries

Claim Ontology Er

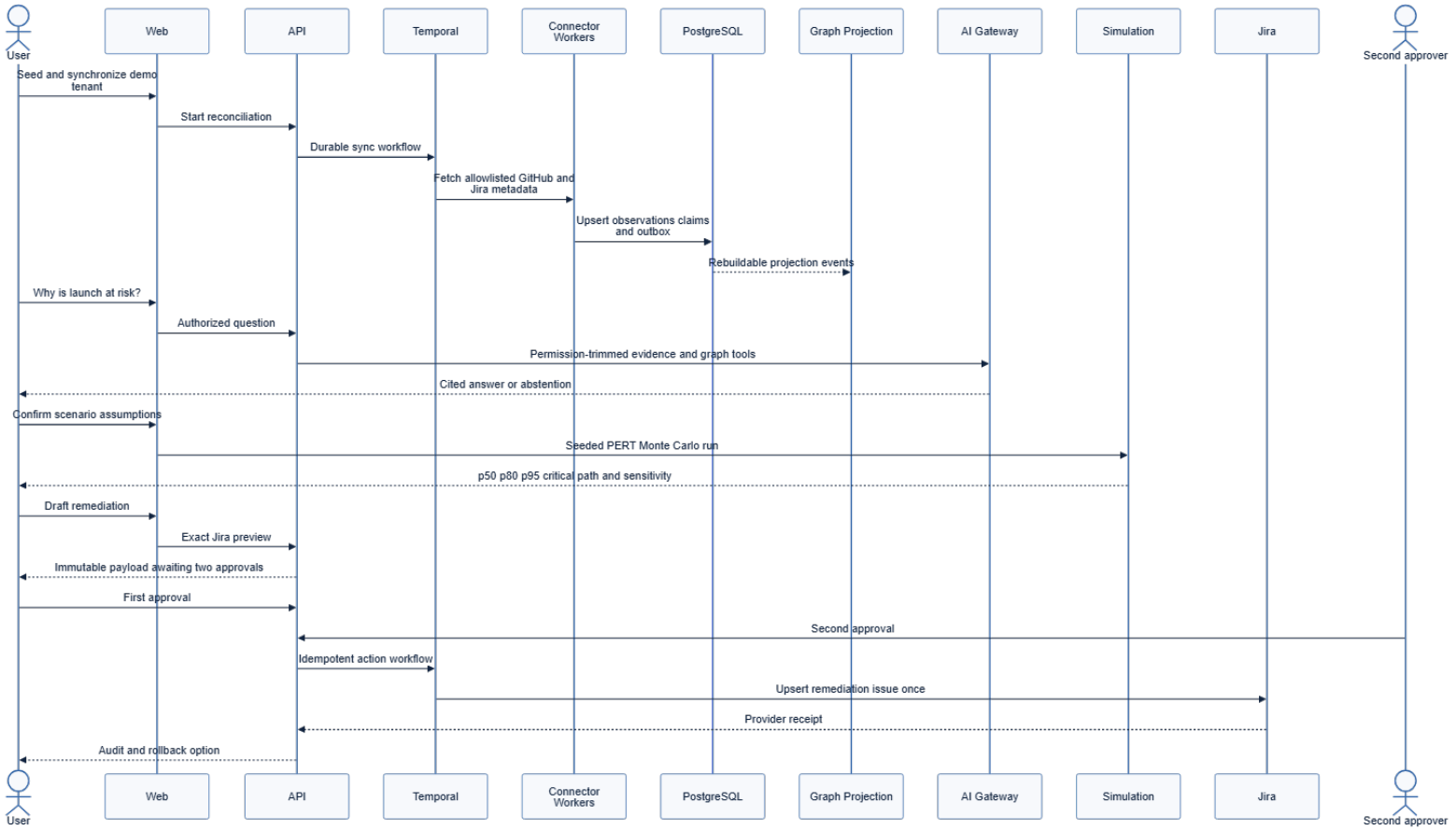
Source: `docs/enterprise-digital-twin/diagrams/04-claim-ontology-er.mmd`



Claim Ontology Er

Hackathon Sequence

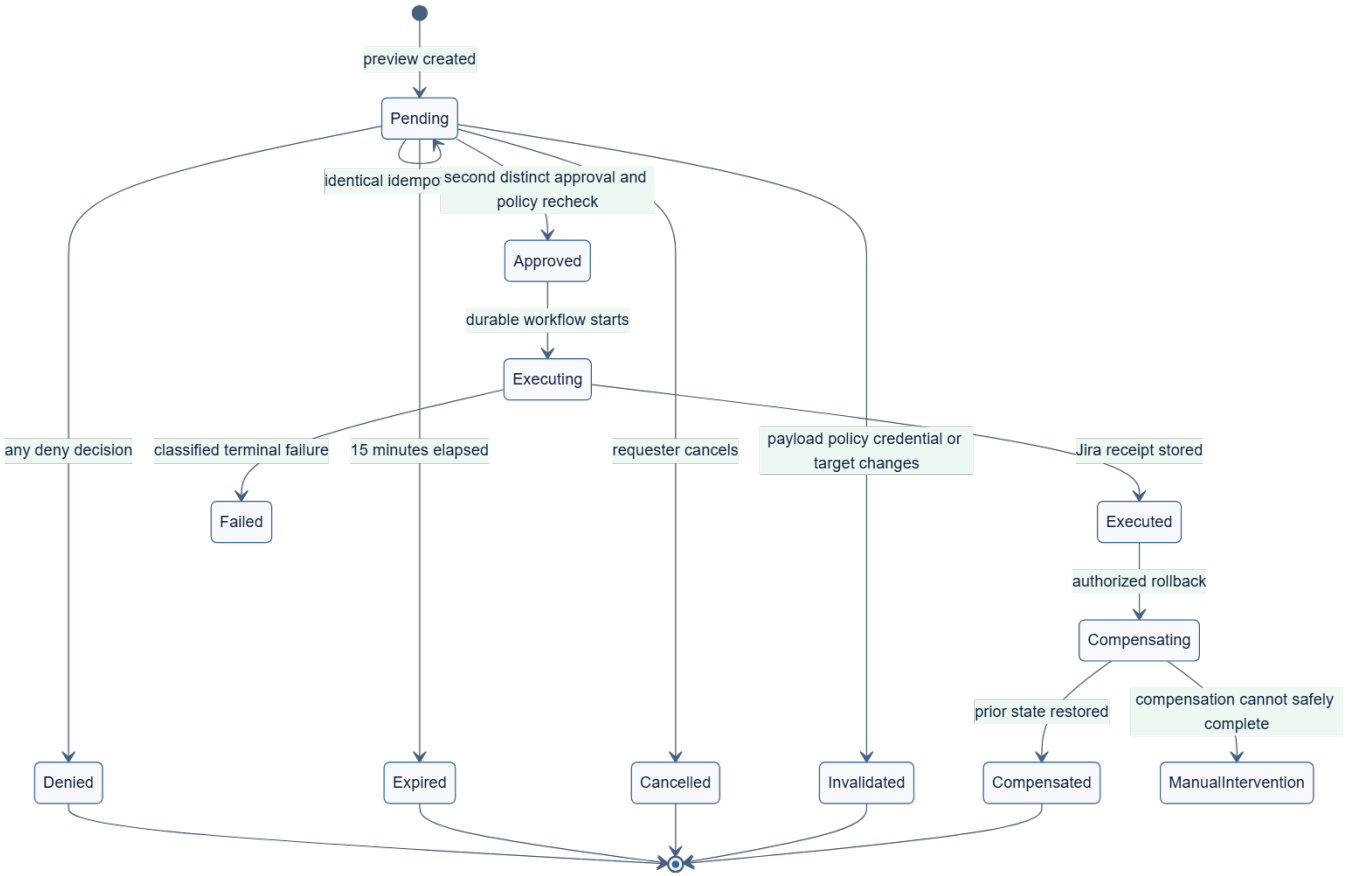
Source: `docs/enterprise-digital-twin/diagrams/05-hackathon-sequence.mmd`



Hackathon Sequence

Approval State

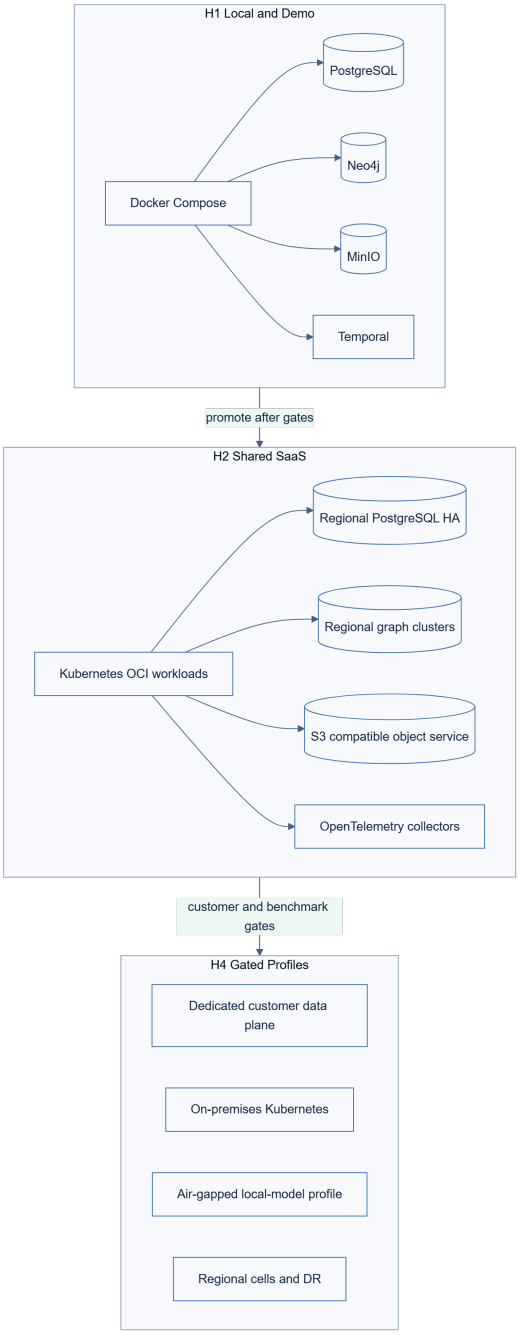
Source: `docs/enterprise-digital-twin/diagrams/06-approval-state.mmd`



Approval State

Deployment Profiles

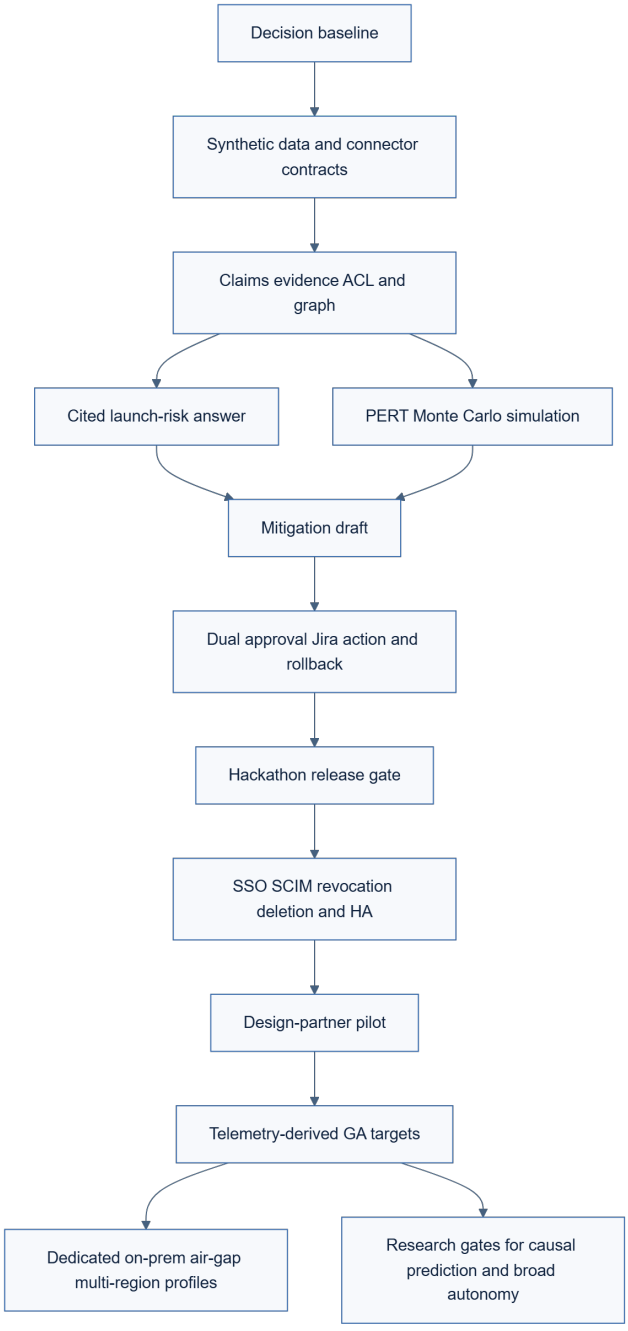
Source: `docs/enterprise-digital-twin/diagrams/07-deployment-profiles.mmd`



Deployment Profiles

Roadmap Dependencies

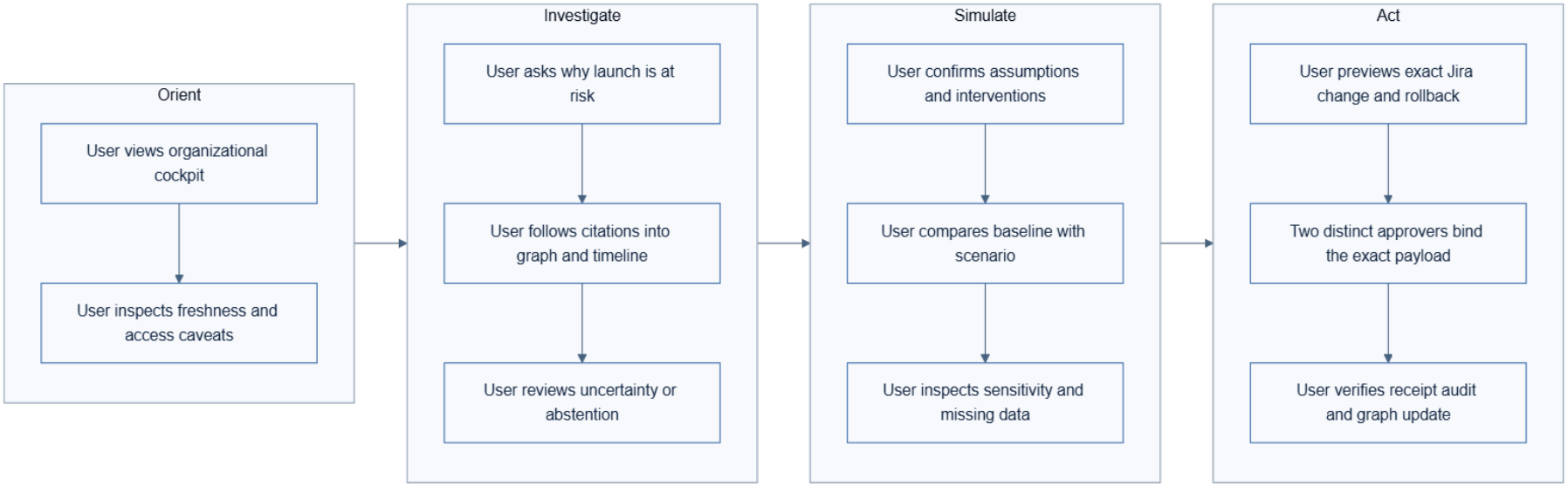
Source: `docs/enterprise-digital-twin/diagrams/08-roadmap-dependencies.mmd`



Roadmap Dependencies

Ux Journey

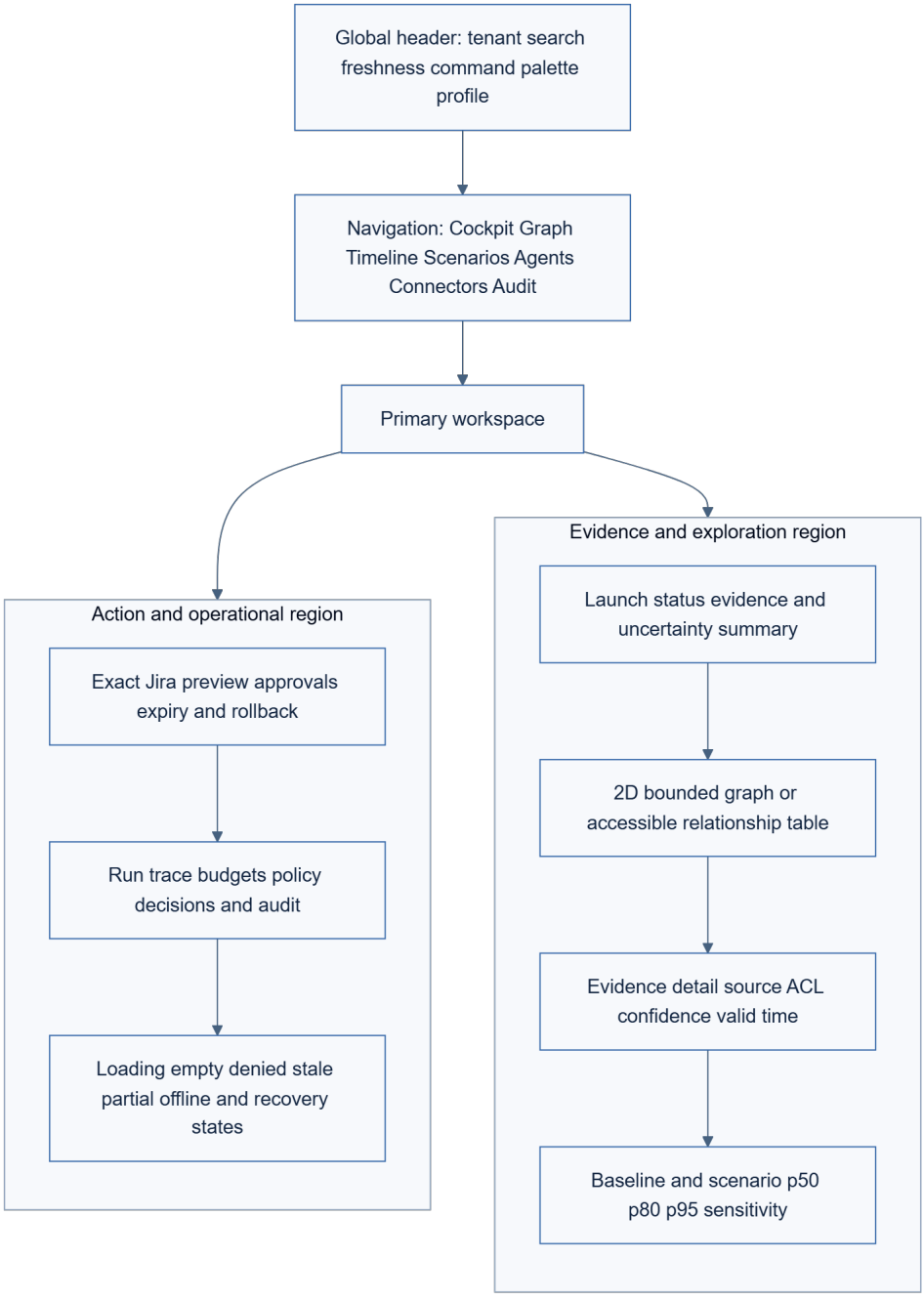
Source: `docs/enterprise-digital-twin/diagrams/09-ux-journey.mmd`



Ux Journey

Screen Wireframe

Source: `docs/enterprise-digital-twin/diagrams/10-screen-wireframe.mmd`



Screen Wireframe

Part VIII - Assurance Reviews

Control Readiness Mapping

Artifact: `docs/enterprise-digital-twin/reviews/compliance-mapping.md`

Control Readiness Mapping

This is a readiness and evidence map, not a certification or legal conclusion.

Capability	Blueprint controls	Evidence owner	H2 evidence
Access governance	CTRL-IAM-001 through CTRL-IAM-004, CTRL-TEN-001 through CTRL-TEN-003	Identity and Security	Identity configuration, access reviews, RLS tests, break-glass records
Data protection	CTRL-DAT-001 through CTRL-DAT-003, CTRL-CRY-001, CTRL-CRY-002	Data Governance	Inventory, classification, key rotation, deletion and restore evidence
Change and supply chain	CTRL-SUP-001	Developer Platform	Reviewed changes, SBOM, signatures, scans, provenance and deployment records
Availability and recovery	CTRL-OPS-001, CTRL-OPS-002	SRE	SLO reports, restore drills, incidents, capacity and failover tests
Monitoring and response	CTRL-AUD-001, CTRL-INC-001	Security Operations	Alert tests, audit verification, incident exercises and postmortems
Vendor and subprocessor governance	CTRL-PRV-001, CTRL-CON-001	Privacy and Procurement	DPA, subprocessor register, risk reviews, exit and deletion evidence
Privacy rights and minimization	CTRL-PRV-001, CTRL-PRV-002, CTRL-DAT-003	Privacy	Purpose records, DPIA screens, DSAR/export/delete tests, retention reports
AI governance	CTRL-AI-001 through CTRL-AI-005, CTRL-ACT-001 through CTRL-ACT-003	AI Governance	Model inventory, evaluations, prompt/tool versions, approvals, incidents and rollback tests

SOC 2 and ISO 27001 also require organizational policies, personnel controls, vendor governance, evidence operation, internal audit, and management oversight outside the software artifact. GDPR role, lawful basis, notice, transfer, retention, and rights decisions are tenant and jurisdiction specific.

Failure Mode and Effects Analysis

Artifact: `docs/enterprise-digital-twin/reviews/fmea.md`

Failure Mode and Effects Analysis

Scores use 1-5 severity, occurrence, and detectability. RPN is their product. Any severity 5 security or tenant-isolation failure blocks release regardless of RPN.

Failure mode	Effect	S	O	D	RPN	Detection and response
PostgreSQL unavailable	Authoritative reads and writes stop	5	2	1	10	Fail closed, readiness false, queue safe connector checkpoints, alert, restore or failover
Object store unavailable	New raw payload persistence stops	4	2	2	16	Do not acknowledge source event until durable; backpressure and retry

Failure mode	Effect	S	O	D	RPN	Detection and response
Neo4j unavailable	Graph exploration and graph-dependent answers unavailable	3	3	1	9	Preserve authoritative writes, mark graph stale, offer evidence search where safe, rebuild projection
Partial projection	Paths or counts omit new claims	4	3	2	24	Checkpoint lag, answer freshness labels, bounded fail-closed policy for high-risk questions
Duplicate or reordered source event	Duplicate facts or state regression	4	3	2	24	Provider revision ordering, observation uniqueness, idempotent reducers, reconciliation
Missed webhook	Stale organizational state	3	3	2	18	Freshness SLO, periodic reconciliation, cursor comparison
ACL revocation lag	Unauthorized derived access	5	2	2	20	High-priority invalidation, policy version checks, cache purge, projection rebuild, incident if breached
Model provider unavailable	Answers and extraction delayed	3	3	1	9	Queue within expiry, fail closed, no unevaluated fallback, visible status
Model hallucination	Unsupported organizational claim	4	3	3	36	Evidence resolution, citation verifier, abstention threshold, continuous eval
Prompt injection	Exfiltration or unintended action	5	3	3	45	Content separation, schema boundaries, tool gateway, approvals, red-team suite
Workflow worker crash	Run pauses or repeats activity	3	3	1	9	Temporal replay, heartbeats, idempotent activities, bounded retry
Action approval expires	User expects action but none occurs	2	3	1	6	Explicit status, no execution, create a new preview
Jira times out after accepting write	Duplicate issue risk	4	2	3	24	Provider correlation and idempotency lookup before retry
Compensation fails	External state differs from intended rollback	4	2	1	8	Manual-intervention terminal state, alert, evidence and runbook
Cache stale or poisoned	Wrong result or authorization leak	5	2	3	30	Tenant-qualified keys, policy version, short TTL, never authoritative, canary tests
Clock skew	Approval or temporal query inconsistency	3	2	2	12	Trusted time service, server timestamps, skew monitoring, leeway bounds
Regional loss	H2 service outage and possible data loss	5	1	2	10	Encrypted backup, restore drill, RPO/RTO alert and projection rebuild

Failure mode	Effect	S	O	D	RPN	Detection and response
Deployment rollback with schema mismatch	Runtime failure or data incompatibility	4	2	2	16	Expand-contract migrations, compatibility gate, feature flags, forward-fix plan

Privacy and AI Impact Assessment

Artifact: `docs/enterprise-digital-twin/reviews/privacy-impact-assessment.md`

Privacy and AI Impact Assessment

Intended purpose

H1 helps an authorized software organization understand project dependencies and compare a launch-delay scenario. It uses deterministic synthetic data. H2 may process customer-approved engineering and project metadata for the same purpose under a documented controller/processor allocation and data processing agreement.

People and data

Potential data subjects include employees, contractors, customers, vendors, and connector administrators. Data classes include identifiers, roles, project assignments, work artifacts, comments, access-control lists, service metadata, model inputs and outputs, approvals, and audit metadata. Secrets, private messages, source-file bodies, Actions logs, health data, emotion inference, and individual productivity scores are excluded from H1.

Necessity and proportionality

- Collect only allowlisted repositories, projects, object types, and fields required for delivery dependency analysis.
- Prefer aggregate team capacity over individual behavior measures.
- Preserve source ACLs and display why a result is visible.
- Use synthetic data for public demonstrations and golden evaluations.
- Do not use customer data for cross-tenant training, retrieval, memory, or analytics without a separate explicit opt-in assessment.

Lifecycle and rights

The data inventory records purpose, category, source, owner, retention, region, subprocessors, derived stores, and deletion method. Access, export, correction, objection, restriction, and eligible deletion requests propagate through observations, claims, objects, graph, vector, search, cache, traces, evaluation sets, and backup expiry. Legal hold is purpose-limited, access-controlled, and visible to authorized administrators.

High-risk exclusions

Through H3 the product does not rank or score individual burnout, attrition, productivity, performance, promotion, compensation, hiring, firing, layoff, health, emotion, misconduct, or union activity. It does not make legal, credit, insurance, financial, or safety decisions. A future proposal requires a separate intended-purpose and jurisdiction assessment, counsel and DPO review, scientific construct validity, fairness evidence, meaningful human review, notice, correction and appeal, monitoring, and kill switch.

Assessment outcome

H1 is acceptable because it uses synthetic data and enforces the same tenant, ACL, minimization, action, and audit paths planned for production. H2 remains conditional on customer-specific purpose, region, contract, retention, subprocessor, worker-notice, and source-scope approval.

Architecture Review and Remediation Ledger

Artifact: `docs/enterprise-digital-twin/reviews/review-ledger.md`

Architecture Review and Remediation Ledger

Review 1 - cross-domain design audit

Finding	Severity	Resolution
Hackathon and enterprise claims were conflated	High	Split H1 demonstrator from H2-H5 and prohibit GA claims without evidence.
Every technology was implied as mandatory	High	Added adopt, conditional, reject matrix and evidence-based introduction triggers.
Exhaustive ontology was not evolvable	High	Added stable core, inherited lifecycle rules, namespaced packages, and migration contract.
Shared graph tenancy relied on an application filter	High	Added query gateway, independent namespaces, negative tests, residual risk, and cell migration trigger.
Broad autonomy lacked an authority ceiling	Critical	Added policy gateway, delegation intersection, two-person exact-payload approval, prohibited actions, budgets, and kill switch.
Simulation could be mistaken for prediction	High	Selected reproducible PERT Monte Carlo, explicit uncertainty, synthetic label, and no individual productivity input.
Compliance language implied certification	Medium	Reframed as control readiness with organizational evidence owners.

All Critical and High findings were remediated in the normative baseline.

Review 2 - security, privacy, AI, and buildability audit

Finding	Severity	Resolution
Derived ACL behavior was underspecified	High	Defined claim-level evidence ACLs, monotonic visibility, revocation invalidation, and side-channel tests.
Approval replay and timeout ambiguity	High	Bound approval to canonical payload and idempotency key; added provider lookup and compensation terminal states.
Trace collection could become a shadow content store	Medium	Added minimization, redacted references, restricted access, and retention.
Cross-tenant learning default was unclear	High	Explicitly prohibited cross-tenant retrieval, resolution, memory, analytics, and training.
H1 scale lacked a test envelope	Medium	Fixed two tenants, 100 identities, 100k nodes, 1M edges, ten users, freshness and latency objectives.

All High findings were remediated. Medium residual risks are owned in `catalogs/risks.yaml`.

Convergence record

The two formal reviews introduced no remaining Critical or High finding after remediation.

Implementation validation record - 2026-07-13

This record is implementation evidence for `1.0.0-rc.1`; it is not the independent `AC-REV-001` sign-off.

Evidence	Result	Boundary
TypeScript compilation	All API, sync-worker, and web type checks passed; all four production workloads built.	Local Node.js toolchain.
Isolated automated suites	25 Python tests, 11 ordinary web tests, 2 sync-worker tests, and 4 API end-to-end cases passed.	The live-only web case is intentionally excluded from the ordinary web suite and executed separately.

Evidence	Result	Boundary
Live cross-workload journey	The live web client drove the API and Python worker with oracle fallback disabled; cited answer, 50,000-trial simulation, exact preview, two distinct approvals, single execution, replay safety, separately approved compensation, 14 audit events, and zero Beacon disclosure passed in 7.81 seconds of test execution.	In-memory authoritative-store profile with real HTTP workload boundaries; no external provider or model effect.
Operator verification	`scripts/verify_live.mjs` returned `status: verified`, simulation engine `pert-mt19937-beta/2.0.0`, replay safety, restored `2026-08-07` due date, 14 audit events, and zero cross-tenant disclosure.	Fresh API state and the real Python simulation worker.
Connector replay	Aster synchronized six sources/four relationships twice to identical state and cursor digests; Beacon synchronized its independent two-source fixture; external effect count remained zero.	In-memory provider simulator. PostgreSQL/S3/Neo4j/Temporal integration is exercised by the Compose profile and CI image gate.
Normative publication	44 Markdown and 60 machine-readable artifacts passed with zero errors/warnings and 100 percent requirement traceability; deliberate trace corruption failed closed.	Clean source build on the local toolchain.
PDF inspection	The 235-page PDF passed metadata/text extraction and visual inspection of the cover, contents, representative dense tables, all ten diagram pages in portrait/landscape, and final provenance page.	Local Poppler render at 144 DPI for diagram inspection.
Dependency audit	Full and production npm audits reported zero Critical/High and the two accepted Moderate PostCSS findings recorded as `RSK-019`.	Source-controlled CSS only; the risk acceptance expires on its recorded trigger/date.
Deployment sources	Compose, Helm, OpenTofu, health checks, and immutable GitHub Action references passed static validation.	Docker is not installed on the local workstation; Compose configuration and all four OCI image builds are mandatory in `.github/workflows/application.yml`.

The in-app browser runtime was unavailable on the local workstation, so this record does not claim interactive browser, screenshot-regression, keyboard, or assistive-technology evidence. Those remain CI/manual release-candidate evidence under CH-14.

Remaining final-release gates

- An engineer independent of the authoring and implementation work must record `AC-REV-001` after reproducing H1 from a clean checkout.
- The application workflow must publish successful Compose configuration and four-image build evidence on a Docker-enabled runner.
- The required browser/accessibility review must be attached before any H2 production or final accessibility claim.

Until those gates are evidenced, the specification remains `1.0.0-rc.1` and MUST NOT be represented as final `1.0.0`.

Residual Risk Acceptance

Artifact: `docs/enterprise-digital-twin/reviews/risk-acceptance.md`

Residual Risk Acceptance

No Critical or High residual risk is accepted for H1 or H2.

Risk	Rationale	Compensating controls	Owner	Expiry or revisit
RSK-008 cloud-neutral abstraction cost	Portability is an explicit product decision	Thin adapters, portable contracts, conformance tests	Platform	H2 provider selection

Risk	Rationale	Compensating controls	Owner	Expiry or revisit
RSK-009 limited polyglot cost	H1 has only TypeScript and Python application runtimes	Contract generation, ownership boundaries, language introduction ADR	Architecture	H3 service extraction
RSK-016 synthetic data external-validity limit	Public demo safety and deterministic verification outweigh realism	No predictive-accuracy claim, golden oracle, H2 design-partner validation	Product	Before H2 prediction claims
RSK-019 framework CSS-tool pin	Next.js 16.2.10 pins PostCSS 8.4.31, which has a Medium stringification advisory; H1 has no untrusted CSS, templates, runtime themes, or style-stringification input	Source-controlled CSS only, zero production High/Critical dependency findings, dependency audit in CI, monthly framework update review	Developer Platform and Security Architecture	2026-08-15, any compatible Next.js release, or any proposal for untrusted style/template input

Acceptance expires automatically if scope, data classes, jurisdictions, deployment mode, or authority expands. A changed risk requires a new review rather than silent continuation.

Threat Model

Artifact: `docs/enterprise-digital-twin/reviews/threat-model.md`

Threat Model

Scope and assets

The assessment covers identity, tenant context, API, connector installations, normalized observations, authoritative claims, object storage, graph/vector/search projections, agent context, model and MCP/tool calls, scenarios, approvals, Jira actions, audit, deployment, and the documentation supply chain.

Highest-value assets are tenant data, source ACLs, connector credentials, encryption keys, policy bundles, action approvals, audit evidence, model/tool configuration, and integrity of simulation results.

Trust boundaries

All browser input, connector payloads, uploaded content, webhooks, model output, MCP output, provider errors, and cross-workload messages are untrusted. Identity assertions become trusted only after issuer, audience, signature, lifetime, nonce, and membership validation. Tenant context is server derived. Model output never becomes authority.

STRIDE analysis

Threat	Example	Required controls	Verification
Spoofing	Forged webhook, token, approver, or service identity	OIDC validation, signed webhook verification, workload identity, MFA, nonce and replay cache	Invalid signature, stale token, wrong audience, and impersonation tests
Tampering	Change action payload after approval or corrupt claims	Canonical hashes, optimistic versioning, exact-payload approval, signed artifacts, hash-linked audit	Payload mutation, concurrent update, audit-chain verification
Repudiation	Approver denies authorizing Jira write	Distinct authenticated approvals, policy version, immutable timestamps and receipts	End-to-end evidence reconstruction
Information disclosure	Cross-tenant ID, graph path, embedding, cache, error, or trace leak	RLS, independent namespaces, ACL monotonicity, redaction, non-enumerable errors	Two-tenant negative suite and canary-secret scans
Denial of service	Graph explosion, model loop, webhook flood, decompression bomb	Traversal limits, quotas, budgets, rate limits, bounded parsers, backpressure, circuit breakers	Load, fuzz, budget, and resource-exhaustion tests
Elevation of privilege	Agent delegation widens tools or tenant	Delegation intersection, external policy gateway, immutable tool manifests, no client tenant selection	Handoff, confused-deputy, and policy-bypass tests

AI and connector abuse cases

- Indirect prompt injection in Jira text requests a secret, another tenant, or a write tool. The content remains a user-data field, structured extraction runs without privileged tools, and the policy gateway denies authority changes.
- A compromised connector sends huge nested payloads or URLs to internal services. Size, depth, format, network, DNS, and egress constraints reject or quarantine it.
- A model invents evidence or tool arguments. Evidence identifiers must resolve under current ACLs, schemas reject unknown fields, and action approval displays the exact canonical payload.
- A remote MCP server changes its tools or output. Tool allowlists, server identity, schema version, approvals, egress checks, and trace records prevent silent authority expansion.
- An evaluator or trace store captures secrets or restricted content. Evaluation datasets are minimized and tenant-scoped; trace exports default to metadata and redacted references.

Residual risk

Pooled graph isolation remains weaker than PostgreSQL RLS. H1 accepts this only with bounded server-owned query templates, tenant-qualified projection assertions, two-tenant tests, and no arbitrary Cypher. Failure of any control blocks release and triggers isolated graph deployment analysis.

Build Provenance

Manifest SHA-256: `e1f17b58c501cb97f8aca9201379fa56cf1930406b353b980e72891b9609eaa9`

Generated by `scripts/build_blueprint.py`; generated editions are not edited directly.